TURING 图灵计算机科学丛书

Springer

信息检索

算法与启发式方法

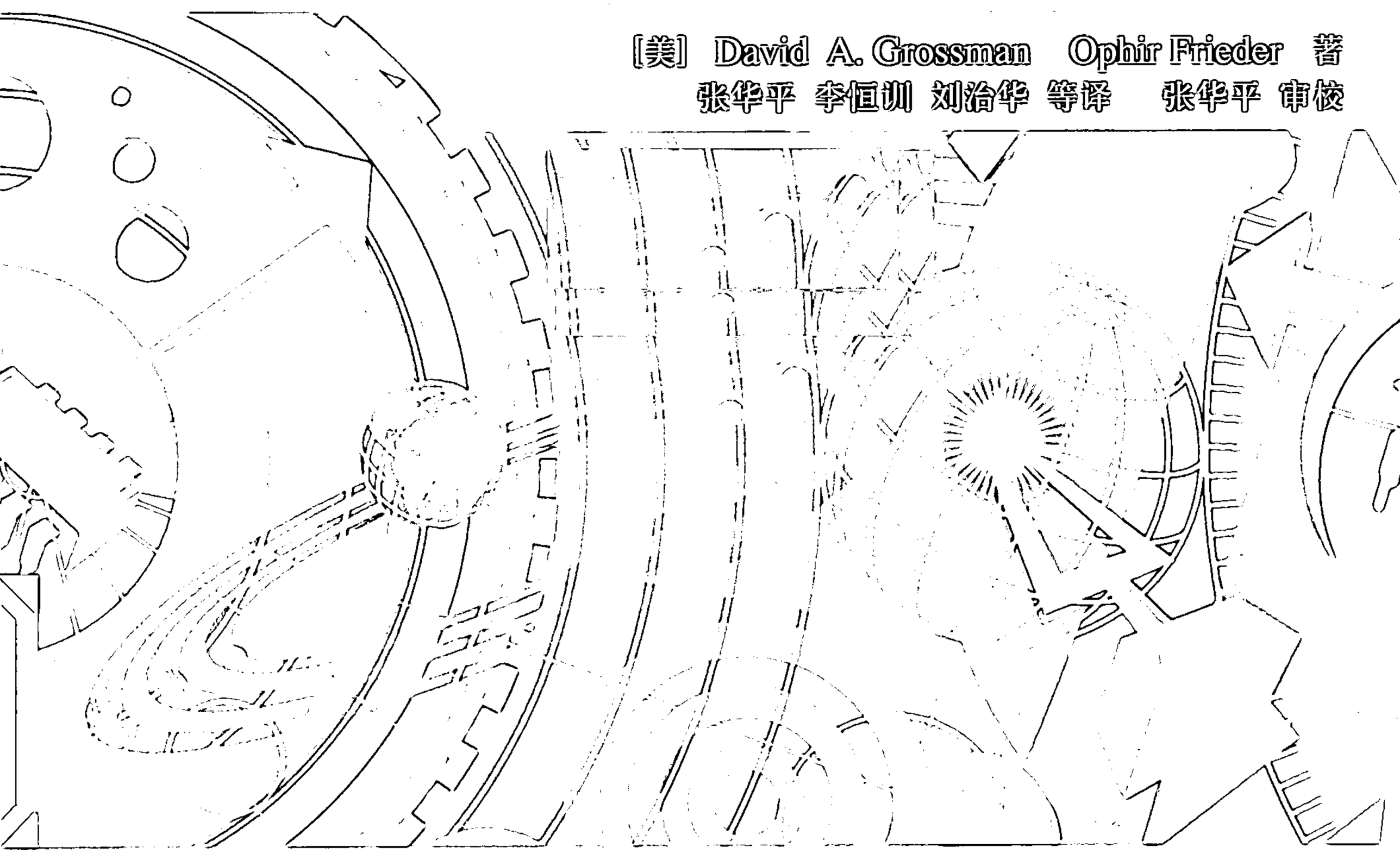
（第2版）

Information Retrieval: Algorithms and Heuristics

Second Edition

[美] David A. Grossman　Ophir Frieder 著

张华平 李恒训 刘治华 等译　张华平 审校



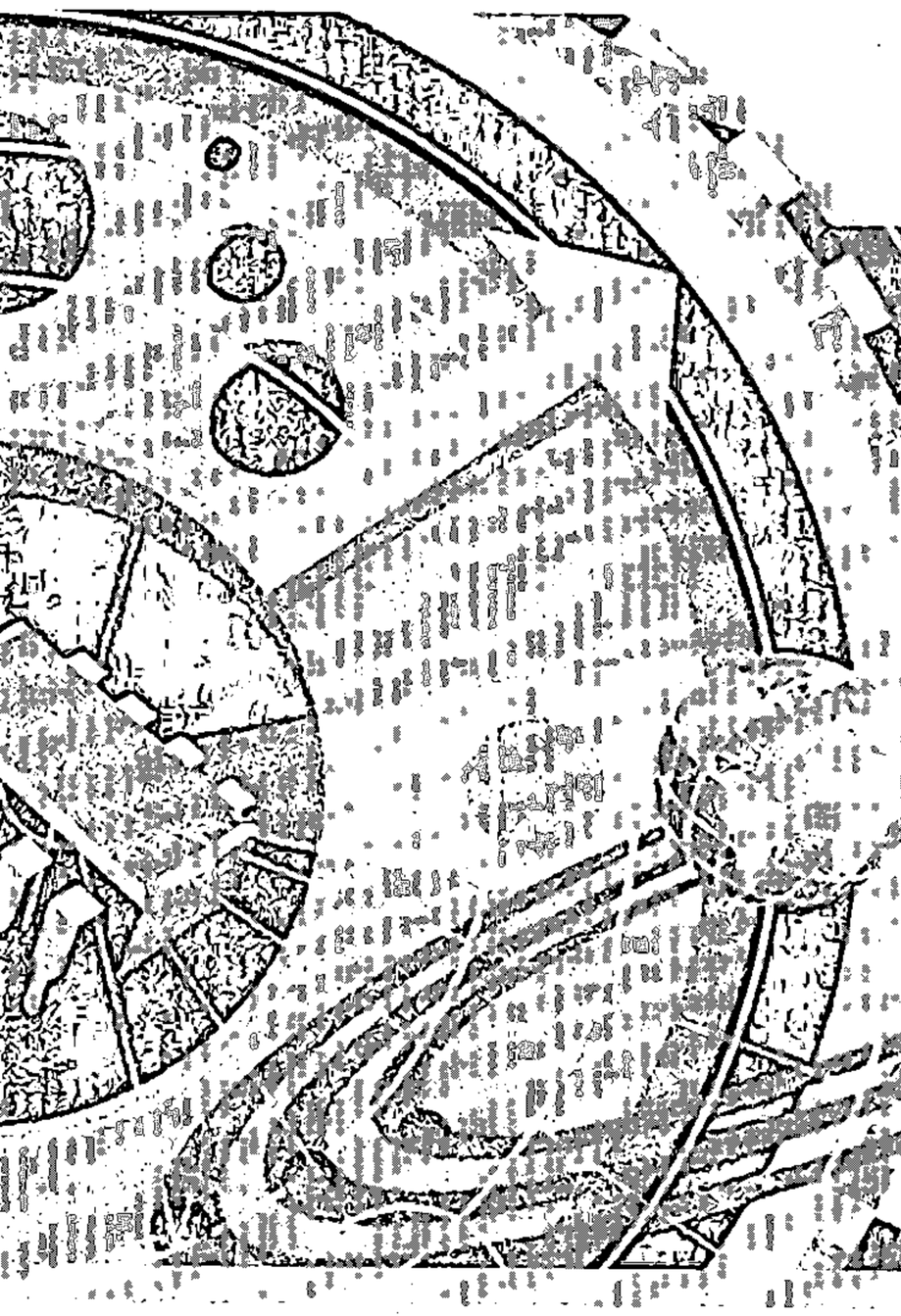
人民邮电出版社
POSTS & TELECOM PRESS

TURING 图灵计算机科学丛书

信息检索
算法与启发式方法
（第2版）

Information Retrieval: Algorithms and Heuristics

Second Edition



[美] David A. Grossman　Ophir Frieder 著
张华平　李恒训　刘治华　等译　　张华平　审校

人民邮电出版社
北　京

图书在版编目（CIP）数据

信息检索 : 算法与启发式方法 : 第2版 / (美) 格罗斯曼 (Grossman, D. A.) , (美) 弗里德 (Frieder, O.) 著 ; 张华平等译. -- 北京 : 人民邮电出版社, 2010.9
(图灵计算机科学丛书)
书名原文: Information Retrieval:Algorithms and Heuristics, Second Edition
ISBN 978-7-115-23575-6

Ⅰ. ①信… Ⅱ. ①格… ②弗… ③张… Ⅲ. ①情报检索 Ⅳ. ①G252.7

中国版本图书馆CIP数据核字(2010)第142198号

内 容 提 要

本书是“信息检索”课程的优秀教材，书中对信息检索的概念、原理和算法进行了详细介绍，内容主要包括检索模型与算法、检索实用策略、跨语言信息检索、查询处理、融合结构化数据和文本、并行信息检索以及分布式信息检索等，并给出了阐述算法的大量实例。

本书有一定的广度和深度，而且所有的内容都用当前的技术阐述，是高等院校计算机及信息管理等专业本科生和研究生的理想教材，对信息检索领域的科研和技术人员也是很好的参考书。

图灵计算机科学丛书

信息检索：算法与启发式方法（第2版）

◆ 著 [美] David A. Grossman Ophir Frieder
译 张华平 李恒训 刘治华 等
审校 张华平
责任编辑 王军花

◆ 人民邮电出版社出版发行 北京市崇文区夕照寺街14号
邮编 100061 电子函件 315@ptpress.com.cn
网址 http://www.ptpress.com.cn
三河市海波印务有限公司印刷

◆ 开本：787×1092 1/16
印张：15.25
字数：390千字 2010年9月第1版
印数：1－3 000册 2010年9月河北第1次印刷

著作权合同登记号 图字：01-2009-4808号

ISBN 978-7-115-23575-6

定价：49.00元

读者服务热线：(010)51095186 印装质量热线：(010)67129223
反盗版热线：(010)67171154

版 权 声 明

Translation from the English language edition: *Information Retrieval: Algorithms and Heuristics, Second Edition* by Davis A. Grossman and Ophir Frieder.

Copyright © 2004 Springer, The Netherlands, as a part of Springer Science+Business Media.

All Right Reserved.

本书简体中文版由Springer Science+Business Media授权人民邮电出版社独家出版。未经出版者书面许可，不得以任何方式复制或抄袭本书的任何部分。

版权所有，侵权必究。

译 者 序

搜索引擎越来越受到普通大众、技术、产业与资本的热力追捧，CNNIC 发布的《第 25 次中国互联网络发展状况统计报告》显示，搜索引擎在网络应用使用率中排名第三，达到了 73.3%。2009 年中国搜索引擎的市场规模达到 69.5 亿元，在经济危机的背景下，搜索引擎营销获得大品牌广告主的认可。

作为网络搜索与挖掘十余年的研发者，我在研发与教学工作中发现，目前搜索引擎技术方面的资料往往有两种不足：一种是过于学术化，关注于一个很窄的话题，读起来生涩难懂，缺乏一定学术背景的读者很难真正理解；另外一种是原理性的介绍，往往比较浅显，缺乏一定的深度，读者读起来总觉得意犹未尽。而本书是难得的一本佳作。一方面，它全面综合了信息检索领域的各类研究，融合了最新的研究成果；另一方面，它采用通俗易懂的行文方式，在关键点上采用了详尽的案例来解释抽象的过程，读者并不需要具备专业的数学背景就很容易掌握信息检索技术的精髓所在。

本书主要作为本科生或研究生信息检索课程的教材，也特别适合信息检索领域从事应用开发的研发人员使用。在国外，它已经被多所高校作为研究生与本科生的教材，广受好评。

我非常有幸应人民邮电出版社图灵公司之邀翻译这本信息检索领域的杰作。对我们来说，这是一次学习过程，也是多年研究工作的总结提炼过程。翻译从来就不是一件轻松的事，加之正赶上工作调动，在时间和精力上受到一定影响。但我们依然克服了各种困难，每周固定开会研讨，字斟句酌，抱着严谨的学术态度，尽可能忠实表达作者的原意，同时兼顾语言之美。在“苦行僧”式的翻译学习过程中，我们往往也会从作者敏捷的思维和精巧的话语中领略到一种无穷的智慧。

在此，我要感谢原作者 Grossman 与 Frieder 的卓越工作，感谢我的研究生忍受我的各种严格要求，感谢人民邮电出版社图灵公司给予我们这次学习的机会。同时，我要感谢北京理工大学计算机学院院长黄河燕教授在此工作过程中给我的支持，也要感谢中科院计算所网络重点实验室主任程学旗研究员对我在信息检索领域的指导。最后，我特别感谢我的妻子，感谢她在这段艰难的时光的默默支持。

本书翻译分工如下：由我负责翻译本书的第 1 章、第 7 章与第 9 章，我的研究生李恒训同学负责翻译第 2 章，刘治华同学负责翻译第 3 章，李恒训同学、秦鹏同学和周鹏同学负责翻译第 4 章和第 5 章，张京阳同学负责翻译第 6 章，蒋骄同学负责翻译第 8 章。另外，我对全书进行了多次全面的审校。

限于水平有限，错误在所难免，恳请读者批评指正。

张华平博士

E-mail: kevinzhang@bit.edu.cn

2010 年 1 月 25 日于北京理工大学计算机语言信息处理研究所

序

正如格劳斯曼（Grossman）和弗里德（Frieder）在“前言”中所述，信息检索（IR）在过去 5 年里取得了相当大的进步。对普通人来说，这种进步充分体现在网络商用搜索引擎的日趋成熟；对从事信息检索的人来说，5 年来的进步拓展了网络搜索问题的研究范畴，并突破了诸多局限，随着基础体系架构和信息检索模型的发展，整个检索过程不断地引入了新的视角，同时开展了一系列令人振奋的应用，比如跨语言检索、P2P 搜索和音乐检索，这些都极大地拓宽了信息检索研究的疆域。数据库与信息检索这两个不同领域的学者逐步达成共识：必须整合非结构化与结构化数据的处理技术，我们才能够真正解决未来社会的信息问题。本书阐述了许多这方面的重要进展，也是迄今为止唯一一本这样做的教科书。

本书让我印象最深的两个例子是信息检索语言模型和跨语言检索。语言模型强大而简洁，而且在搭建很多实验和应用时可以使用许多现成的工具，因此很多研究人员都在采用这一模型，并且语言模型已经成为主流信息检索学术会议研究的重要课题。格劳斯曼和弗里德在第 2 章中很好地概述了这一课题，同时也给出了不同平滑技术的例子。跨语言检索是指采用某种语言检索其他多种语言的文本内容，欧洲和美国政府存在广泛的实际需求，这也一直推动着跨语言检索的迅猛发展。充分利用平行语料库和可比语料库，人们研制了一些方法，这些系统的性能现在已接近（在某些方面甚至已超越）单一语言的检索系统。本书专门增加了一章跨语言检索的内容，清晰地阐述了跨语言检索的主要方法，并给出了示例来具体说明如何在真实数据上执行算法。本书覆盖了最新的研究成果，采用准确直接的语言，同时频繁使用了大量实例，可作为研究生或本科生信息检索课程的首选教材。

W. 布鲁斯·克劳福特（W. Bruce Croft）

2004 年 8 月

致　　谢

本书的第 1 版于 1998 年出版。从那以后，信息检索领域发生了翻天覆地的变化。

我们要感谢那些帮助我们准备第 1 版大部分材料的人。在此依然要深深地感谢 Paul Kantor 和 Don Kraft，感谢他们在本书编写的早期阶段给出的真知灼见。也要特别感谢 Steven Robertson、K.L. Kwok 和 Jamie Callan，本书的一些章节介绍了他们的相关工作，他们对这些章节做了严格的审查；同时，我们还要感谢 Warren Greiff，感谢他极其耐心地反复教我们推理网络的各种细节问题。

本书增加了语言模型和跨语言信息检索相关的内容，为此，我们要特别感谢翟成祥、Doug Oard 和 James Mayfield，感谢他们富有洞察力的点评与建议。

同时，本书还得到了很多人一些非常重要的反馈。通过讲授本科“信息检索”以及研究生“高级信息检索”等课程，我们从学生中得到了诸多反馈，最终提高了本书的易读性，使其更加适合课堂教学。John Juilfs 为我们花了无数的时间，仔细检查了这一版中所有的新增示例，并复查了所有数学方面的内容（当然，我们对剩余的任何错误仍然要负全责，而他帮我们避免了大部分错误）。Abdur Chowdhury 为 5.4 节提供了素材，并进行了仔细的检查。Wei Gen Yee 对第 8 章的修改提供了重要的材料。Steven Beitzel 和 Eric Jensen 为我们更新第 5 章和第 7 章的内容提供了重要的素材。Rebecca Cathey 为第 4 章提供了示例。Michael Saelee 为我们制作并细化了各种图表。最后，管伟反复仔细地检查了所有的示例和符号。

也要感谢 Wendy Grossman、Ben Goldfarb、David Roberts 以及 Jordan Wilberding，他们给出了详尽的点评意见。同样要感谢所有其他读过本书并给出了精彩反馈的人，感谢给我们提供了精神支持的朋友和同事。

如果没有家人和亲朋精神上持续不断的支持，我们也很难完成本书。为此，特别感谢 Mary Catherine McCabe 和 Nazli Goharian，她们为我们牺牲了无数时间，给了我们坚持不懈的鼓励和支持！

我们俩对所有这些人，道以最诚挚的谢意！

前　言

我们在 1998 年写本书第 1 版时，Web 还是比较新鲜的事物。实际上，信息检索是一个比较老的研究领域，只不过还没有引起广泛关注罢了。如今，Google 已成为流行词汇，Google 索引了网页 40 多亿页。1998 年，只有几所学校为研究生开设了信息检索课程；而如今，信息检索在本科阶段都已普及。文献[Goharian 等人，2004]总结了我们讲授本科生信息检索课程的经验，详细分析了在课堂上讨论的每个专题内容，并介绍了课程教学的效果。

信息检索指的是搜索任何形式的信息，包括结构化数据、文本、视频、图像、声音、乐谱、DNA 序列等。事实是，多年来，数据库系统用来搜索结构化数据，信息检索则用来搜索文档。本书的作者原本就从事结构化搜索领域的研究，但是在过去十年的大部分时间里，都在研究文档的检索。对我们来说，客观世界的数据类型本身就是不可知的，因此，我们没有必要特别区分结构化数据与非结构化数据。1998 年，我们在本书第 1 版中有一章内容专门讲述数据整合，书评人则认为，收录该部分内容的唯一原因就是它涉及我们最新的一些研究罢了。而现在，这种评述或者辩解已然没有任何意义了，因为我们已经引入了信息的中间表示结构（mediator），可以对结构化和非结构化数据同时进行操作。而且，XML（eXtensible Markup Language，可扩展标记语言）已经广泛地应用于数据库与信息检索领域。

我们主要关注 ad hoc 信息检索问题[①]。简单来说，ad hoc 信息检索指的是针对用户提交的各种不同查询，搜索出相关的文档集合。像 Google 这样的系统可能已经解决了这个问题，但是，Google 的性能评测并没有公布。一些经典系统的准确率最高也只能达到 40%[TREC, 2003]。在对现有算法深入理解的基础上，我们仍有很大的改进空间。

市面上信息检索教材的内容相对散乱，并不适合我们的日常教学。这些教材在许多关键检索模型的细节上往往避而不谈。推理网络是许多系统都要用到的核心模型，但是，几乎没有教材详细介绍推理网络。另外，许多教材都没有详细描述系统的效率，即单一查询的执行速度。或许，对于那些只关注检索效果的人来说，检索效率能够引起的潜在兴趣特别有限；但是对于从业者来说，对效率的关注可以超越其他所有的指标。

另外，针对每种方法，我们都给出了详细可行的实例。当介绍具体方法时，我们很容易在细节方面轻描淡写，不过，实例可以让我们更忠实地阐述方法的本质所在。我们发现贯穿于整本书的一个实例能让学生们从中受益。值得一提的是，本书每个描述核心检索算法的章节要么经过了算法原创者的评审（我们感谢他们的慷慨奉献，更多的感谢详见致谢），要么经过了精通该算法的专家审校。因此，就我们所知，本书所述检索算法的诸多细节目前还很难在其他出版物中找到。

---

① ad hoc 信息检索是信息检索中一个专门的任务形式，与 routing 相对应，前者的数据集相对稳定，而查询多变；后者反之。——译者注

我们的目标是写一本特别专注于 ad hoc 信息检索的书。为达到这个目标，我们基于模型建立了本领域一套完整的分类体系，主要包括文档和查询比较的算法模型，以及一些可以内嵌到所有算法模型中对性能进行优化的实用策略。本书中介绍了所有的基本方法，还有一系列的工具集。我们提供了足够详尽的说明，阅读本书的学生或者其他读者都可以方便地实现其中的方法或者工具。*Managing Gigabytes* [Witten 等人，1999]一书非常出色地阐述了倒排索引压缩的策略。我们引用了其中最新而且最有效的研究成果，但还是推荐读者将 *Managing Gigabytes* 作为本教材优秀的辅助读物。

在第 2 版中有什么新内容呢？许多核心的检索方法仍没有改变。自 1998 年以来，在信息检索领域引入语言模型的论文不计其数。因此，我们专门增加了语言模型的章节。跨语言信息检索（使用一种语言提交查询，而搜索出另一种语言的文档）在本书第 1 版刚问世时尚处于萌芽期，而如今它已经取得了长足的进步，我们在参考了最近 100 多篇相关文献之后，特别增加了一整章内容来介绍跨语言检索的最新研究进展。

自然而然，我们还讨论了许多当前的热点话题，比如 XML、P2P 信息检索、文本查重、文档并行聚类、不同检索策略的融合以及信息中间表示等。

最后，一些细心的本科生和研究生发现了上一版的一些错误，我们一一作了修正。这里，我们要感谢他们的努力。

本书主要作为本科生或研究生信息检索课程的教材。本书已经在我们的研究生课程中实际使用过，我们结合了同学们的反馈制作了一套与教材配套的幻灯片，可以在课堂教学过程中使用。这些资源可以从 www.ir.iit.edu 上获取。

另外，对于要搭建信息检索系统或相关应用程序的读者来说，如何选择恰当的检索方法和工具集用于产品开发，本书将会非常有用。我们曾收到几个读者来信，反映本书第 1 版对他们有帮助，我们将他们的意见和建议都吸收进了本书新版之中。

虽然我们强调本书的重点是算法而不是商用产品，但是据我们所知，本书中包含了大多数商用产品所采用的方法。我们相信读者或许会发现某些商用产品正在使用本书给出的信息检索方法，还能够将本书作为参考来更多地了解这些产品中采用的技术。

最后，我们注意到信息检索领域每天都在发生新的变化。有关本领域更多最新的研究成果，最好的资源有《ACM 信息系统杂志》（*the ACM Transactions on Information Systems*）、《美国信息科学与技术学报》（*the Journal of the American Society for Information Science and Technology*）、《信息处理与管理》（*Infornmtion Processing and Management*）和《信息检索》（*Information Retrieval*）等杂志。其他相关论文可以查询各种信息检索会议，比如 ACM SGIR（www.sigir.org）、NIST TREC（trec.nist.gov）、ACM CIKM （www.cikm.org）。

目　　录

第 1 章　引言 …… 1

第 2 章　检索模型与算法 …… 7

2.1　向量空间模型 …… 8

2.1.1　相似度计算举例 …… 11

2.1.2　相似度 …… 13

2.2　概率检索模型 …… 14

2.2.1　简单的词项权重 …… 15

2.2.2　非二值独立模型 …… 24

2.2.3　泊松模型 …… 25

2.2.4　文档片段 …… 29

2.2.5　概率模型的关键问题 …… 30

2.3　语言模型 …… 32

2.3.1　平滑 …… 33

2.3.2　语言模型举例 …… 34

2.4　推理网络 …… 40

2.4.1　相关背景 …… 41

2.4.2　链接矩阵 …… 42

2.4.3　相关性排序 …… 44

2.4.4　推理网络实例 …… 45

2.5　扩展布尔检索 …… 47

2.5.1　引入查询权重 …… 48

2.5.2　扩展为任意数量的查询词 …… 48

2.5.3　自动插入布尔逻辑 …… 49

2.6　LSI …… 49

2.6.1　LSI举例 …… 50

2.6.2　选择较优的k值 …… 52

2.6.3　与其他检索模型比较 …… 52

2.6.4　可能的扩展 …… 52

2.6.5　运行时性能 …… 52

2.7　神经网络 …… 52

2.7.1　向量空间 …… 53

2.7.2　相关反馈 …… 53

2.7.3　学习与调整 …… 54

2.7.4　概率检索 …… 54

2.7.5　基于片段的概率检索 …… 55

2.7.6　联合权重 …… 55

2.7.7　文档聚类 …… 56

2.8　遗传算法 …… 56

2.8.1　文档表示形式 …… 58

2.8.2　查询权重的自动赋值 …… 58

2.8.3　自动生成带权重的布尔查询 …… 59

2.9　模糊集检索 …… 59

2.9.1　布尔检索 …… 60

2.9.2　使用概念层次 …… 62

2.9.3　采用区间和提升效率 …… 62

2.10　本章小结 …… 63

2.11　练习题 …… 64

第 3 章　检索实用策略 …… 65

3.1　相关反馈 …… 66

3.1.1　基于向量空间模型的相关反馈 …… 67

3.1.2　基于概率模型的相关反馈 …… 68

3.2　聚类 …… 73

3.2.1　结果集聚类 …… 74

3.2.2　层次聚类 …… 74

3.2.3　不采用预定义矩阵的聚类方法 …… 75

3.2.4　在层次聚类结果中进行查询 …… 77

3.2.5　效率方面 …… 77

3.3　基于段落的检索 …… 78

3.3.1　基于标记的段落划分方法 …… 78

3.3.2　动态段落划分方法 …… 79

3.3.3　合并基于段落的相似度 …… 79

3.4　n元语法 …… 80

3.4.1　D'Amore与Mah方法 …… 80

3.4.2　Damashek算法 …… 81

3.4.3　Pearce与Nicholas方法 …… 81

3.4.4　Teufel …… 81

3.4.5 Cavnar和Vayda……82
3.5 回归分析……82
3.6 同义词表……84
3.6.1 自动构建同义词表……84
3.6.2 使用人工构建的同义词表……90
3.7 语义网络……91
3.7.1 距离计算方法……92
3.7.2 基于"概念"扩展查询词……95
3.7.3 基于约束激活扩散的排序……95
3.8 语言解析……96
3.8.1 单个词……96
3.8.2 简单短语……97
3.8.3 复杂短语……97
3.9 本章小结……100
3.10 练习……100
第4章 CLIR……102
4.1 简介……102
4.1.1 资源……102
4.1.2 评测……103
4.2 跨越语言障碍……103
4.2.1 查询翻译……104
4.2.2 文档翻译……105
4.2.3 短语翻译……105
4.2.4 译文的选择……105
4.2.5 翻译删减技术……107
4.3 跨语言检索模型与算法……107
4.3.1 CLIR中的语言模型……107
4.3.2 双语语料库方法……112
4.3.3 可比语料库方法……113
4.4 跨语言检索实用策略……117
4.4.1 跨语言检索的相关反馈……117
4.4.2 词干还原……118
4.4.3 *n*元语法模型……120
4.4.4 音译名……120
4.4.5 命名实体识别……121
4.4.6 检索融合……122
4.5 本章小结……122
4.6 练习题……123
第5章 检索效率优化……124
5.1 倒排索引……124
5.1.1 构建倒排索引……126
5.1.2 压缩倒排索引……127
5.1.3 变长索引压缩……129
5.1.4 基于倒排表大小的变长压缩……130
5.1.5 索引剪枝……132
5.1.6 在构建索引前对文档重新排序…132
5.2 查询处理……133
5.2.1 倒排索引的修订……133
5.2.2 部分结果集检索……134
5.2.3 简化向量空间……135
5.3 签名文件……136
5.4 重复文档检测……138
5.4.1 精确重复检测……139
5.4.2 近似重复检测……139
5.5 本章小结……141
5.6 练习题……142
第6章 结构化数据与文本的融合……143
6.1 关系模型回顾……145
6.2 相关工作进展……150
6.2.1 独立系统的融合……150
6.2.2 自定义运算符……151
6.2.3 NFN方法……152
6.2.4 使用标准SQL进行文献搜索……153
6.3 信息检索作为关系应用……153
6.3.1 预处理……155
6.3.2 实施案例……156
6.3.3 布尔检索……158
6.3.4 邻近搜索……161
6.3.5 使用标准SQL计算相关度……162
6.3.6 相关反馈在关系模型中的实现…164
6.3.7 关系信息检索系统……164
6.4 使用关系模式进行半结构化搜索……165
6.4.1 背景……165
6.4.2 使用静态关系模式支持XML-QL……165
6.4.3 存储XML元数据……166
6.4.4 跟踪XML文档……167
6.4.5 INDEX关系……167
6.5 多维数据模型……168
6.6 协同器……168

6.6.1　因特网协同器……168
6.6.2　内联网协同器……169
6.7　本章小结……171
6.8　练习题……171
第7章　并行信息检索……172
7.1　并行文本扫描搜索……172
7.1.1　文本硬件扫描……173
7.1.2　并行签名文件……174
7.2　并行索引……176
7.2.1　在连接机上实现并行索引……176
7.2.2　连接机的倒排索引……178
7.2.3　在DAP上实现并行索引……179
7.2.4　并行索引划分……179
7.2.5　在CM-5机上实现并行倒排索引算法……180
7.2.6　在倒排表上执行布尔操作……180
7.2.7　作为RDBMS应用的并行检索……180
7.2.8　并行索引小结……181
7.3　聚类与分类……181
7.4　大型的并行信息检索系统……182
7.4.1　PADRE……182
7.4.2　并行信息检索框架……182
7.4.3　PLIERS……182
7.5　本章小结……183
7.6　练习题……184
第8章　分布式信息检索……185
8.1　分布式检索的理论模型……186
8.1.1　集中式信息检索系统模型……186
8.1.2　分布式信息检索系统模型……187
8.2　Web搜索……189
8.2.1　Web搜索引擎评测……189
8.2.2　高准确率检索……189
8.2.3　查询日志分析……190
8.2.4　PageRank算法……190
8.2.5　Web搜索引擎的效果提升……191
8.3　结果融合……191
8.4　P2P信息系统……192
8.5　其他的体系结构……194
8.5.1　共享磁盘体系结构……195
8.5.2　分布式磁盘体系结构……195
8.6　本章小结……195
8.7　练习题……195
第9章　总结与下一步研究方向……197
参考文献……203
索引……229

第1章 引 言

社会文明伊始，人类就致力于书面形式的沟通。从洞穴石刻到卷轴书写，从印刷出版物到电子图书馆，沟通一直都是人类有史以来最为关注的事情。如今，随着数字图书馆和电子信息通信的涌现，人们的的确确需要不断提高有效管理海量信息的技术。各行各业都需要特别关注信息的溯源、处理、存储和检索等领域理论与实践的研究进展。在本书中，我们主要介绍电子文档查找和检索方面最新的研究成果。

我们介绍的重点就是如何检索出满足用户查询的信息。也就是说，我们讨论的是 ad hoc 信息检索算法与方法，简言之就是信息检索。图 1-1 显示了 ad hoc 信息检索的基本处理流程。在用户查询前，我们需要对静态或相对静态的文档集建立索引。用户提交一个查询请求后，系统会自动给出与查询相关的文档集合，同时按照各篇文档与查询计算出的相关度进行排序，最后将排序后的结果集推送给用户。目前，有许多用来解决文档排序问题的方法（即效果），这是本书讨论的一个重点。另外，还有一些技术用来实现文档的快速排序（即效率），这些也将在本书中给予讨论。

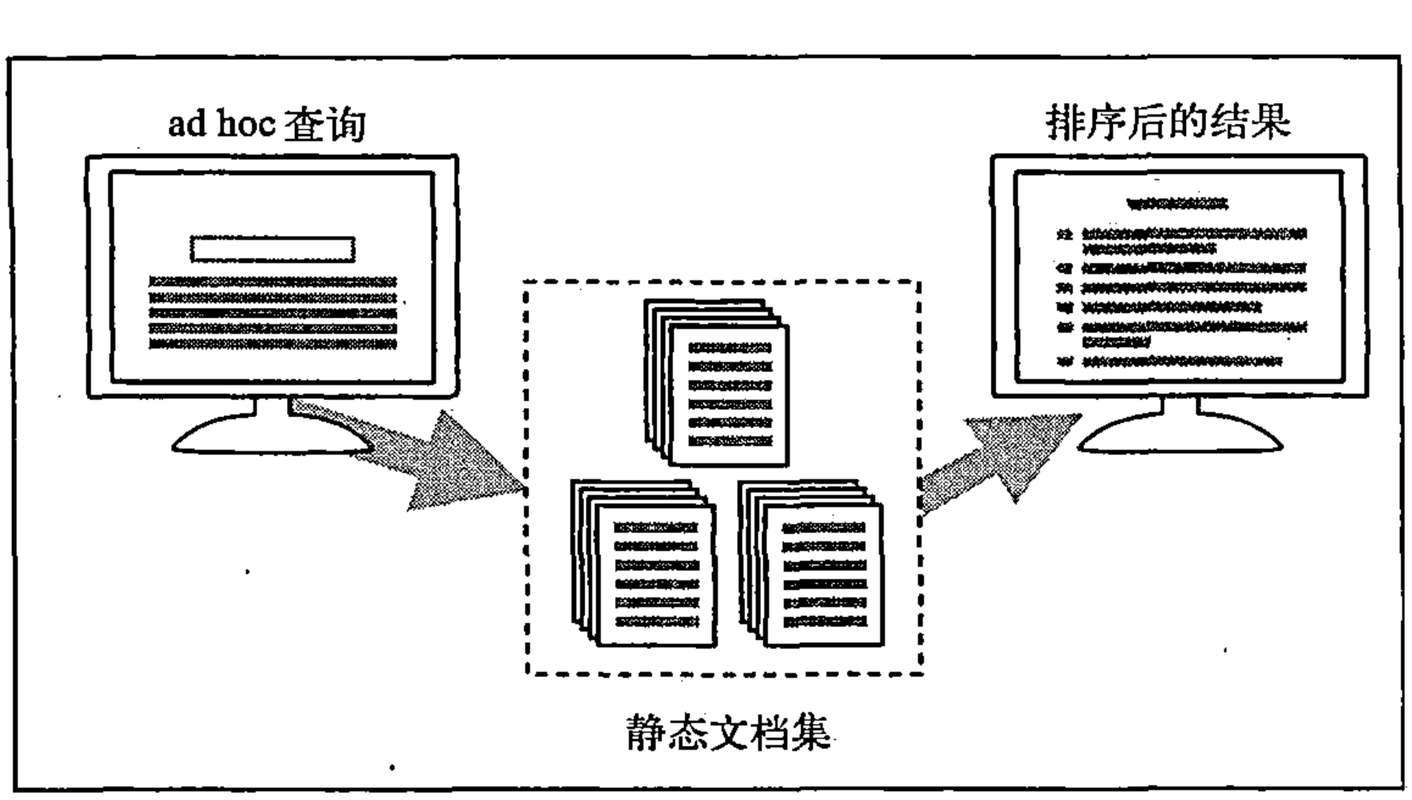


图 1-1　文档检索

信息检索（Information Retrieval，IR）致力于找到相关文档，而不是简单的模式匹配。然而，在信息检索系统的实际评测中，大家往往发现系统会遗漏许多相关文档[Blair 和 Maron，1985]。此外，用户对于信息检索系统精准率的期望也越来越高[Gordon，1997]。

另外一个与 ad hoc 信息检索相关的问题是文档分发（document routing）或文档过滤（document filtering）。两者的区别在于文档分发的查询是稳定不变的，而文档集则不断变化。比如企业往往会根据预定义的查询条件将公司邮件分门别类，分发到不同部门中（也就是关于销售的电子邮件分发给销售部，与市场相关的电子邮件被分发给市场部，等等）。图 1-2 给出了文档分发的示意图。文档分发算法与方法许多文献均有介绍，本书中不予详述。

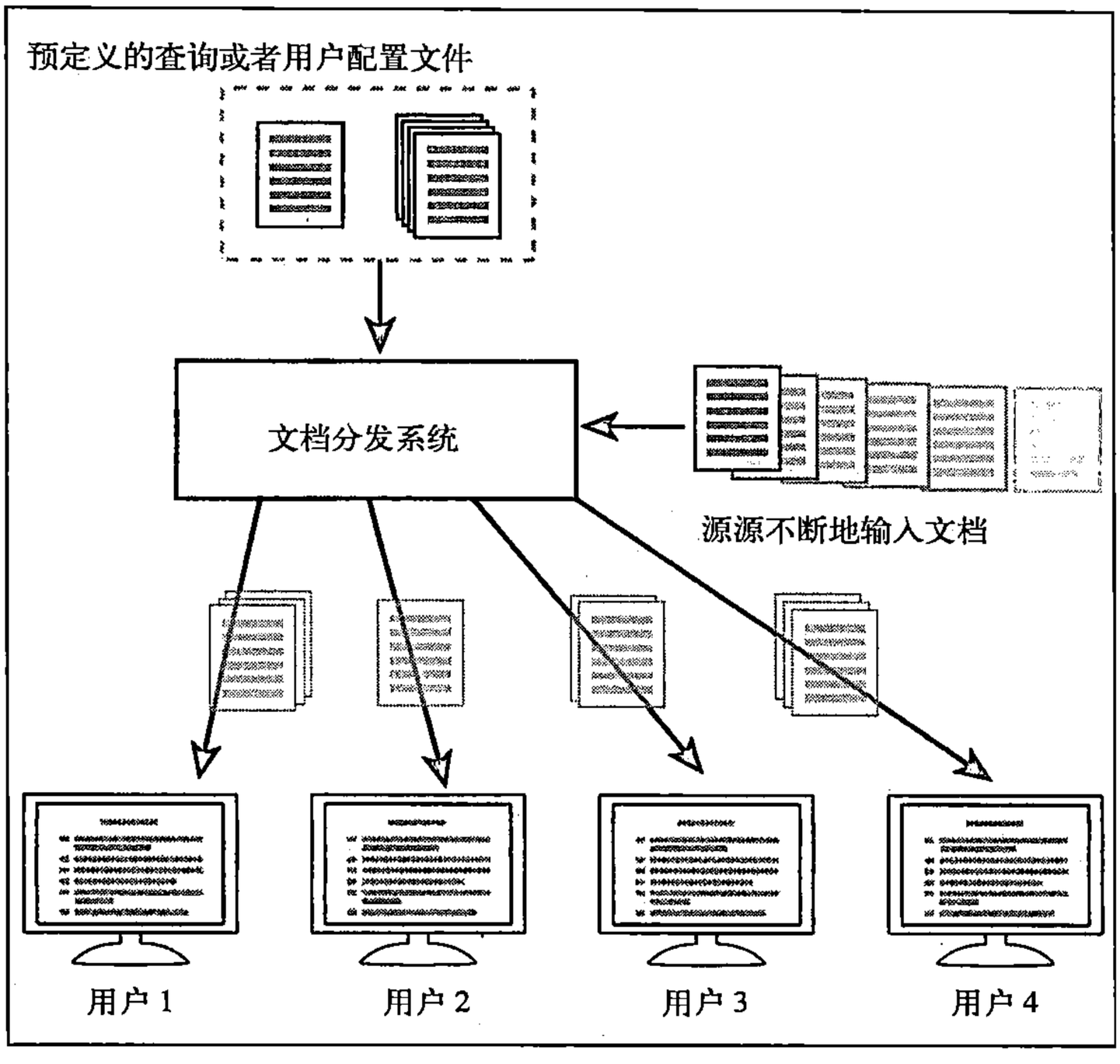


图 1-2　文档分发

针对任何一个查询，文档集合均可以分为关键的几个类别，如图 1-3 所示。在文档集合中，一部分是检索系统返回的文档，一部分是与查询确实相关的文档。理想情况下，这两个文档集应该是完全等同的，即我们只检索出相关的文档。而在现实生活中，系统往往会检索到许多不相关的文档。为了评测检索效果，我们采用两个指标：*准确率*和*召回率*。准确率是检索返回的相关文档数目占检索出文档总数的比率，它反映了检索结果集的质量。然而，准确率并没有考虑到相关文档的总数。比如，某个系统准确率可能很高，如检索返回的 10 篇文档中有 9 篇文档相关（即准确率为 0.9），但相关文档的总数也是至关重要的。如果文档集合内确实仅有 9 个相关文档，那么这个系统将是非常成功的。但是，如果有成千上万的文档是相关的，应该都被检索到而没有被检出，那么这可不是一个好的搜索结果集。

召回率会考虑相关文档的总数，它指的是检索返回的相关文档总数占文档集中相关文档总数的比率。计算所有相关文档的总数不是那么容易的，唯一可信的方法就是人工通读整个文档集并作出判断，这显然是不可行的。人们往往会采用一些近似估计的方法获取该值（参见第 9 章）。[Kantor，1994]综述了各类信息检索性能的评测方法，同时也对信息检索做了一个较好的概述。

在不同的召回率指标下，我们往往需要计算其对应的准确率。以某个查询 q 当做例子来考虑，如图 1-3 所示。对于这个查询，我们估计有两篇相关的文档。现在，假设当用户提交查询 q 时，返回 10 篇文档，其中包含这两篇相关文档。本例中，假定文档 2 和文档 5 是相关的，图 1-4 中的斜线表明在检索返回两篇文档后，我们找到了其中的一篇相关文档，从而达到 50%的*召回率*。此时，我们检索到了两篇文档并且其中一篇是相关的，因此*准确率*也是 50%。

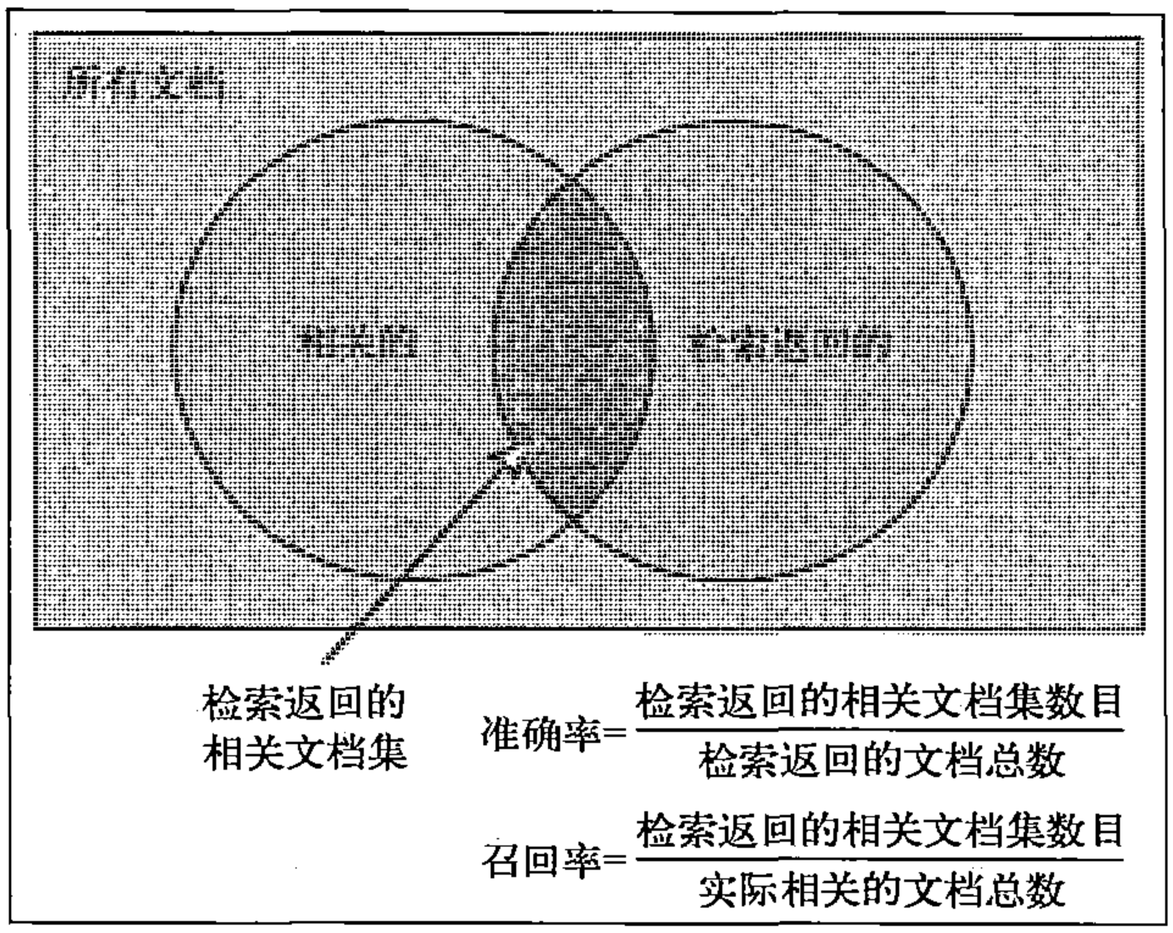


图 1-3 结果集：检索返回的相关文档集、相关文档集以及检索返回的文档集

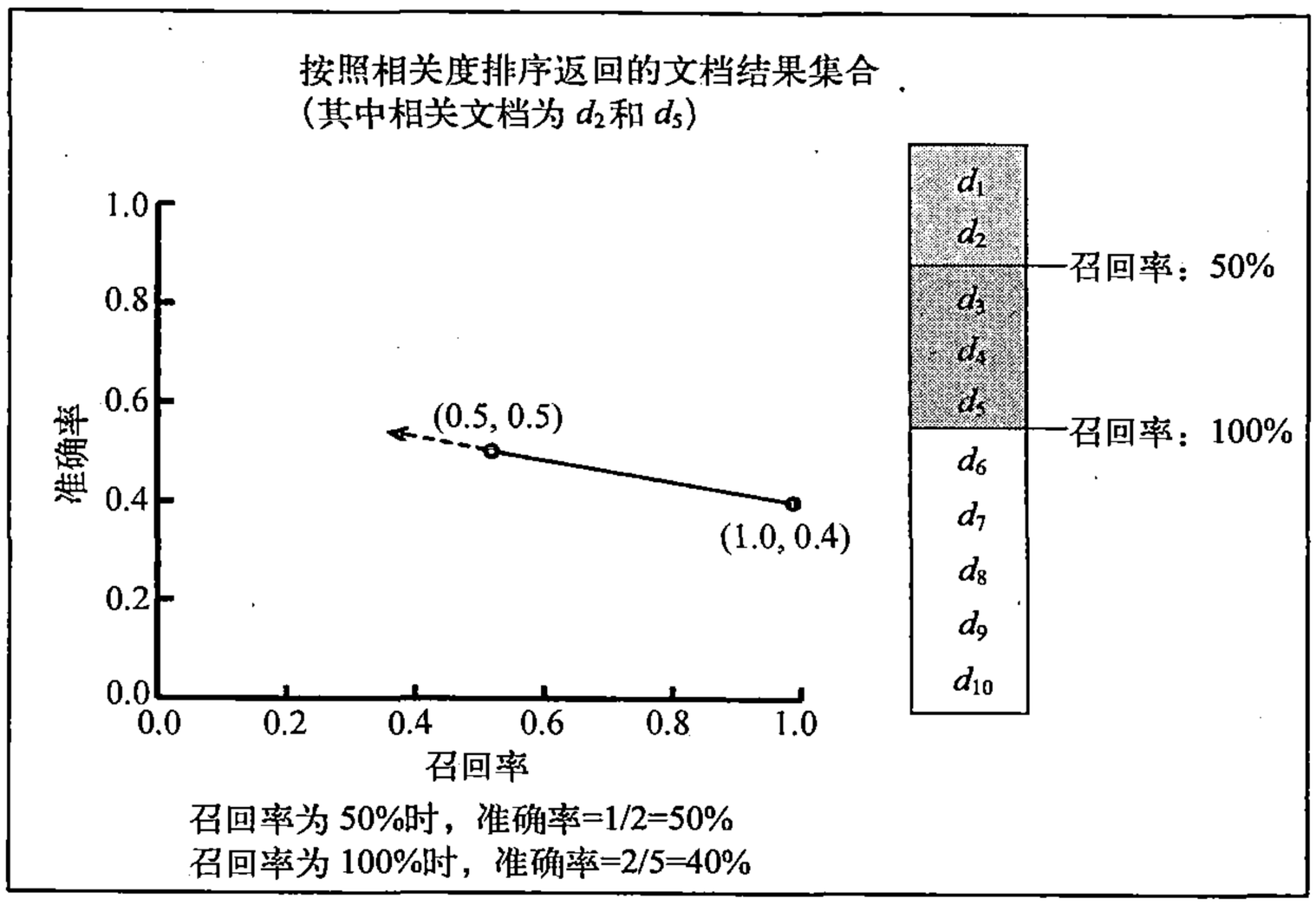


图 1-4 两种召回率以及相应的准确率

为了达到 100%的召回率，我们必须继续检索文档直到所有的相关文档都被返回。在这个例子中，必须返回 5 篇文档才能检索到两个相关文档。此时，准确率为 40%，因为在 5 篇返回的文档中有两篇是相关的。因此，在召回率达到期望值的时候，我们都需要计算相应的准确率。在不同的召回率上计算出准确率，最终形成变化的图形，我们称之为*准确率/召回率曲线*。

图 1-5 给出了一条典型的准确率/召回率曲线。通常情况下，要想取得更高的召回率，我们必须返回更多的文档才能达到期望的召回率。在一个完美的系统中，只检索输出相关的文档。这意味着在任意的召回率条件下，其准确率都为 1.0。最优准确率/召回率曲线如图 1-5 所示。

平均准确率指的是在不同召回率条件下准确率的平均值。在一个标准的文档集上进行测试，当前大部分系统的平均准确率在 0.2 到 0.3 之间。当然，这里有一些比较模糊的因素，因为相关

性还是一个不能清晰定义的概念，但是，有一点是特别明确的：信息检索领域的效果还有很可观的提升空间。

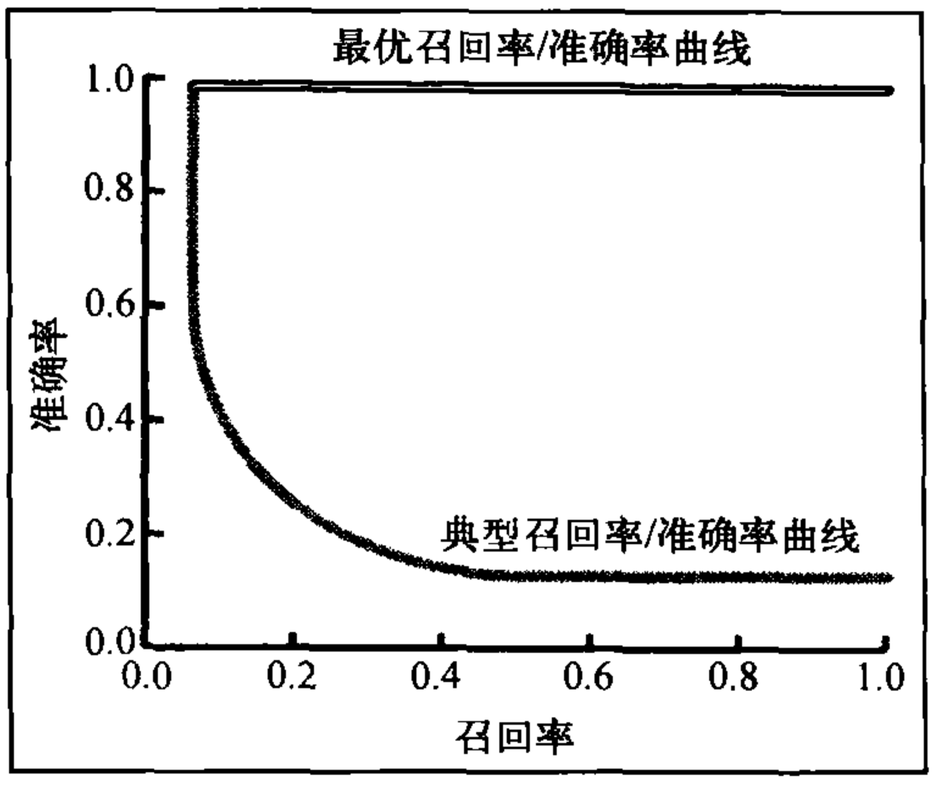


图 1-5　召回率/准确率曲线：典型曲线与最优曲线

仅仅查找到相关文档还是不够的，我们的目标是在可接受的响应时间内检索到相关文档。本书介绍了当前快速检索相关文档的主要方法。如何研制快速高效的信息检索算法？这还需要我们做进一步的探索。

我们详尽地解释了每种算法，针对每个专题，还采用示例的形式阐述了其中最重要的算法。然后，我们转而采用综述的形式对相关的工作以及后续工作给出了相关的参考文献。我们阐述
4 了算法的关键问题，并且为有兴趣的读者提供了进一步学习的参考资料。信息检索研究的主要论文集可以参看[Sparck Jones 和 Willett，1997]。

本书讨论了在大量信息中进行检索的最新算法。很多研究论文或者著作都详尽地论述了这
5 些算法，但是这些文章大都散布于数量繁多的学术杂志上，写作风格迥异。另外，这些文章对读者的知识背景也有各自不同的要求。本书对本领域做了一个相对简洁但又足够细致的综述。

本书的读者并不需要具备专业的数学背景。在涉及具体的数学知识的时候，我们都会快速复习关键点，以便读者充分理解算法和书中给出的详细示例。

我们相信本书对于各种读者来说都是有价值的。熟悉计算机科学核心知识并希望能学习更多信息检索算法的读者应该能从本书中受益匪浅。我们将解释信息检索领域存在的一些基本问题，并具体介绍人们以前是如何处理这些问题的。

正在使用和维护 RDBMS（关系数据库管理系统）的读者也可以看一看这本书。第 6 章具体介绍了一些算法，这类算法将文本检索看做是 RDBMS 的一种应用。因此，我们可以将结构化数据和文本综合起来处理。另外，我们还单独用一节介绍如何利用关系数据库来处理半结构化文档，如采用 XML 标记的文档。

为了引导读者全面理解 ad hoc 信息检索中的关键问题，我们将全书分为几个独立的，但又存在有机联系的部分。第一部分包括第 2 章和第 3 章，全面综述了信息检索领域主要的算法模型与实用策略。所有的模型和策略都紧紧围绕一个关键的问题，即如何提高检索的准确性。第 2 章介绍了 9 种信息检索模型，这些模型要么是专门为信息检索而研制的，要么是为信息检索改造而成的，其目的都是针对特定用户查询，用来提高相关文档排序或评测结果的性能。第 3

章介绍了一些检索相关的工具，这些工具能用来提升第 2 章中任何一个算法的性能。

在第 3 章中，我们重点介绍一些信息检索的技术，这些技术均可以应用于大部分甚至所有的检索模型当中。这里介绍的几个技术与语言密切相关，如语法解析和主题知识库，而其他的很多技术则具有语言无关性，如 n 元处理过程。我们在第 3 章中明确说明了，这里介绍的有些技术实际上就是一个独立的处理系统。在实际应用过程中，这些技术有机地组合在一起可以取得最佳效果。如何精准地判定出这些技术的最优组合策略，采用什么样的执行顺序，实际运行过程中采用什么样的底层模型，才能确保最佳的性能，至今仍然是未知的课题。

介绍完这些提高信息检索准确性的模型和策略之后，我们将注意力转向效率方面。在第 4 章中，我们介绍了多种文档读取访问方法。既介绍了倒排索引的构建与使用，也介绍了其他类型的表现形式，如签名文件。每一种访问模式都有各自的优点和缺点。选择任何一种方案，我们都必须在存储代价、可维护性与查询检索处理速度之间寻求一种综合平衡。介绍完存储访问 6
模式后，我们将介绍几种数据压缩方法。

第 2 章和第 3 章集中阐述了传统信息检索模型的基本原理、常用技术以及处理策略。第 4 章简要介绍跨语言信息检索。第 5 章分析了信息检索中的效率问题。在第 6 章、第 7 章与第 8 章中，我们把重点放在信息检索领域内的 3 个专题上。这 3 个专题分别是数据整合、并行以及分布式信息检索系统。之所以选择讨论这几个专题，是因为它们如今也是业界热切关注的问题。

一般来说，结构化数据和半结构化数据之间有一个清晰的界限。结构化数据大都通过关系数据库管理系统来存储与读取，而文本等半结构化数据一般都是通过信息检索系统来存储与读取的。每一个处理系统都只支持自己的数据存储格式以及相应的处理方法。然而，如今结构化数据和半结构化数据之间的差别正在逐步消失。实际上，我们不再仅仅关注结构化数据和半结构化数据，而是更关注如何采用相同的存储策略来处理文本以及经常出现的非结构化数据，如图像。

为了解决结构化数据和非结构化数据之间的整合问题，Oracle、IBM、Microsoft 等商业公司都在各自的关系数据库系统中加入了信息检索功能，而文本检索厂商（如 Convera、Verity）也在自己的系统中加入了关系处理模块。然而，在所有这些情况下，额外功能的增加都需要另外新增独立的处理模块。在第 6 章中，我们将讨论如何增加处理模块，并给出一种可行的替代方法，该方法将信息检索处理能力作为关系数据库的一种应用。使用该方法，无需额外的软件开发，就能获得关系数据库系统本身具备的一些传统优势（即可移植性、并发性、数据的可恢复性等）。所有的关系数据库厂商在数据库系统中都提供了并行处理功能，因此，如果信息检索系统作为关系数据库系统的应用程序，那么信息检索系统也将具备并行处理能力。

认识到并行能力对信息检索系统的重要性之后，我们将进一步介绍该领域的最新进展。在第 7 章中，我们首先介绍信息检索系统早期的并行方法。这些方法主要利用 SIMD（单指令多数据流）技术，采用多个处理器来高效地并行扫描文本。然而，随着对并行处理技术理解的不断加深，人们研究出了基于倒排索引的方法，倒排索引方法能有效地减少文本扫描过程中不必要的输入/输出开销。我们将讨论各种不同的方法。最后，第 7 章将总结并行信息检索领域最新 7
的研究成果，其中主要介绍文档聚类并行算法。

在当今的信息处理领域，任何一种方法都必须考虑到最常见的应用场景——万维网。因此，在第 8 章中，我们将讨论 Web 检索中必须考虑的一个问题——分布式信息检索系统。我们将对一些早期的基本理论进行综述，最终详细地讨论了 P2P 信息检索。

如何搜索文档集并找到相关的文档，这个问题已经反反复复研究了四十多年。然而，一直没有标准的测试平台来评价各种信息检索算法的优劣，这一情况直到 1990 年 TREC（文本检索会议）的出现才得以改变（该会议由美国国家标准技术研究所主办）。如果没有标准测试数据集和标准查询集，人们就无法客观地比较各种检索算法。许多算法都仅仅在几兆的文档集上运行过，大家都期待现有算法扩展到大规模数据集时仍能取得同样的效果。有一篇重要的论文指出：一些方法在小规模文档集上表现很好，但并不一定能在大规模数据集上取得同样的效果 [Blair 和 Maron，1985]。

我们在第 9 章中简要介绍了 TREC。过去这些年来，人们提出了大量的检索模型、工具以及性能提升的方法，都需要有一套明确一致的测量方法和程序来评价各个系统的准确率、处理时间等方面的性能指标。实际上，这就是 TREC 这个评测论坛横空出世的原因所在，它的部分动机就是提供基准的数据集和查询集。如今，TREC 已经成为比较各种系统和方法优劣的专业会议。稍微遗憾的是，TREC 目前只能评价系统的准确率。我们希望将来 TREC 能增加对处理效率方面的评测。

最后，我们将讨论当前信息检索系统的局限性。我们将回顾信息检索领域已经取得的成就，并对将来的发展方向进行展望。我们特别希望读完本书之后，读者能对信息检索领域产生更浓厚的兴趣。在本书的后续版本中，我们希望能融入您所贡献的聪明才智。

第2章 检索模型与算法

检索模型提供了一种度量查询和文档之间相似度的方法。这些模型基于一种共同的理念：文档和查询共有的词项（term）越多，则认为这篇文档和该查询越相关。语言本身就客观存在着诸多的不确定性，现实中一个相同的概念可能会用多种不同的词项来表达（如 new york 和 the big apple 可能指的是同一含义[①]）。另外，相同的词项也可能有很多种语义（比如 bark 和 duck，它们的名词形式和动词形式的意思迥然不同[②]。我们介绍的一些检索算法采用了相应措施来解决这些语言中的不确定性问题。

检索模型就是一种算法，该算法的处理对象是查询 Q 和文档集合 $\{D_1, D_2, \cdots, D_n\}$，处理过程就是计算每篇文档 D_i $(1 \leqslant i \leqslant n)$和这个查询的相似度 $SC(Q, D_i)$。[注：SC 是 Similarity Coefficient（相似度）的缩写，有时记作 RSV（Retrieval Status Value），用来表示检索状态值]。

常用的检索模型有如下几种。

- **向量空间模型**——我们将查询和文档都表示为词项空间中的向量，进而可以计算这两个向量之间的相似度。
- **概率模型**——基于文档集中每个词项在相关文档中出现的可能性计算出概率。对文档和查询匹配的所有词项计算联合概率，从而得出文档与查询的相似度。
- **语言模型**——针对每一篇文档，建立一个语言模型，并且计算出各文档“生成”查询的概率。
- **推理网络**——我们采用贝叶斯网络来推断文档和查询间的相关性。推理过程基于文档中的“证据”，这种“证据”是文档中可以作出文档相关性推断的依据。推理的可信度即为文档查询的相似度。
- **布尔检索**——在原始的布尔查询结果中，我们对每篇文档都赋予一个得分，这样就可以进行查询结果排序。具体的做法是：将每个查询词项赋予一个权重，进而使用这个权重计算文档与查询的相似度。
- **LSI（隐性语义检索）**——我们使用词项-文档矩阵来表示文档集中出现的词项。这个矩阵通过奇异值分解（Singular Value Decomposition，SVD）来降维，奇异值分解可以过滤掉文档中的噪声，这样两篇有相同语义的文档在多维空间中距离会比较接近。
- **神经网络方法**——网络中包含一系列神经元（或者称作节点），这些神经元在激活查询到文档的触发链接的过程中进行传导。网络中每个链接的权重被传到文档，这些权重汇集起来作为查询和文档之间的相似度。人们往往会预先定义好一批相关文档集和不相关文档集，然后再调整神经网络链接上的权重，以达到“训练”神经网络的目的。

① 纽约的昵称为“大苹果”。——译者注

② bark 做名词的时候，意思为“树皮”，而作动词表示的却是“狗叫”；duck 名词表示的是“鸭子”，而动词表示的是“躲避”。——译者注

- **遗传算法**——我们可以通过进化的方法来得到查找相关文档的最优查询过程。初始的查询可以使用随机的权重，也可以使用估计值。通过修改这些权重来产生新的查询。接近已知相关文档的新查询得以幸存，而“适应度”较小的查询在随后的过程中被淘汰。
- **模糊集检索**——每篇文档被映射到一个模糊集[这种集合不仅包含元素，还包含与每个元素相关的权重，用来表示该元素的隶属度（membership）]。该模型将布尔查询映射为模糊集的相交、合并和补集等操作，这样我们就可以计算出每篇文档与查询相关的隶属度。隶属度即为相似度。

对于一个已知的检索模型，我们可以使用许多不同的实用策略（utility）来提高检索模型的性能。我们将在第 3 章中详细介绍这些实用策略。需要注意的是，一些检索模型和实用策略基于完全不同的数学思想。例如，概率检索模型理论上不能与向量空间模型的同义词表（thesaurus）联合使用，然而，这种结合很有可能提高最终的检索效果。不过，当混合搭配使用基于不同数学模型的检索模型和实用策略时，我们还是要特别谨慎。

10

我们竭尽所能来优化查询，大多数方法是在初始查询中增加或者删除一些词项，其他的方法则仅仅是把重点放在查询的计算范围上（比如，使用子文档或者片段来代替整篇文档）。关键在于每种策略是即插即用的，且可以应用于任意一个检索模型（虽然这种情况很少出现）。

在深入研究每种检索模型的细节之前，我们希望提醒读者注意一点。在力图展现算法的原始形式时，我们有意留下不一致的形式。例如，在实现一个缓慢增长函数时，一些算法的提出者使用 $\ln(x)$而其他人则使用 $\lg(x)$。显然，我们都知道这些函数之间严格来说不过是常量积的关系。虽然它确实会带来些许误解，但是我们感觉展现原始的描述仍然有其优越性。为了清晰起见，我们尽量在不同策略中使用通用符号，并且提供一个可以运行的例子。这些例子都采用相同的查询和文档集，其具体用法与检索模型无关。

2.1　向量空间模型

向量空间模型通过向量的方式来计算相似度，其中每一篇文档用一个向量来表示，而查询同样用一个向量来表示 [Salon 等人，1975]。该模型基于以下思想：大致来说，文章的语义通过所使用的词语来表达。如果可以将文档中的词语表示成一个向量，那就有可能将文档和查询进行比较，从而判定它们内容相似的程度。如果将查询也看做文档，就可以计算出文档和查询的相似度。通过文档中使用的词项进行衡量，我们认为最接近查询内容的文档是最相关文档。图 2-1 说明了向量空间模型的基本理念，其中包含一个查询向量和三个文档向量。

该模型主要涉及两方面工作：一是如何构建一个向量来表示文档中的词项；二是如何构建另一个向量来表示查询中的词项。接下来，我们必须选择一种方法来度量任意文档向量和查询向量的相似度。一种方法是可以比较两个向量的差值向量大小，然而由于查询一般比较短小，这会导致较大的向量看起来往往和大多数查询都不相关。判断两个向量相近性的传统方法是计算这两个向量之间夹角的大小。两个向量的夹角可以通过内积（或称点积）计算出来，然而我们并没有必要真正计算出角度的实际值——任何计算夹角的单调函数都可以达到实用的要求。我们通常使用“相似度”来代替夹角。计算相似度有多种方法，通常来说，内积效果最佳。通过以上的讨论，我们可以得到以下共识：只有文档向量和查询向量指向的方向大体一致，它们才相似。

11

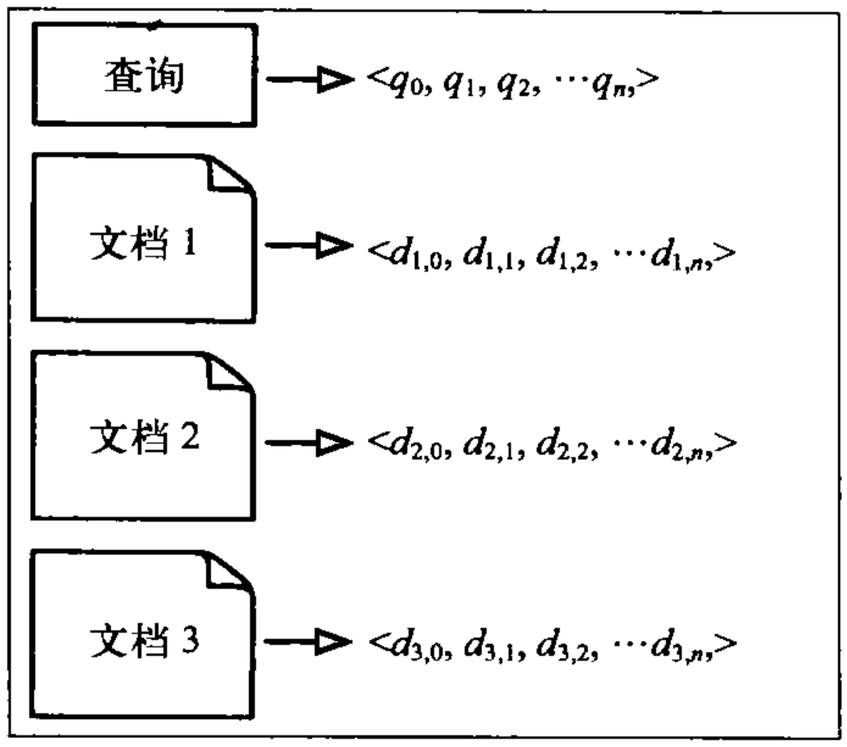


图 2-1 向量空间模型

对于文档集中每一个不同的词项（或概念），我们在向量中只记录一个分量。接下来，我们考虑在文档集中只有两个不同词项 α 和 β 的情况。所有的向量只包含两个分量：第一个分量表示词项 α 的出现情况，第二个分量表示词项 β 的出现情况。构造向量最简单的方法是：当词项出现时，就在对应向量的分量处记 1；如果词项未出现，就在对应的分量处记 0。下面我们看一篇实际的文档，比如文档 D_1，其中词项 α 出现了两次，词项 β 出现了零次。这个文档使用二值表示方法时，对应的向量为<1, 0>。二值表示方法可以用来计算相似度，但是并没有考虑一个词项在文档中出现的次数。通过扩展这种表示形式，我们将词项在文档中出现的频率作为向量中各个分量的值。在这个例子中，向量可以表示为<2, 0>。

图 2-2 给出了一个简单的例子。对于文档集中每个独立的词项，我们需要在向量中定义一个分量。使用一个简单的例子，在一个只包含两个词语的语言中（只有 A 和 I 是有效的词项），所有的查询和文档都可以采用二维空间向量来表示。图 2-2 给出了一个查询和三篇文档，并且给出了它们的向量形式及其图形表示。查询和文档的相似度可以通过计算两个向量的距离得出。在这个例子中，可以认为文档 1 和查询的向量相同，从而文档 1 在查询结果列表中排在第一位。

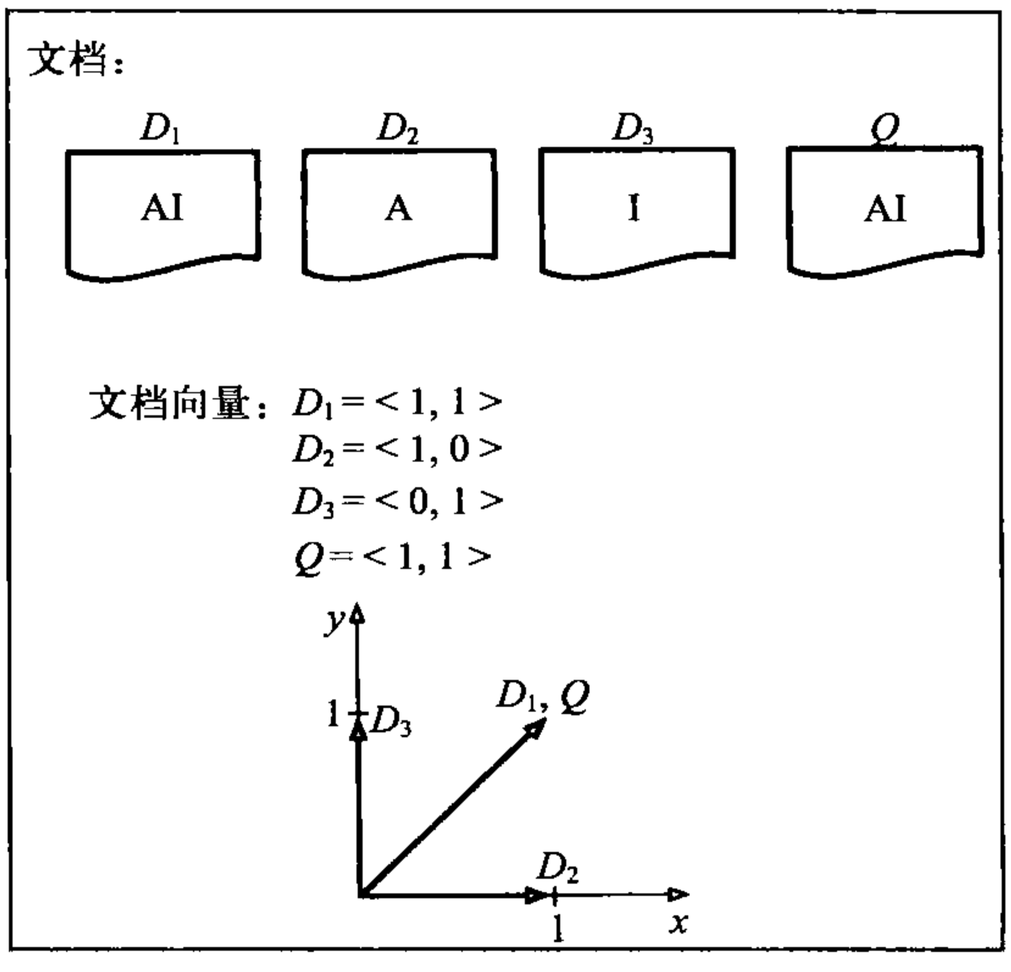


图 2-2 只有两个词项的向量空间模型

除了简单地给出查询词列表外，用户通常还会给出权重，该权重表示一个词项比另外一个词项更重要。这是通过在初始查询中用户人工指定词项权重来实现的。另外一种方法是自动指定权重——通过基于词项在整个文档集中出现的频率。基本思想是：不频繁出现的词的权重应该比频繁出现的词的权重更高。文献[Salton，1969；Salton，1970b]分别采用权重自动赋值与人工赋值方法计算相似度，然后进行查询比较。实验结果表明：自动赋值在查询性能上并不比和人工赋值差[Salton，1969，Salton，1970b]。遗憾的是，这些结果并没有引入词项在整个文档集中的相对权重。

在20世纪70年代，研究者深入研究了不同文档集的权重。研究结论是：如果引入不同文档集的权重，能够提升相关度排序的效果。尽管在实验中使用了相对较小的文档集，作者仍然得出一个结论："迄今为止，信息检索研究中不少事情都可以称为结论确凿，这就是其中之一。"
12 [Robertson和Sparck Jones，1976]

我们采用更加形式化的定义，并采用稍大一些的例子来展示如何使用基于数据集频率的权重。对应于一个给定的词项，其权重使用IDF（逆文档频率）来计算。

为了给每篇文档建立一个对应的向量，可以考虑如下定义。

t——文档集中不同词项的个数。

tf_{ij}——词项t_j在文档D_i中出现的次数，也就是词频。

df_j——包含词项t_j的文档的篇数。

idf_j——$\lg\left(\frac{d}{df_j}\right)$，其中$d$表示所有文档的篇数。这就是*逆文档频率*。

对于每一篇文档向量，都有n个分量，并且对于整个文档集中每个不同的词项，都包含一个词条。向量中的每个分量为在整个文档集中计算出来的每个词项的权重。在每篇文档中，词
13 项权重基于词项在整个文档集中出现的频率情况以及词项在某一个特定文档中出现的频率自动赋值。词项在一篇文档中出现的频率越高，则权重越大；相反，如果词项在所有文档中出现的频率越高，则权重越小。

仅当词项在文档中出现时，文档向量中词项的权重才为非零值。对于一个包含许多小文档的大文档集，文档向量可能会包含大量的零元素。例如，一篇文档集包含10 000个不同的词项，也就是每个文档中要用10 000维的向量来表示。一个给定的只有100个不同词项的文档向量则包含9 900个零分量。

对于文档中词项的权重因素，主要综合考虑词频和逆文档频率。也就是说，我们使用下面的公式计算文档i对应的向量中第j个词条的值：

14
$$d_{ij} = tf_{ij} \times idf_j$$

下面我们来考虑一个包含D_1和D_2两篇文档的文档集，在文档D_1中词"绿色"出现了十次，而在D_2中"绿色"仅出现了5次。如果仅仅查询"绿色"，那么在结果中文档D_1排在文档D_2前面。

当我们在一篇文档检索系统中用文档集中t个不同的词项来查询时，系统将为每个文档计算维度为t的向量$D(d_{i1}, d_{i2}, \cdots, d_{it})$。向量值使用前文所述的词项权重填充。类似地，查询中的词项构建的向量为$Q(w_{q1}, w_{q2}, \cdots, w_{qt})$。

查询Q和文档D_i的相似度可以简单地定义为两个向量的内积。因为查询向量和和文档向量

在长度上是相似的，这种策略也常常被用来计算两篇文档的相似度。我们将在 3.2 节中讨论将 SC 应用到文档聚类中。

$$\mathrm{SC}(Q,D_i)=\sum_{j=1}^{t} w_{qj}\times d_{ij}$$

2.1.1 相似度计算举例

下面我们考虑一个固定的查询和文档集，包含一个查询 Q 和三篇文档组成的文档集：

Q：“gold silver truck”

D_1：“Shipment of gold damaged in a fire”

D_2：“Delivery of silver arrived in a silver truck”

D_3：“Shipment of gold arrived in a truck”

在这个文档集中，有三篇文档，所以 $d=3$。如果一个词项仅在三篇文档中的一篇中出现，那么该词项的 IDF 就是 $\lg(d/df_i)=\lg(3/1)=0.477$。类似地，如果一个词项在三篇文档中的两篇中出现，那么该词项的 IDF 就是 $\lg(d/df_i)=\lg(3/2)=0.176$。如果一个词项在三篇文档中都出现了，那么该词项的 IDF 就是 $\lg(d/df_i)=\lg(3/3)=0$。

三篇文档的每个词项的 IDF 值如下所示：

$idf_a=0$　　$idf_{\text{in}}=0$

$idf_{\text{arrived}}=0.176$　　$idf_{\text{of}}=0$

$idf_{\text{damaged}}=0.477$　　$idf_{\text{silver}}=0.477$

$idf_{\text{delivery}}=0.477$　　$idf_{\text{shipment}}=0.176$

$idf_{\text{fire}}=0.477$　　$idf_{\text{truck}}=0.176$

$idf_{\text{gold}}=0.176$

现在就可以构造文档向量了。因为文档集中出现了 11 个词项，所以我们构造一个 11 维的文档向量。我们可以用上文给出的按字母顺序排列的词项来构建文档向量，所以 t_1 对应第一个词项“a”，t_2 对应“arrived”，依次类推。向量 j 中词项 i 的权重计算方式为 $idf_i\times tf_{ij}$。文档向量如表 2-1 所示。

表 2-1 文档向量

docid	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	0	0	0.477	0	0.477	0.176	0	0	0.176	0	0
D_2	0	0.176	0	0.477	0	0	0	0	0	0.954	0.176
D_3	0	0.176	0	0	0	0.176	0	0	0.176	0	0.176
Q	0	0	0	0	0	0.176	0	0	0	0.477	0.176

$$\begin{aligned}SC(Q,D_1)&=0\times0+0\times0+0\times0.477+0\times0\\&\quad+0\times0.477+0.176\times0.176+0\times0+0\times0\\&\quad+0\times0.176+0.477\times0+0.176\times0\\&=0.176^2\approx0.031\end{aligned}$$

类似地：

$$SC(Q,D_2)=0.954\times0.477+0.176^2\approx0.486$$

$$SC(Q,D_3)=0.176^2+0.176^2\approx0.062$$

因此，检索结果顺序为 D_2, D_3, D_1。

在实现向量空间模型或其他检索模型时，我们通常使用倒排索引（inverted index）来避免在每篇文档中使用冗长的顺序查找来查找查询词。因此，在用户提交查询之前，我们首先构造倒排索引表。从图 2-3 中我们可以看到倒排索引表的结构。n 个词项中的每一项都被存储在称为*索引*的结构中。对于每一个词项，指向的逻辑链接表被称作*倒排表*（posting list）。倒排表记录每篇包含该词项的文档。在图 2-3 中，倒排表中记录了文档标识符和词频等信息。（实际上，人
16 们往往使用比链接表更有效的结构，但从概念上讲都是为了同一目的。）图中的倒排表表明词项 t_1 在文档 1 中出现了一次，在文档 10 中出现了两次。对于一个任意的词项 t_1 的条目，表明该词项在文档 j 中出现了 tf 次。倒排索引的的具体构造细节和用法将在第 5 章中介绍，不过知道倒排索引通常在各种检索模型中用来提高运行效率是非常有用的。

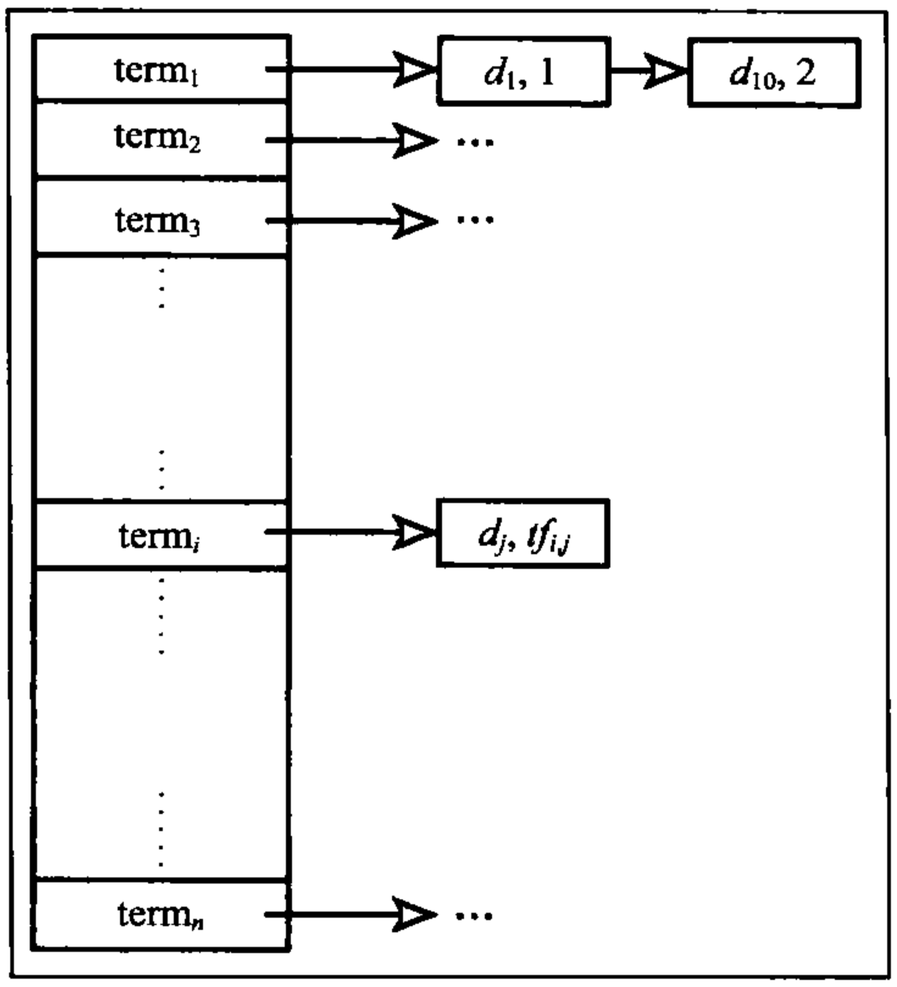


图 2-3　倒排索引

在 20 世纪 60 年代后期的文献[Salton 和 Lesk，1968]中就有关于向量空间模型的早期研究。该模型在 20 世纪 70 年代中期使用非常普遍[Salton 等人，1975]，并且现在仍然是计算查询和文档相似度最广泛使用的方法之一[TREC，2003]。这种方法非常重要，因为检索系统可以据此决定最终将哪些文档展示给用户。通常用户只需要最前面的 n 篇文档，并且这些文档按相似度进行排序。

接下来，科研人员研究了提升基本的 *tf-idf* 权重的词项权重计算方式 [Salton 和 Buckley，1988]。人们研究了许多不同的方法，并且认为用下面的公式计算文档 i 中词项 j 的权重效果出色：

17
$$w_{ij}=\frac{(\lg tf_{ij}+1.0)\times idf_j}{\sum_{j=1}^{t}[(\lg tf_{ij}+1.0)\times idf_j]^2}$$

这样计算权重的出发点是：在已知的查询和文档中，词频很高的匹配词项淹没了其他匹配词项的效果。为了避免这种现象，科研人员提出使用 $\lg(tf)+1$ 来缩小词频的范围。基于该思想的修订版本是在查询和文档中的词项使用不同的权重。

一种被称作 lnc.ltc 的词项权重计算模式非常有效。文档中使用 $1+\lg(tf)\times idf$ 计算权重，查询中使用 $1+\lg(tf)$ 计算权重。标签 lnc.ltc 是如下形式：*qqq.ddd*，其中 *qqq* 指查询权重，*ddd* 指

文档权重。这三个字母：*qqq* 或 *ddd* 是 *xyz* 的形式。

第一个字母 *x* 可以是 *n*、*l* 或 *a*。*n* 表示原始词频或指 *tf*。*l* 表示通过取对数来降低权重，所以可以使用 1+lg(*tf*)。*a* 表示加强权重，所以权重为 $0.5+0.5\times\frac{tf}{tf_{\max}}$。

第二个字母 *y* 表示是否使用 *idf*。*n* 表示不使用 *idf*，*t* 表示使用 *idf*。

第三个字母 *z* 表示是否使用文档长度归一化。通过归一化文档长度，我们试着减小检索中文档长度的影响（见公式 2-1）。在文献[Singhal, 1997]中，*n* 表示不使用归一化，*c* 表示使用标准的余弦归一化，*u* 表示使用临界点长度（pivoted length）归一化。

2.1.2 相似度

人们提出了许多比较查询向量和文档向量的方法，这些方法都经过了证明。这里我们来快速回顾一下。其中最常见的方法是余弦方法，也就是计算查询向量和文档向量之间夹角的余弦值。

$$SC(Q,D_i)=\frac{\sum_{j=1}^{t} w_{qj}d_{ij}}{\sqrt{\sum_{j=1}^{t}(d_{ij})^2\sum_{j=1}^{t}(w_{qj})^2}}$$

因为 $\sqrt{\sum_{j=1}^{t}(w_{qj})^2}$ 在计算每篇文档的相似度时都会出现，向量内积除以文档向量大小后，余弦系数应该给出相同的相关性结果。我们注意到余弦方法通过考虑文档长度来“归一化”结果。通过内积方法，一个比较长的文档可能会得到一个比较高的分数，仅仅因为文档比较长，因此有更多的机会包含查询词——并不一定因为文档是相关的。

18

Dice 系数定义为：

$$SC(Q,D_i)=\frac{2\times\sum_{j=1}^{t} w_{qj}d_{ij}}{\sqrt{\sum_{j=1}^{t}(d_{ij})^2\sum_{j=1}^{t}(w_{qj})^2}}$$

Jaccard 系数定义为：

$$SC(Q,D_i)=\frac{\sum_{j=1}^{t} w_{qj}d_{ij}}{\sum_{j=1}^{t}(d_{ij})^2+\sum_{j=1}^{t}(w_{qj})^2-\sum_{j=1}^{t} w_{qj}d_{ij}}$$

余弦方法通过将向量内积除以文档向量的长度来实现不同文档长度的归一化。余弦方法中假定文档长度对查询没有影响。排除归一化因素，较长的文档更容易被认定为相关的，仅仅因为长文档包含的词多，所以增加了包含查询词的可能性。除以文档向量长度就是不考虑文档长度。

结果是（至少对于 TREC 数据），这种简单的假设是不正确的。拿 50 个 TREC 查询集所有查找到的相关文档来说，Singhal 发现实际上在长文档集中更多文档被判断为相关的[Singhal, 1997]。原因可能是长文档仅仅是有更多的机会包含那些与给定查询确实相关的词项。

为了找到一种调整归一化因子的方法，Singhal 比较了相关的可能性和在文档集中被检索到的可能性，其中已知与查询集相关的文档。理想情况下，如果检索概率和相关概率同时以文档长度为标准进行作图，那么这两条曲线应该基本一致。由于并不是这种情况（实际上两条曲线是相交的），所以肯定存在一个文档长度点使检索概率和相关概率相等。在这个点（被称为临界点）前，文档被检索的概率大于相关概率。在这个点后，文档被检索的概率小于相关概率。一

旦找到临界点，我们就可以使用这个“校正因子”进行归一化调整。这个“校正因子”通过一个线性表达式计算出来，其中在临界点时线性表达式的值等于临界点的值，并且选定一定的斜率来增加短文档归一化之后的值，这样短文档被选定的概率等于相关概率。因此，相似度为：

$$SC(Q,D_i)=\frac{\sum_{j=1}^{t} w_{qj}d_{ij}}{(1.0-s)p+(s)\sqrt{\sum_{j=1}^{t}(d_{ij})^2}}$$

19 这种方法有两个变量：分别为斜率 s 和临界点 p。不过，我们也有可能将斜率 s 表示为临界点的函数。Singhal 在纠正和调整相应的斜率之前，将整个文档集上统计计算出来的平均归一化因子选定为临界点。同时，将归一化因子除以(1.0−s)p。计算相似度的等式如下：

$$SC(Q,D_i)=\frac{\sum_{j=1}^{t} w_{qj}d_{ij}}{(1.0-s)+(s)\sqrt{\frac{\sum_{j=1}^{t}(d_{ij})^2}{\text{avgn}}}} \tag{2-1}$$

其中 avgn 是在任何纠正前的平均文档归一化因子。

临界点模式对于短文档和中等长度的文档还算有成效，但是与归一化前相比，整个算法会更有利于特别长的文档。为了修正这一点，在任何调整前，科研人员提出将文档中不同词项的数量$|d_i|$作为归一化因子。

最后一种调整是针对在特别长文档中出现的词频特别高的情况。首先，使用 1+lg 来限制词频。为了应对长文档，将每个词项权重除以平均词项权重。

新的权重 d_{ij} 为

$$d_{ij}=\frac{1+\lg tf}{1+\lg(atf)}$$

使用新权重，并且除以调整因子的新公式如下：

$$SC(Q,D_i)=\frac{\sum_{j=1}^{t} w_{qj}d_{ij}}{(1.0-s)p+(s)(|d_i|)} \tag{2-2}$$

然后我们计算给定文档集中每篇文档的词项的平均数量，并且将其作为临界点 p。一旦计算完成，就可以使用文档集就上训练出一个很好的斜率。公式 (2-2) 被称为临界点唯一归一化 (pivoted unique normalization)，并且实验表明，在公式 (2-1) 临界点余弦归一化的基础上检索效果得到了提高。修改后的归一化因子使得更可能检索到长文档，并且对于 TREC 查询，性能可以提高 10%。

我们同时可以注意到向量空间模型假设词项之间是独立的。缓解向量空间模型词项独立性

20 问题的一个方法就是改变基向量。尽管改变基向量并不能完全解决问题，但是可以缓解这一问题。其思想为：为每篇文档选一个基本的组合词项（不管词频）。新的基本向量可以相互正交，并且按比例转化为单位向量。文档和查询可以表示为新的基本向量的形式。同时联合使用其他方法（可以是概率），可以避免独立性假设，但是在实际应用中效果并没有显著的提高。

2.2　概率检索模型

概率模型通过计算文档与查询相关的概率来作为文档和查询的相似度。这就使相关性排序

问题降为概率论应用问题。概率论的综述可以查看文献[Fuhr，1992]。

概率论可以用来计算查询和文档的相关性。学术界提出了两种根本不同的方法。第一种依靠使用模式来预测相关性[Maron 和 Kuhns，1960]，第二种使用查询中的每个词项作为线索来判断文档是否相关[Robertson 和 Sparck Jones，1976]。

使用概率论检索文档的研究可以追溯到 Maron 和 Kuhns。他们的研究开创了估计文档是否与一个给定的特定词项相关的研究领域。

所有的概率研究起源于这样一种观点：基于一个词项分别在相关文档和不相关文档中出现的频率来估计该词项的权重。2.2.1 节描述了简单的词项权重模型，2.2.2 节中介绍了非二值独立模型，2.2.3 节和 2.2.4 节分别介绍泊松模型和基于片段的模型，这两种模型在 TREC 数据集上都取得了不错的成绩。最后，2.2.5 节中讲述模型的两大问题——参数估计和独立性假设。

2.2.1 简单的词项权重

词项权重的使用基于 PRP（Probability Ranking Principle，概率排序原理），PRP 假定最佳效果是基于与查询相关的概率估计进行排序 [Robertson，1977]。

关键是对查询的构成分量计算概率，然后使用这些分量作为证据，从而计算出文档与查询相关的最终概率。

我们对查询中的词项赋予权重，权重对应于查询词可以检索到相关文档的概率。我们结合 21
每个词项的权重来计算最终的相关性。

在该领域的大多数论文中，引入了概率论并且讨论了独立性假设的有效性，所以简要回顾一下概率论是恰当的。

下面我们试着预测一支叫做沙罗曼蛇的垒球队是否可以赢得比赛。基于以往的经验，我们可以观察到：他们经常在晴朗的天气，当他们最好的游击手上场时赢得比赛。这意味着有两个证据可以使用：户外的天气情况和好的游击手。对于一场已知的比赛，如果天气晴朗的话，有 75%的可能性可以赢得比赛；如果游击手出赛的话，有 60%的可能性赢得比赛。因此，我们记作：

$$P(\text{win} \mid \text{sunny}) = 0.75$$

$$P(\text{win} \mid \text{good-shortstop}) = 0.6$$

这支球队在两个条件下可以取胜的条件概率记为 p(win | suny, good-shortstop)，这表示“这支球队在天气晴朗且有好的游击手出赛的情况下获胜的条件概率”。我们有两个证据推断沙罗曼蛇队可以获胜。直觉告诉我们两个条件都满足时会比只满足一个条件获胜的概率更大。组合它们的方法也就是“看看成败的机率”。75%获胜的可能性也就是有 25%输掉的可能性，60%获胜的可能性也就是有 40%输掉的可能性。我们假设这两个条件是独立的。

$$P(\text{win} \mid \text{sunny,good-shortstop}) = \alpha$$

$$P(\text{win} \mid \text{sunny}) = \beta$$

$$P(\text{win} \mid \text{good-shortstop}) = \gamma$$

通过贝叶斯公式：

$$\alpha = \frac{P(\text{win,sunny,good-shortstop})}{P(\text{sunny,good-shortstop})} = \frac{P(\text{sunny,good-shortstop}|\text{win})P(\text{win})}{P(\text{sunny,good-shortstop})}$$

因此：

$$\frac{\alpha}{1-\alpha}=\frac{P(\text{sunny,good-shortstop}|\text{win})P(\text{win})}{P(\text{sunny,good-shortstop}|\text{lose})P(\text{lose})}$$

先计算第一项（因为独立性假设）：

$$\frac{P(\text{sunny,good-shortstop}|\text{win})}{P(\text{sunny,good-shortstop}|\text{lose})}=\frac{P(\text{sunny}|\text{win})P(\text{good-shortstop}|\text{win})}{P(\text{sunny}|\text{lose})P(\text{good-shortstop}|\text{lose})}$$

22

类似地：

$$\frac{\beta}{1-\beta}=\frac{P(\text{sunny}|\text{win})P(\text{win})}{P(\text{sunny}|\text{lose})P(\text{lose})}$$

$$\frac{\gamma}{1-\gamma}=\frac{P(\text{good-shortstop}|\text{win})P(\text{win})}{P(\text{good-shortstop}|\text{lose})P(\text{lose})}$$

进行相应的替换和运算，我们得到：

$$\frac{\alpha}{1-\alpha}=\frac{\beta}{1-\beta}\left(\frac{P(\text{lose})}{P(\text{win})}\right)\left(\frac{\gamma}{1-\gamma}\right)\left(\frac{P(\text{lose})}{P(\text{win})}\right)\left(\frac{P(\text{win})}{P(\text{lose})}\right)$$

化简得：

$$\frac{\alpha}{1-\alpha}=\frac{\beta}{1-\beta}\left(\frac{\gamma}{1-\gamma}\right)\left(\frac{P(\text{lose})}{P(\text{win})}\right)$$

假设沙罗曼蛇队是0.500球队俱乐部（也就是说，他们输赢次数一样多），然后假设β和γ的值分别是0.6和0.75，那么可以得到：

$$\frac{\alpha}{1-\alpha}=\frac{0.6}{0.4}\times\frac{0.75}{0.25}\times\frac{0.500}{0.500}=1.5\times3.0\times1.0=4.5$$

由此可以得到α的值为9/11＝0.818。

注意到，组合晴朗的天气和优异的游击手两个条件可以得到比只有任何一个条件都高的获胜概率。

关键在于独立性假设。天气晴朗的可能性和优异游击手的出场是完全独立的。游击手出现的机率不受天气情况而改变。类似地，天气情况也不会受游击手出场与否影响。如果这种独立性被破坏了：假如游击手喜欢晴朗的天气——需要特别考虑这种依赖性。同时独立性假设也要求天气和优异游击手的出现和给定的输赢之间是独立的。

对于信息检索查询，查询中的词项可以看做文档相关的指示器。查询中词项A的出现与否可以预测文档是否相关。因此，经过一段时间的观察，我们发现词项A同时在文档和查询中出现时，文档相关的概率为$x\%$。这样我们就为词项A赋值这个概率。假设词项间是独立的，这样就可以针对查询中的每一个词项来计算其概率。最终，计算所有权重的乘积就可以计算出文档相关的概率。

我们知道，独立性假设在现实中确实不是一个很好的模型。尽管理论上存在问题，但是一

23 些研究者已经研究出为什么使用这些假设的系统性能相当好 [Cooper，1991]。例如，对应查询苹果派，相关文档中包含词项苹果可能比用词项派或者其他随机选择的词项的相关概率更高一些。因此，这就违背了关键的独立性假设。

概率模型的大多数工作都假定词项间是相互独立的，这是因为处理依赖性将引入大量的计算。人们还不清楚考虑依赖性后效果是否有所提升，而且我们注意到只需相对较少的工作即可实现依赖性。这些方法计算量确实相当大，而且更为重要的是，计算量很难估计。获取在相关文档和不相关文档中词项共现的足够多的训练数据也是非常必要的。通常，获取充分的训练数据来估计这些参数是非常困难的。

在图 2-4 中，我们列出了大多数概率模型中所需要的训练数据。当我们提交含有两个词项 q_1 和 q_2 的查询后，返回了 5 篇文档，并且我们评定文档 2 和文档 4 是相关的。在这个评定中，已知的相关（不相关）文档的概率是通过其包含的词项 q_1 计算出来的。类似地，可以计算出 q_2 的概率。显然，这些概率是通过训练数据估计出来的。其主要思想是只要获得足够多的训练数据，那么当用户提交一个查询时，我们就可以很好地估计出与查询相关的文档。

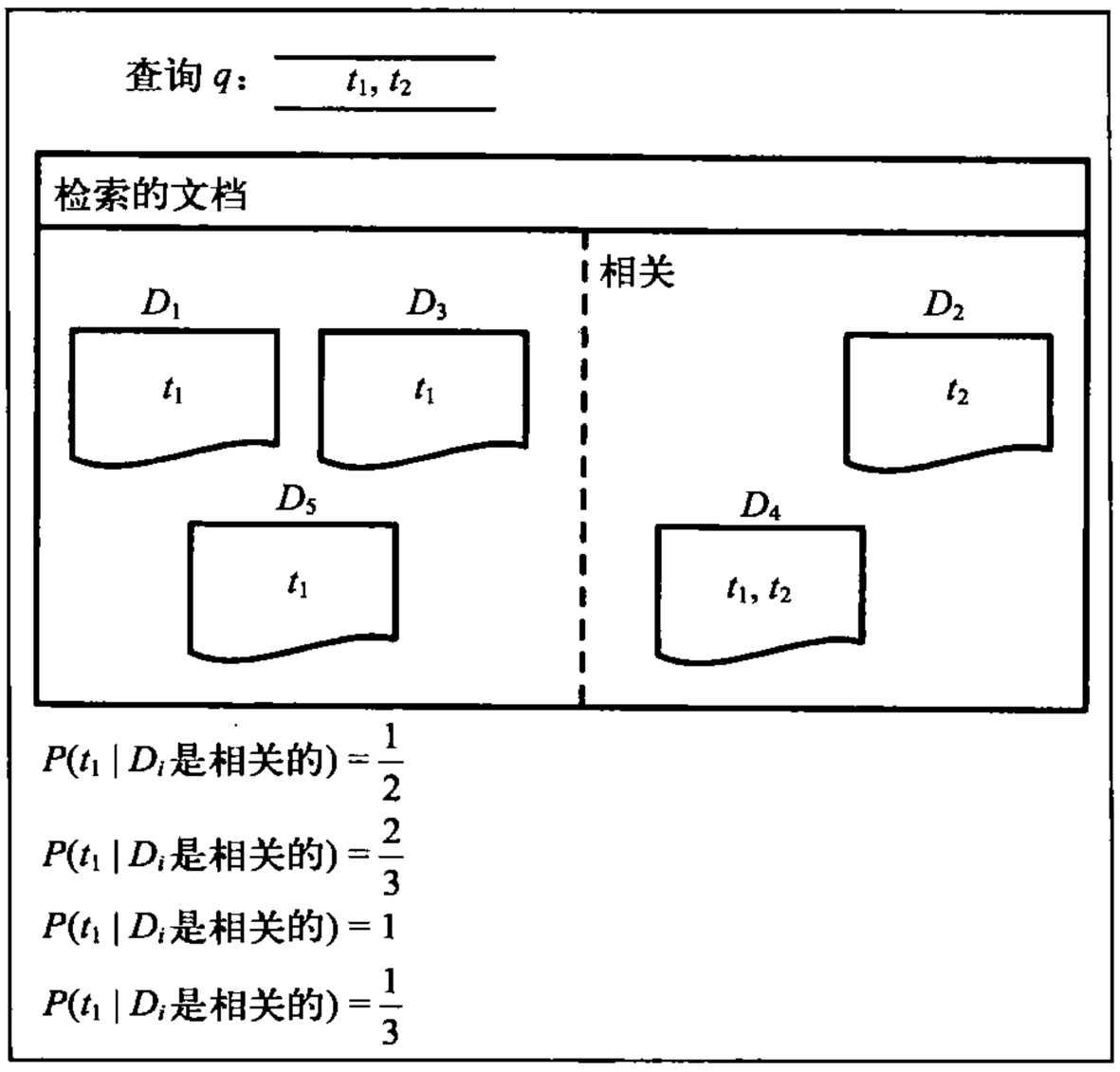


图 2-4 概率检索的训练数据

给定一篇文档 d_i，它包含 t 个词项$(w_1,w_2,\cdots,w_t)$，其中 w_i 表示词项 i 导致文档相关的估计概率。文档 d_i 相关的权重或者“可能性”基于文档中每个词项相关的概率。对于文档中一个已知的词项，它对估计整篇文档相关的贡献可以计算为：

$$\frac{P(w_i \mid \text{rel})}{P(w_i \mid \text{nonrel})}$$

接下来的问题是：怎么利用每个词项相关的概率来估计整篇文档相关的概率？基于已知的独立性假设，我们可以将文档中每个词项出现的概率相乘来得到文档相关的概率，最后将乘积取对数：

$$\lg\left(\prod_{i=1}^{t}\frac{P(w_i \mid \text{rel})}{P(w_i) \mid \text{nonrel})}\right)=\sum_{i=1}^{t}\lg\left(\frac{P(w_i \mid \text{rel})}{P(w_i) \mid \text{nonrel})}\right)$$

我们注意到这些计算值都基于词项在相关文档和非相关文档中独立出现的假设。我们同时假设如果一个词项在一篇文档中出现，那么该词项对于另外一个词项是否在相同文档中出现没

有影响。

既然我们描述了独立的词项可以联合起来估计出整篇文档的相关概率，那么就必须讨论单个词项权重的计算方法。自从引入概率检索模型以来，人们研究了许多种不同的计算给定词项分别在相关和不相关文档中的概率的方法。在 Robertson 和 Spark Jones 1976 年的论文中，他们考虑了许多种方法[Robertson 和 Spark Jones，1976]。他们首先给出了两个相互排斥的独立性假设：

25

I1：词项在相关文档中的分布是独立的并且在所有文档中的分布是独立的。

I2：词项在相关文档中的分布是独立的并且它们在非相关文档中的分布也是独立的。

他们同样提出了两种方法来展现结果集，被称作排序原则。

O1：相关的可能性仅仅基于文档中出现的查询词项。

O2：相关的可能性基于文档中出现的查询词项和未出现的查询词项。

I1 表明词项在文档中随机出现，也就是说，一个词项在一篇文档中出现绝对不会影响到另外一个词项在这篇文档中出现。这与我们先前给出的例子类似，对于给定比赛胜利的情况，一个好的游击手并不会影响到天气情况。这也同样表明所有文档中词项的分布对于所有文档是绝对独立的，也就是说，一个词项在一篇文档中出现绝对不会影响到相同的词项在其他文档中出现。这好比说，在一场比赛中优秀游击手的出场不会影响到他是否在别的比赛中出场。类似地，一场比赛中优秀游击手的出场与否不会影响到其他场比赛的天气情况。

I2 表示相关文档中的词项是独立的，也就是说，这些词项满足 I1 并且在非相关文档中的词项同时满足 I1。回到我们的例子中，就好比说无论输赢，优秀游击手和晴朗的天气的独立性都是保持不变的。

O1 表明文档只有在它们包含匹配查询词时才会被排在前面(即唯一使用的依据是哪个查询词项实际在文档中出现)。我们注意到这种排序假设现在并没有被广泛采用，这是因为考虑查询词项没有在文档中出现的情况是同样重要的。这在实际当中是不易实现的。大多数系统使用倒排索引来标识每个词项以及一个给定文档中所有出现的词项。如果需要找出一篇文档中没有出现的词项，索引可以标识出所有在该文档中没有出现的词项（具体细节我们将在 5.1 节中给予详细讨论)。为了避免去查找文档中未出现的词，我们将不包含任何查询词文档的相关概率估计为零——这与随机文档的相关概率是不同的。零概率并不意味着我们不知道任何信息：这至少意味着我们有一些判定文档非相关的依据。这样我们就可以将基于 O2 的权重计算转换为仅仅计算已出现词的权重。

O2 将 O1 引入的更深入一些，并且 O2 要求我们同时考虑查询词项出现和不出现两种情况。因此，对于需要词项 t_1 和词项 t_2 的一个查询——一篇仅包含一个查询词项的文档应该比同时包

26 含两个词项的文档的排序靠后一些。

在不同的排序原则和独立性假设的组合下，可以得出 4 种权重。给出一个词项 t，考虑以下变量：

N —— 文档集中文档的数量；

R —— 对于已知查询 q 对应的相关文档的数量；

n —— 包含词项 t 的文档数目；

r —— 包含词项 t 的相关文档数目。

选择 I1 和 O1 组合，会得到如下权重：

$$w_1 = \lg\left(\frac{\frac{r}{R}}{\frac{n}{N}}\right)$$

选择 I2 和 O1 组合，得到如下权重：

$$w_2 = \lg\left(\frac{\frac{r}{R}}{\frac{n-r}{N-R}}\right)$$

选择 I1 和 O2 组合，得到如下权重：

$$w_3 = \lg\left(\frac{\frac{r}{R-r}}{\frac{n}{N-n}}\right)$$

选择 I2 和 O2 组合，得到如下权重：

$$w_4 = \lg\left(\frac{\frac{r}{R-r}}{\frac{n-r}{(N-n)-(R-r)}}\right)$$

Robertson 和 Sparck Jones 认为 O2 是正确的，并且 I2 比 I1 更能符合实际情况。因此，w_4 应该最有可能产生最好的结果。接下来，他们用实验结果表明 w_4 和 w_3 效果要比 w_1 和 w_2 好。许多以 w_4 为基础进行后续研究，还加入其他重要因素，如文档的词频和相关文档的长度等。我们将在 2.2.3 节详细讨论这些 w_4 的扩展研究。

当使用不完整的相关性信息时，由于估计相关性的不确实性，我们将权重都加0.5。Robertson 和 Sparck Jones 建议："这个处理看起来有些随意，但是这确实是基于实际统计数据的。"修改后的权重公式看起来如下：

27

$$w = \lg\left(\frac{\frac{r+0.5}{(R-r)+0.5}}{\frac{(n-r)+0.5}{(N-n)-(R-r)+0.5}}\right)$$

据说概论模型的优点是其完全基于概率论。这就意味着其他模型有一定的随意性。或许其他模型实验效果很好，但是由于参数很难估计，所以缺乏坚实的理论基础。或者需要完整的训练数据，或者进行了不准确的估计。

这种争论类似于曾经发生的关系数据库管理系统和面向对象的数据库管理系统。面向对象的数据库管理系统有时被称作"真实世界"的数据模型，但是缺乏坚实的理论基础。另一方面，关系数据库管理系统有着坚实的理论基础支撑，但是有时对于真实数据的建模却存在问题。

1. 举例

我们仍然使用前一节向量空间模型中举的例子。现在我们来演示一下怎么使用 4 种权重来进行相关性排序。

我们再一次给出文档集和查询：

Q："gold silver truck"

D_1：“Shipment of gold damaged in a fire.”

D_2：“Delivery of silver arrived in a silver truck.”

D_3：“Shipment of gold arrived in a truck.”

由于概率模型需要训练数据，所以我们假定这三篇文档就是训练数据，并且认为文档 D_2 和文档 D_3 与该查询相关。

为了计算相似度，首先计算出查询词项的权重，然后计算出匹配词项的权重的和。我们关注以下 4 个变量：

N —— 文档集中文档的数量；

n —— 一个指定词项索引的文档；

R —— 查询 Q 对应的相关文档的数量；

r —— 一个指定词项索引的相关文档的数量。

我们在表 2-2 中给出了每个查询词的值。正如前文所述，对于一个已知的词项，Robertson

28 和 Sparck Jones 提出了 4 种不同的权重等式来估计包含查询词的文档相关的可能性。

表 2-2　每个查询词项的频率

	gold	silver	truck
N	3	3	3
n	2	1	2
R	2	2	2
r	1	1	2

$$w_1 = \lg\left(\frac{\frac{r}{R}}{\frac{n}{N}}\right)$$

$$w_2 = \lg\left(\frac{\frac{r}{R}}{\frac{n-r}{N-R}}\right)$$

$$w_3 = \lg\left(\frac{\frac{r}{R-r}}{\frac{n}{N-n}}\right)$$

$$w_4 = \lg\left(\frac{\frac{r}{R-r}}{\frac{n-r}{(N-n)-(R-r)}}\right)$$

读者注意到在我们的文档集中，由于 $n-r=0$，silver 的权重为无穷大。这是因为 silver 只在相关文档中出现。由于我们只是在预测方式中使用这种方法，Robertson 和 Sparck Jones 建议为每个变量增加一个常量[Robertson 和 Sparck Jones，1976]。新的权重公式为：

$$w_1 = \lg\left(\frac{\frac{r+0.5}{R+1}}{\frac{n+1}{N+2}}\right)$$

$$w_2 = \lg\left(\frac{\frac{r+0.5}{R+1}}{\frac{n-r+0.5}{N-R+1}}\right)$$

$$w_3 = \lg\left(\frac{\frac{r+0.5}{R-r+0.5}}{\frac{n+1}{N-n+1}}\right)$$

$$w_4 = \lg\left(\frac{\frac{r+0.5}{R-r+0.5}}{\frac{n-r+0.5}{(N-n)-(R-r)+0.5}}\right)$$

使用以上公式进行计算，我们得出以下权重：

gold

$$w_1 = \lg\left(\frac{\frac{1+0.5}{2+1}}{\frac{2+1}{3+2}}\right) = \lg\frac{0.5}{0.6} = -0.079$$

silver

$$w_1 = \lg\left(\frac{\frac{1+0.5}{2+1}}{\frac{1+1}{3+2}}\right) = \lg\frac{0.5}{0.4} = 0.097$$

truck

$$w_1 = \lg\left(\frac{\frac{2+0.5}{2+1}}{\frac{2+1}{3+2}}\right) = \lg\frac{0.833}{0.6} = 0.143$$

gold

$$w_2 = \lg\left(\frac{\frac{1+0.5}{2+1}}{\frac{2-1+0.5}{3-2+1}}\right) = \lg\frac{0.5}{0.75} = -0.176$$

silver

$$w_2 = \lg\left(\frac{\frac{1+0.5}{2+1}}{\frac{1-1+0.5}{3-2+1}}\right) = \lg\frac{0.5}{0.25} = 0.301$$

truck

$$w_2 = \lg\left(\frac{\frac{2+0.5}{2+1}}{\frac{2-2+0.5}{3-2+1}}\right) = \lg\frac{0.833}{0.25} = 0.523$$

gold

$$w_3 = \lg\left(\frac{\frac{1+0.5}{2-1+0.5}}{\frac{2+1}{3-2+1}}\right) = \lg\frac{1.0}{1.5} = -0.176$$

silver

$$w_3 = \lg\left(\frac{\frac{1+0.5}{2-1+0.5}}{\frac{1+1}{3-1+1}}\right) = \lg\frac{1.0}{0.667} = 0.176$$

truck

$$w_3 = \lg\left(\frac{\frac{2+0.5}{2-2+0.5}}{\frac{2+1}{3-2+1}}\right) = \lg\frac{5}{1.5} = 0.523$$

30

gold

$$w_4 = \lg\left(\frac{\frac{1+0.5}{2-1+0.5}}{\frac{2-1+0.5}{(3-2)-(2-1)+0.5}}\right) = \lg\frac{1}{3} = -0.477$$

silver

$$w_4 = \lg\left(\frac{\frac{1+0.5}{2-1+0.5}}{\frac{1-1+0.5}{(3-1)-(2-1)+0.5}}\right) = \lg\frac{1}{0.333} = 0.477$$

truck

$$w_4 = \lg\left(\frac{\frac{2+0.5}{2-2+0.5}}{\frac{2-2+0.5}{(3-2)-(2-2)+0.5}}\right) = \lg\frac{5}{0.333} = 1.176$$

结果汇总见表 2-3。

表 2-3　词项权重

	w_1	w_2	w_3	w_4
gold	–0.079	–0.176	–0.176	–0.477
silver	0.097	0.301	0.176	0.477
truck	0.143	0.523	0.523	1.176

通过对给定文档中出现的查询词项的权重相加，就可以得到文档的相似度。表 2-4 给出了 4 种权重计算模式下每篇文档的相似度。对于文档 D_1，“gold”是唯一出现的词项，所以文档 D_1 的权重就是“gold”的权重，也就是–0.079。对于文档 D_2，“silver”和“truck”是出现的词，所以文档 D_2 的权重就是“silver”和“truck”的和，也就是 0.097 + 0.143 = 0.240。对于文档 D_3，“gold”和“truck”是出现的词，所以文档 D_3 的权重就是“gold”和“truck”权重之和，也就是 –0.079 + 0.143 = 0.064。

表 2-4 文档权重

	w_1	w_2	w_3	w_4
D_1	–0.079	–0.176	–0.176	–0.477
D_2	0.240	0.824	0.699	1.653
D_3	0.064	0.347	0.347	0.699

2. 实验结果

最初的关于 4 种权重计算模式的测试是在克兰菲尔德（Cranfield）文档集（包含 1 400 篇文档）上进行的。结果表明第三种权重和第四种权重的性能相当，但是优于第一种权重和第二种权重。还有科研人员在 UKCIS 文档集（包含 27 361 篇文档）上研究第一种权重和第四种权重衡量方式上的不同[Sparck Jones，1979a]。研究发现，使用第四种权重性能提高很多。 31

研究者还使用了其他两组对比实验。第一组只是简单地依据匹配词项的个数排序，第二组使用 IDF 来估计权重。实验表明，这些方法都次于这四种权重的任何一种，但是使用 IDF 比简单地使用匹配词项的频率要好一些。在所有的情况中，文档的排序为 D_2,D_3,D_1——与 2.1 节向量空间模型的排序结果相同。

我们还没有使用词项在给定文档中出现的次数，权重函数只是基于词项在大量文档中是否出现。因此，如果词项 t 在 10 篇相关文档中出现 50 次和词项 u 在相同的相关文档中只出现 10 次的权重是一样的。

3. 引入词频

在原始的概率模型中并没有使用词频。Croft 和 Harper 在概率模型中引入了词频[Croft 和 Harper，1979]。相关度是通过词项在一篇给定文档中出现的概率来估计的，而不是简单地看词项在文章中出现与否。词频用来估计文档中出现的词项的可能性大小。下面给出了新的相似度。

$$SC(Q,D_j)=C\sum_{i=1}^{t}q_i d_{ij}+\sum_{i=1}^{t}P(d_{ij})q_i d_{ij}\lg\left(\frac{N-n_i}{n_i}\right)$$

$P(d_{ij})$表示词项 i 在文档 j 中出现的概率，并且可以只通过词项 i 在文档 j 中出现的频率来估计。遗憾的是，这个频率并不是一个真实的概率，所以要对词频进行归一化处理。归一化词频可以通过下式计算：

$$ntf_{ij}=\frac{tf_{ij}}{\max(tf_{1j},\ tf_{2j},\ \cdots,\ tf_{ij})}$$

归一化词频是给定词项的频率与文档中最大词频的比值。如果词项 i 在文档中出现了 10 次，并且其他词项的最大频率为 100，那么 ntf_{ij} 就是 0.1。

Croft 和 Harper 在克兰菲尔德文档集和 NPL 文档集的 11 429 篇文档上比较了归一化后的词频、未归一化的词频和一个未使用词频的对比系统。和对比系统相比，实验结果表明，使用归 32

一化词频后的系统效果有统计性的显著提升。在大多数情况下，未归一化词频的系统比对比系统的效果还要差。

2.2.2　非二值独立模型

Yu、Meng 和 Park 提出了非二值独立模型（Non-Binary Independence Model），在词项权重计算中很自然地引入了词频和文档长度[Yu 等人，1989]。一旦计算出词项权重，就可以使用向量空间模型（见 2.1 节）计算内积来获得最终的相似度。

简单的词项权重计算方法基于词项是否在相关文档中出现来估计权重。现在使用在相关文档中出现 *tf* 次的词项来估计一篇文档相关的概率，而不是使用词项出现与否来估计相关概率。例如，一个包含 10 篇文档的文档集，其中文档 1 包含词项蓝色一次，而文档 2 包含词项蓝色 10 次。假设文档 1 和文档 2 都是相关的，且其他 8 篇文档是不相关的。使用简单的词项权重计算模型，我们可以计算 $P(\text{Rel}|\text{blue}) = 0.2$，因为蓝色在 10 篇文档中的两篇中相关。

使用非二值独立模型，可以为每个词频计算单独的概率。因此，我们计算蓝色出现一次的概率 $P(1|\text{R}) = 0.1$，因为它在文档 1 中确实只出现了一次。蓝色出现 10 次的概率为 $P(10|\text{R}) = 0.1$，因为它在 10 篇文档中的一篇中出现了 10 次。

为了引入文档长度，权重计算使用文档的大小进行归一化。因此，如果文档 1 包含 5 个词项，且文档 2 包含 10 个词项，那么“蓝色”在相关文档中出现 1 次的概率就相当于原来“蓝色”在相关文档中出现 0.5 次的概率。

我们同样使用一个词项导致一篇非相关文档的概率。最终的权重为词项在相关文档中出现 *tf* 次与词项在非相关文档中出现 *tf* 次的比率。

较为形式化的定义如下：

$$\lg \frac{P(d_i \mid R)}{P(d_i \mid N)}$$

这里 $P(d_i|R)$表示相关文档中第 i 个词项的出现频率为 d_i 的概率，$P(d_i|N)$表示非相关文档中第 i 个词项的出现频率为 d_i 的概率。

33

举例

回到我们的例子，文档和查询如下：

Q：“gold silver truck”

D_1：“Shipment of gold damaged in a fire.”

D_2：“Delivery of silver arrived in a silver truck.”

D_3：“Shipment of gold arrived in a truck.”

表 2-5　词项到文档的映射表

docid	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	1	0	1	0	1	1	1	1	1	0	0
D_2	1	1	0	1	0	0	1	1	0	2	1
D_3	1	1	0	0	0	1	1	1	1	0	1
Q	0	0	0	0	0	1	0	0	0	1	1

因此，我们有三篇文档和一个查询，总共 11 个词（见表 2-5）。训练数据包含相关文档和非

相关文档。我们假设文档 2 和文档 3 是相关的，文档 1 是不相关的（毕竟从旁观者的角度，我们是随意地将其定义为相关）。通过文档长度进行归一化后，如表 2-6 所示。

表 2-6 归一化文档长度

docid	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	$\frac{1}{7}$	0	$\frac{1}{7}$	0	$\frac{1}{7}$	$\frac{1}{7}$	$\frac{1}{7}$	$\frac{1}{7}$	$\frac{1}{7}$	0	0
D_2	$\frac{1}{8}$	$\frac{1}{8}$	0	$\frac{1}{8}$	0	0	$\frac{1}{8}$	$\frac{1}{8}$	0	$\frac{1}{4}$	$\frac{1}{8}$
D_3	$\frac{1}{7}$	$\frac{1}{7}$	0	0	0	$\frac{1}{7}$	$\frac{1}{7}$	$\frac{1}{7}$	$\frac{1}{7}$	0	$\frac{1}{7}$

我们没有对查询进行归一化。查询中出现的词项是：“gold”、“silver”和“truck”。对于文档 D_1，“gold”的权重是：

$$\lg\left[\frac{P\left(\frac{1}{7}\mid R\right)}{P\left(\frac{1}{7}\mid N\right)}\right]=\lg\frac{\frac{1}{2}}{1}=-0.3010$$

在两篇相关文档中，一篇有词频为 $\frac{1}{7}$ 的词，而另外一篇没有，所以 $P\left(\frac{1}{7}\mid R\right)=\frac{1}{2}$。然而，唯一一个非相关文档包含词频为 $\frac{1}{7}$ 的词项“gold”，所以 $P\left(\frac{1}{7}\mid R\right)=1$。

34

对于 D_1 中的“silver”，我们得到

$$\lg\left(\frac{P(0\mid R)}{P(0\mid N)}\right)=\lg\frac{1}{2}=-0.3010$$

我们可以通过这种方法来计算文档中每个词项的权重和已知词频的权重。我们可以构造出对应向量，然后以此计算查询和每个文档的相似度。

在我们的例子中，只需要考虑很少的频率，但是对于一个正常的文档集来说，要考虑很多的频率，特别是使用归一化文档长度后。为了缓解这个问题，我们可以对所有频率进行聚类。因此，所有频率为 0 的文档归为一类，但是对于正词频的文档，区间$(0,f_1],(f_1,f_2],\cdots,(f_n,\infty]$的选择是区间要包含合适的相等的词项数。为了计算权重，$P(d_i|R)$和 $P(d_i|N)$分别用 $P(d_i\in I_j\mid R)$ 和 $P(d_i\in I_j\mid N)$ 代替。I_j表示第 j 个区间$(f_{j-2},f_{j-1}]$。权重变为：

$$\lg\left(\frac{P(d_i\in I_j\mid R)}{P(d_i\in I_j\mid N)}\right)$$

2.2.3 泊松模型

Robertson 和 Walker 研究出一种使用泊松分布来估计相关性概率，并且引入了词频和文档长度的概率模型 [Robertson 和 Walker，1994]。在标准的概率模型中，权重计算如下：

$$w=\lg\left(\frac{p(1-q)}{q(1-p)}\right)$$

其中 p 是词在已知相关文档中出现的概率，q 是在已知非相关文档中出现的概率。

我们使用 p_{tf} 来引入词频。已知相关性下，p_{tf} 表示词频为 tf 的词出现的概率，q_{tf} 表示对应的非相关的概率。下标 0 表示未出现的词项。权重计算就变为：

$$w = \lg\left(\frac{(p_{tf})(q_0)}{(q_{tf})(p_0)}\right)$$

35

研究人员假定：词在文档中随机出现，且符合泊松分布。

$$p(tf) = \mathrm{e}^{-m\frac{m^{tf}}{tf!}}$$

当前文档所阐述的语义内容是否与查询词所表示的概念相同？参数 m 随着不同的语义情况而变化。这导致权重计算如下：

$$w = \lg\frac{\left(p' + (1-p')\left(\frac{\mu}{\lambda}\right)^{tf}\mathrm{e}^{j}\right)\left(q'\mathrm{e}^{(-j)} + (1-q')\right)}{\left(q' + (1-q')\left(\frac{\mu}{\lambda}\right)^{tf}\mathrm{e}^{j}\right)\left(p'\mathrm{e}^{(-j)} + (1-p')\right)}$$

式中　λ ——与词 t 有关的文档的泊松分布参数；

μ ——与词 t 无关的文档的泊松分布参数；

j ——$\lambda - \mu$；

p' ——已知词 t 相关的文档是相关文档的概率；

q' ——与已知词 t 无关的文档是不相关文档的概率。

这种权重计算的难点在于如何在实际中应用：实际应用中并没有关于 4 个参数 p'、q'、λ 和 μ 的直接依据。基于许多实际中容易得到的观察量：词频和文档长度有着类似的形状，研究人员使用了类似形状的曲线和更简单的函数。为了引入词频，我们使用以下函数：

$$w' = w\frac{tf}{k_1 + tf}$$

其中 w 是标准概率权重；k_1 是未知常量，它的值取决于文档集，并且通过实验获取。

该模型同样考虑了文档长度。考虑文档长度因素最简单的方法是修改上式中的 w'，可以使用如下公式替换：

$$k_1 = \frac{k_1 \times d}{\Delta}$$

式中　d ——文档长度；

Δ ——文档平均长度。

这样计算 w' 的公式变为：

36

$$w' = w\left(\frac{tf}{\frac{k_1 \times d}{\Delta} + tf}\right)$$

类似于文档频率的方式，研究人员使用文档和查询间的对称度来引入查询词的词频。调节系数 k_1 用于衡量文档中词频的效果。类似地，另外一个参数 k_3 用来衡量查询词频的效果（q_{tf}）。

最后，我们引入另外一比例因子 k_2，可以更好地逼近二元泊松分布估计。k_2 如下所示：

$$k_2\left(|Q|\left(\frac{(\Delta-d)}{(\Delta+d)}\right)\right)$$

其中 k_2 是一个通过实验确定的常量，$|Q|$ 是查询词项的个数。这一项允许 k_2 为很高的值来加强比平均长度短的文档的权重。将这些修改应用于相似度后，就是：

$$SC(Q,D_i)=\sum_{j=1}^{t}\lg\left(\frac{\dfrac{r}{(R-r)}}{\dfrac{(n-r)}{(N-n)-(R-r)}}\right)\left(\frac{tf_{ij}}{\dfrac{(k_1 dl_i)}{\Delta}+tf_{ij}}\right)\left(\frac{qtf_j}{k_3+qtf_j}\right)+\left((k_2)|Q|\left(\frac{(\Delta-dl_i)}{(\Delta+dl_i)}\right)\right)$$

式中 N —— 文档集中文档的数量；

n —— 文档集中通过一个已知词索引的文档数量；

R —— 对于查询相关的文档数量；

r —— 给定词索引的相关文档数量；

tf_{ij} —— 文档 i 中词 j 的词频；

qtf_j —— 查询 Q 中词 j 的词频；

dl_i —— 文档 i 中词的数量；

$|Q|$ —— 查询中词的数量；

Δ —— 平均文档长度；

k_1, k_2, k_3 —— 调节参数。

k_1 和 k_3 取较小的值可能会削弱文档词频和查询词频的影响。如果二者都是零，那么将会去除文档词频和查询词频的影响。k_1 和 k_3 取较大的值会明显削弱第一项的大小。

分子中引入 k_1+1 和 k_3+1 的因子对全局排序并不起作用，这是因为这些因子对所有文档的 37
作用都一样。然而这确实允许给 k_1 和 k_3 赋很大的值而不会削减第一项的权重。此外，这样可以归一化调节参数。其思想为：当词频为 1 时，就没有必要改变原始的权重。

为了归一化文档长度，在相似度计算的第一项中引入了分母 Δ。如果仅仅是因为文档长，它给出了关于主题更加具体的细节，那么这是明智的。在这种情况下，长文档的权重不应该比短文档的大。然而，文档长的原因可能是它讨论了更多无关的主题。在这种情况下，长文档就应该受到惩罚。新的调节参数 b 可以基于文档集的性质调节查询。这个参数是在涉及 tf_{ij} 的因子通过用 K 代替 k_1 引入的，那么：

$$K=k_1\left((1-b)+b\left(\frac{dl_i}{\Delta}\right)\right)$$

引入调节参数 b。并在分子上加入 k_1+1 和 k_3+1 后：

$$SC(Q,D_i)=\sum_{j=1}^{t}\lg\left(\frac{\dfrac{r}{(R-r)}}{\dfrac{(n-r)}{(N-n)-(R-r)}}\right)\left(\frac{(k_1+1)tf_{ij}}{K+tf_{ij}}\right)\left(\frac{(k_3+1)qtf_j}{k_3+qtf_j}\right)+\left((k_2)|Q|\frac{\Delta-dl_i}{\Delta+dl_i}\right)$$

在[Robertson 等人，1995]的实验中，这些值分别为 k_1=1，k_2=0，k_3=8，b=0.6。

举例

同样使用前文中的例子，我们首先计算 w_4 为：

gold = −0.477

silver = 0.477

truck = 1.176

avgdl = 22/3=7.33

使用同样的参数 k_1，k_2，k_3，b，我们计算 dl_i 的值：

$dl_1 = 7$

$dl_2 = 8$

38

$dl_3 = 7$

对于 D_1，唯一匹配查询的是“gold”，其出现的词频 $tf = 1$，因此 D_1 的相似度仅仅是“gold”的值（D_1 的长度 dl 是 7）。

$$K = 1\times\left((1-0.6)+\frac{0.6\times 7}{7.33}\right)=0.973$$

既然有了 K 的值，那就可以计算 D_1 的相似度。相似度是所有词项的和，但只有词项“gold”是非零值。我们首先使用前面计算得到的 $w_4 = -0.477$ 开始计算。

$$SC(Q,D_1)=-0.477\times\left(\frac{(1+1)\times 1}{0.973+1}\right)\left(\frac{8+1}{8+1}\right)=-0.477\times\left(\frac{2}{1.973}\right)=-0.484$$

对于文档 D_2，匹配查询的词项“silver”和“truck”的和为非零值。

$$K = 1\times\left(0.4+\frac{0.6\times 8}{7.33}\right)=1.055$$

对于“silver”，tf_{12}=2：

$$w_4 = 0.477\times\left(\frac{(1+1)\times 2}{1.055+2}\right)\left(\frac{(8+1)\times 2}{8+2}\right)=1.124$$

对于“truck”，tf_{22}=1：

$$w_4 = 1.176\times\left(\frac{(1+1)\times 1}{1.055+1}\right)\left(\frac{(8+1)\times 1}{8+1}\right)=1.145$$

$$SC(Q,D_2)=1.124+1.145=2.269$$

对于 D_3，$dl = 7$，所以 K=0.973（同 D_1）。两个词“gold”和“truck”都出现了一次。对于“gold”，$tf_{13} = 1$。

$$w_4 = -0.477\times\left(\frac{(1+1)\times 1}{0.973+1}\right)\left(\frac{(8+1)\times 1}{8+1}\right)=-0.484$$

对于“truck”，$tf_{23} = 1$。

$$w_4 = 1.176\times\left(\frac{(1+1)\times 1}{0.973+1}\right)\left(\frac{(8+1)\times 1}{8+1}\right)=1.192$$

39

$$SC(Q,D_3)=-0.484+1.192=0.708$$

通过表 2-7，我们再次对比考虑词频的相似度和不考虑词频的相似度，排序结果同样是 D_2, D_3, D_1。

表 2-7 泊松模型：最终的相似度

	无 *tf*	*tf*
D_1	–0.477	–0.484
D_2	1.653	2.269
D_3	0.699	0.708

2.2.4 文档片段

在文献[Kwok，1990]中，作者给出了一种标准概率模型的改进版。算法的前提是基于文档不同的片段（Component）进行文档排序。例如，一篇文档可以划分成多个段落，并且每一个段落都可以计算相似度。一旦计算完每一个段落的相似度，就需要有种方法来综合每个段落的相似度，从而对整篇文档排序。Kwok 提出了使用几何平均的方法（对于 n 个数，取其乘积的 n 次方根）来有效计算每个部分的平均值。

对给定的片段进行排序的方法有多种，并且片段的大小也可以不同。如果把整篇文档看做一个片段，那么我们就回退到了传统的概率信息检索模型。

把给定片段的基本权重定义为片段相关的概率与片段无关的概率的比值，也就是：

$$w_{ak} = \ln\left(\frac{r_{ak}}{1-r_{ak}}\right) + \ln\left(\frac{(1-s_{ak})}{s_{ak}}\right)$$

r_{ak}和 s_{ak}可以通过以下三种方式的任意一种来计算。

- **使用自相关来初始化参数。**一个片段与其自身相关，结果是：

$$r_{ak} = \frac{q_{ak}}{L_a}$$

$$s_{ak} = \frac{F_k}{N_w}$$

其中 L_a是文档集中片段的数量，F_k是词项 k 在文档集中出现的次数。N_w是文档集中不同词的个数。

- **逆文档集词频（ICTF）。**s 的估计要好一些，因为不相关文档可能要比相关文档更多一些。因此，在整个文档集中，词频比较低的词的 s_{ik}值就比较小。假定 r_{ak}的最初估值比较差，并且仅仅指定为常量 p。使用常量 r_{ak}导致整个权重大约等于 s_{ik}。s_{ik}是从整个文档集的估计值中去除一篇相关文档 d_{ik}后计算出来的。也就是通过使用与假定的“相关文档”相匹配的词的数量 d_{ik}来计算。s_{ik}的计算公式如下：

$$s_{ik} = \left(\frac{F_k - d_{ik}}{N_w - L_i}\right)$$

在我们的权重计算中，使用参数 p 得出下面的权重，这个权重与 IDF 非常接近：

$$w_{ik} = \ln\left(\frac{p}{1-p}\right) + \ln\left(\frac{1-s_{ik}}{s_{ik}}\right)$$

□ 本质上讲，权重的计算要基于用户的反馈（相关反馈的内容见3.1节）。一旦我们得到了每一个片段的权重，那么接下来就是如何综合起来计算整篇文档的权重——无论是采用以查询为中心的方法，还是采用以文档为中心的方法，还是两者结合的方法。拿以查询为中心为例，查询是已知的，所有的权重都是以与查询的相关性计算的。在求出每一片段的权重后，使用几何平均来计算。这就使权重变为：

$$\sum_{i=1}^{k}\left(\frac{d_{ik}}{L_i}\right)w_{ak}$$

以文档为中心的计算方法是计算查询的每一个片段，然后分别计算与文档的相关性权重，再取平均值。计算如下：

$$\sum_{i=1}^{k}\left(\frac{q_{ik}}{L_i}\right)w_{ik}$$

接下来我们考虑综合策略。综合策略只是以查询为中心的权重和以文档为中心的策略之和。公式如下：

$$\sum_{i=1}^{k}\left(\frac{d_{ik}}{L_i}\right)w_{ak}+\sum_{i=1}^{k}\left(\frac{q_{ik}}{L_i}\right)w_{ik}$$

41

片段理论与基于词项的查询效果相当，并且在相关反馈下效果更好。综合以查询为中心和以文档为中心的计算方法优于分别使用以查询为中心的计算方法和以文档为中心的计算方法。

2.2.5　概率模型的关键问题

通常，概率模型必须设法解决两个基本问题：参数估计和独立性假设。参数估计是指这样一种问题：精确的概率计算需要基于相关性估计。如果没有良好的训练数据集，通常很难精确地估计参数。第二个问题是独立性假设。大家都很清楚词“new”的出现增加了词“york”出现的可能性[①]。尽管在现实中，独立性假设并不符合真实情况，但是许多概率模型都要求这种独立性假设。

1. 参数估计

在20世纪70年代就有明确的文献记载，需要进行很好的参数估计。最初的实验是将文档集简单地分为奇数部分和偶数部分，然后进行权重计算。奇数部分作为训练数据，在获取相关信息后用于偶数部分文档的检索。对于大多数应用，相关性的先验概率是不知道的。后续的论文使用了缩减的相关性信息[Sparck Jones，1979b]。这些论文研究使用最相关的一篇或两篇文档来作为训练数据，而不再使用所有的相关文档。对于比较小的克兰菲尔德数据集，使用较少的最相关文档的效果与使用所有相关文档的效果相当。然而遗憾的是，在较大的UKCIS数据集上，只使用两篇最相关文档的效果远差于使用所有相关文档的效果。

原始模型并没有表明如何开始计算。一旦获取到相关性信息（通过训练集），就可以进行下一步的检索。在一个在线系统中并不能提前得知用户提交哪个查询，也就不能使用先前给出的权重计算公式。这些都需要参数 r 和 R 的值，而这些值只能通过提交查询然后检索所有相关文档获得。当然，也可以使用别的方法进行初始查询，然后要求用户对初始检索结果的相关性进

① “new york”表示纽约市，而“new”本身也是一个单词。——译者注

行判断。这些信息将用于随后的查询。这项技术叫做相关反馈，将在 3.1 节给予详细介绍。

使用概率权重作为实现相关反馈的方法使概率模型退化为一个对于任何检索模型都可以使用的实用工具。系统中可以使用余弦值来进行初始的排序，然后使用概率权重进行相关反馈。

使用没有任何先验相关性信息的概率模型产生的问题可以参见文献[Croft 和 Harper, 1979]。在这种情况下，很显然概率模型就可以成为不需要任何其他协助就可以进行文档排序的检索模 42
型。关键问题是我们假设（没有任何相关信息）每个词引起相关的概率是相等的。因此，我们得到下面的相似度计算公式：

$$SC(Q,D_i)=C\sum_{i=1}^{t}q_i d_{ij}+\sum_{i=1}^{t}q_i d_{ij}\lg\frac{N-n_i}{n_i}$$

式中 N——文档集中文档的数量；

n_i——词 i 索引的文档的数量；

d_{ij}——若词 i 在文档 j 中出现，则该值为 1；

d_{ij}——若词 i 在文档 j 中未出现，则该值为 0；

q_i——若词 i 在查询中出现，则该值为 1；

q_i——若词 i 在查询中未出现，则该值为 0。

C 是常量，可以根据检索的不同而调节。在大的文档集上，词项权重 $\frac{N-n_i}{n_i}$ 非常接近逆文档频率 $\frac{N}{n_i}$（N取较大值）。因此，整个表达式非常接近在向量空间模型中使用的 *tf-idf*。

作者比较了这种方法计算的相似度，还有余弦系数和只通过每个词项的 IDF 求和得到的权重系数。新的相似度的效果要更好，但是值得注意的是，作者仅仅是在较小的克兰菲尔德文档集上做的测试。

最近，在计算无相关性信息或者仅带有少量相关性信息的相似性权重问题上的研究，Croft 和 Harper 的工作有所完善[Robertson 和 Walker，1997]。他们注意到，在某些情况下，Croft 和 Harper 的权重计算模式将导致负权重。

在 Robertson 和 Sparck Jones 最初的模型中，两种概率决定了权重大小。第一个是 p 的值，用于估计文档包含一个给定词项并且相关的概率。参数 q 估计一篇文档包含给定词项并且不相关的概率。在先前的模型中，我们假设 p 和 q 为常量，但是 Robertson 和 Walker 允许 p 根据已知的相关性依据的函数而变化。 43

特别地，不带先验信息的权重计算函数给出了逆文档频率（或者有一些轻微的变化）。另外一个极端，带有大量相关性信息的权重计算函数由相关性信息决定。Croft 和 Harper 提出的公式可能会导致负权重（当一个词在文档集中超过半数出现时）。Robertson 和 Walker 提出了新的公式，这个公式可以进行调节，并且可以对权重 p 和 q 进行独立估计。也就是，文档相关的前提下，只通过 p 来影响权重，文档不相关的前提下，只通过 q 来影响权重。新公式如下所示：

$$w=\frac{k_5}{k_5+R}\left(k_4+\lg\frac{N}{N-n}\right)+\frac{R}{k_5+R}\left(\lg\frac{r+0.5}{R-r+0.5}\right)$$
$$-\frac{k_6}{k_6+S}\left(\lg\frac{n}{N-n}\right)-\frac{S}{k_6+S}\lg\left(\frac{(s+0.5)}{(S-s+0.5)}\right)$$

式中　R——相关文档的数量；

r——通过已知词索引的相关文档数量；

S——不相关的文档数量；

s——包含已知词的不相关文档的数量；

k_4, k_5, k_6——调节常量，其中 $k_4 \geqslant 0$。

公式前两项给出了根据相关性信息计算的权重，后两项给出了根据非相关性信息计算的权重。（注意：如果没有先验信息（$R=S=0$），那么等式就退化为 $k_4+\lg\frac{N}{n}$）。k_4的值表示查询词的优劣程度，k_5和 k_6分别表示对相关性信息和非相关性信息的敏感度。公式中可以使用基于统计的参数，使用 R和 S的平方根代替 R和 S来调节对应项。

2. 独立性假设

通过简单地综合文档中的词项概率来计算整篇文档的相关概率的关键前提是词项的独立性假设。因为这是一个错误的假设，所以有研究人员认为这是一个“错误的理论”[Cooper，1991]。其实推理网络和逻辑回归法都是用来解决独立性假设问题的，这些将分别在 2.4 节和 3.5 节进行详细讨论。

44

20 世纪 70 年代末、80 年代初的论文开始研究独立性假设的不足[Rijsbergen，1977；Yu 等人，1983]，但是他们都需要词共现的信息，而词共现信息的计算相当耗时。Van Rijsbergen 建议应该通过简单的聚类算法将相关词进行聚类，然后就可以获取每组之间的依赖性[Rijsbergen，1977]。

随着计算机计算速度的提升，这些方法可能马上变为可行的。据我们所知，这些修正方法都没有在大的数据集上做过测试。

2.3　语言模型

统计语言模型是一种“生成”一段文本的概率机制。因此，模型需要定义所有可能的词序分布。最简单的语言模型是一元语言模型，它只需要每个词的分布。更复杂的语言模型需要更多的上下文信息（例如词的前文）来预测下一个词[Charniak，1993；Rosenfeld，2000]。

尽管使用语言模型进行语音识别和机器翻译有二十多年的历史[Hodjat 等人，2003]，语言模型用于信息检索仅起源于 1998 年[Ponte 和 Croft，1998]。语言模型的核心思想为：文档可以通过它们生成查询的可能性的大小来排序。我们考虑朗读一篇文档，如果说话人读出该文档中的所有词语，那么他再说出查询中的词语的概率是多少呢？在形式上，相似度可以简单地定义为：

$$SC(Q, D_i)=P(Q|M_{D_i})$$

其中 M_{D_i}是文档 D_i中隐含的语言模型。

我们需要明确定义前面所讲的“生成”查询的准确含义。也就是说，我们需要为查询构造一个概率模型。一种方法（在[Ponte 和 Croft，1998]中提出）是对任何词项的出现与否使用独立的伯努利事件建模，并且将生成整个查询看做观察到查询中的词项和没有观察到查询外的词项的联合事件。在这种情况下，生成查询的概率为所有查询中出现的词项和没有出现的词项的概率的乘积。也就是：

$$SC(Q,D_i)=\prod_{t_j\in Q}P(t_j\mid M_{D_i})\prod_{t_j\notin Q}(1-P(t_j\mid M_{D_j}))$$ 45

我们可以通过多种方式来估计模型中的 $P(t_j\mid M_{D_i})$。一种直接的方法是：

$$p(t_j\mid M_{D_i})=p_{ml}(t_j\mid M_{D_i})$$

其中 $p_{ml}(t_j\mid M_{D_i})$表示词项分布的最大似然估计（例如，相对词频），如下所示：

$$p_{ml}(t_j\mid M_{D_i})=\frac{tf(t_j,D_i)}{dl_{D_i}}$$

其中 dl_{D_i} 是文档 D_i 的长度。

语言模型的基本思想可以参见图 2-5。现在可以使用相似度，但是存在比较大的问题。如果查询中的词项在文档中没有出现，整个相似度就为零。下面我们考虑一下我们前面所用的一个查询和三篇文档的小例子。 46

Q：“gold silver truck”

D_1：“Shipment of gold damaged in a fire”

D_2：“Delivery of silver arrived in a silver truck”

D_3：“Shipment of gold arrived in a truck”

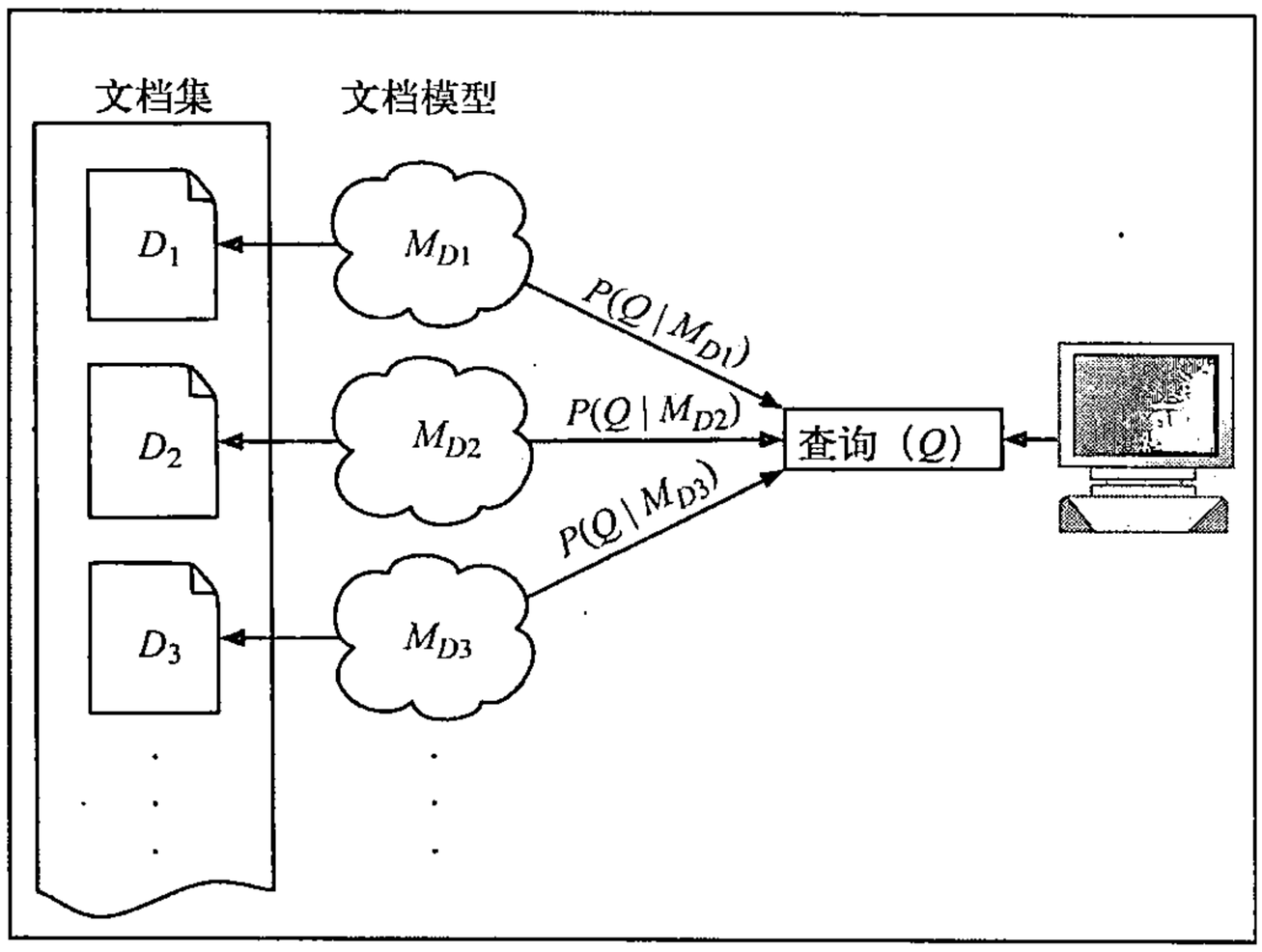


图 2-5 语言模型

词项“silver”在文档 D_1 中没有出现。类似地，“silver”在文档 D_3 中也没有出现，并且“gold”在文档 D_2 中也没有出现。因此，这个例子将导致三篇文档的相似度都为零。

因此，最大似然估计如下：

$$p_{ml}(\text{silver}\mid M_{D_j})=\frac{tf(\text{silver},D_i)}{dl_{D_i}}=0$$

2.3.1 平滑

为了避免查询中的词项在文档中没有出现引起的问题，现在有许多种这些词项估计非零值

的平滑方法。一种方法是假设查询词项可能在这个模型中出现，但是不会高于它在其他文档中出现的比率。最初提出比值 $\frac{cf_t}{cs}$，其中 cf_t 表示词项 t 在文档集中出现的次数，cs 表示词项所有词项在文档集中出现的次数。在这个例子中，“silver”的值估计为 2/22=0.091。

另外一种调整方法源于以下事实：这些文档模型仅仅基于单个独立的文档。这些数据量对于构建一个模型来说非常小。研究者提出使用大的样例数据（整个文档集）来进行如下估计：

$$p_{\text{avg}}(t)=\frac{\sum p_{ml}(t\mid M_d)}{df_t}$$

其中 df_t 是词 t 的文档频率，其倒数就是我们在 2.1 节中计算的逆文档频率。

为了提高估计词项权重的效果，最小化估计风险是有可能的。首先我们定义 $\overline{f_t}$ 为文档中词 t 的平均词频，它的计算公式为：$\overline{f_t}=p_{\text{avg}}(t)\times dl_d$。通过使用几何分布可以得到其风险率：

$$R_{t,d}=\left(\frac{1.0}{1.0+\overline{f_t}}\right)\times\left(\frac{\overline{f_t}}{1.0+\overline{f_t}}\right)^{tf_{t,d}}$$

第一种相似度计算公式描述的是：在信息检索使用的语言模型中，我们对查询中没有出现的词项使用平滑因子 $\frac{cf_t}{cs}$；如果该词在查询中存在，则在小规模文档模型上估计 w 的值，并引

47 入风险函数作为混合参数。现在词项权重的计算公式如下所示：

$$P(t\mid M_{di})=\begin{cases}p_{\text{ml}}(t,d)^{(1-R(t,d))}\times p_{\text{avg}(t)}^{R(t,d)} & \text{如果 } tf(t,d)>0\\ \dfrac{cf_t}{cs} & \text{其他}\end{cases}$$

2.3.2　语言模型举例

下面我们使用前面使用的通用例子来计算相似度。我们的文档集中有三篇文档，共有 22 个词。所以，cs=22。D_1、D_2 和 D_3 的词数分别为 7、8 和 7。表 2-8 给出了 dl_d 的值。

表 2-8　文档中的词数

	D_1	D_2	D_3
dl_d	7	8	7

表 2-9 中给出了 df_t 的值，也就是词 t 的文档频率。我们使用与 2.1.1 节中同样的词序。

表 2-9　文档频率

	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
df_t	3	2	1	1	1	2	3	3	2	1	2

表 2-10 给出了文档集中词项 t 的原始数量 cf_t。

表 2-10　每个词在整个文档集中的词频

	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
cf_t	3	2	1	1	1	2	3	3	2	2	2

表 2-11 给出了文档 d 中词 t 的原始词频 $tf_{t,d}$。首先，我们需要计算 $p_{\mathrm{ml}}(t|M_d)$，也就是词 t 在文档 d 中词项分布下的概率的最大似然估计。对于每一个词项 t，$P_{\mathrm{ml}}(t \mid M_d)=\dfrac{tf_{t,d}}{dl_d}$，见表 2-12。 48

表 2-11 原始词频

	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	1	0	1	0	1	1	1	1	1	0	0
D_2	1	1	0	1	0	0	1	1	0	2	1
D_3	1	1	0	0	0	1	1	1	1	0	1

表 2-12 每个词的最大似然估计

$P_{\mathrm{ml}}(t \mid M_d)$	D_1	D_2	D_3
A	0.143	0.125	0.143
Arrived	0	0.125	0.143
Damaged	0.143	0	0
Delivery	0	0.125	0
Fire	0.143	0	0
Gold	0.143	0	0.143
In	0.143	0.125	0.143
of	0.143	0.125	0.143
shipment	0.143	0	0.143
silver	0	0.250	0
truck	0	0.125	0.143

第二步，我们计算所有包含词 t 的文档的平均概率。公式为：

$$P_{\mathrm{avg}}(t)=\frac{\sum_{d(t\in d)} P_{\mathrm{ml}}(t \mid M_d)}{df_t}$$

对于词“arrived”，它仅在文档 D_2 和 D_3 中出现，所以：

$$P_{\mathrm{avg}}(\text{arrived})=\frac{P_{\mathrm{ml}}(\text{arrived} \mid M_{D_2})+P_{\mathrm{ml}}(\text{arrived} \mid M_{D_3})}{df_{\text{arrived}}}$$

使用我们先前的估计值：我们知道 $P_{\mathrm{ml}}(\text{arrived} \mid M_{D_2})=0.125$，$P_{\mathrm{ml}}(\text{arrived} \mid M_{D_3})=0.143$，$df_{\text{arrived}}=2$。因此，$P_{\mathrm{avg}}(\text{arrived})=(0.125+0.143)/2=0.134$。表 2-13 给出了剩下的词的计算方式。

表 2-13 每个词的平均概率

	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
$P_{\mathrm{avg}}(t)$	0.137	0.134	0.143	0.125	0.143	0.143	0.137	0.137	0.143	0.250	0.134

第三步，我们计算文档 d 中词 t 的风险值。首先，我们要计算文档中词 t 的平均频率 $\overline{f}_t$，$\overline{f}_t=P_{\mathrm{avg}}(t)\times dl_d$。表 2-14 给出了每个词 t 的 $\overline{f}_t$。接下来，我们使用下述风险函数来得到公式(2-3)。 49

$$R_{t,d}=\left(\frac{1.0}{1.0+\overline{f}_t}\right)\times\left(\frac{\overline{f}_t}{1.0+\overline{f}_t}\right)^{tf_{t,d}} \tag{2-3}$$

表 2-14　每个词的平均词频

f_t	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	0.958	0.938	1.000	0.875	1.000	1.000	0.958	0.958	1.000	1.750	0.938
D_2	1.096	1.071	1.143	1.000	1.143	1.143	1.096	1.096	1.143	2.000	1.071
D_3	0.958	0.938	1.000	0.875	1.000	1.000	0.958	0.958	1.000	1.750	0.938

表 2-15 给出了每个文档的每个词的风险值。

表 2-15　每个词的风险值

$R_{t,d}$	D_1	D_2	D_3
a	0.250	0.249	0.250
arrived	0.516	0.250	0.250
damaged	0.250	0.467	0.500
delivery	0.533	0.250	0.533
fire	0.250	0.467	0.500
gold	0.250	0.467	0.250
in	0.250	0.249	0.250
of	0.250	0.249	0.250
shipment	0.250	0.467	0.250
silver	0.364	0.148	0.364
truck	0.516	0.249	0.250

现在我们使用风险值作为混合参数来计算 $P(Q \mid M_d)$，即前文提到的对于一个给定的文档模型产生查询的概率。计算包含两步。首先，我们要计算 $P(t \mid M_d)$，计算方法可以参见公式 (2-4)。表 2-16 中的斜体词匹配查询词。对于所有其他出现的词项，我们使用平滑估计，

50　即 $P(t \mid M_d) = \dfrac{cf_t}{cs}$。

$$P(t \mid M_d) = P_{ml}(t \mid M_d)^{(1.0-R_{t,d})} \times P_{\text{avg}}(t)^{R_{t,d}} \tag{2-4}$$

最后，我们使用公式（2-5）计算出最后的相似值 $P(Q \mid M_d)$。表 2-17 给出了实际的值。

$$P(Q \mid M_d) = \prod_{t \in Q} P(t \mid M_d) \times \prod_{t \notin Q} (1.0 - P(t \mid M_d)) \tag{2-5}$$

表 2-16　每个词项的期望概率（平滑后）

$P(t \mid M_d)$	D_1	D_2	D_3
a	0.141	0.128	0.141
arrived	0.091	0.127	0.141
damaged	0.143	0.045	0.045
delivery	0.045	0.125	0.045
fire	0.143	0.045	0.045
gold	***0.143***	0.091	***0.143***
in	0.141	0.128	0.141
of	0.141	0.128	0.141
shipment	0.143	0.091	0.143
silver	0.091	***0.250***	0.091
truck	0.091	***0.127***	***0.141***

表 2-17 使用语言模型计算的相似值

	D_1	D_2	D_3
$P(Q \mid M_d)$	0.000409	0.001211	0.000743

因此，正如我们所讨论过的其他模型，最后的排序结果为 D_2, D_3, D_1。

尽管查询中的其他词匹配文档 D_1（如：gold），但对于这篇文档，相似值却为零。这是因为其中有一个词没有出现。我们刚讲到的这个模型基于 Ponte 和 Croft 的原始模型，该模型对查询词的出现与否进行了建模，但是忽略了查询词频（也就是查询中词出现的次数）。因此，没有考虑词“silver”在查询中出现一次或两次对查询结果的影响。其他的语言模型的研究（包含早期 TREC-7 中的研究）对每一个查询词使用一元语言模型进行建模，这也就引入了词频[Miller 等人，1999；Hiemstra 和 Kraaij，1998]。

特别地，对于查询 $Q=q_1,q_2,\cdots,q_m$,概率 $p(Q|D)$引入了 $tf(t,Q)$，即查询词的词频（见公式(2-6)）。 51

$$p(Q \mid D) = \prod_{i=1}^{m} p\left(q_i \mid M_D\right) = \prod_{t \in Q} p(t \mid M_D)^{tf(t,Q)} \tag{2-6}$$

这个多项式模型更像语音识别中使用的语言模型。关于二者区别的探讨可以参见文献[McCallum 和 Nigam，1998；Song 和 Croft，1999]。当我们使用多项式语言模型时，我们将检索问题转化为语言模型的估计，即估计 $p(w|M_d)$。

参数估计中有许多平滑函数可以使用，具体细节可以参见文献[Chen 和 Goodman，1998；Manning 和 Schutze，1999]。Good-Turing 平滑方法使用公式（2-7）给出的变换来调整原始的词频。

$$tf^* = (tf+1)\frac{E(N_{tf+1})}{E(N_{tf})} \tag{2-7}$$

这种估计是为了改进初始的语言模型[Song 和 Croft，1999]。遗憾的是，这种估计需要知道在一篇文档中词频相同的词项的数量，而其计算量相当庞大。

在[Zhai 和 Lafferty，2001b]中给出了三种有效的平滑方法。这三种方法是 Jelinek-Mercer 平滑方法[Jelinek 和 Mercer，1980]、基于狄利克雷先验分布的贝叶斯平滑方法[MacKay 和 Peto，1995]以及绝对折扣平滑方法[Ney 等人，1994]。通常对于长查询，Jelinek-Mercer 平滑方法优于基于狄利克雷先验分布的贝叶斯平滑方法和绝对折扣平滑方法。对于仅检索标题的查询，实验证明基于狄利克雷先验分布的贝叶斯平滑方法优于 Jelinek-Mercer 平滑方法和绝对折扣平滑方法。

概括起来讲，平滑方法就是已知一篇文档的情况下，计算一个词出现的概率的能力，也就是计算 $p(t|d)$。平滑方法用于减少在文档中已观察到的词的概率，并且提高文档中未观察到的词的概率。作为最后一招，未观察到的词的概率可以简单地赋值为该词在整个文档集上构造的语言模型的概率 $p(t|C)$。

Jelinek-Mercer 平滑方法在最大似然模型和文档集模型上进行线性插值。参数 λ 用来控制两个模型之间的影响度。较形式化的定义如下：

$$p_\lambda(t \mid d) = (1-\lambda)P_{ml}(t \mid d) + \lambda P(w \mid C)$$

绝对折扣平滑方法只是将文档中出现的词的词频减去一个常量来降低词出现的概率。较形式化的定义如下：

52

$$p_\delta(t\mid d)=\frac{\max(tf(t,d)-\delta,0)}{\sum_t tf(t,d)}+\sigma P(t\mid C)$$

其中$\delta\in[0,1]$是一个常量，且定义如下：

$$\sigma=\delta\frac{|d|_u}{|d|}$$

其中$|d|_u$表示文档d中不同词（unique term）的数量，$|d|$表示文档中词的数量。

通常对于长查询来说，Jelinek-Mercer平滑方法优于基于狄利克雷先验分布的贝叶斯平滑方法和绝对折扣平滑方法。对于仅检索标题的查询，实验证明基于狄利克雷先验分布的贝叶斯平滑方法优于 Jelinek-Mercer 平滑方法和绝对折扣平滑方法。公式（2-8）给出了在贝叶斯方法中使用狄利克雷先验分布的模型。

现在回到我们的例子，由于我们的例子中仅包含三个词，所以将其作为标题检索。因此，接下来我们给出一个简单的例子来阐述如何使用基于狄利克雷先验分布的贝叶斯平滑方法。

$$p_\mu(t\mid d)=\frac{tf(t,d)+\mu P(t\mid C)}{\sum_t tf(t,d)+\mu}\tag{2-8}$$

其中$tf(t,d)$是词t在文档d中出现的次数。$\sum_t tf(t,d)$表示文档d中词的总数。$P(t|C)$表示在文档集上的语言模型，计算方式为词 t 在文档集 C 中出现的次数除以文档集 C 中的总词数，即$\frac{\sum_d tf(t,d)}{cs}$。

文档集中未出现的词的平滑可以使用拉普拉斯方法[Katz, 1987]。因此，$P(t|C)=\frac{1+\sum_d tf(t,d)}{N+cs}$，其中$N$是文档集中不同词的数量。在这个例子中，由于所有的查询词都在文档中出现了，所以我们只使用$\frac{\sum_d tf(t,d)}{cs}$。通常来讲，文档集上语言模型的平滑只有在$cs$特别大时效果才显著。

最后，μ是平滑调节参数。实验结果表明，文档集之间的最佳先验值差别巨大。然而，这个值经常保持在2 000左右。为了使我们的例子可读性更强，我们将μ赋值为3（在这个小的测试文档集中，只有三篇文档，μ赋值为3消减了需要呈现的有效位数，从而为读者提供了更有

53 价值的例子）。

首先，对于每一个词t，我们计算$\sum_d tf(t,d)$。表2-18给出了$\sum_d tf(t,d)$的值。在我们例子中的文档集上，一共有3篇文档，22个词。所以，$cs=22$。使用$\frac{\sum_d tf(t,d)}{cs}$，可以得到$P(t|C)$，如表2-19所示。其次，表2-20再一次给出了文档d中所有词的总数。最后，表2-21再一次给出了$tf(t,d)$。现在，我们通过使用以下公式计算平滑后的概率：$p_\mu(t\mid d)=\frac{tf(t,d)+\mu P(t\mid C)}{\sum_t tf(t,d)+\mu}$。图2-22给出了新的概率。在这个例子中，$\mu$取值为3可以减少需要展现的有效位数。新的相似度计算方式为$P(Q\mid d)=\prod_t p_\mu(t\mid d)$，表2-23给出了结果值。例如，文档1和查询的相似度可以计算为查询中三个词的平滑概率的乘积。对于文档1，就是$0.137\,8\times0.009\,1\times0.009\,1=0.000\,011\,4$。

表 2-18 每个词的 $\sum_d tf(t,d)$

	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
$\sum_d tf(t,d)$	3	2	1	1	1	2	3	3	2	2	2

表 2-19 每个词的 $P(t|C)$

	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
$P(t\|C)$	0.136	0.091	0.045	0.045	0.045	0.091	0.136	0.136	0.091	0.091	0.091

表 2-20 每篇文档的 $\sum_t tf(t,d)$

	D_1	D_2	D_3
$\sum_t tf(t,d)$	7	8	7

表 2-21 每个词的 $tf(t,d)$

$tf(t,d)$	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	1	0	1	0	1	1	1	1	1	0	0
D_2	1	1	0	1	0	0	1	1	0	2	1
D_3	1	1	0	0	0	1	1	1	1	0	1

表 2-22 每个词的 $p_\mu(t|d)$

$p_\mu(t\|d)$	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_1	0.141	0.027	0.114	0.014	0.114	0.127	0.141	0.141	0.127	0.027	0.027
D_2	0.128	0.116	0.012	0.103	0.012	0.025	0.128	0.128	0.025	0.207	0.116
D_3	0.141	0.127	0.014	0.014	0.014	0.127	0.141	0.141	0.127	0.027	0.127

表 2-23 使用狄利克雷先验分布的最终相似度

文档	D_1	D_2	D_3
$SC(Q,D_i)$	0.000 011 4	0.000 259 0	0.000 172 8

因此，通过这种方式，文档相关性排序为 D_3,D_2,D_1。下面我们简要介绍一下另外两种平滑方法。Jelinek-Mercer 平滑方法是一种在整个文档集模型上的最大似然估计模型的线性插值方法，通过系数 λ 来控制每个模型的影响。

$$P_\lambda(t|d)=(1-\lambda)P_{ml}(t|d)+\lambda P(t|C)$$

54

其中 $P(t|C)$的含义与狄利克雷先验分布中使用的含义相同。对于参数 λ，对于不同的查询，我们选择不同的最优值。实验表明，对于较短的查询，λ 取较小的值（大约在 0.1 附近）效果很好；而对于长查询，λ 取较大的值（大约在 0.7 附近）效果很好。表 2-24 给出了本例中的最终相似值。

表 2-24 使用 Jelinek-Mercer 平滑方法的最终相似度

文档	D_1	D_2	D_3
$SC(Q,D_i)$	0.000 314	0.000 443	0.000 381

这里要注意的是，与我们所有的排序结果一致，文档 2 排在文档 3 前，文档 3 又排在文档 1 之前。绝对折扣方法通过在可观察到的词的个数中减掉一个常量来降低可观察到的词的概率。下面的公式表明如何引入折扣：

$$p_\delta(t \mid d) = \frac{\max((tf(t,d) - \delta), 0)}{\sum_w tf(t,d)} + \sigma P(t \mid C)$$

其中 $\delta \in [0,1]$，并且 $\sigma = \frac{\delta \mid d \mid_u}{\mid d \mid}$。$\mid d \mid_u$ 是文档 d 中不同词的数量，$\mid d \mid$ 是文档中所有词的数量。

与 Jelinek-Mercer 平滑方法的一个主要不同是：实验表明，δ 的最优值一直稳定在 0.7 附近。这
55 对于标题类查询和长查询都成立。在我们的例子中，使用绝对折扣的最终相似度见表 2-25。

表 2-25　使用绝对折扣平滑方法的最终相似度

文档	D_1	D_2	D_3
$SC(Q, D_i)$	0.001 215	0.021 716	0.005 727

我们简要介绍一下语言模型近期被用作一种检索模型的情况。尽管在语音识别中使用语言模型是一种古老的技术，但是在检索模型中使用语言模型却没有很长时间。我们的探讨大多基于 Ponte 和 Croft 的开创性工作，他们对词项的出现与否进行建模。我们也讨论另外一种多项式模型，它将查询看做从一元语言模型中得出的样本（即词的多项式分布）。在每种情况下，我们都会给出如何基于使用文档模型产生查询的概率给一篇文档打分。检索问题也就转化为估计一篇文档的语言模型。一种简单的方法是通过使用相对频率来估计。这种方法有一个明显的问题，就是文档中未出现的词项的概率就赋值为零。平滑方法就是为了避免这种情况，这可以提高语言模型的准确率。许多不同的平滑方法都是可行的。目前来说，性能较好的是狄利克雷先验分布。

使用这些基本语言模型的检索效率与向量空间模型这样的传统检索模型一样好[Zhai 和 Lafferty，2001b]，并且效果上也一样好，或者比调整好参数的传统模型效果更好。或许使用语言模型进行检索最主要的优势在于可以自动调整参数，详细说明可以参见文献[Zhai 和 Lafferty，2002]。

科研人员研究了更为复杂的语言模型，其中一些科研人员试图提升基本的语言模型的建模方法（即查询似然方法）：[Stong 和 Croft，1999]通过获取有限的依赖来超越一元语言模型进行建模，[Berger 和 Lafferty，1999]通过引入翻译模型来获取词之间的关联性。其他一些人则以不同的方式使用语言模型。例如，[Lavrenko 和 Croft，2001]将语言模型引入到经典的概率信息检
56 索模型中，他们没有使用相关性判断，而是使用了一种新的方法进行相关性建模。

最近的研究主要集中于消减模型的大小[Hiemstra 等人，2004]。近期的另外一种方法是通过文档聚类（见 3.2 节）代替整个文档集来构建语言模型[Liu 和 Croft，2004]。

还有一种方法的基本思想与伪相关反馈非常相近。在一些其他的研究中，文献[Zhai 和 Lafferty，2001b；Zhai 和 Lafferty，2001a]同时构造了查询模型和文档模型，并且对距离函数进行建模（即 KL 距离）来计算两个模型之间的距离。除了文档和查询是基于一元语言模型来表示外，这与向量空间模型非常相似。这些模型或者更为高级的模型的主要优点是它们可以更自然地进行相关反馈（见 3.1 节）。这些模型中大多数模型的性能优于基本的查询似然评分方法。更准确的、更强壮的语言模型仍然是一个非常活跃的研究方向。

2.4　推理网络

推理网络使用证据推理来估计一篇文档与一个查询的相关概率。研究人员使用 2.2 节中讨

论过的概率检索模型建模，并且引入了文档和查询相关的证据来优化该模型。在这一节中，我们首先简要概述推理网络，然后阐述使用推理网络进行相关性排序的具体方法。

2.4.1 相关背景

推理网络的本质是获取关联关系，并利用这些关系来推理其他关联关系。这就将大量的计算需求转化为估计一个事件发生的概率，大大地降低了计算量。

二值推理网络使用取值为 true 或者 false 的事件。先验概率表明事件发生的可能性。假设我们知道事件 A、B、C、D 和 E 发生的概率分别为 $P(A=\text{true})=a$，$P(B=\text{true})=b$，$P(C=\text{true})=c$，$P(D=\text{true})=d$，$P(E=\text{true})=e$。这些事件是独立的，也就是说，不管其他事件是否发生，这些事件发生的概率不会受其他事件的影响。较形式化的表示就是，对于 b、c、d、e 所有可能的组合，有 $P(A|b,c,d,e)=P(a)$。假定事件 F 依赖于事件 A、B、C、D 和 E，并且我们想要计算在事件 A、B、C、D、E 发生的情况下事件 F 的概率。图 2-6 给出了该例子的推理网络示意图。

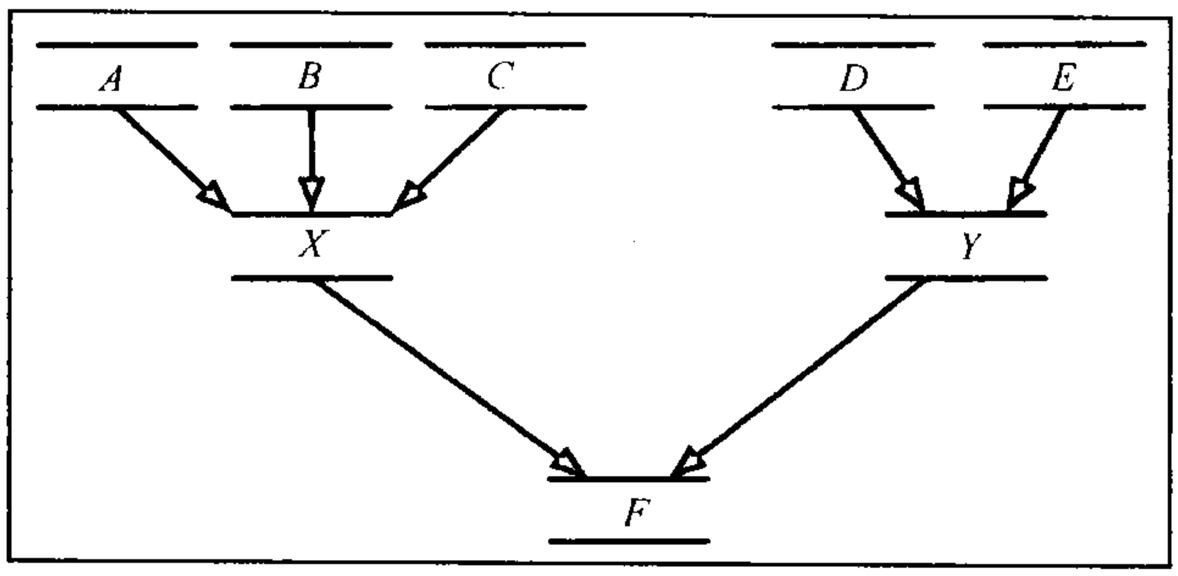


图 2-6 简单的推理网络

在不使用推理网络的情况下，计算量是指数级的，需要考虑事件 A、B、C、D 和 E 的所有可能，即 2^5 种组合。我们使用[Greiff，1996]中给出的符号体系，用 **R** 表示{1，2，3，4，5}所有 2^5 个子集的集合。用 p_i 表示第 i 个事件发生的概率，在我们的例子中，事件 1、2、3、4 和 5 对应 A、B、C、D 和 E。用 P_i 表示第 i 个事件的状态值（发生或不发生），也就是 P_1 表示事件 A 是否发生，P_2 表示事件 B 是否发生，以此类推。最后，事件 A、B、C、D 和 E 中一个事件或多个事件的组合直接影响事件 F 是 true 的概率。这个概率记作 α_R，定义如下： 57

$$\alpha_R = P(F=\text{true} \mid P_1,\cdots,P_5)$$

其中：$i \in R \rightarrow P_i$ 为 true，

$i \notin R \rightarrow P_i$ 为 false。

假定事件 A、B、C、D 和 E 是相互独立的，我们来计算 $P(F=\text{true})$的概率，公式如下：

$$P(F=\text{true}) = \sum_{R\subseteq\{1,\cdots,5\}} \alpha_R \prod_{i\in R} p_i \prod_{i\notin R}(1-p_i) \tag{2-9}$$

对于一个简单的五值问题，我们最终要进行 32 元计算。推理网络自然而然地引入中间变量来解决客观存在的指数级运算问题。它使用局部推理来获取最终结果。

假设我们知道事件 A，B 和 C 会导致事件 X 的发生，事件 D 和 E 会导致事件 Y 的发生。现在，我们可以使用事件 X 和 Y 推导出事件 F。图 2-6 举例说明了简单的推理网络。

下面我们来举一个例子：我们试图预测沙罗曼蛇队是否可以赢得一场垒球比赛。假设这依 58

赖于临场的教练和当值的裁判员。推理网络最顶层的节点对应于指定的教练或者裁判员是否出场。节点斯科特、戴维和凯萨琳（节点 A、B 和 C）对应于教练，节点吉姆和曼尼（节点 D 和 E）对应于裁判员。现在事件“优秀教练出场”（节点 X）依赖于节点斯科特、戴维和凯萨琳，事件“优秀裁判员出场”依赖于节点吉姆和曼尼。显然，事件“沙罗曼蛇队获胜”（节点 F）依赖于节点 X 和 Y。

在我们的例子中，一个裁判员出场不会影响到另外一个裁判员的出场，但这肯定会影响到裁判对于沙罗曼蛇队是否“友好”。类似地，指定的教练出场与否不会影响到其他教练的出场，但这确实会影响到整个教练组的出场。同样，教练的出场与否不会对裁判的出场造成影响。

为了计算 F，我们可以使用公式（2-9），也可以使用推理网络。推理网络可以充分利用事件 X 和 Y 内在的逻辑结构。首先，我们使用三个父节点——A、B 和 C——来计算 $P(X=\text{true})$，这需要 2^3 次计算。父节点对于子节点的影响因子用变量 α 来表示，其下标为二值变量组合，它表示在不同父节点的组合下子节点事件发生的可能性。例如，α_{111} 表示在三个父节点事件都发生的情况下子节点发生的概率。$P(X=\text{true})$计算为：

$$\begin{aligned}P(X=\text{true}) = {} & \alpha_{111}abc + \alpha_{110}ab(1-c) + \alpha_{101}a(1-b)c + \\ & \alpha_{100}a(1-b)(1-c) + \alpha_{011}(1-a)bc + \\ & \alpha_{010}(1-a)b(1-c) + \alpha_{001}(1-a)(1-b)c + \\ & \alpha_{000}(1-a)(1-b)(1-c)\end{aligned}$$

现在我们使用父节点 D 和 E 来计算 $P(Y=\text{true})$：

$$P(Y=\text{true}) = \alpha_{11}de + \alpha_{10}d(1-e) + \alpha_{01}(1-d)e + \alpha_{00}(1-d)(1-e)$$

一旦我们知道事件 X 和 Y 的先验概率，那么事件 F 发生的概率为：

$$P(F=\text{true}) = \alpha_{11}xy + \alpha_{10}x(1-y) + \alpha_{01}y(1-x) + \alpha_{00}(1-x)(1-y)$$

为了计算事件 F 发生的概率，我们对事件 X 进行了 8 次加法运算，而对事件 Y 进行了 4 次加法，最后对事件 F 做了 4 次加法运算。因此，我们只进行了 16 次加法运算，而不采用推理网络则需要进行 32 次计算。这里的关键在于：给定事件 X 和 Y，事件 F 独立于事件 A、B、C、D 和 E，即：

59

$$P(F=\text{true} \mid a, b, c, d, e, x, y) = P(F=\text{true} \mid x, y)$$

一旦给推理网络最顶层的节点赋予初始值（这也称为先验概率），网络中的该节点就被实例化，这个节点所链接的节点都可以顺延向下进行计算，直到网络底端。这些节点随之被激活。此时，我们可以计算这个激活节点是否发生的可信度。针对该节点可信度的计算，我们需要综合其所有父节点发生的概率。在这一点上，我们假设其父节点对于当前节点事件发生的影响是等概率的。在推理网络中，链接矩阵表明了父节点（通过它们自身，或者与其他父节点共同作用）对子节点影响度的大小。

2.4.2　链接矩阵

推理网络提供的另外一种功能是引入子节点依赖父节点的程度。假设我们知道有一个特别的裁判员——曼尼，他比其他的裁判员对沙曼蛇队友好。因此，节点 D 和 E 对节点 Y 的贡献可能就不同。

链接矩阵包含父节点的 2^n 种组合的表头（对于二值推理网络，每个节点取值仅为“真”或

“假”），并且只包含两行。

在这个例子中，节点 Y 的链接矩阵表示判断员 D 和 E 对 Y 的影响，如下表所示：

	DE	$D\overline{E}$	$\overline{D}E$	$\overline{D}\overline{E}$
Y true	0.9	0.8	0.2	0.05
Y false	0.1	0.2	0.8	0.95

从这个矩阵中我们可以看到，在友好的裁判员曼尼值场（D）且吉姆缺席（$\overline{E}$）的情况下，则比赛中出场的裁判员对我们友好的置信度为 80%。我们使用记号 $L_i(Y)$来表示链接矩阵中第 i 个条目的值，用来标识 Y 的值是否为真，而 $\overline{L_i(Y)}$ 用来决定 Y 是否为假。

在上面给出的链接矩阵中，所包含的条目在计算中作为额外的参数。一个给定带有 n 个父节点的节点 N，使用先前使用的集合 $\mathbf{R}$，其可信度的新的计算公式如下：

$$P(N=\text{true})=\sum_{\mathbf{R}\subseteq\{1,\cdots,n\}} L_i(N)\prod_{i\in\mathbf{R}} p_i\prod_{i\notin\mathbf{R}}(1-p_i)$$

p 个父节点的链接矩阵包含 2^p 个条目。再者，链接矩阵度量了每个父节点对推理的贡献度。我们可以选择链接矩阵，这样我们就可能采用闭合式来计算。对于任意节点 N 的矩阵 $\boldsymbol{L}_{\text{AND}}$（最简单的矩阵）见下表[Turtle, 1991]：

60

	P_{000}	P_{001}	P_{010}	P_{011}	P_{100}	P_{101}	P_{110}	P_{111}
N true	0	0	0	0	0	0	0	1

表中每一项以二值形式给出，给出的 1 或 0 表示 p 个父节点中每个节点是否存在（本例中 p 为 3）。除了当所有 p 个父节点都存在时，所有父节点的组合计算结果为零。$P(N$=true)的值为 $(p_1)(p_2)(p_3)$，其中 p_i 表示父节点 p_i 为真的概率。现在，我们仅需要计算单个元素就可以得到最终结果，而原来在最坏的情况下需要 2^N 次计算。其他闭合形式的链接矩阵基本上是求父节点先验概率的平均值。

为了在前文提到的推理网络的基础上给出一个链接矩阵的例子，我们假设 A 出场的概率为 70%，B 和 C 出场的概率分别为 60%（$P(A)=0.7$，$P(B)=0.6$，$P(C)=0.6$）。对于裁判员，假设 $P(D)=0.8$，$P(E)=0.4$。有了 A、B、C、D 和 E 的概率，接下来我们就可以计算 X 和 Y 为真的概率。为了计算 $P(X=\text{true})$，我们需要 X 的链接矩阵。现在，假定链接矩阵可以推导出所有父节点概率平均值的闭合式。

$$\frac{\sum_i^n p_i}{n}$$

$$P(X=\text{true})=\frac{0.7+0.6+0.6}{3}=0.633$$

现在我们计算 Y 的概率，上面给出的链接矩阵可以用来得到下式：

$$P(Y=\text{true})=L_{11}(Y)de+L_{10}(Y)d(1-e)+L_{01}(Y)(1-d)e+L_{00}(Y)(1-d)(1-e)$$

$$P(Y=\text{true})=0.9\times0.8\times0.4+0.8\times0.8\times0.6+0.2\times0.2\times0.4+0.05\times0.2\times0.6$$

$$P(Y=\text{true})=0.288+0.384+0.016+0.006=0.694$$

现在，我们计算出了 X 和 Y 为真的可信度，假定我们使用的是未加权求和的链接矩阵。这是一种闭合形式的链接矩阵，其计算节点 F 的可信度的方法是直接将所有父节点的概率求和取

61 平均。F的最终值如下：

$$P(F=\text{true})=\frac{0.694+0.633}{2}=0.6635$$

在该例中，我们总共只有 3 个父节点来计算 X，4 个父节点来计算 Y，两个父节点来计算 F。如果不使用闭合形式的链接矩阵，X 节点需要计算 2^3 次，Y 节点需要计算 2^2 次，F 节点需要计算 2^2 次，共计 16 次——远远小于不使用推理网络所需要的 2^5 次计算。

2.4.3 相关性排序

在 Turtle 的博士论文中，他创造性地提出了在信息检索中使用推理网络[Turtle，1991]。文档被表示为推理网络中的节点，每篇文档到文档中的每个词表示为链接。当一篇文档被实例化后，文档链接的所有词的节点也被实例化。一个简单的三层网络就将文档词节点直接链接到查询节点。图 2-7 展示了这种三层网络。对于查询中的每个词，词节点和查询节点间存在某个链接。这里要注意的是，这是用于信息检索的最简单的推理网络。三层推理网络由文档层、词项层和查询层组成。需要注意的是：只有在某一层中包含词的概括或概念，这种基本的算法才会起作用。这一层可以位于词项层和查询层中间。词项到概念的链接可以通过语义处理或者使用知识库生成（见 3.6 节和 3.7 节）。在推理网络中使用概念层，我们可以将推理网络适用于匹配问题，因为查询词不需要直接匹配文档中的词，只要概念匹配上就可以了。

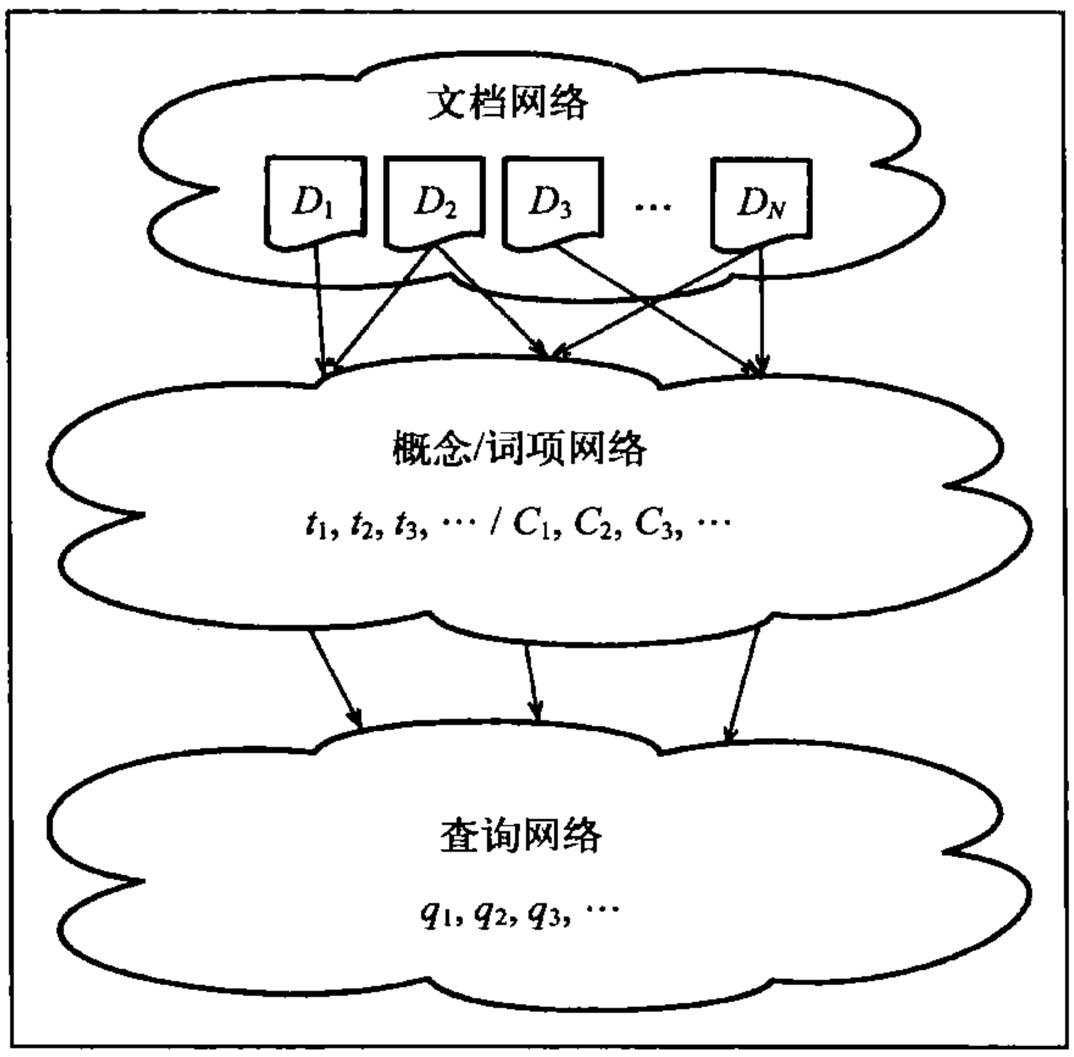


图 2-7　基于文档-词-查询的推理网络

图 2-8 给出了一个带有实际节点和链接的推理网络的例子。该推理网络给出了一个查询、三篇文档以及对应的网络，还有文档层到词项层的每个词的链接。

在我们的讨论中，我们主要关注三层推理网络，并且在文档被实例化后开始处理。为了能够对文档进行实例化，假设我们可以观察到文档 1（D_1），这就可以将文档 D_1 的所有节点实例化。我们一次只用一篇文档实例化网络。因此，该层的链接矩阵闭合式等同于文档中某个词是否在文档中出现。通常，会使用贴近于 *tf-idf* 的变量。

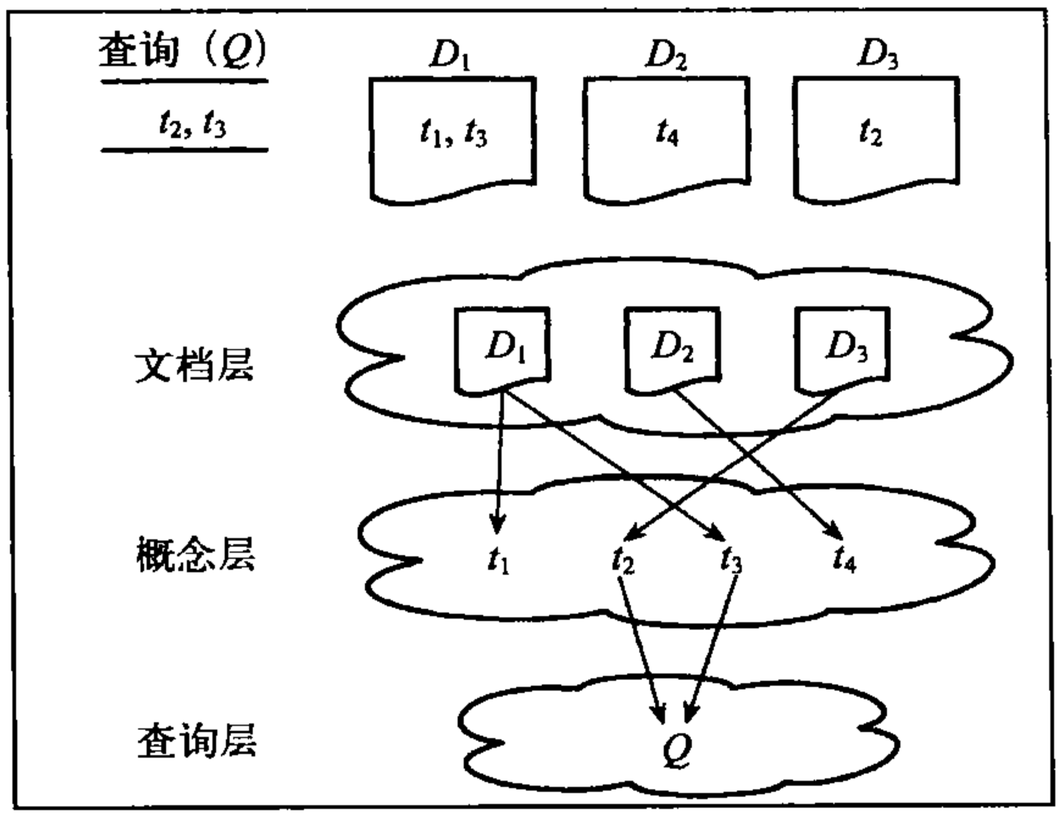


图 2-8 推理网络

接下来，处理继续直到遍及整个网络。所有从刚刚被激活的词节点发出的链接被实例化，接着某个查询节点被激活。我们可以计算出在查询已知文档 D_1 时查询节点的可信度。这个可信度就可以作为文档 D_1 的相似度。我们的处理直到所有文档被实例化后结束。 62

2.4.4 推理网络实例

现在我们使用推理网络来计算文档的相似度，测试文档集依然如下：

Q：“gold silver truck”

D_1：“Shipment of gold damaged in a fire.”

D_2：“Delivery of silver arrived in a silver truck.”

D_3：“Shipment of gold arrived in a truck.”

我们需要在已知文档 D_i 中的证据的情况下，评价查询的可信度。假设可信度与文档频率成正比，与文档集频率成反比，这使我们考虑使用词频 *tf* 和逆文档频率 *idf*。不管怎样，二者都要归一化到区间[0,1]，*tf* 通过除以文档的最大词频归一化，*idf* 通过除以最大的可能的 *idf*（见表 2-26）。

表 2-26 样例文档集的初始值

	a	arrived	damaged	delivery	Fire	gold	in	of	shipment	silver	truck
idf	0	0.41	1.10	1.10	1.10	0.41	0	0	0.41	0.41	0.41
nidf	0	0.37	1	1	1	0.37	0	0	0.37	0.37	0.37
D_1	1	0	1	0	1	1	1	1	1	0	0
D_2	0.5	0.5	0	0.5	0	0	0.5	0.5	0	1	0.5
D_3	1	1	0	0	0	1	1	1	1	0	1

在我们的三篇文档中，每个词都出现了 1 次、2 次或者 3 次。文档集的总大小为 3，所以： 63

$tf = 1$, $idf = \ln(3/1) = 1.10$

$tf = 2$, $idf = \ln(3/2) = 0.41$

$tf = 3$, $idf = \ln(3/3) = 0$

除了词项“silver”在文档 D_2 中出现了两次外，其他每个词在文档中或者出现一次或者不出现。对于词和文档的每一种组合，我们计算 $P_{ij}=P(r_i=\text{true}|d_j=\text{true})$。Turtle 使用公式 $P_{ij}=0.5+0.5$

64 $(ntf_{ij})(nidf_i)$来计算特定文档中给定词的可信度。实例化一篇文档规定对指定的词节点的所有成员的支持度相等。任何没有支持的节点（没有文档实例化）的可信度为0。

我们构造的链接矩阵的每一词项都描述了其父节点的支持度。对于查询中的概念，给出了如下的链接矩阵。

“gold”的链接矩阵为：

	$\overline{D_1 D_3}$	$\overline{D_1} D_3$	$D_1 \overline{D_3}$
false	1	0.315	0.315
true	0	0.685	0.685

该矩阵标明在每一列描述的条件下取假值的可信度（第一行）和取真值的可信度（第二行）。当实例化一篇文档时，我们把它看做真值。每一次只能实例化一篇文档。表中的数字表示 P_{ij} 的值，或者在已知文档 j 被实例化后，词 i 为真的可信度。

如果给文档 D_3 赋值为真，那么可信度可以计算为：

$$P_{ij} = 0.5 + 0.5(ntf_{ij})(nidf_i) = 0.5 + 0.5 \times 0.369 \times 1 = 0.685$$

这是在文档 D_3 为真的情况下从链接矩阵中发现的。在这种情况下，链接矩阵具有闭合形式。因此，它不需要事先存储或计算。该矩阵只需要考虑三种可能性：两个文档都为假，D_1 为真而 D_3 为假，D_3 为真而 D_1 为假。由于“gold”没有在文档2中出现，所以就没有必要考虑 D_2 为真时的可信度，这是因为从 D_2 到表示“gold”的节点并没有链接。同样，由于每次只能给一篇文档赋值为真，所以也就没有必要考虑 D_1 和 D_3 同时为真的情况。

类似地，可以构造“silver”的链接矩阵。“silver”只在文档 D_2 中出现过，所以唯一需要考虑的情况是 D_2 是否赋值为真。链接矩阵计算如下：

$$P_{ij} = 0.5 + 0.5(ntf_{ij})(nidf_i) = 0.5 + 0.5 \times 0.369 \times 1 = 0.685$$

类似地，可以构造“truck”的链接矩阵。“truck”的两个父节点为 D_2 和 D_3。

	$\overline{D_2 D_3}$	$\overline{D_2} D_3$	$D_2 \overline{D_3}$
false	1	0.315	0.408
true	0	0.685	0.592

65 当 D_2 取值为真、D_3 取值为假时，我们有

$$P_{ij} = 0.5 + 0.5(ntf_{ij})(nidf_i) = 0.5 + 0.5 \times 0.5 \times 0.369 = 0.592$$

现在我们给出了所有用来计算一个词在已知一篇文档实例下可信度的链接矩阵。下面我们就要构造一个查询节点的链接矩阵。

选择这个矩阵确实比较自由。用户接口可以允许用户选择一些比其他的更重要的词语。如果是这种情况，查询节点的链接矩阵可以相应地权重化。在 Turtle 的论文[Turtle，1991]中，给出了一个简单的矩阵，这个矩阵使用 g、s 和 t 来表示词项“gold”、“silver”和“truck”，其形式如下：

	$\overline{gst}$	g	s	gs	t	gt	st	gst
false	0.9	0.7	0.7	0.5	0.5	0.3	0.3	0.1
true	0.1	0.3	0.3	0.5	0.5	0.7	0.7	0.9

这个矩阵的理论基础是“gold”和“silver”在值上是相等的，而“truck”更重要一些。同

样，甚至在没有词出现的情况下，文档相关的可信度为较小的值（0.1）。类似地，如果所有的词项都出现了，仍有一些疑惑（0.1）。最后，我们为多个词项的出现引入可信度。

现在我们实例化 D_1，也就是 Bel(gold)为 0.685——在已知文档 D_1 中，“gold”为真的可信度——Bel(truck)=0.5，Bel(silver)=0.5。注意，在本例中“Bel(x)”表示“x的可信度”。

在这一点上，所有的词节点都被实例化了，所以查询节点现在也可以实例化了。

$$\begin{aligned}\text{Bel}(Q \mid D_1) &= 0.1\times0.315\times1\times1+0.3\times0.685\times1\times1+0.3\times0.315\times0\times1+\ 0.5\times0.685\times0\times1\\ &\quad +0.5\times0.315\times1\times0+0.7\times0.685\times1\times0+0.7\times0.315\times0\times0+0.9\times0.685\times0\times0\\ &= 0.031+0.206=0.237\end{aligned}$$

我们可以使用链接矩阵项 $L_i(Q)$直接从前面例子所给出的公式推导出来。

实例化 D_2，使得 Bel(gold) = 0，Bel(silver) = 0.685，Bel(truck) = 0.592。D_2的可信度为：

$$\begin{aligned}\text{Bel}(Q \mid D_2) &= 0.1\times1\times0.315\times0.408+0.3\times0\times0.315\times0.408+0.3\times1\times0.685\times0.408\\ &\quad +0.5\times0\times0.685\times0.408+0.5\times1\times0.315\times0.592+0.7\times0\times0.315\times0.592\\ &\quad +0.7\times1\times0.685\times0.592+0.9\times0\times0.685\times0.592\\ &= 0.013+0.084+0.093+0.283=0.473\end{aligned}$$

66

将 D_3赋值为真使得 Bel(gold) = 0.685，Bel(silver)=0，Bel(truck) = 0.685。D_3的可信度如下：

$$\begin{aligned}\text{Bel}(Q \mid D_3) &= 0.1\times0.315\times1\times0.315+0.3\times0.685\times1\times0.315+0.3\times0.315\times0\times0.315\\ &\quad +0.5\times0.685\times0\times0.315+0.5\times0.315\times1\times0.685+0.7\times0.685\times1\times1.68\\ &\quad +0.7\times0.315\times0\times0.685+0.9\times0.685\times0\times0.685\\ &= 0.01+0.065+0.108+0.328=0.511\end{aligned}$$

对于这个例子中的所有链接矩阵，我们假定每个父节点对子节点贡献的概率是相等的。也就是如果存在两个父节点，不管哪个父节点，子节点的概率更大一些。最近的研究表明在每个父节点是否出现两种情况下产生链接矩阵的闭合形式的概率是不同的[Greiff 等人，1997]。关于推理网络在相关性排序的拓扑，目前只是用到了皮毛而已。在推理网络中对普遍的子文档进行分组，或者对文档集分组，或者在推理网络中聚类都是有可能的。同样地，相同的文档可以使用不同表现形式。据我们所知，这个领域的研究还很少。

2.5 扩展布尔检索

传统的布尔检索本身对相关性排序并没有提供很好的支持，这是因为文档要么符合布尔查询请求，要么不满足布尔查询请求。我们可能检索到所有符合查询请求的文档（通过按时间顺序排序），但是并没有计算这些文档与查询的相关性的大小。

文献[Fox，1983]给出了布尔检索的一种可以进行相关性排序的扩展方法，并且这种方法的基础在文献[Salton，1989]中给出了完整的描述。其基本思想为对查询和文档中的每一个词都赋予权重。与简单地查询匹配的词项集合不同，词项权重可以用来进行文档相关性排序。下面我们考虑一个请求(t_1 OR t_2)的查询，在这个查询中，包含 t_1的文档被赋于权重 w_1，包含 t_2的文档被赋予权重 w_2。

如果 w_1和 w_2都等于 1，那么包含这些词的文档很可能会被排在列表的最前面，而不包含这些词的的文档将会排在列表最后面。计算相似度的一种简单方法是计算(w_1,w_2)到原点的欧氏距离。因此，对于包含词 t_1和 t_2（权重分别为 w_1和 w_2）的文档，相似度为：

67

$$SC(Q,d_i)=\sqrt{(w_1)^2+(w_2)^2}$$

如果 w_1 和 w_2 分别为 0.5 和 0.5,，那么 SC 的值就是：

$$SC(Q,d_i)=\sqrt{(0.5)^2+(0.5)^2}=\sqrt{0.5}=0.707$$

只有在 w_1 和 w_2 分别为 1 时，SC 的值最大。在这种情况下，我们计算 $SC(Q,d_i)=\sqrt{2}=1.414$。如果我们想调整相似度的范围区间为 0 到 1，那么可以加入归一化系数 $\sqrt{2}$，这样相似度变为：

$$SC(Q_{t_1\vee t_2},d_i)=\frac{\sqrt{(w_1)^2+(w_2)^2}}{\sqrt{2}}$$

上述公式中假设我们提交的查询中默认包含布尔“或”运算：（$t_1\vee t_2$）。当然，我们可以直接推导出布尔“与”运算的计算公式。我们计算到点(1,1)的距离，而不是到原点的距离。越是接近点(1,1)的查询，越有可能符合“与”操作。其形式化的定义如下：

$$SC(Q_{t_1\wedge t_2},d_i)=1-\frac{\sqrt{(1-w_1)^2+(1-w_2)^2}}{\sqrt{2}}$$

2.5.1　引入查询权重

我们再来深入思考一下包含查询词 t_1 和 t_2（权重分别为 w_1 和 w_2）的文档。前面我们只是简单地假定布尔查询形式为(t_1 OR t_2)或者(t_1 AND t_2)。现在我们对查询赋予权重 q_1 和 q_2。加入权重的相似度计算公式如下：

$$SC(Q_{q_1\vee q_2},d_i)=\frac{\sqrt{(q_1^2w_1)^2+(q_2^2w_2)^2}}{\sqrt{q_1^2+q_2^2}}$$

$$SC(Q_{q_1\wedge q_2},d_i)=1-\frac{\sqrt{q_1^2(1-w_1)^2+q_2^2(1-w_2)^2}}{\sqrt{q_1^2+q_2^2}}$$

2.5.2　扩展为任意数量的查询词

对于二维的欧氏距离，我们使用 2 范数。为了在多维空间中计算到原点的距离，我们使用
68 L_p 向量范数。参数 p 根据评价相关度权重的重要性而不同。对于由权重为 q_1 的词 t_1 和权重为 q_2 的词 t_2 组成的查询 Q，由权重分别为 w_i 和 w_j 的同样的词组成的文档 D_i，新的相似度定义如下：

$$SC\left(D,Q_{(q_i\vee q_j)}\right)=\left[\frac{q_i^pw_i^p+q_j^pw_j^p}{q_i^p+q_j^p}\right]^{\frac{1}{p}}$$

$$sim\left(D,Q_{(q_i\wedge q_i)}\right)=1-\left[\frac{q_i^p(1-w_i^p)+q_j^p(1-w_j^p)}{q_i^p+q_j^p}\right]^{\frac{1}{p}}$$

当 p 等于 1 时，公式就退化为向量的点积。当 p 的值趋于无穷大时，这就转化为一个不带权重的普通的布尔查询系统。最初的测试发现布尔查询优于向量空间模型（即 p=2 时），但是这些测试只是在较小的数据集上进行的，并且比向量空间模型的计算更复杂。

2.5.3 自动插入布尔逻辑

我们能够处理的检索模型都不需要用户书写复杂的布尔表达式。因此，当使用“或”运算时，可以使用仅包含词的查询。查询系统可以自动赋予权重（使用类似 *tf-idf* 的方法），并且文档可以通过在每个词中间插入“或”操作进行排序。传统的向量空间模型在计算排序列表时，会在文档的每个词中隐含地包含“或”操作。至少包含一个查询词的任何文档的排序得分都大于 0。

相反，更为高级的算法可以在一系列词中间自动加入“与”操作和“或”操作[Fox，1983a]。基于最坏情况下文档频率的求和，这种算法估计了检索集的大小。如果词 t_1 出现在 50 篇文档中，词 t_2 出现在 100 篇文档中，那么估计整个查询结果将返回 150 篇文档。这种估计仅在 t_1 和 t_2 不同时出现在一篇文档中成立。

使用最坏情况下的求和，查询中的词通过文档频率排序。频率最高的词将放在“停用词”表。我们仅将频率最高的两个词放入“停用词”表。接下来我们将“停用词”表中的词合并成两个一组，查询中只加入每一组词中检索结果集最小的词，这种处理直到检索集的大小低于查询阈值。

69

2.6 LSI

在称作 LSI 的检索模型中，矩阵运算是其基础[Deerwester 等人，1990]。前提是由于越来越多的传统检索模型（即向量空间模型、概率模型和扩展的布尔模型）直接匹配关键词，所以这些模型都有问题。由于同一个概念可以用多个不同的关键词来描述，实践证明这种匹配是失败的。作者引用了一个研究成果，该研究认为：两个人使用相同的关键词表述同一概念的几率只有 20%。

查找与文档中表达的潜在语义相近的内容并不是新提出的目标。早在 20 世纪 70 年代早期，人们就提出了使用正则形式进行自然语言处理[Winograd，1983；Schank，1975]。在这里进行应用的基本思想并不是想要找到一种规范的知识表达形式，而是进行矩阵运算，特别是奇异值分解。这样我们就可以过滤出文档中的噪音，以便两篇语义上相同的文档（不管它们是否有匹配词）在另外一个多维空间上映射得近一些。

这个处理过程还是比较简单的。我们构建词项–文档矩阵 $\boldsymbol{A}$，其中 location(i,j)表示词 i 在文档 j 中出现的次数。这个矩阵奇异值分解的结果是矩阵 $\boldsymbol{U}\sum\boldsymbol{V}^{\mathrm{T}}$，其中 $\sum$ 是一个对角矩阵。矩阵 $\boldsymbol{A}$ 每一行中的元素表示词项，每一列中的元素表示文档。$\sum$ 中的值被称作奇异值。然后可以通过奇异值的大小进行排序，选择其中前 k 个元素作为矩阵 $\boldsymbol{A}$ 的“隐性语义”向量，其余的奇异值赋值为 0。我们仅将前 k 列保存在 $\boldsymbol{U}_k$ 中，将前 k 行记录保存在 $\boldsymbol{V}_k^{\mathrm{T}}$ 中。将其他的结果赋值为 0 后，我们得到一个新的矩阵 $\boldsymbol{A}'$ 来近似计算 $\boldsymbol{A}=\boldsymbol{U}\sum\boldsymbol{V}^{\mathrm{T}}$。

我们通过计算 $\boldsymbol{U}_k$ 中两个对应行的内积来比较两个词项的大小，通过计算 $\boldsymbol{V}_k^{\mathrm{T}}$ 中对应两列的内积大小来比较两篇文档的大小。

在查询–文档相似度计算上，我们将查询视为一篇文档，并且计算其奇异值分解。然而，奇异值分解的计算量相当庞大，所以不建议这样做。现在有方法可以近似地计算 $\sum$，并且避免了计算奇异值分解的开销。对于一个不经常更新的文档集，实际上经常定时计算奇异值分解。

2.6.1　LSI举例

70

为了给出隐性语义索引的示例，我们再次使用先前使用的查询和文档集的样例：

Q：“gold silver truck”

D_1：“Shipment of gold damaged in a fire.”

D_2：“Delivery of silver arrived in a silver truck.”

D_3：“Shipment of gold arrived in a truck.”

在下面给出的在词项-文档表中，我们可以从数字列中得到矩阵 A。

	D_1	D_2	D_3
a	1	1	1
arrived	0	1	1
damaged	1	0	0
delivery	0	1	0
fire	1	0	0
gold	1	0	1
in	1	1	1
of	1	1	1
shipment	1	0	1
silver	0	2	0
truck	0	1	1

下一步是对矩阵 A 进行奇异值分解，也就是 A 作为表达式 $U\sum V^{\mathrm{T}}$ 乘积的结果。在这个例子中，A 等于下面的矩阵的乘积：

$$\begin{bmatrix} -0.4201 & 0.0748 & -0.0460 \\ -0.2995 & -0.2001 & 0.4078 \\ -0.1206 & 0.2749 & -0.4538 \\ -0.1576 & -0.3046 & -0.2006 \\ -0.1206 & 0.2749 & -0.4538 \\ -0.2626 & 0.3794 & 0.1547 \\ -0.4201 & 0.0748 & -0.0460 \\ -0.4201 & 0.0748 & -0.0460 \\ -0.2626 & 0.3794 & 0.1547 \\ -0.3151 & -0.6093 & -0.4013 \\ -0.2995 & -0.2001 & 0.4078 \end{bmatrix} \begin{bmatrix} 4.0989 & 0 & 0 \\ 0 & 2.3616 & 0 \\ 0 & 0 & 1.2737 \end{bmatrix} \begin{bmatrix} -0.4945 & -0.6458 & -0.5817 \\ 0.6492 & -0.7194 & -0.2469 \\ -0.5780 & -0.2556 & 0.7750 \end{bmatrix}$$

然而，我们并不打算准确地找到矩阵 A。实际上，我们需要的仅仅是找到矩阵 A 排名最高的 k 个近似结果。我们只是想要最大的 k 个奇异值（$k<3$）。k 值的选择和 $\sum$ 中奇异值的数目是有些随意的。在这个例子中，我们选择 k=2。现在，我们有 $A_2=U_2\sum_2 V_2^{\mathrm{T}}$。基本上，我们只需

71

要使用 U 的前两列以及 $\sum$ 和 V^{T} 的前两列。

新的乘积为：

$$\begin{bmatrix} -0.4201 & 0.0748 \\ -0.2995 & -0.2001 \\ -0.1206 & 0.2749 \\ -0.1576 & -0.3046 \\ -0.1206 & 0.2749 \\ -0.2626 & 0.3794 \\ -0.4201 & 0.0748 \\ -0.4201 & 0.0748 \\ -0.2626 & 0.3794 \\ -0.3151 & -0.6093 \\ -0.2995 & -0.2001 \end{bmatrix} \begin{bmatrix} 4.0989 & 0 \\ 0 & 2.3616 \end{bmatrix} \begin{bmatrix} -0.4945 & -0.6458 & -0.5817 \\ 0.6492 & -0.7194 & -0.2469 \end{bmatrix}$$

为了得到一个 $k \times 1$ 维数组，现在引入查询。查询向量 q^T 的构建使用与起初的矩阵 A 相同的构建方法。现在通过变换 $q^T U_2 \sum_2^{-1}$ 将查询向量映射到一个二维空间。

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}^T \begin{bmatrix} -0.4201 & 0.0748 \\ -0.2995 & -0.2001 \\ -0.1206 & 0.2749 \\ -0.1576 & -0.3046 \\ -0.1206 & 0.2749 \\ -0.2626 & 0.3794 \\ -0.4201 & 0.0748 \\ -0.4201 & 0.0748 \\ -0.2626 & 0.3794 \\ -0.3151 & -0.6093 \\ -0.2995 & -0.2001 \end{bmatrix} \begin{bmatrix} 0.2440 & 0 \\ 0 & 0.4234 \end{bmatrix} = \begin{bmatrix} -0.2140 & -0.1821 \end{bmatrix}$$

我们可以使用相同的变换来将我们的文档向量映射到二维空间，但是 $\mathbf{V}_2$ 的行中包含文档的坐标。因此：

$$D_1 = (-0.4945 \quad -0.0688)$$
$$D_2 = (-0.6458 \quad 0.9417)$$
$$D_3 = (-0.5817 \quad 1.2976)$$

最后，我们已经准备好使用余弦系数来计算相关度，如下所示：

72

$$D_1 = \frac{(-0.2140)(-0.4945) + (-0.1821)(0.6492)}{\sqrt{(-0.2140)^2 + (-0.1821)^2}\sqrt{(-0.4945)^2 + (0.6492)^2}} = -0.0541$$

$$D_2 = \frac{(-0.2140)(-0.6458) + (-0.1821)(-0.7194)}{\sqrt{(-0.2140)^2 + (-0.1821)^2}\sqrt{(-0.6458)^2 + (-0.7194)^2}} = 0.9910$$

$$D_3 = \frac{(-0.2140)(-0.5817) + (-0.1821)(-0.2469)}{\sqrt{(-0.2140)^2 + (-0.1821)^2}\sqrt{(-0.5817)^2 + (-0.2469)^2}} = 0.9543$$

2.6.2　选择较优的 k 值

k 的值表示经过奇异值分解后保留的列的数量，而且是通过实验确定的。研究人员使用仅包含 1 033 篇文档和 30 个查询的 MED 数据库，在 9 种不同召回率水平上，根据不同的 k 值绘制出平均准确率。从 20 开始，准确率显著增加到 100 左右，接下来开始趋于平稳。

2.6.3　与其他检索模型比较

研究人员对因子为 100 的 LSI、基本的 *tf-idf* 向量空间模型、扩展的布尔检索模型进行了对比。在 MED 文档集上，LSI 的准确率比其他两种模型高出 30%。对于科学文摘的 CISI 文档集，LSI 的准确率并不高。经审查，作者发现 LSI 的词项选择和 *tf-idf* 实验是非常不同的。LSI 方法并不使用词干还原或者停用词表。当两种方法都使用相同的词项时，LSI 与 *tf-idf* 模型旗鼓相当。文献[Dumais，1994]在 TIPSTER 文档集上做了更多关于 LSI 的研究。这项研究表明：LSI 比传统的向量空间模型的性能要稍微好一些，其平均准确率为 0.24，而向量空间模型的平均准确率为 0.22。

2.6.4　可能的扩展

LSI 相对简单，文献中并没有描述太多的扩展。LSI 的重点是文档的语义表达，许多查询词项可以描述一篇相关文档，但并不一定会在文档中出现，LSI 对这种现象更具有适应性。

2.6.5　运行时性能

LSI 方法的运行时性能确实引起了人们的严重关切。使用向量空间模型或者概率检索模型时，我们使用倒排索引来快速计算相似度，不需要去查找文档集中的每一篇文档（除非查询词项在每篇文档中都出现了）。使用 LSI 模型，由于查询只表示另外一篇文档，我们不可能使用倒
73 排索引，因此必须与所有文档进行比较。

奇异值分解本身的计算量也相当复杂。我们注意到研究人员提出了许多专门用来计算奇异值分解值的并行算法，可以参考文献[Berry，1992]。对于由 N 篇文档组成的文档集，秩为 k 的奇异值矩阵 $\sum$，算法的时间复杂度为 $O(N^2K^3)$。在文献[Letsche 和 Berry，1997]中给出了使用 LSI 进行信息检索的许多并行算法实现的详细情况。

2.7　神经网络

神经网络由节点和链接组成。从本质上讲，节点由输出值和输入值组成。当节点被激活后，输出值沿着链接传递到其他节点。沿着链接传递的值等于发送节点的输出值乘以链接权重，因此链接是有权重的。一个节点的输入值是所有输入链接的权重之和。神经网络是按照层次结构构建的，这样网络接收到的数据是分阶段被激活的，在某个阶段，该层的数据传递给下一个层。算法基于训练数据去学习，对应不同的训练数据修改链接的权重。最早使用神经网络实现信息检索的工作，可参见文献[Belew，1989]。这项工作现在仅仅作为参考文献所引用，但是它所阐述的基本层次方法一直为后来者所采纳。

对于 ad hoc 的查询检索，我们用神经网络方法来实现向量空间检索和概率检索。此外，我

们还可以使用神经网络来实现相关反馈。

首先，使用神经网络方法实现向量空间检索还未能引起大家广泛关注。正如我们在 2.1 节所讨论的，向量空间模型完全可以不使用神经网络而单独实现搜索。然而，神经网络提供了一种学习能力，在这种模式下，我们可以基于相关信息对神经网络进行调整。从这个角度来说，在为终端用户服务会话的过程中，神经网络可用来适应或者学习用户的行为偏好。

2.7.1 节阐述了如何使用神经网络方法实现向量空间模型。2.7.2 节介绍了如何实现相关反馈。2.7.3 节介绍了如何使用学习算法来训练信息检索所用到的神经网络。2.7.4 节介绍了如何实现概率模型。2.7.5 节介绍了在神经网络中怎样使用不同粒度的词项。2.7.6 节使用了从向量空间模型和概率模型中派生的权重。

2.7.1 向量空间

为了使用神经网络来实现向量空间模型，我们构建了一个包括三类节点的网络：QUERY（查询层）、TERM（词项层）和 DOCUMENT（文档层）[Crouch 等人，1994]。节点之间的链接 74
分为查询–词项链接和文档–词项链接。查询和词项之间的链接表示该词在查询中出现过。该链接的权重是该词的 *tf-idf*。文档和词之间的链接表示该词在给定文档中出现过，同样，链接权重可用 *tf-idf* 来表示。

前馈网络通过激活给定的节点来工作。当一个节点的输出值大于某个阈值时，该节点就被激活。一开始，通过设置一个查询节点的输出值为 1 而激活该节点。该节点的所有链接被激活并且与之相连的 TERM 节点会得到新的输入权重。该条链接发送一个(*tf*)(*idf*)*1 的值，因为它传送的是该链接的权重和发送节点的输出值（在这里就是 1）的乘积。在这种情况下，TERM 节点接收链接的权重为 *tf-idf*。接收节点的权重计算方式为所有输入链接的和。对于一个只包含一个激活查询的词节点，只有一条链接将被激活。该 TERM 节点的输出值为 *tf-idf* 的权重。在下一个阶段，TERM 节点将被激活并且所有连接到文档节点的链接都将被激活。DOCUMENT 节点包括所有链接到文档中每个词的权重的和。对于一个有 t 个词的文档集，文档 j 对应的 DOCUMENT 节点的权重如下：

$$\mathrm{DOC}_j = \sum_{i=1}^{t} (tf_{ij})(idf_j)$$

DOCUMENT 节点现在有了与之关联的权重，该权重是度量文档与给定查询的相关度。我们可以将这个权重简单地等同于 2.1 节中介绍的相似度系数——直接求点积。

2.7.2 相关反馈

为了实现相关反馈，网络中将加入一组新链接。这组新链接从 DOCUMENT 节点反向指向 TERM 节点。在初始检索之后，文档–词项链接已经激活了。图 2-9 阐明了一个样例查询和三个文档的训练过程。以新方式定义的 TERM 节点增加了输入链接，输入添加在 TERM 节点已存在的对应的查询位置上。如果没有相关反馈，神经网络分为两个阶段。第一个阶段是 QUERY 节点发送信息给 TERM 节点。第二个阶段是 TERM 节点发送信息给 DOCUMENT 节点。

如果使用相关反馈，神经网络还要接着前两个阶段继续处理。第三个阶段是对于用户认为相关的 DOCUMENT 节点把信息反向发送到 TERM 节点。文档是否相关是人工标注的，或者是认为前 n 篇文档是相关的。最后，在第四个阶段，如果 TERM 节点超过一定的阈值参数，它将 75

被激活。我们使用 TERM-DOCUMENT 链接来发送在对 DOCUMENT 节点进行相关反馈阶段新得到的权重。此时，我们重新计算 DOCUMENT 节点的得分，这个得分反映了一次相关反馈迭代后的结果。

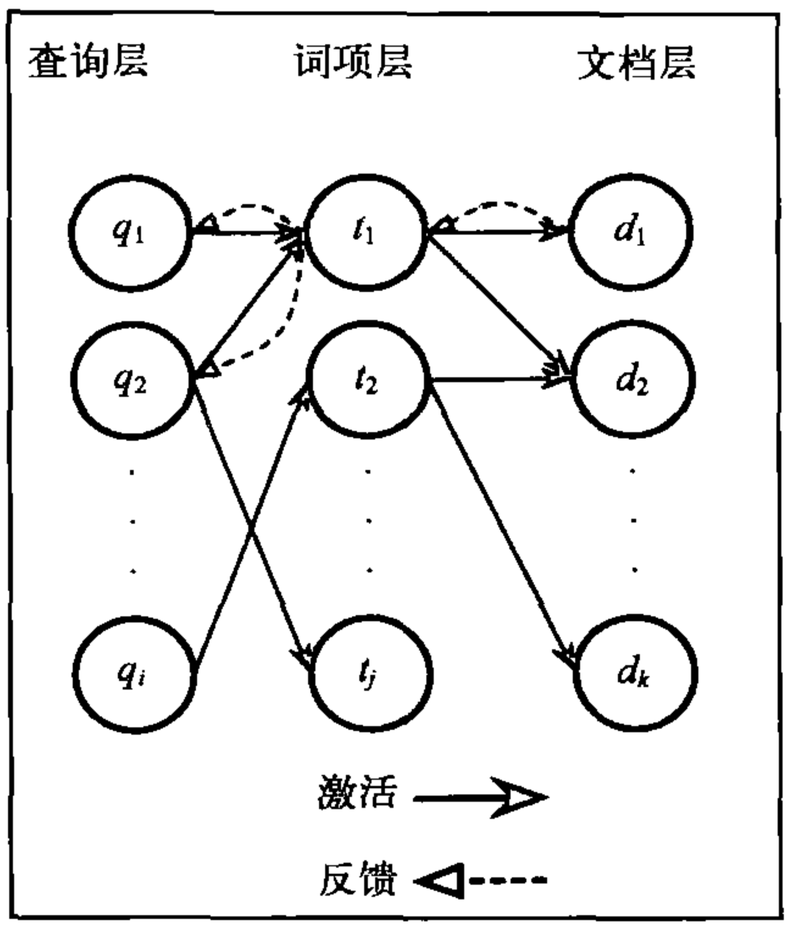


图 2-9　带反馈的神经网络

在 MEDLARS 和 CACM 数据集上的最初实验结果显示：对于 MEDLARS，平均准确率提高了 15%，而对于 CACM，平均准确率降低了 11%。正如前文所提到的，这些数据集都非常小。使用其他不考虑在相关反馈之前检索到的文档的评价方法，在 CACM 上的平均准确率到达了 21%，而在 MEDLARS 上的平均准确率高达 60%。

2.7.3　学习与调整

我们可以调整词和文档之间链接的权重，这样后来的查询就可以充分利用相关性信息。一般的向量空间相关反馈使用相关信息来调整查询的得分。在这种算法中，随后的查询被看做一个新事件，不使用之前得到的任何相关评价信息。

为了在随后的查询中使用相关性信息，文档节点增加了一个称作学习标志的新标记。如果用户判断该文档是相关的，就把学习标志赋值为 1；如果用户没有判断，就将其赋值为 0；如果

76 用户判断该文档为不相关的，就将其赋值为−1。然后，我们基于用户评价和现有文档的权重的差值对词项−文档链接进行调整。如果用户判断权重很高的文档为相关文档，不会引起神经网络太大的改变。例如：一个权重为 0.95 的文档的 δ 值为 1−0.95＝0.05，这将引起每一个词项−文档链接的权重增大 5%。如果用户判断权重很低的文档为相关文档，将导致神经网络作出较大调整。

引入权重学习的算法的结果与简单的相关反馈并没有什么本质的差别，但是这种利用相关反馈的潜能使其区别于传统的向量空间相关性排序。

2.7.4　概率检索

基于神经网络的标准概率检索模型可以参见文献[Crestani，1994]。在 2.2.1 节，我们详细介绍过 Sparck Jones 给出的标准词项权重。这种权重计算方式如下：

$$\lg \frac{r_i(N-n_i-R+r_i)}{(R-r_i)(n_i-r_i)}$$

式中 N——文档总数；

R——相关文档数；

r_i——包含词 i 的相关文档数；

n_i——包含词 i 的文档数（相关或不相关）。

这个权重为一个词在相关文档中出现的频率与它出现在整个文档集中出现的频率的比值。在整个文档集中出现频率不高，而在相关文档中出现频率很高的词就会被赋予很高的权重。我们使用这些权重作为查询-词项链接和词项-文档链接的权重，并且从本质上取代在向量空间模型中的权重——*tf-idf*。链接的求和操作与先前相同，这样就会求出一个跟标准概率检索模型算出来的权重非常相似的值。

在训练数据上，我们使用标准的反向传播学习算法重新计算权重。一旦训练完毕，我们使用神经网络计算排在前 10 位的词项，并修改包含这些词项的查询。

在 Cranfield 数据集上，基于神经网络的算法比现有的一种概率相关反馈系统——News Retrieval Tool 效果一直要差[Sanderson 和 Rijsbergen，1991]。作者解释说一方面是由于训练数据不足，另一方面作者也注意到，神经网络中的大量链接使得网络变得庞大，并且消耗很大的计算资源。

77

邮

电

2.7.5 基于片段的概率检索

文献[Kwok，1989]提出了使用神经网络的基于不同片段的检索模型。网络仍然是三层结构，只是权重不同。我们将查询 a 的查询-词项链接的权重赋值为 $w_{ka}=q_{ak}/L_q$，其中 q_{ak} 表示在查询 a 中词 k 出现的频率。将文档 i 的文档-词项链接的权重赋值为 $w_{ki}=d_{ik}/L_i$，其中 d_{ik} 表示在文档 i 中词 k 出现的频率。L_q 表示查询中词的个数，L_i 表示文档 i 中词的个数。将词项-查询链接的权重赋值为 w_{ak}，文档-词项链接的权重赋值为 w_{ik}。以查询为中心的方法是通过激活文档节点并向前传播到查询实现的。以文档为中心的方法是通过激活查询节点并向前传播到到文档实现的。

Kwok 使用神经网络扩展了他最初的工作[Kwok，1995]。基本的方法就是使用一种新的学习算法。这种新的学习算法使得基于训练数据集改变神经网络内部的权重成为可能。文献中给出了根据相关反馈给查询添加新词的学习算法。其他的算法不需要增加任何查询词，而只是修改权重。这些算法在规模超过 300MB 的 TIPSTER 数据集的 Wall Street Journal 部分上测试过。Kwok 继续给出了没有训练数据集的学习算法、基于相关信息的训练和查询扩展。

没有训练集，我们可以将一些初始的估计参数赋值为一个常数（参见 2.2.4 节）。使用文档-词项链接和词项-文档链接的相关信息的训练过程在 2.7.2 节已介绍过。

2.7.6 联合权重

文献[Boughanem 和 Soule-Depuy，1997]使用了一种类似“输入-词项-文档-输出”这种层次结构的神经网络。据我们所知，这是在大规模文档集上使用神经网络的最早文献。

关键词权重用来表示文档中词出现的权重，我们综合了基于向量空间模型的临界点文档长度归一化和基于概率模型的文档长度归一化的权重（见 2.1.2 节和 2.2.3 节）。

词 i 指向文档 D_j 的链接的权重为：

$$w_{ij} = \frac{(1+\lg(tf_{ij}))*\left(h_1+h_2*\lg\frac{N}{df_i}\right)}{h_3+h_4*\frac{d_i}{\Delta}}$$

78

式中　tf_{ij} ——词项 i 在文档 j 中的权重；

df_i ——包含词项 i 的文档数；

d_j ——文档 j 中包含的词项（不包含停用词）的长度；

Δ ——文档的平均长度。

我们可以通过在 TREC-5 数据集上训练得到调节参数 h_1、h_2、h_3 和 h_4。假定前 12 篇文档为相关文档，并且使用这 12 篇文档进行相关反馈，而且提供额外的词项。我们还假定第 500～1 000 篇文档不相关。这种方法在 TREC-6 数据集上的平均准确率为 0.177 2，这使得在所有 TREC-6 的参赛者中表现最佳。

2.7.7　文档聚类

文献[Macleod 和 Robertson，1991]提出了一种用于文档聚类的神经网络算法。这种算法的性能与本质上都是层次化方法的串行聚类方法具有可比性。在并行机器上，这种神经网络算法更快一些，这是由于层次算法本身都是串行的。

该算法首先将每一个类关联到网络中的一个节点。接着每个节点计算（并行）现有文档和与该节点关联的类的中心向量的相似度。首先，我们计算出新加入的文档 X 和现有类的中心向量的相似度。神经网络的输入节点对应于每一类。如果相似度 s_1 高于阈值 s_{1avg}，那么输入节点被激活。将 X 加入到某个类，经过参与竞争的较少量重新计算后，该节点环回到本身。没有足够接近新加入的文档的节点不会被激活。

对于所有在第一轮中获胜的节点，将会进行新一轮的传递，重新计算相似度。这种处理一直继续直到仅有一类超过阈值。此时，我们计算出不同的相似度 s_2 来确保获胜的类与新加入的文档相当接近。如果足够接近，这篇文档将被加入到类中，并且该类的中心向量会被更新。否则，为新加入的文档新建一个类。

该算法的性能与在 3.2 节中描述的单链接法、完全链接法、组平均法和 Ward 的方法相当。

79　考虑到该算法是非层次的，并且可以并行实现，所以该算法比相对应的计算量巨大的层次聚类方法更有实际价值。

2.8　遗传算法

遗传算法基于遗传与进化的原理。在文献[Chen，1995]中概述了如何将遗传算法用于信息检索。Chen 在文献中回顾了遗传算法的步骤。

- 设定初始种群
- 循环
  - 评估
  - 选择

- 繁殖
- 交叉
- 变异

- 收敛

初始种群由待解决问题的所有解组成，并且由适应度函数来度量给定解的相对“适应度”。这里要注意的是，对于查找一个已知查询的相关文档问题，相似度就是一个很好的适应度函数。我们选择一些解（更倾向于那些最适合的解）可以幸存，然后继续到下一代。问题的解对应于遗传学中的染色体，解中的每一部分对应于遗传学中的基因。

下一代通过选择幸存者的染色体进行繁殖。选择是通过适应度函数进行的。我们将 F 值作为每个适应度函数的和：

$$F = \sum_{i=1}^{population} fitness(v_i)$$

其中 *population* 是初始解的个数。表 2-27 给出了一个例子，其中包含初始群体及其对应适应度函数。适应度函数 F 的总体数量为 100，种群大小为 5。为了产生下一代，我们随机选择的 5 个值，根据其在 F 中所占的比例，选择的概率也会有所不同。

表 2-27 简单的适应度函数

i	*fitness(i)*
1	5
2	10
3	25
4	50
5	10

表 2-28 给出了种群中每一项占总体的比例。为了生成大小为 5 的新的下一代个体，我们在 0 到 1 之间随机选了 5 个值。种群中每个成员的选择区间基于其适应度函数所占的比例。如果随机数值是 0.50，其区间就会落在(0.40,0.90]，并且我们会选择成员 4。对于一个已知的成员，适应度的大小本质上决定了该成员是否会存活到下一代。在这个例子中，成员 4 的适应度函数占了适应度函数的一半，这样成员 4 被选入到下一轮的比例就是 $1-\left(\frac{1}{2}\right)^5=\frac{31}{32}$。

80

表 2-28 选择区间

i	*fitness(i)*	$\frac{fitness(i)}{F}$	选择区间
1	5	0.05	[0,0.05)
2	10	0.10	[0.05,0.15)
3	25	0.25	[0.15,0.40)
4	50	0.50	[0.40,0.90)
5	10	0.10	[0.90,1.0]

对于幸存者来说，可能会发生两种改变——交叉或者变异。交叉在两个幸存者中间发生，并且通过交换两个幸存者的组成部分来实现。下面我们给出一个例子：假设第一个幸存者使用 11111 表示，第二个幸存者使用 00000 表示。接着我们随机选择一点（比如说第三位）。经过交

叉后，两个新的后代为 11100 和 00011。第一个孩子是从第一个个体的前三位 111 和第二个个体的后两位 00 派生的。第二个孩子是从第二个个体的前三位 000 和第一个个体的后两位 11 派生的。

接下来，我们通过随机抽检幸存者的基因进行变异。在遗传算法中，变异的概率是一个参数。在信息检索中实现遗传算法，基因以字节表示，变异的结果是其中一个字节的值被改变了。在这个例子中，在第二个孩子的第二个字节发生了随机变异，这导致第二个孩子第二位的值从
81 0 变为 1，也就是变为 01011。

这种处理一直进行下去，直到不再满足适应度函数，或者新产生的个体序列并不比其前一代个体要好。这就是所谓的收敛。有些算法在达到预先指定的代数后并没有达到收敛就结束了。

2.8.1　文档表示形式

最早将遗传算法应用到信息检索中的研究见文献[Gordon，1988]。遗传算法应用到信息检索中的关键问题是如何对文档进行较好的表示。因此，最初的种群由文档集中每篇文档的多种表现形式组成。每种表现形式是映射用户最有可能选择的词或短语的向量。接着，我们标识出查询的固定集合，然后就可以使用遗传算法来生成每篇文档的最佳表示形式。

查询的表示形式固定不变，但是文档的表示形式通过遗传算法进行优化和修改。我们使用 Jaccard 相似度作为给定表示形式的适应度。一个给定表示形式的总适应度为：每个训练查询在对应的文档表示形式上的相似度的平均值。然后文档表示通过上文中提到的交叉变换和变异来“进化”。综上所述，所有查询和所有文档表示的平均相似度就会提升。Gordon 表明在四十代以后，相似度将会提升近 10%。

首先，我们处理已知相关文档的查询集合，接下来修改算法来引入非相关查询的处理。算法生成的每一代计算两个集合。对于每种表现形式，一个集合对应相关的查询，而另外一个对应非相关的查询。接下来我们选择幸存者。选择依据是那些可以最大程度地提高相关查询的平均得分的个体，还有那些最大程度地消减非相关查询的平均得分的个体。经过四十代后，平均增长率接近 20%，平均消减率达到 24%。我们不能确定这种方法的可扩展性，这是因为所有的查询和文档来自于一个大小为 18 的文档集，其中每篇文档有 18 种不同的表示集合。这些实验结果肯定会被认为是不确定的。

然而应该注意的是，尽管我们把这种算法称为文档索引算法，其实可以直接用于文档检索。我们可以使用许多种不同的自动生成的表现形式，如仅使用词项，仅使用短语，不同的词干还原算法，等等。经过一些训练和进化，我们就可以得到一种最优的表示文档的形式。此外，我
82 们注意到这种策略可能对于 ad hoc 查询处理来说更适合文档传输的应用。

2.8.2　查询权重的自动赋值

在遗传算法中加入查询权重可以参见文献[Yang 和 Korfhage，1994]。这种方法在克兰菲尔德文档集上测试过。文献[Yang 和 Korfhage，1993]中的测试使用了 TIPSTER 文档集中的 DOE 部分（和一些扩展在大文档集上所必需的相关修改）。本来原始的查询并没有考虑任何权重。为原始查询词随机生成 10 个权重集合，这就简单地构成了原始的种群。实际上，在种群中存在 10 个查询。

遗传算法接着扩展每个查询，并且标识出适应度函数。首先要计算查询到每篇文档的距离，接着检索到查询对应的前 x 篇文档（x 的大小由距离阈值决定。距离阈值用来决定结束检索文档

的条件，上限为 40 篇文档)。适应度函数基于检索到前 x 篇文档的相关性估计：

$$\text{fitness}(i) = 10R_r - R_n - N_r$$

式中 R_r—— 检索到的相关文档数；

R_n—— 检索到的非相关文档数；

N_r—— 未检索到的相关文档数。

从根本上说，一个好的适应度值可以更好地检索到相关文档。由于在较大的文档集上检索所有相关文档非常困难，所以我们通过赋予额外的权重——常量 10 来检索相关文档。我们仅仅选择那些适应度高于平均值的个体。接下来，我们使用赋予权重的适应度来进行繁殖，这样适应度比较高的个体最有可能进行繁殖。交叉变异在一个查询向量和另外一个查询向量之间进行交换某些部分来完成。这种处理一直继续下去，直到检索到所有相关文档。当然，前提是原始查询能够查找到一些相关文档，并且基于用户的相关性反馈，我们也可以查找到其他相关文档。

在克兰菲尔德文档集上的测试表明，在用户反馈后，平均准确率比先前的相关性反馈方法提高到了 20%。在接下来使用 DOE 文档集的论文中，实验表明遗传算法在每一遍中都可以继续找到新的相关文档。这是非常有趣的，因为每次查询中改变的仅仅是查询权重。查询中并没有添加或者删除一些词。Yang 和 Korfhage 确实使用了一个相对较大的文档集（TIPSTER 文档集的 DOE 部分），但是仅仅测试了两个查询。 83

2.8.3 自动生成带权重的布尔查询

在文献[Kraft 等人，1994；Petry 等人,1993]中给出了用遗传算法构建查询的研究。其主要思想是：对于一个给定的查询请求，最优查询可以通过单个查询词项集合进化得到。假设我们已知与一个查询相关的文档集，这些文档中的所有词项可以用作遗传算法的初始种群。接着每个词都是一个查询，另外可以使用相似度来作为适应度（Kraft 等人使用的 Jaccard 系数)。查询词的变异结果是查询词的带权重的布尔组合。文献中提出了三种适应度函数。第一种就是召回率：

$$E_1 = \frac{r}{R}$$

其中 r 是检索到的相关文档数，R 为所有已知的相关文档数。

第二种适应度函数是准确率和召回率的组合：

$$E_2 = \alpha(\text{recall}) + \beta(\text{precision})$$

其中 α 和 β 是任意权重，recall 表示召回率，precision 表示准确率。

结果表明不管是适应度函数 E_1 还是 E_2，都可以生成检索到所有相关文档的查询（在 50 代之后)。由于 E_2 引入了准确率，检索到的非相关文档的数量由平均值 33 降至平均值 9（每种测试都使用了三种实现方法)。

这项研究表明，遗传算法可以用来产生带权重的布尔查询。然而遗憾的是，这项工作是在较小的文档集上使用了两个查询（由 483 篇文档组成的 CACM 文档集)，所以我们也不确定这种算法是否可以扩展到较大的文档集上。

2.9 模糊集检索

模糊集最早出现于文献[Zadeh，1965]。模糊集提出了使用隶属度函数来标明元素在集合中

的隶属度，而不是假设元素是某个集合的成员。对于信息检索，模糊集是非常有效的，因为它可以描述一篇文档是“关于”什么内容的。

描述文档是关于什么内容的一组元素的集合本身就具有不确定性。关于“医药”的且与诉讼间接有关的文档，或许可能是关于“医疗事故”的文档。尽管将“医疗事故”作为集合的一
84 个元素实际上并不精确，但是将其从集合中排除掉也是不精确的。模糊集就是一种隶属度，其中每个元素的隶属力度本来就更精确。在这个例子中，描述文档概念的集合的形式如下：

$$\mathbf{C}=\{\ (\text{medicine}, 1.0),\ (\text{malpractice}, 0.5)\}$$

由于每个元素还附带其隶属度，所以集合 **C** 是一个模糊集。在模糊集 $\mathbf{C}=\{c_1,c_2,\cdots,c_n\}$ 中包含的概念可以形式化地表示为：

$$\mathbf{A}=\ (c_1, f_{\mathbf{A}}(c_1)),\ (c_2, f_{\mathbf{A}}(c_2)),\ \cdots,\ (c_n, f_{\mathbf{A}}(c_n))$$

其中 $f_{\mathbf{A}}: \mathbf{C} \rightarrow [0, 1]$ 表示隶属度函数，用于标识集合中元素的隶属度。

对于有限集合，模糊集 **A** 表示为：

$$\mathbf{A}=\left\{\frac{f_{\mathbf{A}}(c_1)}{c_1},\frac{f_{\mathbf{A}}(c_2)}{c_2},\cdots,\frac{f_{\mathbf{A}}(c_n)}{c_n}\right\}$$

接下来我们给出了模糊集的基本操作：求交集和并集。从根本上说，求交集的方法是取相同元素的两个隶属度函数的最小值，并集就是取相同元素的两个隶属度函数的最大值。

我们使用下面的定义来求交集、并集和补集。

$$f_{\mathbf{A}\cap\mathbf{B}}(c_i)=\text{Min}(f_{\mathbf{A}(c_i)}, f_{\mathbf{B}(c_i)}),\forall c_i \in \mathbf{C}$$

$$f_{\mathbf{A}\cup\mathbf{B}}(c_i)=\text{Max}(f_{\mathbf{A}(c_i)}, f_{\mathbf{B}(c_i)}),\forall c_i \in \mathbf{C}$$

$$f_{\mathbf{A}'}(c_i)=1-f_{\mathbf{A}(c_i)},\forall c_i \in \mathbf{C}$$

2.9.1　布尔检索

在 20 世纪 70 年代末期，研究人员对布尔检索进行了扩展，提出了模糊集检索，其概要介绍可以参见文献[Salton，1989]。我们可以将文档中的词看做模糊集来计算布尔的相似度，这是因为这些词在文档中出现的频率可以视为隶属度。

下面我们考虑由文档集中所有文档组成的集合 $\boldsymbol{D}$。模糊集 $\boldsymbol{D}_t$ 可以看做描述所有包含词 t 的文档的集合 $\boldsymbol{D}$。这个集合可以记作：$\boldsymbol{D}_t=\{(d_1,0.8),(d_2,0.5)\}$。这表明文档 d_1 包含词 t，且其隶属度
85 为 0.8；文档 d_2 包含词 t 且其隶属度为 0.5。

类似地，集合 $\boldsymbol{D}_s$ 可以定义为所有包含词 s 的文档。这个集合可以记作：$\boldsymbol{D}_s=\{(d_1,0.5),(d_2,0.4)\}$。

计算 $s \vee t$ 需要计算 $\boldsymbol{D}_s \cup \boldsymbol{D}_t$，计算（$s \wedge t$ 需要计算 $\boldsymbol{D}_s \cap \boldsymbol{D}_t$）。这些计算可以通过使用并集的最大值和交集的最小值实现。因此：

$$s \vee t = \boldsymbol{D}_s \cup \boldsymbol{D}_t = \{(d_1,0.8),(d_2,0.5)\}$$

$$s \vee t = \boldsymbol{D}_s \cup \boldsymbol{D}_t = \{(d_1,0.5),(d_2,0.4)\}$$

我们可以通过应用这些操作的结果来构造更复杂的布尔表达式。最终，我们可以得到包含文档及其相似度的一个集合。

这种方法的一个问题是，这种模型不允许我们给查询词赋予权重。我们可以通过在集合中每个元素的隶属度上乘以查询词权重从而在模型中引入查询词权重。另外一个问题是权重很低

的词会决定相似度。隶属度很低的词最终是相似度计算的唯一因素。比如下面这种情况，文档1包含词 s 和词 t，并且词 s 的隶属度为 0.000 1，词 t 的隶属度为 0.5。在请求 $s \wedge t$ 的查询时，文档1的得分就是 0.000 1。特别是查询中包含较多的词项时，这种低隶属度的词就决定了整个相似度的权重。这个问题的一种解决方法是定义一个阈值 x，当其值低于 x 时，隶属度函数值变为 0。

模糊集举例

下面在我们的例子中应用模糊集布尔检索模型。查询“gold silver truck”是不适当的，这是由于这个查询是用来进行相关性排序的，现在我们修改成布尔请求形式“gold OR silver OR truck”。我们将每一篇文档作为一个模糊集。为了得到每个词的隶属度，我们使用文档中的词频和文档长度的比率。因此，我们的文档集就变成了模糊集的集合：

$$D_1 = \{(\text{a},0.143),(\text{damaged},0.143),(\text{fire},0.143),(\text{gold},0.143),(\text{in},0.143),(\text{of},0.143),(\text{shipment},0.143)\}$$

$$D_2 = \{(\text{a},0.125),(\text{arrived},0.125),(\text{delivery},0.125),(\text{in},0.125),(\text{of},0.125),(\text{silver},0.25),(\text{truck},0.125)\}$$

$$D_3 = \{(\text{a},0.143),(\text{arrived},0.143),(\text{gold},0.143),(\text{in},0.143)(\text{of},0.143),(\text{shipment},0.143),(\text{truck},0.143)\}$$

86

为了计算 $Q(\text{gold} \vee \text{silver} \vee \text{truck})$，我们查找包含这些词的文档。“gold”在文档 D_1 和文档 D_3 中出现，隶属度都为 0.143。“silver”仅在文档 D_2 中出现，隶属度为 0.25。类似地，“truck”在文档 D_2 和文档 D_3 中出现，其隶属度分别为 0.125 和 0.143。应用最大的集合隶属度来实现模糊操作 OR，我们得到：

$$Q(\text{gold} \vee \text{silver} \vee \text{truck}) = \{(D_1,0.143),(D_2,0.25),(D_3,0.143)\}$$

接下来根据每一篇文档的隶属度，文档的排序结果是 D_2,D_1,D_3。我们再举另外一个例子，考虑查询： $\text{truck} \wedge (\text{gold} \vee \text{silver})$。对于这个查询，我们要确定 $D(\text{truck})$和 $D(\text{gold} \vee \text{silver})$——我们将这两个集合记作集合 **A** 和集合 **B**。

$$\mathbf{A} = D(\text{truck}) = \{(D_2,0.125),(D_3,0.143)\}$$

我们依然采用前面的方法计算 $D(\text{gold} \vee \text{silver})$，取每篇文档中出现的词的隶属度的最大值。从前面的计算中，我们可以得到：

$$\mathbf{B} = D(\text{gold} \vee \text{silver}) = \{(D_1,0.143),(D_2,0.25),(D_3,0.143)\}$$

我们采用隶属度的最小值来计算集合 **A** 与集合 **B** 的交集。这样我们得到：

$$\mathbf{A} \cap \mathbf{B} = D(\text{truck} \wedge (\text{gold} \vee \text{silver})) = \{(D_2,0.125),(D_3,0.143)\}$$

此时，我们还没有引入任何查询权重。现在我们修改例子，在每一个隶属度上乘以每一个查询词项的 idf。我们使用下面的查询词项权重：

$$\text{gold} = \lg\frac{3}{2} = 0.176$$

$$\text{silver} = \lg\frac{3}{1} = 0.477$$

$$\text{truck} = \lg\frac{3}{2} = 0.176$$

现在我们计算 $D(\text{gold} \vee \text{silver} \vee \text{truck})$。文档 D_1 仅包括“gold”，其隶属度为 0.143。“gold”

的查询词项权重为 0.176，所以 D_1 的权重化后的隶属度变为 $0.143 \times 0.176=0.025$。文档 D_2 中包括“silver”和“truck”。“silver”的隶属度为 0.25，权重为 0.477，所以权重化的隶属度为 $0.25 \times 0.477=0.119$。类似地，对于“truck”，权重化的隶属度为 $0.125 \times 0.176=0.022$。由于是在求
87 并集，所以取最大值，于是 D_2 的隶属度为 0.119。

对于文档 D_3，“gold”和“truck”都出现了，而且隶属度都为 0.143，并且其权重都为 0.176。因此，权重化的隶属度为 $0.143 \times 0.176 = 0.025$。模糊集 $D(\text{gold} \vee \text{silver} \vee \text{truck}) = \{(D_1,0.025),(D_2,0.119),(D_3,0.025)\}$。对于查询 $D(\text{truck} \wedge (\text{gold} \vee \text{silver}))$，我们需要再次确定 $D(\text{gold} \vee \text{silver})$和 $D(\text{truck})$。使用权重化的隶属度，得到：

$$\mathbf{A} = D(\text{truck}) = \{(D_2,0.022),(D_3,0.025)\}$$

$$\mathbf{B} = D(\text{gold} \vee \text{silver}) = \{D_1,0.025),(D_2,0.119),(D_3,0.025)\}$$

然后我们取隶属度的最小值来计算交集，得到：

$$\mathbf{A} \cap \mathbf{B} = D(\text{truck} \wedge (\text{gold} \vee \text{silver})) = \{(D_2,0.022),(D_3,0.025)\}$$

2.9.2　使用概念层次

FIRST 系统（模糊信息检索系统）中使用了概念层次的模糊逻辑推理的方法[Lucarella 和 Morara，1991]。

概念网络用来表示在文档中和查询中找出的概念以及这些概念之间的关系。概念网络是一种图，其中每个顶点表示一个概念，两个顶点间的有向边表示两个概念的关联度。接着文档就可以表示为概念的模糊集：

$$d_1 = \{(C_1,w_1),(C_2,w_2),\cdots(C_3,w_3)\}$$

这表示文档 1 包含概念 C_1、C_2 和 C_3，每个概念的权重为 w_1、w_2 和 w_3。链接关系被定义为模糊传递，这样一来，如果 C_i 链接到 C_j，并且 C_j 链接到 C_k，那么 C_i 到 C_k 的权重定义为：

$$F(C_i,C_k) = \text{Min}(F(C_i,C_j),F(C_j,C_k))$$

为了计算两个概念之间的权重，可以取沿着二者路径的链接的最小值，然后在概念层次的根节点上加入查询 Q。对于链接到查询的每一个概念，我们可以获取给定文档的概念的权重。为此，查找概念图中所有从概念到这篇文档的路径，并且取从源概念到文档的最小链接值。每一个路径的结果是单个的值。然后应用汇总规则来计算源概念的权重，规则为每条不同路径返
88 回的最大值。在文献[Lucarella 和 Morara，1991]中给出了一个详尽的例子。注意查询涉及不止一个初始概念，在布尔运算的每一对参数中，我们适当应用了模糊操作。

这种方法和向量空间模型进行了比较。实验使用了 300 篇文档的意大利语测试集和人工整理的包含 175 个概念的概念图。实验选择了 10 个查询，FIRST 系统的准确率与向量空间模型相当，但是召回率高于向量空间模型。

2.9.3　采用区间和提升效率

在文献[Chen 和 Wang，1995]中，通过使用区间而不是真实值作为概念的权重，扩展了 Lucarella 的模型。另外，还加入了一种有效的改进：提出了概念矩阵。

概念矩阵是一个 $C \times C$ 的矩阵，其中 $M(C_i,C_j)$表示 C_i 和 C_j 之间的关联强度。这种强度定义为一个单个的值或者一个强度所在的区间。我们可以通过连续的矩阵乘法来计算 M 的传递闭包

T。一旦计算出矩阵 $\boldsymbol{T}$，$T(C_i,C_j)$表示 C_i到 C_j的关联强度，其中这个强度为从 C_i到 C_j所有路径的最大值。

对于使用较多概念的概念网络，尽管 T的初始计算是耗费大量时间的，但是 T有效地计算出了相似度。首先，一个新的大小为 $D\times C$ 的矩阵将所有文档映射到概念。这个矩阵的 t_{ij}项表示文档 i和概念 j之间的关联强度。t_{ij}的值可以直接得到或者间接得到。如果概念在给定文档中出现，就可以直接得到；如果概念在给定文档中没有直接出现，则要间接得到——在这种情况下，我们可以使用矩阵 $\boldsymbol{T}$来获取概念权重。

给定一篇文档 D_i和查询 Q，文档 D_i中包括概念$(c_{i1},c_{i2},\cdots,c_{in})$，查询 Q中包括概念$(x_1,x_2,\cdots,x_n)$，我们可以计算查询中出现的所有概念的相似度（查询中没有出现的概念的值为“－”）：

$$SC(Q,D_i)=\sum_{q(j)\neq"-"\wedge j=1}^{n} T(t_{ij},x_j)$$

其中 $T(x,y)=1-|x-y|$。

函数 T度量在文档中发现的概念的强度和在查询中用户输入的概念的强度的差异。文档强度为从文档到概念路径上的强度的最小值。较小的差异导致 T的值比较大，并且相似度只是累加给定查询中每个概念的差异。

对于区间来说，新的函数 S计算区间高端和低端间的平均距离。例如，区间[3,5]与区间[2,6]比较的结果距离是 $\frac{|3-2|+|5-6|}{2}=1$。接下来我们将这种差异求和得到基于区间的相似系数。

89

2.10 本章小结

本章中，我们介绍了对查询结果列表进行排序的 9 种不同的检索模型。概率模型、向量空间模型和推理网络都使用了统计的方法，本质上都依赖于查询和文档间的匹配词。

向量空间模型将查询和文档都表示为向量的形式。文档间的相似度、文档和查询间的相似度都是根据两个向量间的距离来计算的。例如，最普遍的一种相似度计算方法是使用余弦，它将文档和文档间或者文档和查询间的不同使用这两个向量夹角的余弦值来表示。

概率模型使用引入了一些关键性假设的基本概率理论来估计相关性概率的大小。概率模型总是被批判需要先验的相关性信息效果才会比较好。我们描述了许多种避免这个问题的方法，包括 Kwok 的创造性方法，它使用自相关性来获取初始参数。

语言模型是相对比较新的模型，在先前的语音识别中其效果非常出色。在过去的几年里，研究人员发表了大量关于语言模型的论文，现在我们可以很明确地将其列为最流行的检索模型之一。

使用证据，推理网络使用贝叶斯推理网络来推断文档相关的概率。由于推理网络可以同时使用向量空间模型和概率模型建模，这使它看起来功能非常强大。事实上，科研人员还没能完全研究出不同的推理网络拓扑结构。

LSI 直接将问题处理为一个查询的相关文档有时会包含大量的词，这些词在意思上与查询是相同的，但是语法却不同，这是我们所讨论的唯一一种这样的检索模型。通过在词项-词项矩阵中估计“隐性语义”的特征值，LSI 可以精确地计算出文档与查询相关度的得分，而不用关心是否出现不匹配词的问题。

神经网络和遗传算法在机器学习中的应用都非常广泛，但在文档排序中的应用才刚刚起步。计算资源的限制已经阻碍了这些模型的广泛使用，但是这些模型检索文档的方法与其他模型完
90 全不同，这使得它们很有潜力激起人们的研究兴趣。

模糊集和扩展的布尔模型是使用历史比较久远的模型，这两个模型将布尔模型在相关性排序上进行了扩展。其他检索模型需要用户提交查询词列表而不是先前的布尔表达式，然后才能返回一个结果列表。在某些应用中，用户更喜欢布尔请求。对于已经使用布尔请求好多年的用户来说，这确实是非常正确的。

那么，哪种模型是最好的呢？这仍然是备受争议的领域，因此这有待于进一步的研究。据我们所知，所有模型间并没有进行直接的“白刃战比拼”。尽管 TREC（文本检索会议）的任务是评价完整的信息检索系统，但是一个系统除了包含检索模型和一些实用策略外，还包括一些其他的实现细节。到目前为止，还没有给出在给定的统一系统框架上进行检索模型间比较的结论。在文献[Salton，1989；Kowalski 和 Maybury，2000]中给出了关于这些检索模型的其他想法和评价，并且有些早期的论文是关于这方面的成就的，可以参见文献[Spark Jones 和 Willett，1997]。

2.11　练习题

(1) 请说明如何将推理网络用于向量空间模型或者概率模型。

(2) 在因特网上下载《爱丽斯梦游仙境》[①]。请编写程序来计算每个词的 idf，并验证其符合齐普夫分布。

(3) 设计一个新的检索模型，可以允许用户实现布尔检索请求。查询结果应该以与查询的相似度进行排序。将你的检索模型与扩展的布尔检索模型进行比较。

(4) 请说明在向量空间模型中添加新文档或者改变现存文档的影响。哪些值必须重新计算？如何在模型上只做轻微修改，就可以使模型更加适合添加新文档？

(5) 设计一个详细的例子（使用我们的标准查询：gold、silver 和 truck），使用基于片段的概率模型在我们的标准文档集上计算查询和每个文档之间的相似度。

(6) 设计一个详细的例子（使用我们的标准查询：gold、silver 和 truck），使用 Jelinek-Mercer 平滑方法的语言模型在我们的标准文档集上计算查询和每个文档之间的相似度。

91
~
92 (7) 有研究人员建议融合两种检索模型的检索结果来得到更优的检索结果。请给出两条融合两个任意检索模型结果的启发式规则，并且给出这个方法本身存在的优点和缺点。

① Alice in wonderland，一个童话故事。——译者注

第3章 检索实用策略

对于一个信息检索模型来说，我们可以采用许多实用策略来改进检索结果。大多数实用策略通过增加或删除初始查询的词项来优化查询，另一些则只是用子文档或片段（而不是整个文档集）来聚焦查询的中心。关键在于，每种策略对于任何一种检索模型都要做到即插即用（虽然这种情况很少出现）。

实用策略有如下几种。

- **相关反馈**——我们将初始查询检索出的排名靠前的文档认为是相关文档，然后对这些文档进行进一步的分析处理。我们可以通过人工干预的方式来判定相关文档，或者直接假定返回的前 n 篇文档就是相关文档。随后，通过各种不同的技术对相关文档中的词项进行排序，将其中排名靠前的 t 个词加入到原始查询中。
- **聚类**——通过自动或人工的方式，将文档或词项聚成不同的类别。我们只在与查询相关的信息类别中检索，这样就减小了搜索空间，在搜索之前就排除了不相关的文档。
- **基于片段的检索**——这种方法的前提是，大多数相关文档都包含不相关的部分片段，而且相关的片段往往会比较集中。因此，查询与文档中的片段（片段可能交叉重叠，也可能是非重叠的）进行匹配，然后将与每个片段计算出的结果归并为最后的查询相似度。片段的长度究竟是固定不变还是动态调整，一般取决于片段定位算法。另外，还有一些方法仅仅是简单地按照文档中的句子、段落或者其他自然划分的片段进行排序。
- **语言解析**（名词短语处理、词干还原等）——简单的词项匹配不一定总能产生好的结果，识别并使用短语匹配在计算上比相似运算会简单得多。采用语法规则或者已知短语列表就可以识别 New York 这样的短语，并将这些短语作为单个词项来处理。其他的语言解析技术可以去除词语通用的前缀与后缀形式，从而匹配查询和文档中词根相同但前后缀不同的词项。
- ***n* 元语法**（*n*-gram）——我们将查询划分为 n 元语法的形式（n 个重叠或不重叠的字符序列），然后对查询和文档进行匹配，这样可以对拼写错误或光学字符识别（OCR）错误进行模糊匹配，达到匹配算法更“模糊”更健壮的目的。另外，n 元语法还有不依赖于语言的优点。
- **同义词表**——同义词表可以从文本中自动生成，也可以人工编撰。该方法的关键不仅是生成同义词表，而且要用它来扩展查询或文档，从而改进检索效果。
- **语义网络**——某个独立的概念会与其他相关的概念关联，最终形成概念间的层次关系。其中，概念间的连接通过权重来表示其关系的强度。Wordnet[Beckwith 和 Miller，1990]就是语义网络的一个例子，当然还有其他类型的语义网络。自动创建语义网络是当前研究的一个热点。当前主要的挑战是如何使用语义网络来扩展查询或文档，从而包含更多词项来描述查询的语义内容。

- **回归分析**——我们可以采用统计方法来确定查询与相关文档间匹配特征的参数，因此，可以使用回归分析法来确定精确的参数，从而优化相似度的计算。

3.1　相关反馈

相关反馈是目前应用比较广泛的信息检索策略，其前提是：执行多次检索，用户在每次检索后，根据上一次的检索结果进行提炼，形成新的查询。其一般过程是，用户从上一次的检索结果中找出相关文档，并根据这些相关文档向本次查询增加新词项。另外，对于查询中已有的词项，根据用户反馈重新确定其权重。本过程如图 3-1 所示。

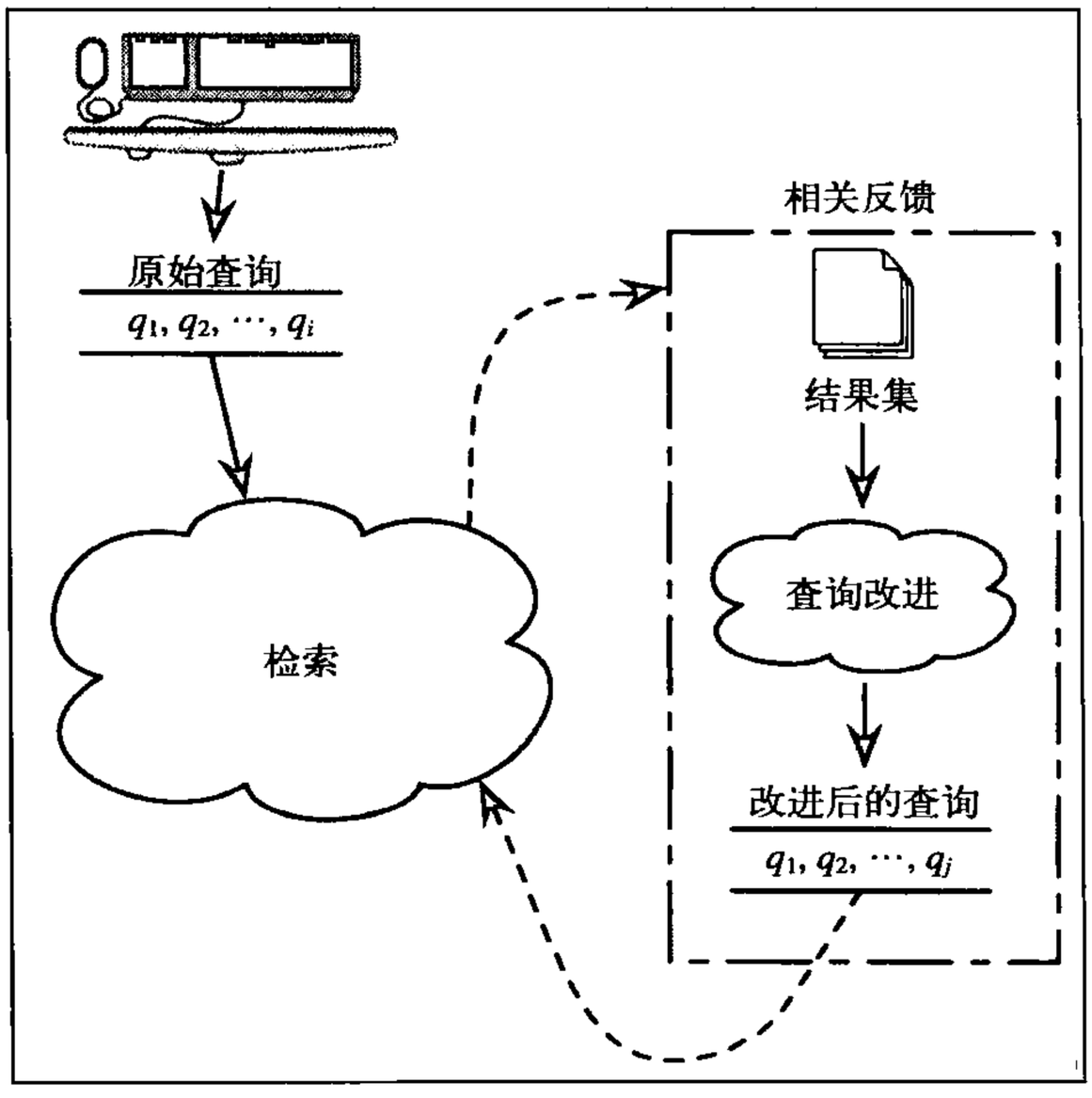


图 3-1　相关反馈过程

另一种方法是不询问用户，而只是假设结果中排在前面的文档即为相关文档。相关反馈使用人工（询问用户）或者自动（假设结果中排在前面的文档是相关文档）反馈修改初始查询，
94 然后使用新查询进行检索。

例如，初始查询“find information surrounding the various conspiracy theories about the assassination of John F. Kennedy”（“请找出约翰·F.肯尼迪刺杀事件各种阴谋相关的信息”）既有有用的关键词又有无效的噪音。最有用的关键词有“assassination”和“John F. Kennedy”，与许多查询（查询以检索词集合表示）一样，该查询也有无效的信息，比如“various”和“information”可能不是停用词（即出现频率很高但在信息检索系统中常被忽略的词，如“a”、“an”、“and”和“the”等），但是它们对检索相关文档却没什么帮助。我们可以这样来确定有效词：首先使用初始查询中的所有词来检索，并由用户确定相关文档；随后，在排名靠前的文档中出现的一些相关词，我们可以视之为“好”的查询词，并加入下一步的查询中。

假设排名非常靠前的文档中含有词项“Oswald”，那么从理论上讲，将词项“Oswald”加到初始查询中就能改进准确率和召回率。类似地，如果某篇文档（由用户确定是相关文档且排

名靠前）中出现词“assassination”的频率比较高，那么在查询中就应该提高该词的权重。 95

增加、删除词项或修改原始查询中已有词项的权重都可以通过向量空间模型实现。概率模型在相关反馈中只能改变已存在词项的权重，却不能增加词项。虽然相关反馈具体的实施方法在很大程度上依赖于检索模型，但是相关反馈（即执行查询，收集用户信息，优化查询语句，如此重复迭代）的基本原理在任意检索模型中都是通用的。

3.1.1 节将介绍向量空间模型在相关反馈中的应用，3.1.2 节则介绍概率模型在相关反馈中的应用。

相关反馈已经成为研究的热点，很多可调参数对搜索结果的影响立竿见影。大多数的反馈算法都基于这样的前提：使用排在前面的 x 篇文档中的前 t 个词项。确定 x 和 t 的最优值以及迭代次数的最佳值，已成为大量研究人员的主要课题。在 2004 年，以 x 和 t 的最佳值判别为主题的夏季研讨会已经成功举行，详见[Harman and Buckley, 2004]。

3.1.1 基于向量空间模型的相关反馈

Rocchio 在他的原创论文中首次对相关反馈进行了阐述[Rocchio，1971]。有意思的是，他提出的基本方法一直沿用至今，并没有根本性的改变。

Rocchio 用向量空间模型对文档进行排序。用向量 $\boldsymbol{Q}$ 表示查询，向量 $\boldsymbol{D}_i$ 表示第 i 篇文档，则查询向量和文档向量间的相似度表示为 $SC(\boldsymbol{Q}, \boldsymbol{D}_i)$，其中 SC 即为相似度。如 2.1 节所述，SC 是文档向量和查询向量的内积或者它们之间夹角的余弦值。用户提交查询向量 $\boldsymbol{Q}$ 后，系统检索文档集；然后用户确定相关文档；检索系统返回集合 **R** 与集合 **S**，其中集合 **R** 包含 n_1 个相关文档向量，集合 **S** 包含 n_2 个不相关的文档向量。最后，Ricchio 使用下面的公式从上一次查询 $\boldsymbol{Q}$ 生成新的查询 $\boldsymbol{Q}'$：

$$\boldsymbol{Q}' = \boldsymbol{Q} + \frac{1}{n_1}\sum_{i=1}^{n_1} R_i - \frac{1}{n_2}\sum_{i=1}^{n_2} S_i$$

其中，R_i 和 S_i 分别是集合 **R**、**S** 中的元素。

如上式，相关文档集合中的文档向量添加到前一次查询向量 $\boldsymbol{Q}$，然后在新的查询向量中减去不相关文档集合中的向量。如果所有文档都相关，上式中的第三项就没有必要出现了。为了 96
保证新信息不会完全覆盖原始查询，向量修改时利用相关和不相关文档的数量进行规范化。检索过程迭代进行，其中由 Q_i 到 Q_{i+1} 为一次迭代，可以根据需要对此过程进行反复迭代。

上式的依据是相关文档中包含能与原始查询相匹配的词项。通过添加相关文档集合中的向量，查询与相关文档中相匹配的词的权重增加；而通过减去不相关文档的文档向量，查询中与不相关文档相匹配的词的权重则降低；并且，在原始查询中不存在的词项（该词在查询向量中对应的值为 0）也添加到查询中。

除了使用 n_1 和 n_2，也可以使用任意权重作为规范化因子，则上面的公式变为：

$$\boldsymbol{Q}' = \alpha\boldsymbol{Q} + \beta\sum_{i=1}^{n_1}\frac{R_i}{n_1} - \gamma\sum_{i=1}^{n_2}\frac{S_i}{n_2}$$

并不是所有的相关文档或者非相关文档都必须要使用。增加阈值 n_a、n_b，分别表示相关文档向量和不相关文档向量的阈值。结果如下：

$$\boldsymbol{Q}' = \alpha\boldsymbol{Q} + \beta\sum_{i=1}^{\min(n_a,n_1)}\frac{R_i}{n_1} - \gamma\sum_{i=1}^{\min(n_b,n_2)}\frac{S_i}{n_2}$$

α、β、γ称为Rocchio权重，它们在每年的TREC会议中都频繁被提及，其最优解通过实验获得，目前比较通用的情况是忽略不相关文档（即$\gamma=0$）而仅使用相关文档。Ide继Rocchio之后在研究中使用了这种方式，其公式定义为：

$$Q'=\alpha Q+\beta\sum_{i=1}^{n_1}R_i-S_1$$

公式中只使用了排名最高的不相关文档，而不是所有不相关文档向量的和。Ide将此称为Dec-Hi（降低最不相关文档的权重）方法。另一种更简单的权重计算方法则把基于文档向量数量的规范化因子省略掉，即α、β、γ都设为1[Salton，1971b]，新的公式变为：

$$Q'=Q+\sum_{i=1}^{n_1}R_i-\sum_{i=1}^{n_2}S_i$$

原始查询只检索到不相关文档时，将会出现一个十分有意思的现象。凯利（Kelly）在文献
97 [Salton，1971b]中介绍了这一现象。该文建议对文档集中频率最高的概念增加任意权重。该方法可以进一步拓展为对增加权重最高的重要度。因为很重要的词容易被周围的噪音所淹没，所以增加权重保证了词项在一定程度上获得正确的权重并产生相关文档。注意，这个方法只能用在人工反馈而不能用于自动反馈中。自动反馈仅仅假设（按定义）检索结果的前n篇文档是相关文档。

3.1.2　基于概率模型的相关反馈

2.2节介绍了基本的概率模型，其核心思想是：文档中的词视为文档与查询相关的证据。假设词间是相互独立的，则文档与查询相关的概率即为文档中每个词项与查询中词项匹配的概率的乘积。

概率模型非常适合应用于相关反馈中，因为在计算词权重的时候，我们特别需要了解与查询相关的已有文档数目。一般来说，原始的概率模型需要训练数据，训练数据应该包括是否相关等信息。一旦计算出词项权重，我们将这些词和权重应用于另一个测试文档集。

相关反馈不需要训练数据。将基于概率模型的相关反馈看做一个实用检索策略而不是检索模型，可以嵌入到任一目前已知的检索模型中。通过任一检索模型进行初始查询后，就可以将反馈阶段获得的相关信息进行合并。

例如，概率检索模型中权重的基本定义是：

$$w_i=\lg\frac{\dfrac{r_i}{R-r_i}}{\dfrac{n_i-r_i}{(N-n_i)-(R-r_i)}}$$

式中　w_i——在特定查询中词项i的权重；

R——与查询相关的文档数量；

N——文档集中的文档数量；

r_i——包含词项i的相关文档的数量；

n_i——包含词项i的文档数量。

R和r在初始查询的时候是个未知数，直到我们获得了相关信息的训练数据为止。实际上，
98 不可能存在领域无关的训练数据集。用其他检索模型（比如向量空间模型）进行初始查询，得

到前 n 篇文档，在这前 n 篇文档中出现的相关文档的数量可以作为 R 的估计值，r 就是在这 R 篇文档中包含某词项的文档数量。这样，概率检索模型需要训练数据的问题通过相关反馈就解决了。

1. 初始化估计

基于概率模型的相关反馈的初始化估计方法千差万别。有些方法仅将 idf 相加的和作为初始估计值。Wu 和 Salton 提出一种很有意思的扩展方法，这个方法需要用到训练数据。对于给定的词项 t，要估计出现该词的文档数量，需要知道在其他查询中与词项 t 相关的文档数目。在检索之前估计 r_i 的值的公式如下：

$$r_i = a + b \lg f$$

式中，f 表示词在所有文档中出现的频率。

这个公式将频率映射到相关文档数的估计值。频率表示对于一个给定词，其相关文档的数目。通过几个样本点，a 和 b 可以用最小二乘拟合计算得到；然后，r_i 的估计值可以通过对已知的 f 进行运算得到，从而根据 r_i 可以得到词项的初始权重（IW）的估计值；最后通过各个词项的初始权重计算出相似度。在文献[Wu 和 Salton，1981]中，作者得出结论（使用非常小的数据集）：idf 计算代价非常小，IW 的引入使准确率和召回率有轻微下降。然而，我们还不知道在 TREC 数据集中使用 IW 会出现什么样的效果。

2. 计算新的查询权重

文献[Robertson 和 Sparck Jones，1976]提出了在相关反馈中使用的、基于基本权重策略的许多变种方法。[Wu 和 Salton，1981]首次发掘出基于概率模型的相关反馈的潜力，Wu 和 Salton 主要将 Sparck Jones 公式[Robertson 和 Sparck Jones，1976]应用于相关信息的获取。他们通过使用下面的公式计算相似度来改进该方法。用向量 $\boldsymbol{Q}$ 表示查询，向量 $\boldsymbol{D}_i$ 表示包含 t 个词项的文档，我们就可以通过下面的公式计算出相似度。d_i 的取值为 0 或 1，取 1 代表第 i 个词项出现，0 代表第 i 个词项未在文档中出现。K 是常数。

99

$$SC(\boldsymbol{Q}, \boldsymbol{D}_i) = \sum_{i=1}^{t} d_i \lg \frac{p_i(1-u_i)}{u_i(1-p_i)} + \mathrm{K}$$

使用上述公式需要估计 p_i 和 u_i 的值，最简单的方法是取 $p_i = \dfrac{r_i + 0.5}{R+1}$，$u_i = \dfrac{n_i - r_i + 0.5}{N - R + 1}$。其中 p_i 表示文档中出现第 i 个词项时该文档是相关文档的概率，等式右边则是检索出的含有第 i 个词项的相关文档数量与相关文档总数目的比值，用该比值作为 p_i 的估计值是合理的。而等式右边的 0.5 只是一个调整因子。类似地，u_i 的估计值表示为含有第 i 个词项的不相关文档数与不相关文档总数的比值。将 p_i、u_i 代入上式，可得到计算权重的一个经典算法 [Robertson 和 Sparck Jones，1976]，正是我们在 2.2.1 节中“举例”一节所述的 w_2：

$$\frac{\dfrac{r_i + 0.5}{R+1}}{\dfrac{n_i - r_i + 0.5}{N - R + 1}}$$

采用相关反馈方法，提交初始查询后，检索执行后，在初始结果集中便有可能存在相关文档。通过用户检查前几项结果，从而使 R 和 r_i 的估计更准确（N 和 n_i 的值在检索前已知）。然后计算出新查询词项的权重并再次执行查询。Wu 和 Salton 实验了 4 种不同的计算新查询的方法。

(1) 直接使用第一次检索后计算出的词项权重生成新查询。

(2) 不仅使用第一次检索后计算出的词项权重，而且结合旧的权重生成新查询。Wu 提出生成新查询的方法如下：

$$Q' = \frac{1-\beta}{Q} + (1-\beta)(T)$$

其中，Q 包含旧的词项权重，T 包含第一次迭代后计算出的词项权重。β 的大小表示初始权重的重要性，其值为出现第 i 个词项的相关文档数与整个集合中相关文档总数的比值，即 $\beta = \frac{r_i}{R}$。检索出相关文档比较多的查询应该加大新权重的比例，相反，检索出的相关文档较少的查询则需降低新权重的比例。

(3) 结合前一次查询中的所有词项和相关文档中出现的所有词项来扩展查询。在新查询中，所有旧词项（既包含在初始查询中又包含在相关文档中的词项）的权重与第一次迭代时的计算
100 方法一样；包含在初始查询中但不存在于任何文档中的词项权重不变。这是与 Sparck Jones 工作的根本不同之处，原因在于这里对查询进行了扩展并重新计算权重。在此之前，概率模型在检索中的应用依赖于旧词项的权值重新计算，并不允许增加新词项。

(4) 结合初始权重和新权重扩展查询。这与上面所述的第（2）个相似。假设 $q_1,\cdots,q_m$ 表示初始查询中 m 个词项的权重，查询过后，出现了 n 个新词项，新查询的计算方法如下：

$$Q' = \frac{1-\beta}{Q}(q_1, q_2, \cdots, q_m, 0, 0, \cdots, 0) + \beta(q_1, q_2, \cdots, q_m, q_{m+1}, \cdots, q_{m+n})$$

并且，p_i 和 u_i 的计算方法有所改进。新的估值公式如下：

$$p_i = \frac{r_i + \frac{n_i}{N}}{R+1}$$

$$u_i = \frac{n_i - r_i + \frac{n_i}{N}}{N-R+1}$$

这里，关键是 idf（$\frac{n_i}{N}$）代替了简单的 0.5 作为调节因子。

Wu 和 Salton 发现第（4）种方法（即结合新旧权重，并且扩展查询词项的方法）最有效。修改后的 p_i 和 u_i 并没有产生多大的不同。

Salton 和 Buckley 完美地概述了相关反馈，其中介绍了相关反馈的 12 种变化方法[Salton 和 Buckley，1990]，包括：进行词干还原的 Rocchio 方法、Ide、传统的 Sparck Jones 概率方法（见 2.2.1 节）以及 Wu 和 Salton 给出的扩展概率方法[Wu 和 Salton，1981]。所有的 12 种方法都在 6 个小测试集（CACM、CISI、CRAN、INSPEC、MED 和 NPL）进行测试。这 6 个测试集在更大的 TIPSTER 测试集出现之前被研究人员广泛使用。测试时使用了不同的参数，所以总共有 72 种不同的反馈方法。最终 Ide Dec-Hi（削减排序最高元素的权重的方法）取得最好的结果——在 3 个测试集上获得第一、在其他测试集上分获第二、第六。

3. 部分扩展查询

Wu 和 Salton 在 1981 年的工作中，要么保持初始查询词项不变而重新计算权重，要么将初始结果集中的所有词项添加到新查询中并计算权重[Wu 和 Salton，1981]。在文献[Harman，1988]

中，首次提出了仅选择结果集的前几篇文档中出现的部分词项作为扩展的方法。在该论文中，从检索结果集的前 10 篇文档中人工选择出相关文档，随后提取出这些相关文档中的所有词项，接下来的问题是哪些词项应该作为扩展添加到新查询中呢？Harman 基于 6 个不同的标准将这些词项进行排序，选择前 20 个词项作为扩展。排序标准是决定效果好坏的关键因素，这 6 种不同的标准在一个较小的被称为克兰菲尔德的测试集上进行了测试。 101

6 个排序标准大部分使用了噪声度量因子 n。第 k 个词项的噪声度量因子的计算公式为：

$$n_k = \sum_{i=1} N \frac{tf_{ik}}{f_k} \log_2 f_k tf_{ik}$$

式中 tf_{ik} ——词项 k 在第 i 篇文档中出现的次数；

f_k ——词项 k 在整个文档集中出现的次数；

N——整个集合中所有词项的数量。

对于许多文档中频率不高、但在整个文档集中频率较高的词，应该增大其噪声度量因子。但对于整个文档集中频率较高的词项，其噪声度量因子一般较小。它与 idf 相似，区别在于该公式还引入多个文档中的频率。

排序标准中用到的其他变量还有：

p_k ——相关文档集中包含词项 k 的文档数量；

rtf_{ik} ——词项 k 在相关文档集中出现的次数。

噪声度量因子的一种变形是 rn_k，其定义为相关文档集上的噪声。计算方法为：

$$rn_k = \sum_{i=1} p_k \frac{tf_{ik}}{f_k} \log_2 f_k tf_{ik}$$

将 rn_k、n_k 和 p_k 进行不同的组合形成排序标准。文章中使用的 6 种排序标准是：

- n_k
- p_k
- rn_k
- $n_k \times rtf_k$
- $n_k \times f_k \times p_k$
- $n_k \times f_k$

102

排序标准 $n_k \times f_k \times p_k$ 在平均准确率上产生了最好的效果，其平均准确率提高了 9.4%。这个标准与有实际物理意义的度量基准 $p_k \times idf$ 非常相似，其中 p_k 从直观上看在此处非常合适（即相关文档集中出现频率较高的词项加入到查询中极有可能效果较好）。然而，对于在整个文档集中频率较高的词项来说，p_k 就不太合适了。这也是 p_k 与 n_k 组合使用时不能取得较好效果的原因。

文献[Harman，1992]测试了另外的 6 种排序标准，这 6 种标准如下所示。

- $\frac{(RT_j)(df_i)}{N}$，RT_j 指查询 j 检索出的所有文档数量，df_i 是包含词项 i 的文档频率或者文档数量，N 是集合中所有的文档数量。对于初始结果集中出现在多篇文档中的词项，这个标准增加了其权重。
- $\frac{r_{ij}}{R_j} - \frac{df_i}{N}$，$r_{ij}$ 是查询 j 检索到的含有词项 i 的相关文档数量，R_j 是查询 j 的所有相关文档

数量。对于在许多相关文档中都出现但是在整个文档集中频率不高的词项，这个标准增加了其权重。

- $W_{ij}=\log_2\frac{p_{ij}(1-q_{ij})}{(1-p_{ij})q_{ij}}$，$W_{ij}$是查询$j$中词项$i$的权重。这个标准基于在2.2.1节所述的Sparck Jones的统计权重，对于查询j，其相关文档中含有词项i的概率是p_{ij}；对于查询j，其不相关文档集中出现词项i的概率为q_{ij}。其计算方法如下：

$$p_{ij}=\frac{r_{ij}+0.5}{R_j+1.0}\qquad q_{ij}=\frac{df_i-r_i+0.5}{N-R_j+1.0}$$

- $idf_j(p_{ij}-q_{ij})$，其理论基础是一个假设：词项i的重要性与该词项所导致的相关与不相关文档平均得分之间的差异成正比。文献中没有指明词项权重的计算方法，所以在这个标准中使用了idf_j。更详细的说明请参见[Robertson，1990]。
- $W_{ij}(p_{ij}-q_{ij})$，词项权重的计算方法如上面所述。
- $\lg(RTF_i+1)(p_{ij}-q_{ij})$，其中$RTF_i$是词项$i$在检索出的相关文档中出现的次数。

103 从根本上说，第3个标准比第4、5、6个效果好，但差别不大。第1、2个标准效果较差。

排序标准确定后，接下来要研究的问题就是确定要添加到新查询中词项的数目。一般这个数目最大定义为20。在TREC评测中，有些小组喜欢使用“大规模查询扩展”（massive query expansion）（即将第一阶段产生的所有词项都添加到查询中），而另一些小组只使用这些词项的一个子集[Buckley等人，1994；Salton和Buckley，1990]。TREC中有些小组使用了50个词项和20个短语并取得了好的效果。

文献[Lundquist等人，1997]在TIPSTER测试集上提出了另外的排序技术，并发现$p_k\times nidf$的效果较好。变量$nidf$是使用临界点文档长度归一化了的idf（见2.1.2节），并且结果显示使用前10个词项（词或者短语）可以使平均准确率比使用前50个词和20个短语提高了31%。

4. 反馈迭代次数

相关反馈迭代多少次效果最好？ Salton在1971年首次进行了最优迭代次数的实验[Salton，1971d]。在1990年，他使用的72种不同的相关反馈方法都假设相关反馈只迭代一次。Harman研究了在相关反馈中使用多次迭代的效果，详情请参见文献[Harman，1992]。

Harman的研究过程是：第一次检索出10篇文档，随之获得这10篇文档中相关文档的数量，然后再检索出10篇文档。这个过程迭代6次，如果某一次迭代过程中没有出现相关文档，则整个搜索过程终止。在迭代过程中，Harman使用了三种不同的方法更新词项权重，并根据得到的相关文档数量是不变还是渐增的而选择一种方法使用。每次迭代使用基本的检索算法检索出前10篇文档、确定前20个词项，并重新计算其权重。

共进行了三种不同的测试实验。

- 累加计数——在相关文档范围内，统计相关文档数以及词频。在迭代过程中，统计数据不断累加。
- 重置计数——在相关文档范围内，统计相关文档数以及词频。每次迭代之后，统计数目重置。
- 重置计数，重置每次的查询——重置相关文档数量和词频，以便新查询仅包含本次迭代产生的词项。

在每种方法中，每次迭代后都会发现相关文档的数量是递增的。然而，在前两次迭代中，大多数的相关文档都已经被检索出来了。平均而言，第 3、4、5、6 次迭代平均发现的新相关文档数目少于一篇。三种方法在相关反馈的每次迭代中的效果相差不大。 104

5. 用户交互

如前所述，相关反馈的初始阶段总是假设由用户确定与查询相关的文档。后来，又假设前 *n* 篇文档是相关的并且只使用这些文档。Spink 做了一项有意思的研究，在使用前 *n* 篇文档中的词项进行查询扩展时，将选择哪些词项加入到查询的权力交给用户[Spink，1994；Spink，1995]。在该研究中，用户还要决定增加词项的哪些信息用于反馈。对查询词项来说，可供选择的来源有：

- 原始查询；
- 用户交互——搜索之前与专家用户或“媒介”交互以确定扩展的词项；
- 中间结果——搜索过程中专家用户的意见；
- 知识库；
- 相关反馈——根据用户或者专家意见选择扩展词项。

用户从相关反馈得到的搜索词项(由 40 多个查询得到)中选择 48 个词项(占总词项的 11%)。在这些用户当中，最终用户选择了 15 个，而专家选择了 33 个。这表明高级用户更容易利用相关反馈。

另外，这项研究还标出了用户发现的词项在文档中位于哪一部分。有大约 85%的用于相关反馈的词项来自文档的标题或者叙述性片段，只有 2 个词项来自文档的摘要。研究最后得出结论：新系统应重点关注来自标题和描述性片段的词项，并将这些词作为相关反馈阶段的词项来源。

3.2 聚类

文档聚类将文档按内容分类，以减少查询时所需要的搜索空间。例如，一个既包含医学又包含法律内容的文档集，经过聚类后，所有的关于医学的文档被归为一组，法律文档则归为法律类（见图 3-2）。关于法律信息的查询就被定向（自动或者人工）到法律类文档集。 105

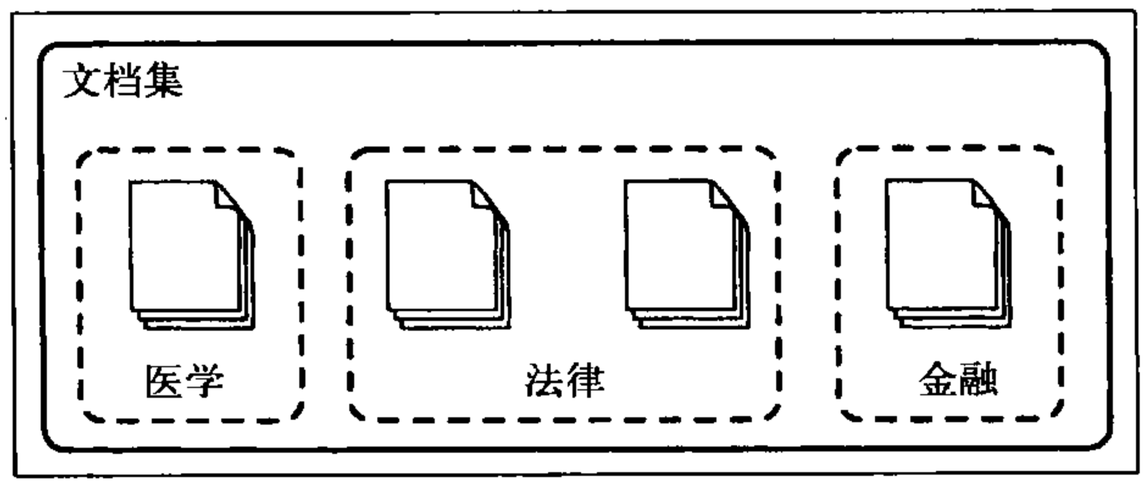


图 3-2 文档集

到目前为止，已有多种不同的聚类算法。在许多情况下，将查询自动指向某一文档类非常困难，所以如何评价聚类算法非常具有挑战性。在本节中，我们将文档聚类作为 ad hoc 检索的辅助策略，所以我们重点讨论聚类算法本身及这些算法在改进 ad hoc 检索和人工查询处理的准确率和召回率上的潜力。

计算过于复杂是限制聚类算法广泛应用的另一个因素。许多聚类算法都要使用相似度矩阵，这类矩阵的元素是任意两篇文档间的相似度。比如，含有 1 000 000 篇文档的文档集的矩阵为 1 000 000 阶，含有 $\frac{1\,000\,000^2}{2}$ 个不同的元素。传统检索也存在同样的问题，并且因为这个问题使文档间相似度的计算非常复杂。换句话说，将每一篇文档看做维数为词典中词项数的向量，计算两个文档间的相似度需要对两个向量中所有不为零的分量进行比较。由于许多词仅出现在单个文档中，文档向量非常稀疏，所以计算代价很高。

应用于检索的聚类算法的计算复杂度必须适应于大规模数据。通过使用数字阵列处理器（Digital Array Processor，DAP）并行进行初始化工作以改进聚类算法的性能[Rasmussen 和 Willett，1989]；然后，这些算法在通过环形网络连接的并行机器上执行[Ruocco 和 Fieder, 1997]。

文献[Salton，1989]详细地综述了聚类算法，聚类算法按自顶向下或自底向上的方法形成类簇（cluster）。自顶向下的方法先将整个文档集看做一个类（cluster），然后向下分为更小的类。
106 自底向上的方法将每篇文档看做大小为 1 的类，这些类逐级相互组合形成越来越大的类。由于自底向上方法的聚类结果分为不同的层次（随着不同的类别拼凑到一起，于是就出现了层次），所以又被称为层次聚类。

其他的聚类算法，比如应用广泛的 K-Means 算法，首先随机选取几个中心，随后在一个迭代过程中反复调整它们，直至达到终止条件。一些研究发现：层次聚类算法尤其是使用组平均法（group-average）的层次聚类算法由于对文档进行了完全的两两比较，所以产生了更好的效果[Larsen 和 Aone，1999；Willet，1988；Dubes 和 Jain，1988]。最近的研究表明这并不是在所有的评价方法中都是正确的，并且一些层次与重叠算法相结合的方法取得了更好的效果[Steinbach 等人，2000；Zhao 和 Karypis, 2002]。由于这些研究实验不同，评测方法不同，使用的文档集（经常很小）不同，所以很难确定到底是哪个聚类算法更好。

3.2.1　结果集聚类

文献[Lu 等人，1996]将聚类作为相关反馈的辅助策略。这种情况下，我们只需对查询结果（比原始文档集小得多的文档集合）进行聚类，并且在相关反馈过程中，我们只选择较大类中的新词。

最近，[Zeng 等人，2004]基于结果集上的关键短语对 Web 搜索结果进行聚类。首先，对结果集中的文档进行语言解析，识别出长度为两个词的短语；然后将这些短语的特征作为各种学习算法（比如：线性回归、逻辑回归和支持向量回归）构建的模型的输入；最后，识别出关键短语后，将它们用于构建类。起初类被认为是包含最关键短语的文档集。例如，含有关键短语“New York”的所有文档一开始被置为一个类别。最终，这些初始的聚类再根据文档相似度进行归并。

3.2.2　层次聚类

首先计算出 $N \times N$ 阶的文档相似度矩阵。每篇文档作为一个类别，然后重复执行以下两个
107 步骤，直至形成一个类。

- 找出具有最大相似度的两个类。
- 合并这两个类，并重新计算新合并的类与其他类之间的相似度。

在上述过程中，小的类别合并成大类，这样所有的类形成了一个层次。

假设有文档 A、B、C、D 和 E，它们之间的相似度矩阵已知，并且每篇文档是一个类：{{A}{B}{C}{D}{E}}。

现在假设文档 A 和文档 B 之间的相似度最大，所以聚类后变为：{{A，B}{C}{D}{E}}。

这个过程不断地迭代执行，最终将只剩下一个类：{A，B，C，D，E}。然而，我们也就能够得到形成这个类别的过程：在聚类过程中，由于类 A 和类 B 合并形成了类{AB}，所以在类别层次中节点{AB}是节点{A}和{B}的父节点。

层次聚类算法因第一步中{A}和{B}合并方法的不同而有所区别。当它们合并之后，就要计算其他文档与新形成的类{AB}之间的相似度。

1. 单链接聚类

两个类之间的相似度取两个类中任两篇文档之间相似度的最大值，两篇文档各来自不同的类。因此，对于含有 8 篇文档的类 A 和含有 10 篇文档的类 B，它们之间相似度的计算过程为：首先计算 A 中 8 篇文档与 B 中任意一篇文档的相似度，再从这些相似度中选取最大值作为 A 和 B 之间的相似度。

2. 完全链接

类间相似度取两个类中任两篇文档之间相似度的最小值，这两篇文档分别来源于两个类。

3. 组平均法

在组平均法（Group Average）形成的类中，类内某成员比其他成员间的相似度均值比该成员与其他类的成员间的相似度均值大。要确定一个节点是否属于某一类时，需要计算该节点与其他所有节点的平均相似度。当该节点加到某一类后其平均相似度比该节点加到其他任一类后 108 的平均相似度都大时，该节点即归入到该类中。

4. 沃德法

合并算法在合并两个类时，总是尽量使每篇文档到类中心的距离之和的增量最小[El-Hamdouchi 和 Willett，1986]。类中心是用向量表示的类的平均向量。假设向量 $\boldsymbol{D}_i=<t_1, t_2,\cdots, t_n>$表示第 i 篇文档，$\boldsymbol{C}=<c_1, c_2,\cdots, c_n>$表示类中心，则中心向量的第 j 个分量即为所有文档向量的第 j 个分量的平均值：

$$c_j = \frac{\sum_{i=1}^{n} t_{ij}}{n}$$

因此，如果类 A 与 B 或 C 合并的话，先要计算出 AB、AC 的类中心以及类中任一文档到类中心的最大距离。最后选择最大距离最小的类进行合并。

5. 层次聚类算法分析

文献[El-Hamdouchi 和 Willett，1989]描述了前面介绍的 3 种层次聚类算法的实现，并发现：沃德法（Ward's method）执行时所耗时间最长，单链接聚类与完全链接聚类算法的执行时间相差不大。

文献[Burgin，1995]对几种不同的聚类研究进行了概述。在大多数研究中，单链接聚类算法形成的类比较分散、效果不好。但从其中选择最好的类作为相关文档的来源与完全链接聚类、沃德法、组平均法效果很相近。单链接聚类算法效果的持续降低需要引起注意。

3.2.3 不采用预定义矩阵的聚类方法

目前，也存在不需要相似度矩阵（一个 $N\times N$ 阶矩阵，其中每个元素表示两篇文档间的相

似度）的聚类方法。这些方法依赖于输入文本的顺序，相同的文档集如果输入顺序不同，其结果也是不同的。

1. 单遍聚类法

单遍聚类法（One-Pass Clustering）仅访问一遍整个文档集。首先假设输入的第一篇文档是一个大小为 1 的类。当有新文档输入时，计算新文档与所有已存在的类之间的相似度，这个相
109 似度即为新文档与类中心之间的距离。当新文档与某类之间的相似度超过一定的阈值时，该文档并入该类。这个方法与文档的输入顺序相关性极大，顺序为 1, 2, …, 10 的输入与其他（10!−1）种输入产生的结果相差很大。

由于聚类后可能会形成极其庞大的类，所以有必要将其分为规模更小的类。同样，太小的类也应该合并。

2. Rocchio 聚类

Rocchio 于 1966 年提出了一种聚类算法[Rocchio，1966]。在该算法中，扫描所有的文档并将其分为已聚类的（clustered）和分散的（loose）。当扫描到新文档时，先假设其为类的中心，以检测其文档密度，如果有 n_1 篇文档与它的相似度不小于 p_1、有 n_2 篇文档与其相似度小于 p_2，则该文档符合作为类中心的条件。Rocchio 最常用的相似度度量方法是余弦。如果新文档符合作为类中心的条件，则该类的中心改为该新文档，类中原先的文档根据与新中心的距离确定是否仍保留在类中。该新文档标记为已聚类的。

如果文档超出了阈值，则其状态由已聚类的变为分散的。扫描完所有文档后，仍会有标记为分散的文档存在，这些文档会加入到一个类中，该类中文档到类中心的距离是所有类中最小的（这一步与单遍聚类方法相同）。

在 1971 年，Grauer 和 Messier 在他们的文章[Grauer 和 Messier，1971]中介绍了 Rocchio 算法的几个参数。

- 每个类的最大和最小文档数量。
- 文档不属于某一类的相似度下限。在最后处理未聚类的文档阶段，会使用这个下限。
- 稠密度参数（n_1, n_2, p_1, p_2）。
- 相似度。

3. K-Means 算法

目前应用非常广泛的 K-Means 算法是一种划分算法，将 k 个中心（centroids）进行移动，反复迭代，直到达到终止条件为止。通常，第一次迭代时随机选择 k 个中心，当文档离某一类的中心最近时，将其加入到该类中，然后重新计算中心。算法终止的条件是：中心在两次迭代
110 间的变化小于用户定义的阈值，或在一次迭代中达到了一定的信息增益[Willett，1990]。

4. Buckshot 聚类算法

Buckshot 聚类算法的时间复杂度为 $O(kn)$，其中 k 是类的数量，n 是文档的数量。在聚类类别数目要求较少的应用中，聚类的时间复杂度接近于 $O(n)$。其他算法需要计算文档两两之间的相似度矩阵，时间复杂度为 $O(n^2)$，因此，Buckshot 聚类算法对时间性能的提升是非常明显的。

Buckshot 聚类算法首先随机选择 $\sqrt{kn}$ 篇文档作为样本，然后采用层次聚类算法（任何一种聚类算法都可以）对这 $\sqrt{kn}$ 篇文档进行聚类。通过这种方法，可以从聚类层次中的某一层找到 k 个类。层次聚类算法需要使用文档间的相似度矩阵，所以这个过程的时间复杂

度为 $O(\sqrt{kn^2}) = O(kn)$。确定了这 k 个类的中心后，扫描其余的文档，并且根据新加入的文档与 k 个中心的相似度将其聚到相应的类别中。计算 k 个中心的时间复杂度为 $O(kn)$，而将文档聚到 k 个类的时间复杂度也是 $O(kn)$，所以整个算法的时间复杂度为 $O(kn)$。需要注意的是，Buckshot 聚类算法每次执行的结果会不同，因为在计算 k 个中心时所选择的样本是一个随机的文档集合。Buckshot 算法的详细介绍及其分析请参见文献[Cutting 等人，1992]。

5. 非负矩阵分解

最近，有研究人员提出用非负矩阵分解（NMF）方法进行聚类。非负矩阵分解方法提供了一个隐性语义空间（见 2.6 节），空间中每一维代表一个类的主题。文档表示为该空间中的多维向量，其投影值最大的维度，即为其所对应的类别[Xu 等人，2003]。

3.2.4 在层次聚类结果中进行查询

一旦生成了层次聚类结果，在信息检索的时候，我们必须确定应该搜索层次中的哪一个部分。自顶向下的搜索从树根开始，搜索时计算查询向量与每一个子树中心的相似度，然后对相似度最大的子树展开搜索。如此循环执行，直到搜索到树叶或者聚类类别的大小小于预定义的阈值为止。

自底向上的方法从树叶（即文档）开始向上搜索。早期的研究表明：从包含小类[①]的树叶开始搜索比从大类开始效果要好。文献[Willett，1988]研究了 3 种不同的自底向上的方法，其主要思想如下所示。

111

- 如果有一篇相关文档，那么从该相关文档所在的类开始搜索。
- 如果没有相关文档，那么首先采用标准向量空间模型在整个文档集中进行查询，并且假定排名靠前的文档是相关文档，然后从包含这部分文档的类开始搜索。
- 从最底层类（该类的中心与查询的距离最近）开始搜索。

一旦确定叶子节点或者最底层类，则其父节点相继加入到结果集中，直到结果集的大小达到一定的阈值。

将上述 3 种自底向上的方法与一种更简单的方法进行比较，该方法仅搜索最底层的节点。将最底层类的中心与查询进行比较并获得排名靠前的 n 个类，然后对这 n 个类进行扩展生成结果集。

3.2.5 效率方面

虽然本章的重点是检索效果，但是因为聚类算法的效率导致其应用的局限性，我们不得不大概考虑一下其效率问题。许多聚类算法一开始都需要计算文档两两之间的相似度矩阵。比如对于包含 1 000 000 个文档的文档集，其相似度矩阵有 $\frac{1\,000\,000^2}{2}$ 个元素。文献[Voorhees，1986]提出了改进聚类效率的算法，但是目前 TREC 任务中还没有人对整个文档集进行聚类。

1. 并行文档聚类算法

另一种提高聚类算法运行时性能的方法是使用并行处理器（见第 7 章）。最早的并行聚类算法是在数字阵列处理器（Digital Array Processor，DAP）上实现的，该工作采用了并行处理来提高聚类算法的运行效率 [Rasmussen 和 Willett，1989]。这些算法在由 Torus 网络连接的并行机器

① 指类中文档数比较少。——译者注

上实现[Ruocco 和 Frieder，1997]。Buckshot 聚类算法（见“Buckshot 聚类算法”一节）的并行版本在一个 16 台工作站的网络中达到了接近线性的处理性能。这使 Buckshot 算法可以扩展到更大规模的文档集上，同时层次聚类算法有了可并行执行的版本[Jensen 等人，2002]。也有一些研究工作专注于并行层次聚类算法[Zhang 等人，1996；Guha 等人，1988]，但是这些算法要么
112 需要耗费巨大的运算量，要么没有在文档聚类中进行实际评测。还有一些研究工作着重发展层次文档聚类的并行算法[Olson，1995]，然而这些算法只适用于几种专用连接的网络，对于目前通用的简单总线连接的并行网络架构是否可用还不明确。

还有些研究者提出将聚类作为帮助实现相关反馈的策略[Lu 等人，1996]。只对查询检索出的结果（一个小得多的文档集）进行聚类，然后将从大类中获取的新词作为相关反馈。

2. 使用缩略文档向量进行聚类

聚类过程中最耗时的步骤是计算新文档与所有已存在类之间的距离。此步骤一般的作法是：先计算出每个类的中心向量，再计算新文档向量与每个类中心向量的夹角的余弦，并将其作为相似度。后来的一些工作表明：向量只选用文档中几个有代表性的词，也能够实现聚类，参见文献[Schutze 和 Silverstein，1997]。

减小文档向量大小的一个方法是使用隐性语义索引（见 2.6 节）来判别最重要的成分。另一种删减文档向量更简单的方法是去掉权重低于一定阈值的词。与不删减的文档向量相比，通过隐性语义索引方法，采用 20、50、150 个词，或者通过简单删减方法采用 50 个词来作聚类，其效果并没有显著性的差别。

3.3　基于段落的检索

基于段落的检索算法[Callan，1994]的前提是：相关文档只有很小的部分（即文档内的相关段落）包含与查询相关的信息。因此在计算整篇文档与查询的相似度时，文档中的噪声部分（即不相关部分）可能会掩盖相关的片段。

以本书为例，本节是唯一与查询为“基于段落的检索”相关的节。如果将本书看做一篇文档，本节对全书与查询之间相似度的影响极小。

因为文档一般都自然地划分为章、节以及小节，所以在初始查询时，充分利用这些划分并进行简单的排序是合理的。而相似度系数的计算必须要将基于段落检索的结果进行“综合”。

假设一篇文档 D_1 包含 4 部分：A、B、C 和 D，而 C 是文档中与所有查询内容相关的唯一一部分。由于 A、B 和 D 包含的多数词项与查询不匹配，所以文档 D_1 与查询 Q 的相似度 $SC(Q, D_1)$在极大程度上导致不相关。考虑到文档长度和相对很小的匹配词数量（即便是语义相关的词
113 数量也很小），所以文档与查询的相似度很小，这个相似度正反映了上述推理。在基于段落的检索中，先分别计算 4 个相似度：$SC(Q, A)$、$SC(Q, B)$、$SC(Q, C)$和 $SC(Q, D)$，然后合并这 4 个不同的相似度。我们稍后会介绍这种合并的几个不同技术。

基于段落的聚类研究主要关注如何划分段落以及如何将每个段落相似度合并成一个。下面的各节将针对每一个问题进行讨论并演示各领域早期的相关工作。

3.3.1　基于标记的段落划分方法

基于标记的段落划分方法利用段落标题、段落缩进以及段落间的间距来划分段落。文献

[Zobel 等人，1995]使用了从美国联邦纪事的长文档中提取的 SGML 标记。

在一些类似的工作中使用了段落标记。为了避免出现特别长或特别短的段落，长段落根据大小分割成多个小段，而小段落则被合并到一起。在文献[Callan，1994]中，对段落的界限进行了限定：最少包含 50 个词，最多不超过 200 个词。在文献[Zobel 等人，1995]中则是不断地合并段落，直到超过预定义的阈值 p 为止。这两个文献的工作对检索效果都有所改进，但是在联邦纪事中得出的结论要谨慎看待，因为这个特殊的数据集上的相关文档相对较少。这个方法直观上感觉很好，但是其实际效果有限，其主要原因是：文档作者本身就会很容易在段落标记和章节标记方面犯错，并且我们可能很难进行较好的语义划分(即一个段落可能会描述很多个概念)。

3.3.2 动态段落划分方法

动态段落划分使用多种不同的方法自动发现好的划分。这些方法根据各个特定的查询对文档进行不同的划分[Callan，1994]。其中一个方法就是在文档中查找能匹配查询的词，然后围绕这个词划分出一个段落。如果该词在文档中的位置为 n，则段落 A 从 n 开始到 $n+p$ 为止，其中 p 是一个可变的段落长度；下一个段落 B 与 A 重叠，并从 $n+\frac{p}{2}$ 开始。图 3-3 展示了重叠和没有重叠的段落之间的区别：与查询相匹配的词的位置是 10，给定一个较小的段落长度为 50，则在该匹配词前后的段落有[10, 60]、[35, 85]、[60, 110]，等等。其中[i, j]表示段落由位置 i 开始，在位置 j 结束。重叠段落的目的是避免将相关文本切割成不同的部分。 114

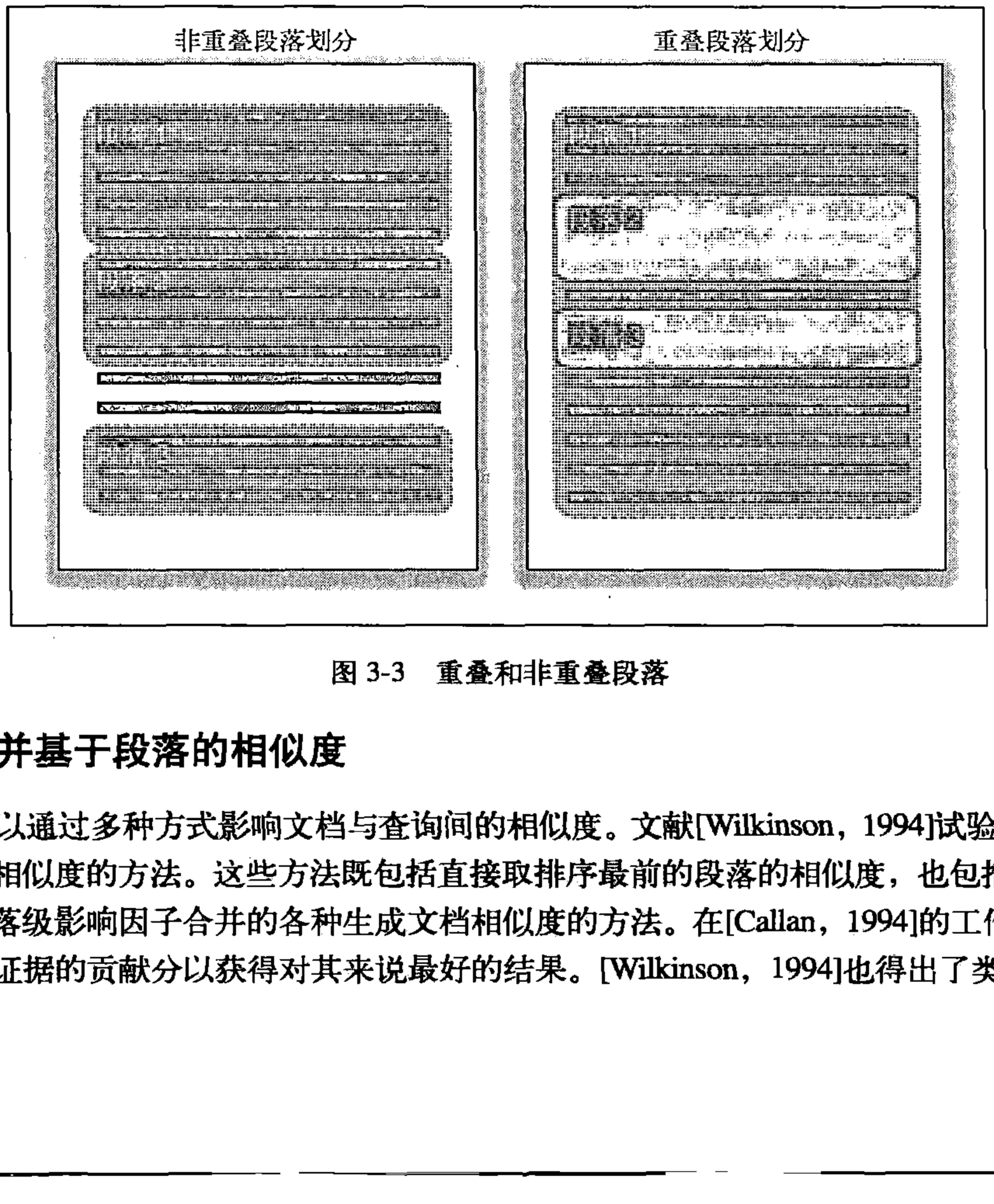


图 3-3 重叠和非重叠段落

3.3.3 合并基于段落的相似度

段落可以通过多种方式影响文档与查询间的相似度。文献[Wilkinson，1994]试验了 20 种不同的合并段落相似度的方法。这些方法既包括直接取排序最前的段落的相似度，也包括将文档级影响因子与段落级影响因子合并的各种生成文档相似度的方法。在[Callan，1994]的工作中，使用文档和段落级证据的贡献分以获得对其来说最好的结果。[Wilkinson，1994]也得出了类似的结果。

3.4　n元语法

基于词的搜索技术一般使用倒排索引或扫描文本（倒排索引的创建和搜索的具体介绍见第5章）。另外，查询时进行词的准确匹配对于有错误的文档集来说效果较差，这主要是没有考虑
115 错误的原因——OCR错误或者拼写错误。为增加信息检索应对噪音的能力，引入了n元语法。首先将词分解为n个字的片段，然后设计匹配算法，以判断是否匹配这些片段。

n元语法也用在拼写错误的检测与更正方面[Pollock和Zamora，1984；Thorelli，1962；Zamora等人，1981]，同时也可用于文本压缩[Yannakoudakis等人，1982]。关于拼写自动更正技术的综述请参见文献[Kukich，1992]。另外，n元语法还可以用来判别文章的作者[Kjell等人，1994]。基于n元语法的传统信息检索算法可参见文献[D'Amore和Mah，1985；Damashek，1995；Pearce和Nicholas，1993；Teuful，1988；Cavnar和Vayda，1993]。

3.4.1　D'Amore与Mah方法

最初信息检索中关于n元语法的研究请参见文献[D'Amore和Mah，1985]。他们研究n元语法的动机是：潜在的未知词无穷无尽，因此我们很难基于词建立数学模型。采用n元语法，给定n的值，我们就可以只考虑固定大小的n元组。因此，我们就可以建立数学模型来评估出索引中的噪声，并确定合适的文档相似度计算方法。

D'Amore和Mah方法在向量空间模型中采用n元组来代替词，我们需要做的事情就只是计算每个n元组的权重。不是只使用词频，而是引入了比例法（scaling method）归一化文档长度。D'Amore和Mah这样做的根据是：长文档比短文档包含更多的n元组，所以应该根据长度来压缩计算。

D'Amore和Mah利用n元组在一篇文档中出现的次数来计算其权重。首先简单地假设n元组以等概率出现并符合二项分布，即n元组“ABC”发生的概率不会大于“DEF”的概率。目前被普遍认可的在以词为单位的处理中存在的齐普夫分布在n元组中不再存在。D'Amore和Mah注意到：n元组不是以等概率出现的，但是从文档集中去掉高频词后，n元组比词更接近于二项分布。

在一篇特定的文档中，D'Amore和Mah计算出n元组出现次数的期望值。该期望值是文档中所有n元组数量（即文档长度）与该n元组出现概率的乘积，n元组出现的概率为其出现次数
116 与文档中所有n元组数量之比。D'Amore和Mah继续应用二项分布以获得期望的方差，接下来可以算出n元组的标准方差。文档j中的n元组i的最终权重为：

$$w_{ij}=\frac{f_{ij}-e_{ij}}{\sigma_{ij}}$$

式中　f_{ij}——文档j中的n元组i的频率；

e_{ij}——文档j中n元组i出现次数的期望值；

σ_{ij}——标准方差。

n元组的权重表示标准差与期望值的差距，目的是给出现次数远大于预期的n元组赋予高权重，而对出现次数与期望相同的n元组则赋予低权重。

D'Amore 和 Mah 做了多次实验以验证二项分布模型适应于 n 元组。遗憾的是：在大型标准语料库上，他们并没有用这个方法实现基于词的方案，以进行对比测试。

3.4.2 Damashek 算法

Damashek 在 D'Amore 和 Mah 研究的基础上进行扩展，使用基于五元组的相似度度量方法[Damashek，1995]。Damashek 的算法基于向量空间模型，但是在计算相关性时采用了另一种方法。我们通常会使用停用词表和词干还原来对出现的 n 元组去噪实现标准化，与此不同的是，Damashek 引入中心向量来消除噪音。计算查询与文档间相似度时，采用向量夹角的余弦的方法，其公式如下：

$$SC(Q,D)=\frac{\sum_{j=1}^{t}(w_{qj}-\mu_Q)(w_{dj}-\mu_D)}{\sqrt{\sum_{j=1}^{t}(w_{qj}-\mu_q)^2\sum_{j=1}^{t}(w_{dj}-\mu_D)^2}}$$

式中，μ_Q 和 μ_D 分别表示查询语言和文档语言的中心向量。权重 w_{qj} 和 w_{dj} 分别表示查询向量和文档向量中相应词的权重。每个 n 元组在中心向量中相应的值为该 n 元组的出现次数与所有 n 元组总数的比值，这与 D'Amore 和 Mah 的方法相同。该值并不作为 n 元组的期望概率，而仅仅表示整个文档集中该 n 元组出现的频率。单文档向量中某特定 n 元组的权重为该 n 元组在该文档中出现的次数与该文档中所有 n 元组的总次数之比。这种“文档内频率”使用文档长度进行 117
归一化，而中心向量则采用整个文档集上 n 元组的总数进行归一化。

在这里，我们不需要去除停用词，也不需要支持词干还原（理论上来说，停用词已由中心向量表示，所以不必再去除），因此，该算法仅需要对文档进行全部扫描，不用进行任何语言解析而获取 n 元组。这使得该算法是语言无关的。另外，中心向量提供了一种文档中“普通”n 元组的滤除方法。我们将保留下来的 n 元组逆向添加到词项集中，自动识别为关键词，用来表示文档。这种逆向处理过程的详细介绍，请参见文献[Cohen，1995]。采用 Damashek 算法，分别测试了英语、德语、西班牙语、格鲁吉亚语、俄语、日语，从而证明了算法的语言无关性。

3.4.3 Pearce 与 Nicholas 方法

作为 Damashek 研究的扩展，Pearce 和 Nicholas 使用 n 元组生成超文本链接[Pearce 和 Nicholas，1993]。通过计算已选择文本块与其余文档之间的相似度来获得链接信息。当用户选择一个文本块后，检测其中的五元组，并采用这些五元组来生成表示该文本块的向量。然后，计算该向量与其余文档之间的余弦相似度，排名靠前的文档自动推荐给用户，如同动态定义了其中的超文本链接。该方法对 Damashek 工作主要的提升地方是：在超文本周边，实现了相关的用户接口。构建向量以及使用中心向量来消除噪声的基本思想并没有变化。

3.4.4 Teufel

Teufel 也使用向量空间模型和 n 元组来计算相似度[Teuful，1988]。停用词和词干还原算法作为一种消除 n 元组集合噪声的较好方法，在 Teufel 的思想中仍然提倡使用。然而 Teufel 与其他方法的不同之处在于：其相似度度量方法会强制增强类似文档间的相似度。也就是说，如果文档 A 与文档 B 相似，而文档 B 与文档 C 又相似，那么文档 A 与文档 C 也大致相似。典型的相似度[比如内积、Dice、Jaccard（见 2.1.2 节）]都是非传递性的。Teufel 使用了一个新的度量系

数 H：

$$H = X + Y - (XY)$$

其中 X 是直接相似度（在此例中使用的是 Dice 方法，但是 Jaccard、余弦、内积也适用于上式），
118 Y 是一个“非直接的”相似度，以使 H 有传递性。通过非直接的相似度度量方法，文档 A 与文档 C 就相似了。关于非直接相似度度量方法的具体描述，请参见文献[Teuful，1991]。

在 INSPEC 文档集上，这种方法获得了较好的准确率与召回率。尽管该算法依赖于词干还原和停用词表，Teuful 仍宣称其算法具有语言独立性。

3.4.5 Cavnar 和 Vayda

还有一些研究者也使用了 n 元语法[Cavnar，1993；Cavnar 和 Vayda，1993]。他们主要使用 n 元语法来识别邮政地址，因为 n 元语法对地址中的错误有较好的还原能力。这里使用了一个简单的扫描算法对匹配的 n 元组进行计数，该匹配是指查询与文档中单行文本之间的匹配。该算法没有使用任何权重，但是因为以单行文本为单位进行处理，所以不需要对文档长度进行归一化。该算法适应于文档中的相关部分出现在单行文本中的情况。

Cavnar 的方法是目前唯一记录在案的在大规模标准化语料库上进行测试的解决方案。在整个 TIPSTER 文档集上，平均准确率在 0.06 到 0.15 之间。但是值得注意的是，在文档集的 AP 部分，平均准确率达到了 0.35。因为在 AP 集上取得的成果，Cavnar 没有进行再深入地调整算法，然而遗憾的是，在整个文档集上其算法的表现相对较差。对于这个结果，作者们说：“为什么在 AP 集上算法的恢复性能与其他文档集有如此差别，目前还不明确。”

3.5 回归分析

另一种估计相关概率的方法是引进一些变量，这些变量描述查询与相关文档匹配的特征。然后，回归分析用来确定与训练数据匹配的确切参数。例如，要通过一个人的年龄来预测其预期寿命，可以通过已知数据确定一个等式：

年龄	预期寿命
45	72
50	74
70	80

我们可以用最小二乘多项式回归方法来确定 α 和 β 的值，然后根据下式，利用年龄（A）来预测预期寿命（LE）：

119 $$LE = \alpha A + \beta$$

根据上式，已知一个年龄，就可以预测其寿命。现在如果要预测一个人有心脏病的概率，我们可能会获得以下数据：

年龄	预期寿命	心脏病
45	72	是
50	74	否
70	80	是

我们根据以上数据点描出一条直线或曲线，如果有一个新人出现并要求预测其得心脏病的概率，那么我们需要检测曲线上对应于其年龄的点。这个例子与文档检索更相似。因为在文档检索中，我们要确定“查询-文档”匹配的特征以判断文档是否相关。这里有一个问题要注意，训练数据中相关性一般只用一个二进制位（1或0）表示，而很少用像人所理解的文档相关的“程度”来表示。在前面的例子中，我们有一个基本的独立性假设：年龄与预期寿命不相关（这个假设对该例子来说是错误的）。逻辑回归分析一般用于估计二分变量，即值域是一个小集合的变量（即性别、是否有心脏病、是否相关文档）。

现在我们回到信息检索，主要问题是确定能表征文档相关性的变量。

匹配的词数	查询大小	文档大小	相关?
5	10	30	是
8	20	45	否

文献[Fontaine，1995]中使用的6个变量如下：

- 查询中匹配词数的平均值；
- 查询中词数的平方根；
- 文档中词匹配词数的平均值；
- 文档中词数的平方根；
- 匹配词的平均 *idf*；
- 查询中匹配词的总数。

文献[Cooper 等人，1992]给出了多项式回归的简要概述以及逻辑回归的应用。然而，使用逻辑回归时要求用于分析的变量是相互独立的。因此，我们把逻辑回归分析分为两个阶段。首先，我们要确定由相互独立的变量构成的综合特征。这里，我们假设上述6个变量中的1～3、4～6个分别构成两个综合特征。逻辑回归的两个处理阶段如下所示。 120

第一阶段：

对每个综合特征进行逻辑回归分析：

$$\lg O(R \mid C_1) = c_0 + c_1 X_1 + c_2 X_2 + c_3 X_3$$
$$\lg O(R \mid C_2) = d_0 + d_1 X_4 + d_2 X_5 + d_3 X_6$$

其中，系数 c_0、c_1、c_2、c_3 用于估计综合特征 C_1 的相关性。同样，d_0、d_1、d_2、d_3 用于估计 C_2 的相关性。

第二阶段：

在分阶段逻辑回归分析的第二阶段，我们要改正由综合特征数量所导致的错误。随着综合特征的增加，错误概率也会增大。假定有 N 个综合特征，则逻辑回归需要计算：

$$\lg O(R \mid C_1, C_2, \cdots, C_N) = e_0 + e_1 Z + e_2 N$$

其中 Z 可以由综合特征的和得到，也可以根据下式得到：

$$Z = \sum_{i=1}^{N} \lg O(R \mid C_i)$$

第二阶段回归分析要用到第一阶段得到的结果。需要注意的是，再增加其他阶段进行更深入的分析也是可能的。

上述回归分析完成后，实际计算相似度的处理就很快了。综合特征只与词在文档中出现与否有关，所以可以预先计算。对查询与文档中匹配数量的计算可以在查询时进行，但该计算涉及各个系数，这些系数在逻辑回归分析时结合预先计算的查询段来计算。更深入的实施细节请参见文献[Fontaine，1995]。

该方法的问题是各系数能否在通用方式下计算，而通用方式可以适应文档集的变化。该方
121 法比较吸引人的方面是通过实验可以确定最好的综合特征，也可以避免基本的独立性假设；而且，该方法能够改正由初始的逻辑回归引起的错误。

3.6　同义词表

使用同义词表来提升信息检索系统的效率是最直观的改进方法中的一个。甚至从 20 世纪60年代第一个信息检索系统出世以来，研究人员就开始关注加入同义词表来改进准确率和召回率。使用同义词表扩展查询的过程见图 3-4。

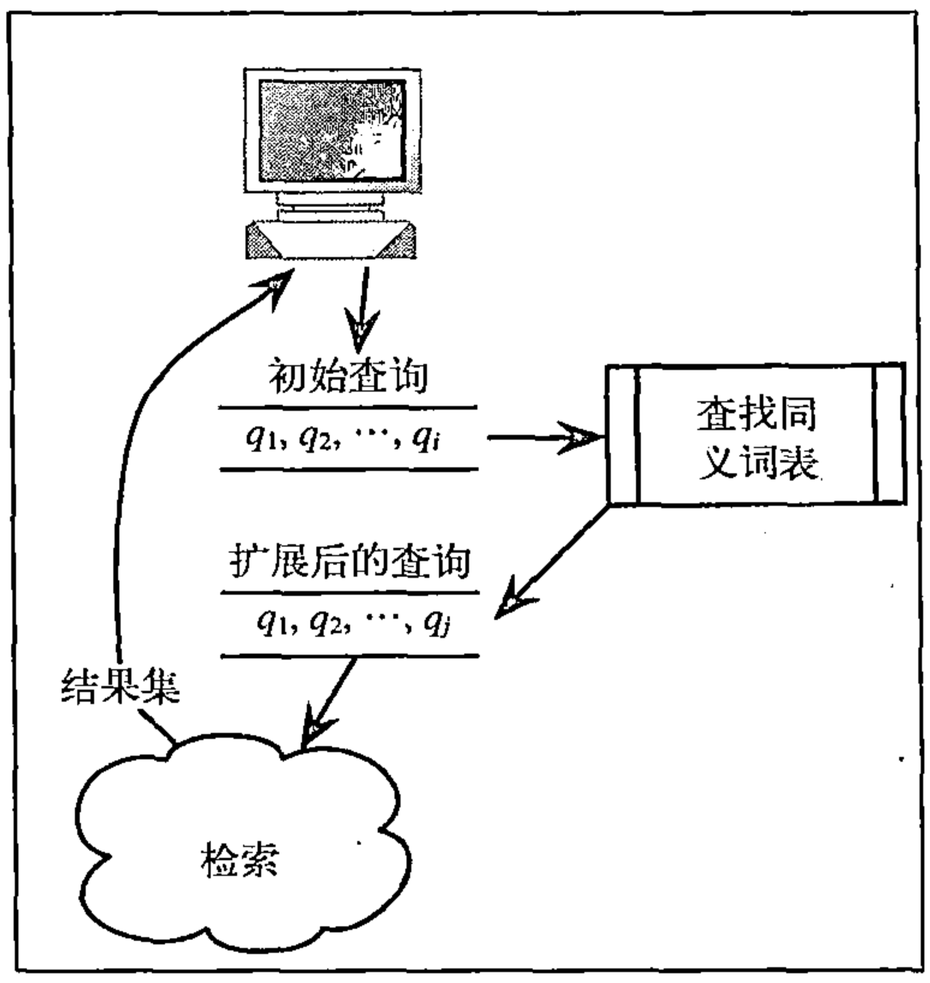


图 3-4　使用同义词表扩展查询

乍看上去，同义词表可能会用于辅助解决一个关键问题——两个人很少用相同的词（即一个人说他去参加“party”，而另一个可能会说去“gathering”）描述同一个概念。当语义相同的词恰好在某个特定句子中意义不同时，依赖于查询词与文档词匹配数量的统计方法就不是很好用。例如，查询与“dogs”相关的信息时，可能也会对与“canines”相关的文档感兴趣。

122 与查询相关的文档可能与查询中的任何词都不匹配。通过同义词表，可以用一个普通词表示所有的同义词，也可以将一个查询扩展成包含所有的同义词。直观上看，这样应该会得到较好的性能，但是结果并不尽如人意。本节主要介绍两方面的内容：一是人工构建同义词表，这是非常耗费人力的方法；二是解决信息检索领域最具挑战性的问题——自动构建同义词表。

3.6.1　自动构建同义词表

人工创建的同义词表对通用词的覆盖较好，但它缺乏面向专门领域的词。比如，一个医学

文档集合中的许多词在通用同义词表中就不会出现。为了避免人工构建无穷无尽的特定领域同义词表，出现了自动构建同义词表的方法。

1. 词共现度

关于自动创建同义词表的早期研究请参见文献[Salton，1971c]。该方法的关键是将每个词表示为向量，然后使用相似度来比较词语，而相似度可以是欧几里得距离或向量间的夹角。

要构建一个给定词 t 的同义词表，与 t 相关的词 u 要满足：$SC(t,u)$大于预先定义的阈值。这里需要注意的是：该计算的时间复杂度为 $O(t^2)$[①]，所以在构建相关词列表时，我们通常要限制给定词的范围。这种限制主要是指：给定词的词频不能太大以至于成为停用词，但是词频又不能太小，以致小到几乎没有多个同义词。

现在来看下面的例子：

D_1：“a dog will bark at a cat in a tree”

D_2：“ants eat the bark of a tree”

词-文档共现矩阵的结果见表 3-1。

计算词 i、j 间相似度的方法为：从数量为 N 的文档集中分别为词 i、j 计算一个 N 维向量。该向量对应于表 3-1 中的一行，这样，“bark”与“tree”之间的内积相似度为：

$$SC(\text{bark,tree}) = <1 \quad 1> \cdot <1 \quad 1> = 2$$

对应于表 3-1 中词之间的相似度矩阵如表 3-2 所示。

表 3-1 词-文档共现矩阵

term	D_1	D_2
a	3	1
ants	0	1
at	1	0
bark	1	1
cat	1	0
dog	1	0
eat	0	1
in	1	0
of	0	1
the	0	1
tree	1	1
will	1	0

表 3-2 词间相似度矩阵

term	a	ants	at	bark	cat	dog	eat	in	of	the	tree	will
a	0	1	3	4	3	3	1	3	1	1	4	3
ants	1	0	0	1	0	0	1	0	1	1	1	0
at	3	0	0	1	1	1	0	1	0	0	1	0
bark	4	1	1	0	1	1	1	1	1	1	2	1
cat	3	0	1	1	0	1	0	1	0	0	1	1
dog	3	0	1	1	1	0	0	1	0	1	1	1
eat	1	1	0	1	0	0	0	0	1	0	1	0
in	3	0	1	1	1	1	0	0	0	0	1	1
of	1	1	0	1	0	0	1	0	0	1	1	0
the	1	1	0	1	0	0	1	0	1	0	1	0
tree	4	1	1	2	1	1	1	1	1	1	0	1
will	3	0	1	1	1	1	0	1	0	0	1	0

① 这里的 t 指的是词表总数。——译者注

相似度矩阵是对称的，即 $SC(t_1, t_2)$与 $SC(t_2, t_1)$是相同的。前提是这两个词相似或者与它们所伴随的词相关。在上面的例子中，“tree”和“bark”在两篇文档中共同出现了两次，因此，这个词对的相似度最高。此方法的一个简单扩展是使用词干代替整个词（词干还原的相关内容
123 详见 3.8.1 节）。其中词干还原是重点，经过词干还原后，“cat”与“cats”是相同的。对于相对频率较低的词，在词间相似度矩阵中使用 *tf-idf* 方法可以增大它们的共现度权重。

词间相似度矩阵的早期研究见文献[Minker 等人，1972]。该论文总结了 20 世纪 60 年代使用词聚类方法的大部分工作，并介绍了一些其他的实验[Salton，1971c；Sparck Jones 和 Jackson，1968；Sparck Jones 和 Barber，1971]。这些论文普遍认为词间相似度矩阵是可以构造出来的，然后各种聚类算法就可以构建相关词的类。

构建完类后，就可以使用它们扩展查询。原始查询中的每个词都可以在类中找到，如果某
124 个词出现在某个类簇中，则将该簇中部分或者全部（这取决于阈值）词扩展到查询中去。这期间有大量的工作都专注于两点：一是研究不同的聚类算法，二是研究不同的阈值以确定每个类的词数。最终的结果表明：使用词聚类方法对查询进行扩展并没有提升使用词权重的简单查询的性能。

其他一些关于词间相似度矩阵的研究可参见文献[Chen 和 Ng，1995]。文献[Chen 等人，1995]创建了一部面向专门领域的同义词表，它是关于“秀丽隐杆线虫”的信息，用于支持分子生物学理论。在这里，使用了词与短语来构建词间相似度。权重计算时使用 *tf-idf* 和另一个因子 p_i，其中 p_i表示短语 i 中的词数。因此，一个包含两个词的短语的权重是单个词的两倍。权重的新计算公式如下：

$$w_{ij} = tf_{ij} \times \lg\left(\frac{N}{df_i} \times p_i\right)$$

使用这个新权重，可以产生非对称的词间相似度。因为词 t_i 或 t_j 中的任一个频率较大时，用对称方法计算出的 t_i 和 t_j 间的相似度非常不准确，所以对排序作用不大。非对称方法可以对任意词 t_i（不管频率是高是低）与其他词进行排序。对于列表中的每一个词使用阈值，生成其相关词的列表，该相关列表与该词所在的列表不同——这个方法可应用于所有的词。

$SC(t_i,t_j)$的计算方法如下：

$$SC(t_i,t_j) = \left(\frac{\sum_{k=1}^{n} \min(tf_{ik}, tf_{jk}) \lg\left(\frac{N}{df_{ij}} \times p_j\right)}{\sum_{k=1}^{n} w_{ik}}\right) \times w_j$$

其中 df_{ij}是词 i 与词 j 共同出现的次数。两个额外的权重因子 p_j和 W_j使该方法是非对称的。与前面所述的一样，p_j是词 j 的大小，并且其值较小。在其他权重相同的情况下，相似度 SC(food,apple pie)>SC(food,apple)，因为 *“apple pie”* 的 p_j值大于 *“apple”*。权重因子 W_j在不背离词 i 与词 j 的关系的情况下，增加低频词的权重。W_j的计算方法如下：

$$W_j = \left(\frac{\lg\left(\frac{N}{df_j}\right)}{\lg N}\right)$$

举例来说，有词“york”以及与它有关的词“new”和“castle”。假设词“new”的频率比“castle”高。在其他权重都相同的情况下，W_j会导致如下结果：

$$SC(\text{york, castle}) > SC(\text{york, new})$$

125

上面没有考虑词“york”的频率。问题的关键是：给定一个词（即“york”），我们要生成同义词表或者是相关词列表。当我们派生出“new”的相关词表时，我们很可能会发现词“york”的频率比“castle”要低，所以有：

$$SC(\text{new, york}) > SC(\text{new, castle})$$

注意我们可以用上述方法计算“york”和“castle”的相对频率。在这里

$$SC(\text{new, york}) = SC(\text{york, new})$$

高频率的词“new”掩盖了词“york”与“castle”之间的真正差别——或者至少这是本方法的前提。例如，“new york”被当成短语的可能性比较大，但是使用我们的例子中的方法并不能得到这样的结果。

至此为止，我们已经定义了 $SC(t_i, t_j)$。由于相似度是非对称的，现在我们给出 $SC(t_j, t_i)$的定义：

$$SC(t_j, t_i) = \left(\frac{\sum_{k=1}^{n} \min(tf_{ik}, tf_{jk}) \lg\left(\frac{N}{df_{ij}} \times p_i\right)}{\sum_{k=1}^{n} W_{jk}} \right) \times W_i$$

对于一个给定的词，通过阈值我们只获取排名在前 100 的词作为其相关词，然后呈现给用户。对于较小的文档集，同义词表有助于提升召回率。到目前为止，在自动检索中还没有通用的准确率和召回率测试。

2. 词上下文

与词共现度方法不同，另一些研究者使用词上下文（词周围的内容）构造的向量来表示该词[Gauch 和 Wang, 1996]。这种向量存在的问题是向量不能表示词义的差别。而同义词表将词关联到不同的意思。在下面给出的例子中，“bark”有两个完全不同的意思。典型的同义词表对“bark”的解释是：

bark——surface of tree（树皮，名词）

bark——dog sound（狗叫，动词）

理想情况下，自动生成的同义词表能够区分不同的同义词列表。词间相似度矩阵不能专门区分不同意义的同义词，并且 Gauch 和 Wang 也没有进行这方面的尝试，而是用向量中临近词的相对位置来表示该词[Gauch 和 Wang，1996]。

126

在这里，判断词相似的关键不再是它们同时出现在相同的文档中，而是出现在相同的上下文中，也就是说与它们临近的词非常相似。

“Bark”声音方面的意思是狗吠，与作为“树皮”意思的“bark”会有不同的上下文。我们来看下面的 3 个句子：

S_1：“The dog yelped at the cat.”

S_2：“The dog barked at the cat.”

S_3：“The bark fell from the tree to the ground.”

在句子 S_1 和 S_2 中，“yelped”与“barked”是同义词，它们出现在完全相同的上下文中。“bark”的另一个意思也出现在这样的上下文中是不可能的。“bark”以“树皮”的意思出现在左侧比两个词的中间的可能性要大得多。

我们需要识别出上下文中的词集合以确定词的上下文。给定目标词周围的这些词是否出现决定了表示该目标词的向量的内容。在文献[Gauch 和 Wang，1996]中，作者假设频率最高的词是最合适的上下文用词，所以他们将前 200 个频率最高的词（包括停用词）作为上下文用词。上下文的窗口大小为 7，这个窗口包含了目标词左侧的 3 个词和右侧的 3 个词。表示目标词 i 的向量格式如下：

$$T_i = < v_{-3} v_{-2} v_{-1} v_1 v_2 v_3 >$$

其中每个向量 v_i（i=−3, −2, −1, 1, 2, 3）是一个 200 维的词向量，表示目标词周围给定位置的上下文。向量 v_{-3} 中的每个元素代表的是这 200 个词出现在目标词左侧的第三个位置上的情况。类似地，v_3 是总共 200 个词在目标词右侧第三个位置的出现情况。

向量 v_i 按前面的格式连接到一起构成表示目标词的向量 T_i。举一个简单的例子，我们从文档集 S_1、S_2、S_3 中建立词“bark”和“yelp”的上下文向量。为了简化起见，我们假设已经对“bark”和“barked”进行了词干还原，并且“the”和“at”分别是上下文向量中仅有的第一个和第二个元素。对于我们的测试文档集，可以得到上下文向量：

127

$$T_{\text{bark}} = [< 00 >< 10 >< 10 >< 01 >< 10 >< 10 >]$$

$$T_{\text{yelp}} = [< 00 >< 10 >< 00 >< 01 >< 10 >< 00 >]$$

因为 S_1 与 S_2 完全匹配，所以上述两个向量非常相似，唯一的不同是由 S_3 中“bark”的其他意思产生的。

本例中将上下文用词出现的次数作为上下文向量的元素。在文献[Gauch 和 Wang, 1996]中，针对出现次数低于期望值的上下文用词，作者加大其权重。其方法是使用互信息作为向量 v_i 的第 j 个分量的实际值。即：

df_{ij} ——上下文用词 j 与目标词 i 共同出现的频率；

tf_i ——数据集合中目标词 i 出现的总次数；

tf_j ——数据集合中上下文用词 j 出现的总次数。

$$v_{ij} = \lg\left(\frac{N df_{ij}}{(tf_i)(tf_j)} + 1\right)$$

如果上下文用词与目标词共同出现的频率高于预测值（该预测值通过这两个词的频率进行计算），那么用这种方法就可以增加该上下文用词的权重。

Gauch 和 Wang 取前 200 个上下文用词，上下文窗口大小为 7，因此每个目标词向量的大小为 1 200。然后对这些向量采用标准余弦法计算它们两两之间的相似度，所有与当前词[①]相似度高于阈值的词都被保留下来。然后在保留下来的目标词中选择最终加入到同义词表的词，这个过程非常困难。经过多次实验后，他们选择了 4 000 个目标词。

高于某一特定阈值的前 n 个词用来扩展查询。遗憾的是，与未扩展的查询相比，经扩展的

① 当前词指本次需要计算其同义词集的词。——译者注

查询的平均准确率并没有提高多少。

对于使用词间相似度矩阵自动构造同义词表一再失败的分析见文献[Peat 和 Willett, 1991]。作者发现，使用词共现度生成同义词表的一个关键问题是频率相对较高的词总是与其他高频词同时出现。结果就是由此构建的同义词表中常用词通常与另一个常用词相关（比如，“hairy”可 128
能会与“furry”相关）。尽管这些常用词确实有相关关系，但是它们不能提高查询的准确率和召回率，因为它们相对较高的频率导致它们并不能提供较好的区分度[①]。

有趣的是，早期的论文显示随机选择扩展词有时比词间相似度矩阵生成的词更有效[Smeaton 和 Rijsbergen，1983]。按照齐普夫分布[Zipf，1949]，大多数词是低频词（超过一半的频率为 1），所以如果随机选择的词频率较低，那么这样做比选择高频但没有区分度的词更有利。

3. 奇异值分解法聚类

Schutze 和 Pedersen 使用词聚类和奇异值分解法来生成同义词表[Schutze 和 Pedersen, 1997]。首先对于频率在 2 000~5 000 之间的词计算出一个矩阵 A。矩阵 A 中包含了这些词在一个大小为 k（这里 k 为 40）的窗口中同时出现的次数。然后，这些词被聚集成 200 个 A-类（聚类方法为组平均法，详见“组平均法”一节）。例如，一个 A-类 g_{A1} 为(t_1, t_2, t_3)，而另一个类别 g_{A2} 可能会是(t_4, t_5)。

接下来，对于频率最高（该频率基于词与类别内所有词共同出现次数）的 20 000 个词生成另一个矩阵 $\boldsymbol{B}$。接上例，如果词 t_j 与词 t_1 共同出现 10 次，与 t_2 共同出现 5 次，与 t_4 共同出现 6 次，则 $\boldsymbol{B}[1, j]$=15，$\boldsymbol{B}[2, j]$ = 6。我们注意到通过聚类减小了矩阵 $\boldsymbol{B}$ 的大小，并且提供了更多的训练信息。矩阵 $\boldsymbol{B}$ 的行对应于 $\boldsymbol{A}$ 中的类别，列对应于词。矩阵 B 的大小为 200×20 000。这 20 000 个列又通过 Buckshot 聚类算法（详见“Buckshot 聚类算法”一节）被聚成 200 个 B-类。

最后，对于数据集中所有的词生成矩阵 $\boldsymbol{C}$。元素 $C[i, j]$表示词 j 与 B-类中第 i 类中的所有词共同出现的次数。生成矩阵 $\boldsymbol{C}$ 后，即对 $\boldsymbol{C}$ 进行奇异值分解产生奇异值来表示该矩阵。这里所说的方法与隐性语义索引（详见 2.6 节）中的技术类似。在这里，因为仅有 200 列，所以奇异值分解更容易进行。

将上下文向量（这些向量对应于奇异值分解中的一列）相加得到的向量表示一篇文档。而上下文向量用于与查询进行比较。

另一种使用上下文向量矩阵的技术是基于查询的上下文向量对其进行聚类。该技术被称为词因式分解（word factorization）。首先将查询划分为 3 个独立的类，然后对每一个词执行查询操作并产生一篇文档，该文档对 3 个类别综合来说是排在最前的。这要求该文档对 3 个类别来 129
说都排在前面，以获得综合排名最高。该方法的前提是查询一般涉及两个或三个概念并且相关文档包含所有概念的相关信息。

总体来说，这个方法非常有发展前景，它运行在一个大小合理的数据集上（在 TIPSTER 的 B 类部分上使用词因式分解，平均准确率从 0.27 提高到了 0.32——整体提高了 18.5%）。

4. 仅使用文档聚类生成同义词表

Crouch 提出了自动构造同义词表的另一种方法[Crouch，1989；Crouch，1990]。首先，通过文档聚类算法将文档集划分为相关的类，这个过程需要计算文档间相似度。文献使用了完全链接聚类算法，但是也可以使用其他聚类算法（关于聚类算法的更多信息，详见 3.2 节）。

然后提取出每个类中的词。由于这些词来自类中不同的文档，要获得给定类中的词集合需

① 也就是说在查询中增加这类词对结果没什么影响。——译者注

要使用不同的运算符。我们来看包含下面所给的词的文档：

$$D_1 = t_1, t_2, t_3, t_4$$
$$D_2 = t_2, t_4$$
$$D_3 = t_1, t_2$$

包含上述文档的类可以表示为所有的词$\{t_1,t_2,t_3,t_4\}$、各文档交集$\{t_2\}$或者其他操作的结果，其他操作指的是取得类别中包含某个词的文档数量。Crouch发现简单的聚类算法效果最好。表示类的词集可以看做同义词表中的一个类，该同义词表通过上述方法自动生成。保留类中的好词，去掉其他的词，这个处理使用一个基于文档频率（关于文档频率的具体介绍请参见2.1节）的词区分函数来执行。

对查询的扩展基于同义词表中的类。对于出现在与同义词表类中的词匹配的查询中的任意一个词，将该词所在类中所有词添加到查询中去。在小数据集上的测试结果表明该方法对平均准确率有提高：在小ADI测试集上提高了10%，在Medlars测试集上提高了15%。比较遗憾的是，这两个结果都是在较小数据集上进行的，而文档聚类算法计算代价很大，时间复杂度为
130 $O(N^2)$，其中N是集合中文档的数量。

3.6.2　使用人工构建的同义词表

尽管人工构建同义词表耗时超乎想象，一些研究人员还是探索研究了使用这类同义词表改进准确率和召回率的方法。

1. 使用人工同义词表扩展相关排序

1971年在哥伦比亚大学图书馆服务学院，研究人员开发了使用计算机辅助人工构建同义词表的系统。其算法本质上就是一个简单的同义词表编辑器[Hines和Harris，1971]。

人工同义词表一般用于面向特定领域的同义词表。首先要召集一些专家，请他们确定特定领域词汇之间的关系。Ghose和Dhawle发现构建社会科学领域的同义词表比自然科学领域的同义词表困难得多，原因是社会科学领域的专业词汇有歧义[Ghose和Dhawle，1977]。

文献[Wang等人, 1985]介绍了一系列的人工同义词表（都由学生完成）。这些同义词表都是根据两个词之间的关系生成——比如*dog is-a animal*。将这些同义词表合并成一个，最终得到7组关系，这7组关系如下：

- 反义关系；
- 非反义的其他所有关系；
- 所有关系；
- 部分-整体关系和集合关系；
- 共现关系（co-location）；
- 同类和同义关系；
- 词形变化关系。

反义是指两个意义相反的词之间的关系（例如，白天与晚上），is-part-of表示部分和整体的关系（例如，“车胎”与“汽车”）。共现关系表示的是两个同时频繁出现在一个短语或句子中的词之间的关系。同类和同义关系即同义词。词形变化关系将相同语义核心的不同词关联到一起，比如canine与dog。在一项研究中，在一个小文档集上进行了对单个或所有词添加这些关系的
131 实验，该关系由该研究的领导者判定。该研究使用了除反义关系外的所有关系，取得了最好的

平均准确率与召回率，但是整体性能提升有限。

1993 年，有项研究采用了一部同义词表，该词表包含 3 种不同的关系：等价（同义）、层次（is-a）和关联关系[Kristensen，1993]。在一个由芬兰新闻报纸文章构成的相当大的（227 000）文档集上的测试显示，召回率从 47%上升到了 100%，同时准确率仅从 62.5%下降到 51%。令人欣慰的是该项工作是在大数据集上进行的，然而，为测试而人工构建的同义词表仅包含 1 011 个概念，总共 1 573 个词。测试仅使用了 30 个查询，这显然是针对同义词表选择了效果好的词，才产生了这么好的效果。

众所周知，专业词库是高度专业化的，这个结果与 TREC 会议中人工跟踪的类似，TREC 会议的参加者可以手动修改原始查询以包含更多有区分度的词。使用同义词、意义更少的词以及相关词进行搜索，在召回率为 50%的基础上总是能有 10%到 20%的提升。联合搜索（使用所有的值）的性能提升则更高，其平均准确率能提高 50%。上述研究工作是少数不使用 TIPSTER 集但测试集规模仍很大的研究的一个代表。然而，这不能保证在一个更一般化的数据集上使用更一般化的同义词表仍会取得这么好的结果。

2. 使用人工同义词表扩展布尔查询

前面介绍的所有研究都是使用同义词表改进相关排序的。而 Lee 等人提出了扩展布尔检索模型的方法，该方法在布尔请求中包含同义词表信息[Lee 等人，1994]。扩展布尔模型的具体介绍请参见 2.5 节。通过试验不同的 p 值，发现 p 为 6 时（该值是 Salton 针对扩展布尔检索提出的建议值[Salton，1989]）效果最好。结果显示本方法在效果上有轻微提升。

3.7 语义网络

语义网络的基本思想是：知识可以用通过不同关系相互连接的概念来表示。语义网络就是一个简单的节点和边的集合。边根据其所表示的关系的类型进行不同的标记。节点的实际信息（比如其个性特征：颜色、大小等）通常存储在一个被称为框架的数据结构中。框架中的每个实体称作槽[Minsky，1975]。 132

单词“rose”的框架格式如下：

(**rose**

(*has-color* red)

(*height* 2 feet)

(*is-a* flower)

)

这里框架 *rose* 是语义网络中的一个独立节点，包含到节点 *flower* 的一个 *is-a* 连接。槽 *has-color* 与 *height* 存储“rose”的属性。

自然语言理解系统已发展为读取自然语言文本并建立语义网络来表示文本中包含的知识[Schank, 1975；Schank 和 Lehnert, 1977；Gomez 和 Segami, 1989；Gomez 和 Segami, 1991]。目前的研究表明许多概念极难表示（最棘手的问题通常是由时间或空间引起的）。句子“A rose is a flower.”、“A rose is red.”中包含的信息既容易表示也易于存储，但是语义网络在存储如下信息时有困难：“The rose grew three feet last Wednesday and was taller than anything else in the garden.”在语义网络中存储玫瑰在不同时期的尺寸以及它的相对位置是很困难的。关于这方面的具体讨

论，请参见文献[Lenat 和 Guha, 1989]中的“表示的难题”（Representational Thorns）一节，该节的主要内容是介绍一个被称为 *Cyc* 的大规模知识表示工程（该工程中常识推理的很大一部分是由人工处理的）。

尽管在语义网络中存储复杂知识存在一些问题，但还是有人研究使用何种语义网络来改进信息检索。这些工作虽然成果有限并且严重依赖于语言，但还是存在改进潜力的。

语义网络所要解决的问题是：在文档与查询相关的情况下，查询中的词与文档中的词仍然不匹配的问题。摒弃对构成查询词与文档的特征间的匹配，语义网络计算词间的语义距离并将其加入到网络中。这样做的依据是意思相同的词在语义网络中也会出现在相对较近的位置上。*激活扩散*是计算两个词之间语义距离的一个方法。

同义词表与语义网络存在着很密切的关系。从信息检索系统的立场上看，同义词表使用相
133 关词扩展用户查询以期相关词能匹配文档，也是在解决不匹配的问题。语义网络通过包含表示“is-a-synonym-of”或“is-related-to”的连接可以将同义词表包含进来，但是语义网络可以表示更复杂的信息，比如 is-a 层次关系在同义词表中就无法表示。

WordNet[Beckwith 和 Miller, 1990]作为信息检索研究的应用十分广泛的工具，就是一个语义网络。WordNet 是对公众开放的并包含专门为单词而设计的框架（一些语义网络包含更多具体概念的框架，比如 big-and-hairy-person）。WordNet 的相关资料可以在网页 www.cogsci.princeton.edu/~wn 上找到。

在 WordNet 中包含一个词的多种语义对应的条目。此外还存储了多种单词间的关系，这包括：同义、反义（粗略地看，是指一个词的对立意思）、下义词（词典关系如“is-a”）以及部分名词（part-of）。绝大部分名词是“is-a”层次关系，而反义关系一般用于表示形容词之间的关系。

有意思的是，像使用承继（entailment）和连带（troponyms）这样比较少见的关系表示动词间的关系，用承继关系关联的两个动词表示第一个动词导致第二个动词的发生，如“buy”东西意味着你要为之“pay”，因此“buy”和“pay”用承继关系进行关联。如果两个活动室存在“承继”关系，但又是同时发生的，比如词对（limp，walk），那么这两个活动用“连带”关联。用于搜索 WordNet 的软件的更多描述，请参见[Beckwith 和 Miller, 1990]。

使用 WordNet 中的同义词集扩展查询词可以提高查询的有效性，这在理论上是可行的。然而 Voorhees 在他的早期工作[Voorhees, 1993]中，对查询效果的改进却以失败告终。甚至人工对同义词集进行筛选，扩展后的查询效果也没有提高。失败的关键因素是查询词的专用性，以至于在 WordNet 中经常查不到，比如“National Rifle Association”在 WordNet 中就查不到。另一个失败的原因是，扩展加入的词因为多义性大大降低了查询的有效性。随着 WordNet 的不断改进，更近期的研究通过更细致的选择短语，得到了较小（约 5%）的提升[Liu 等人，2004]。

语义网络用于扩展布尔检索和自动相关排序，我们将在本节接下来的内容中介绍这些方法。

3.7.1　距离计算方法

计算语义网络中两个节点之间的距离时，可以使用激活扩散算法：两个指针分别指向待计算距离的节点，跟踪这两个节点的链接，直到这些链接之间存在交集。我们用这两个节点之间
134 的最短路径来计算距离。注意这里没有使用简单的最短路径算法[①]，是因为同样两个节点间可能

① 直接取两节点直接相连的路径。——译者注

存在多条链接。节点 a 与 b 之间的距离计算公式为：

Distance(a,b) = 连接 a 与 b 的最小边数

1. R-距离

计算两个节点集合之间的距离是十分复杂的问题。理想情况下，两个集合是线性对齐关系。“large rose”与“tall flower”便是这样一个例子：large 与 tall 有可比性，并且 rose 与 flower 也具有可比性。问题是我们很难将所有集合都排列成像上述例子中具有的对应关系。因此，文献[Rada 等人, 1987]中定义的 R-距离利用集合中所有的实体，并取两个集合间所有可能距离的平均值作为集合间的距离。

如果我们将文档视为一系列通过“AND”连接在一起的词的集合，将查询看做是析取范式中的布尔表达式，那么 R-距离就定义了布尔查询与文档间距离的计算方法。另外，将 NOT 用于短语产生的距离是能从短语得到的最深层次的距离[①]。因此，对于查询 Q——((a AND b AND c) OR (e AND f))与文档 D——(t_1 AND t_2)，它们之间的相似度计算如下：

$$c_1 = \frac{d(a,t_1)+d(a,t_2)+d(b,t_1)+d(b,t_2)+d(c,t_1)+d(c,t_2)}{6}$$

$$c_2 = \frac{d(e,t_1)+d(e,t_2)+d(f,t_1)+d(f,t_2)}{4}$$

$SC(Q, D)$为 MIN(c_1, c_2),即为 c_1、c_2之间的最小值。实质上，首先计算查询中的每个短语与整个文档之间的距离，然后取文档与短语间的最小距离作为相似度。

一般地，给定析取范式形式的查询 Q，词集合为($t_1, t_2, \cdots, t_n$)的文档 D，以及表示短语 i 中第 j 个词的 c_{ij}，则 Q 与 D 之间的相似度定义为：

$$SC(Q,D) = \min(SC_1(c_1,D), SC_1(c_2,D), \cdots, SC_1(c_m,D))$$

$$SC_1(c_i,D) = \frac{1}{mn}\sum_{i=1}^{n}\sum_{j=1}^{m} d(t_i,c_{ij})$$

$$SC(Q,D) = 0, \text{如果} Q = D$$

135

2. K-距离

在 R-距离提出之后，文献[Kim 和 Kim，1990]提出了一个被称为 K-距离的距离计算方法。该方法对语义网络中的边赋以不同的权重,首先查找两节点之间的最短路径（同样是使用激活扩散算法），然后将最短路径上的边权重相加即得到节点间的距离。其形式化定义如下：

$$d_{ij} = w_{t_i,x_1} + w_{x_1,x_2} + \cdots + w_{x_n,t_j}$$

上式表示词 t_i 与 t_j 之间的距离，其中从 t_i 到 t_j 的最短路径为 t_i，x_1，x_2，…，t_j。

作者将 NOT 视为特殊情况，具体介绍请参见[Kim 和 Kim，1990]，但其基本思想是：对于连接了被引用为 NOT 的节点的边（一般称为孤立边），在相当大程度上增加其权重。对这些边增加权重后，任何包含这个节点的路径都比不包含这类节点的路径长得多。

要计算两个有带权边的集合 **A** 与 **B** 之间的距离，K-距离算法首先计算集合 **A** 与 **B** 中每个节点之间的最小距离，然后计算这些最小距离的平均值。因为边的权重可能会在不同方向上取不同的值，所以对 **A** 到 **B** 的距离与 **B** 到 **A** 的距离再取平均值。对于前面例子中的查询 Q：

((a AND b AND c) OR (e AND f))

① 即使用了短语的所有可能因素来计算距离。——译者注

假定文档 D 只有两个词：(t_1 AND t_2)，则 Q 与 D 之间的相似度计算如下：

$$c_1 = \frac{\min(d(a,t_1),d(a,t_2)) + \min(d(b,t_1),d(b,t_2)) + \min(d(c,t_1),d(c,t_2))}{3}$$

$$c_2 = \frac{\min(d(e,t_1),d(e,t_2)) + \min(d(f,t_1),d(f,t_2))}{2}$$

$SC(Q,D)$仍取 $\min(c_1,c_2)$，然后计算出 $SC(D,Q)$，最后计算 $SC(Q,D)$与 $SC(D,Q)$的平均值以得到最终的相似度。

以析取范式表示的查询 Q 与词集合为$(t_1,t_2,\cdots,t_n)$的文档 D 之间的 K-距离定义为：

$$SC(Q,D) = \frac{SC_1(Q,D) + SC_1(D,Q)}{2}$$

136

$$SC_1(Q,D) = \min(SC_2(c_1,D), SC_2(c_2,D), \cdots, SC_2(c_m,D))$$

$$SC_2(c_i,D) = \frac{1}{n}\left(\sum_{j=1}^{n} \min(d(c_{ij},t_j))\right)$$

$$SC(Q,D) = 0, \text{如果} Q = D$$

R-距离满足三角不等式，即 r-dist(a,c)不大于 r-dist(a,b)+r-dist(b,c)。而 K-距离不满足三角不等式，但是它利用了语义网络中边的权重。

3. 在信息检索模型中引入语义距离

Lee 等人将语义网络的距离方法引入到扩展布尔检索模型中，并称之为 KB-EBM（Knowledge Base-Extended Boolean Model，基于知识库的扩展布尔模型）[Lee 等人，1993]。其思想是：以 2.5 节所介绍的扩展布尔检索模型为基础，修改权重计算方法，使其包含语义网络中两个节点间的距离。

扩展布尔模型使用函数 F 表示文档中一个词的权重。如前所述，我们用 w_i 表示该权重，但严格来说应该表示为 $F(d,t_i)$。Lee 等人通过语义网络修改这个权重，其他与扩展布尔检索模型保持一致，未作任何变化。这非常简洁地处理了 NOT 的情况。

原始的距离函数 $d(t_i,t_j)$返回的是两个节点之间的最短路径长度，由此可以推导出两个词之间语义概念上的接近程度。这里需要的是两个词之间的概念距离（conceptual distance），与原始距离函数成反比关系。因此，新的 F 函数使用如下的距离公式：

$$\text{distance}^{-1}(t_i,t_j) = \frac{\lambda}{\lambda + \text{distance}(t_i,t_j)}$$

首先，如果给定文档中的词没有权重，函数 $F(d,t_i)$计算词 t_i 到文档中所有其他节点的平均距离，并将其作为文档 d 中词 t_i 的权重。新函数 F 为：

$$F(d,t) = \frac{\sum_{i=1}^{n} \text{distance}^{-1}(t_i,t)}{1 + \frac{\lambda}{\lambda+1}(n-1)}$$

如果文档中的词已有权重，则函数 F 改为包含权重 w_i。w_i是文档 d 中第 i 个词的权重。

$$F(d,t) = \frac{\sum_{i=1}^{n} \text{distance}^{-1}(t_i,t) w_i}{1 + \frac{\lambda}{\lambda+1}(n-1)}$$

137

4. 距离算法评价

所有 3 种距离算法都在 4 个数据集——9 篇、6 篇、7 篇以及 7 篇文档——分别进行了评测。因为评测中没有计算准确率与召回率，所以通过比较每种算法产生的排序来评价算法优劣。有些情况下，使用数据集 MESH（一个医学语义网络），另外一些情况使用 CRCS（Computing Reviews Classification Scheme）。总体来说，因为测试数据集较小并且缺少准确率与召回率，所以评价这 3 个方法十分困难。我们介绍这几种算法是因为它们都有使用语义网络的能力。尽管目前大部分的研究不再关注布尔查询，但是如果把自然语言查询中的各个词看做是布尔 OR，则所有这 3 种算法都是可用的。在更大的数据集上，使用通用语义网络（如 WordNet）对这 3 种算法进行评测将是十分有意义的。

3.7.2 基于“概念”扩展查询词

前面介绍的方法是在语义网络中计算查询词与文档中词的距离，然后把该距离作为一个计算因素，与该方法不同，将语义网络作为一个同义词表，用语义网络中与查询词“相近”的词对其进行替换，然后生成“概念”向量来代表查询，替换掉基于词的向量。这种方法首先出现在 20 世纪 70 年代早期。1988 年，[Giger，1988]使用这种方法对一个已有的布尔检索系统进行改进，并因此提出了一个算法，作者使用“概念”将原先布尔系统中的词进行替换。通过查找语义网络与系统中原有词有连接的节点，获得用于替换的“短语”。该文献使用同义词表而不是网络这个词，但是词间不同关系与我们对于语义网络的定义是一致的。

文献[Chen 和 Lynch，1992；Chen 等人，1993]介绍了一个系统，它使用自动生成的语义网络。网络的生成使用了两种不同的聚类算法，一种是标准余弦算法（见 3.2 节），而另外一种算法由作者提出并能在语义网络中生成非对称的链接。用户可以人工遍历该语义网络以获得针对查询的“好”词，同时该语义网络可以在人工对新文档创建索引时用于发现合适的词。

3.7.3 基于约束激活扩散的排序

1987 年出现了两篇很有意思的论文，之后在基于知识的信息检索研究中被频繁引用[Cohen 和 Kjeldsen，1987；Kjeldsen 和 Cohen，1987]。这些论文介绍了 GRANT 系统，该系统会根据研究主题自动判别出最优的研究资助机构。在该研究中，人工创建的语义网络包含 4 500 个节点（研究主题）和 700 个资助机构，语义网络中的边连接了资助机构与研究主题，其中网络中的连接是基于资助机构感兴趣的研究主题来划分的。 138

对于一个给定的主题，首先激活由该节点传出的链接，开始激活扩散过程。当搜索到资助机构时，这个激活过程停止。在该过程的每一步，激活都要受约束条件的约束。搜索到第一个连接之后，就开始应用 3 个约束条件。第一个是距离：如果路径长度大于 4，则不再继续搜索该路径。第二个约束条件是扇出（*fan-out*）：如果一条路径到达某个节点,该节点出去的链接大于 4 条，则停止对该路径的搜索。这是因为该节点太普通，所以用处不大并且会导致搜索过程在许多用处较小的方向上进行。第三个约束条件是一个给链接打分的规则。该分数被称作支持度（endorsement）。最后，对结果根据这些分数的累加和进行排序。例如，如果一个研究者的研究主题是某个通用领域的子领域或者是具体领域，则通用领域的资助者与该研究主题之间链接的支持度增加。资助数据库系统研究的机构也会资助时序数据库系统的研究。更形式化的描述为：

request-funds-for-topic(x) and IS-A(x,y) → request-funds-for-topic(y)

反过来，如果某个研究主题是资助机构所资助领域的泛化，则其支持度减小。资助数据库系统研究的机构对研究主题计算机科学可能不会有兴趣。

该系统所使用的最佳优先搜索是优先搜索可信度最高的连接。当资助机构的数量达到一定的阈值时，搜索过程结束。GRANT 系统在使用中进行测试，结果表明其优于另一个使用中的简单关键词匹配系统。原先需要几个小时的搜索在这里仅在几分钟内即可完成。在一个包含 23 个查询的小集合上进行了更正式的测试。然而语义网络与文档集都相对较小，所以很难从这个结果中得出一般性的结论。总而言之，GRANT 系统因为使用了语义网络，所以引起很多人的兴趣，但是基于特定领域的规则对其语义网络进行了约束。

3.8　语言解析

使用符号集合来表示文本块是每个信息检索系统都具有的基本特征。仅仅简单罗列查询与文档中所有的符号非常容易导致重大的语义错误。例如，某查询要查询与 computer chips 相关的
139 信息，当其与描述 potato chips 的文档进行匹配时，就会出现语义错误。我们将在 3.8.1 节介绍人工或自动使用简单的语言元符号的方法（单个词），尽管这些方法看起来很简单并且将文本视为简单的词袋（bag of words），但它们一般都很容易实现且高效，而且其效率在 TREC 评测中经常不亚于甚至超过许多复杂的方法。更多关于 TREC 会议的介绍请参见第 9 章。

比单个词更深入的方法是，在文档检索中使用短语来代替词元。短语包含了词袋所不具备的一些语义，并且产生二词词对（一般情况下是多词短语），所以如果要查询关于 New York 的信息将不会出现关于 new Duke of York 的结果。3.8.2 节将介绍短语识别的一些简单方法。

在 3.8.3 节中，我们将介绍更复杂的短语识别方法。这些方法都基于 NLP（自然语言处理）中通用的一些算法，包括词性标注器、语言解析器以及启发式信息抽取。这一节与 NLP 的整个领域都相关，所以其内容要展开来讲可以写成一本书。我们将仅简要概述现有启发式方法，并且重点介绍那些已直接应用于信息检索系统的方法。

总而言之，语言解析是影响系统性能的关键因素。在复杂的 NLP 方法中，语言解析的研究更加深入，但是到目前为止，与简单方法相比，这些方法在性能上并没有质的提升。关于将 NLP 方法应用于信息检索系统的早期工作综述，请参见文献[Lewis 和 Sparck Jones，1996]。

3.8.1　单个词

单个词是最简单的方法，搜索文档时需要人工干预并为每一篇文档赋予一系列的相关词。这个方法存在的问题是为文档选择合适的有区分度的关键词通常很困难，并且，如果系统有分类的话——例如美国国会图书馆的分类体系，就更难将文档归入唯一的一个领域中了。更不必说，人工对文档进行分类的代价极其高昂了。因此，很早人们就发现人工选择关键词并不比自动选择的方法好多少[Salton，1971d]。

在 20 世纪 60 年代早期，人们就已经知道扫描是很好的方法，那么下一步要做的就是对文本内容进行规范化，以避免由前缀、后缀或者大小写引起的低级匹配错误。现在，大多数的信息检索系统都将所有的文本转换为单一的大小写格式（都是大写或都是小写），所以句子中的第一个词不会因为大写而与查询产生低级匹配错误了。

词干还原是通过去掉词的前缀或后缀来规范化单词。其思想是：在查询中包含“throw”的用户应该会希望文档中的“throwing”、“throws”等词也能与之匹配。词干还原算法已有 20 多 140
年的历史，其中 Porter and Loving 算法最为常用[Porter，1980；Loving，1968]。这些算法仅去除掉一些常见的前缀和后缀，但问题是两个看似很不同的词可能会有相同的词干。如果词干还原器对 *“-ing”* 和 *“-ed”* 都一律去除，这就会导致词 *“red”* 和 *“ring”* 的词干只剩下 *“r”*。一种方法是 KSTEM 方法，该方法利用词典以确保生成的每个词干都是有效的词[Krovetz 和 Croft，1989；Krovetz，1993]。另一种方法使用基于语料库的统计方法（本质上基于词的共现度）来识别词干，并且该方法独立于语言[Croft 和 Xu，1994]。在相关排序方面，这些词干还原器比传统的词干还原器表现出了更好的性能。

我们将相关文档中相对没有意义的词称为停用词，它没有存储到索引中。根据文献[Francis 和 Kucera，1982]，停用词大约占整个文档集中词的 40%。去掉停用词可以减少索引创建、时间和空间开销，但可能会降低响应某些查询的能力。例如，对于仅包含停用词的查询短语（如“to be or not to be”）来说，如果去掉停用词就变为空串了。尽管如此，研究者还是在频繁使用停用词列表，并且一些研究的全部内容就是确定好的停用词列表[Fox，1990]。

最后，在处理特殊字符时还要用到其他的语言解析规则，例如在处理像连字符、省略号、逗号等特殊符号时仅使用前面介绍的规则就会出现问题。关于这个问题的一些基本规则请参见文献[Adams, 1991]，但是该文献没有说明其准确率与召回率。许多 TREC 会议论文都介绍了将语言解析器进行删减并做简单修改后，准确率和召回率有所改进（原作者也都承认这个事实）。然而，对基于单个词的语言解析、特殊字符的处理以及这些处理所产生的对准确率与召回率的影响，我们还不是很了解。

3.8.2 简单短语

许多 TREC 系统将任意两个不被停用词、标点符号或特殊字符分隔的词对作为短语。随后，低频率短语被滤除，不进行存储。在许多 TREC 系统中，频率低于 25 次的短语都被滤除。这大大减少了短语数量，从而也减少了存储空间[Ballerini 等人，1996]。

引入短语之后，接下来的问题就是如何将它们用于相关排序中。有的系统仅将短语扩展到查询中，而另一些系统并不这样做，而是在文档长度归一化的计算中不包括这些短语[Buckley 等人，1995]。这样进行归一化的原因是短语中的词已经计算过了。在许多系统上都进行了仅使用短语或词的测试，结果表明短语可以用于扩展词项，但是不能代替词项。因此，查询 *“New York”* 141
应该转换为对 *“new”*、*“york”*、*“New York”* 进行搜索。一般来说，以这种方式使用短语比简单词项在准确率和召回率上会有大约 10%的改进。

3.8.3 复杂短语

自从 20 世纪 60 年代早期开始，就已经有研究者在响应用户查询中应用 NLP 方法。事实上，NLP 系统常常与信息检索系统是完全不同的，这是由于 NLP 系统试图通过建立一个表示文档的正则结构来理解文档内容。使用正则结构的目的是减少语言本身固有的歧义。检索关于 walking 信息的查询应该能匹配这样的文档：文档描述了 are moving slowly by gradually placing one foot in front of the other 的人。

NLP 系统用确定性的正则结构——该结构生成的第一步便是对文档进行句法分析——来识

别文档中的关键元素（主语、谓语、宾语等），以存储关于 walking 和 moving slowly 的信息，然后建立一个单独的结构来表示文档。简单原语——包括一大类动词——由 SChank 提出[Schank, 1975]，例如使用简单原语 PTRANS（physically transport，物理传输），句子 John drove to work 与句子 John used his car to get to work 就有相同的简单结构 John PTRANS work。

尽管 NLP 技术取得了进步，但是知识表示中的许多问题导致构建一个满足需要的正则结构是极其困难的。CYC 项目在过去的 15 年里人工创建了一个知识库，这个过程中很难确定大量文本中知识的准确意义[Lenat 和 Guha, 1989]。

完整的 NLP 系统产生了令人想象不到的效果是：许多在语言理解中效果不理想的工具在信息检索系统中却非常有用。我们可能不能为文档创建完美的知识表示结构，但是通过使用与 NLP 系统中相同的词性标注器和语法分析器，我们能够开发出识别文档中关键词的几个算法。

1. 词性标注与词义消歧的应用

词性标注器一般基于统计或规则的方法，其目的是切分文本并确定切分出的符号串的词性。文献[Church, 1988]介绍了一种方法，该方法使用已标记的语料库来统计下面两个量：一个给定词被赋予某个词性标记的频率以及不同词性标记序列出现的频率。例如，在某个上下文中，*duck*

142 可能作为一个名词（在池塘中游泳的一种生物）出现的概率为 80%，作为一个动词（避开扔向你的头的球的路线）出现的概率为 20%。还有，“名词 名词 动词”出现的概率可能为 10%，而“名词 名词 名词”出现的概率则可能为 30%。有了这两类数据（由已标注的训练语料库得到），就可以创建一个动态规划算法来优化某一处理步骤中符号串的词性标注。DeRose 在文献[DeRose，1988]中介绍了 Church 词性标注器的一个改进。基于规则的词性标注器根据规则的命中序列标注词性，更详细的介绍请参见[Brill, 1992]。

词性标注器可以用于识别短语，其中一种应用是识别出名词序列（如 Virginia Beach）或者是名词前面的形容词序列（如 big red truck）[Allan 等人，1995；Broglio 等人, 1994]。词性标注器的另一个应用是修改匹配过程，这样文档与查询中的词相匹配就要求其词性也相同。按这种方法，作为动词的 duck 与作为名词的 duck 就不再匹配。尽管这在理论上是合理的，但是却没有产生什么较好的效果。其中一个原因是一些单词（如 *bark*）在相同的词性下仍然有许多不同的意思。在句子 A dog's bark is often stronger than its bite 与句子 Here is a nice piece of tree bark 中，bark 都是名词，但是意义却迥然不同。文献[Krovetz，1993]介绍了词义标注器的早期发展状况，该方法使用基于词典的词干还原器来识别词的语义。关于首字母缩写词的语义消歧的近期研究请参见文献[Zahariev，2004]。

2. 语法分析

沿着 NLP 工具由简单到复杂的路线，我们接下来介绍语法分析。语法分析的目的是识别句子中的语法成分，比如主语、谓语、宾语等。对于结构简单的句子来说，问题没那么严重。简单句 Whales eat fish 的主语是 Whales，谓语是 eat，宾语是 fish。一般情况下，语法分析在词性标注之后进行。

随后，语法分析使用了两个不同的方法。一个方法应用语法分析器，首先尝试使用增强转换网络（Augmented Transition Network，缩写为 ATN，本质上就是一个非确定的有限状态自动机）：主语-谓语-宾语作为该网络的一个状态转换序列。问题是复杂的句子通过自动机转换后会产生许多不同的路径，并且一些句子会递归地调用有限状态自动机，因为他们包含的子结构也

143 是主谓宾齐全的句子。例如，在句子 Mary, who is a nice girl that plays on the tennis team, likes

seafood 中，主要结构为 Mary likes seafood，还包含子结构 Mary plays tennis。然后，使用基于规则的方法是在实施规则的基础上进行分析。

其他的分析算法，如 Gomez 的 WUP（Word Usage Parser），使用词典查找每个词，然后每个词生成特定的状态序列[Gomez, 1988]。换句话说，ATN 是基于各个词的出现动态生成的。尽管 WUP 比 ATN 算法快得多，但是需要大量的人工干预来构建词用法词典。一些词法分析器（比如 Apple Pie Parser）基于轻量级分析，该分析先进行一遍快速扫描以确定关键元素，然后使用规则方法，但是对于复杂的句子，该分析器不能进行完整的分析。

语法分析完成后，信息检索系统就会使用其生成的语法结构。语法分析器的简单用法是选择几个语法成分（比如主语或者谓语）作为查询的有效成分来与文档进行匹配。用这种方法生成的短语能匹配英语中的许多变化。例如查询是 American President 与包含 President of America、president who is in charge of America 等的短语相匹配。在一项引入识别偏正短语（例如“American + president”）方法的系统中，基于一个专利数据集的评测结果表明其在平均准确率上提高了 16%[Osborn 等人，1997]。在 TREC-5 数据集上，基于对简单短语（基本上是“形容词-名词”短语或“名词-名词”短语）的词干还原对索引进行分割，结果偏正短语、人名都被分割出来单独建立了索引[Strzalkowski 等人，1997]。这些串经过合并处理并最终使系统的平均准确率有 20% 的提升。

到目前为止，语法分析尽管大大增加了信息检索系统的运行时性能，但并没有在效率上有质的提升。

3. 信息抽取

MUC（Message Understanding Conference）关注的主题是信息抽取——在非结构化的文档中查找各种结构化数据的方法。人名、地名、数量等的识别是 MUC 会议的基本问题，并且已有许多算法试图解决这个问题。这些算法既不是基于规则的，也不是统计算法。这种算法的第一步大多都是生成句子的语法结构，或者最少也要进行词性标注。这些算法的具体信息请参见 MUC 会议记录。最近计算语言学协会的自然语言学习特别兴趣小组（CoNLL-2003）发布了一个关于语言独立的命名实体识别任务。关于该任务的所有信息请参见 http://cnts.uia.ac.be/signll/conll.html。在该任务中，语言独立算法用来处理英语和德语的标准测试数据集。在 4.4.5 节中，我们将对此进行更具体的介绍。 144

命名实体识别器能识别人名、机构名以及地名。接下来，我们给出一个简单的例子：我们用 BBN 公司的一个基于规则的抽取程序来获得这篇新文档。该抽取程序使用人工创建的规则进行信息抽取，抽取规则使用词周围的词来确定该词是否应该被抽取。首先，我们给出抽取前的文本——关于吉他手 Allen Collins 的一个段落。

```
<TEXT>
Collins began his rise to success as the lightning-fingered guitarist for the Jacksonville
band formed in 1966 by a group of high school students. The band enjoyed national fame in
the 1970's with such hits as "Free Bird," "Gimme Three Steps," "Saturday Night Special"
and Ronnie Van Zant's feisty "Sweet Home Alabama."
</TEXT>
```

下面是由抽取程序产生的输出内容，诸如 PERSON 和 LOCATION 这样的标签已经标记。

```
<TEXT>
<ENAMEX TYPE="PERSON">Collins</ENAMEX> began his rise to success as the lightning-fingered
```

```
guitarist for the <ENAMEX TYPE="LOCAT1ON">Jacksonville</ENAMEX> band formed in <TIMEX
TYPE="DATE">I966</TIMEX> by a group of high school students. The band enjoyed national fame
in the <TIMEX TYPE="DATE">1970s </TIMEX> with such hits as "Free <ENAMEX TYPE="PERSON">
Bird </ENAMEX>," "Gimme Three Steps," "Saturday Night Special" and <ENAMEX TYPE="PERSON">
Ronnie Van Zant</ENAMEX>'s feisty "Sweet Home <ENAMEX TYPE="LOCATION">Alabama</ENAMEX>."
</TEXT>
```

在这个例子以及许多其他我们都人工检查过的例子中，抽取器的效果都不错。现在许多抽取程序在准确率和召回率上都比20世纪90年代的提高很多[Sundheim, 1995]，但是还没有达到完美的效果。我们注意到，短语“Free Bird”中的词“Bird”被标记为标签PERSON。

有了抽取出的数据之后，就有可能给用户展示文档集中的人名、地名和机构名。这些可以作为推荐查询词提供给用户。

抽取程序的最简单应用是识别文档中的关键短语。引入抽取程序后，信息检索系统可以增加抽取出的词的权重。考虑到抽取程序直到最近才在处理大批量文本时达到足够快的速度，因
145 此抽取程序在信息检索领域的研究还处在初始阶段。

3.9　本章小结

我们介绍了8种信息检索实用策略，并强调每种策略无论是单独还是与其他策略结合，都可以集成到任何检索模型中。大多数策略都对词匹配错误进行修正，换句话说，这个错误就是在仅有极少词与查询中的词在语义上相匹配时，文档仍是高度相关的。相关反馈、同义词表和语义网络3个策略都直接用来修正这个问题，方法是试着发现文档与查询中准确匹配的词的相关词。语法分析和n元语法通过词片段代替词来避免匹配错误，分割词能够避免由拼写错误或者前后缀的丢失或误加导致的匹配错误。

基于段落的检索仅关注文档的相关部分。因此，来自文档不相关部分的词的错误匹配已经不存在并且不会在很大程度上降低相似度。聚类算法也试图将用户引导到相关类别的文档中，因此也避免了不相关文档的错误匹配。

回归分析基于相关文档的历史来估计相似度。尽管这需要以前的相关信息，但它提供了一个不错的调整检索模型的机会。

哪个策略最重要？或许这样问更合适：对于一个给定的检索模型，哪个策略或组合策略的性能最好？这些问题的答案都不会很直接。相关反馈一般用于大多数参加TREC会议的系统。文档聚类因为超出目前大多数计算资源的计算能力，所以没有广泛使用。同义词表和语义网络在同一基准下仍显示出极大的性能提升。语法分析在所有信息检索系统中发挥着关键的作用，因为信息检索系统的许多工作需要各种各样的词干还原器。n元语法因为会导致倒排索引迅速增长，所以没有得到广泛应用，但是它提供了解决拼写错误的能力。

总之，需要更多的测试来确定：哪个策略在给定检索模型下性能最好，需要哪个度量方法在何种程度上来确定给定策略的改进效率。

3.10　练习

(1) 使用*Alice in Wonderland*的文本作为数据源，用代码实现某个检索模型以对文档进行排

序。如果某篇文档与查询中的 i 个词相匹配，则将该文档与查询的相似度置为 i（为简单起见，一篇文档定义为书中的 10 行）。在该模型中实现自动相关反馈以提取出供给定查询扩展的 10 个新词。使用 *idf* 作为新词的排序依据。

146

- 从 10 个新词中选出 5 个“好”词（即与查询直接相关的词）生成新的查询。
- 从 10 个新词中选出 5 个“不好”的词生成新的查询。

(2) 用你选择的检索模型做一个演示实例，演示如何修改语法分析器能够从根本上改变结果（即相关文档排序发生变化）。

(3) 为书中的词 *teacup* 实现一个自动同义词表生成算法，并给出 3 个与该词最相关的词。

(4) 给出 10 个词干还原满足用户功能需求的例子。给出 10 个词干还原产生了用户不期望的作用的例子。

(5) 不仅匹配词语而且匹配词的意义是改进信息检索系统的一种方法。该方法的一个例子是：对于含有名词 duck 的查询，包含“She tried to duck to avoid the ball thrown at her”的文档不是相关文档。在你常用的 Web 搜索引擎中执行 5 次查询，并且在每次查询时找出使用该方法能够避免其出现在结果中的文档。

147

第4章

CLIR

CLIR（Cross-Language Information Retrieval，跨语言信息检索）正迅速成为信息检索研究界中成熟的技术。CLIR 的目标是允许用户使用语言 L 表达出查询意图，但可以检索到采用另一种语言 L' 撰写出的文档（见图 4-1）。其思想是：用户希望根据一种语言的查询来获得多种语言的文档集合。当然，隐含的假设是：用户可以理解多种语言的检索结果。否则，系统需要将文档翻译为用户可以理解的语言。CLIR 技术和多语种处理的研究综述，可参见 [Oard 和 Diekema，1998；Haddout，1999]。

4.1 简介

CLIR 与单语言检索的主要区别是：CLIR 中的查询与文档集合不能够直接匹配。信息检索需要从语言风格、音调、词语使用方法以及其他一些语言特征等方面对查询与文档进行匹配，除了这些客观的困难之外，CLIR 还必须跨越查询与文档的语言障碍。4.2 节主要介绍跨越语言障碍的核心问题。4.3 节描述跨语言检索的算法模型。4.4 节主要讨论跨语言检索的实用策略。

4.1.1 资源

跨语言检索系统的实现需要大量资源。大多数方法使用双语词项列表、词项字典、可比语料库以及并行语料库。

149

*可比语料库*指的是由语言 L 写的文档以及主题相同但采用语言 L' 撰写的文档所组成的集合。这里的关键是，文档恰好是由不同的语言编写的，而不是文学意义上的相互翻译。例如，在官方语言为 L 以及官方语言为 L' 的两个国家，分别用其各自的语言刊登了同一则新闻。两国报纸分别写自己的文章，而不是把别人的文章翻译为自己的语言，这种情况就属于可比文档。可比语料库的另一个关键是可比语料库的主题必须相同。关于药物的法语书籍与关于法律的西班牙语书籍，不存在可比性。如果两本书都是关于药物或法律，则它们具有可比性。我们将在 4.3.3 节中讨论可比语料库在 CLIR 中的应用。

*并行语料库*提供语言 L 写的文档，该文档是用语言 L' 写的文档的直接译本，反之亦然。其关键在于用语言 L 写的每篇文档都是对应语言 L' 文档的直译。因此，可以在文档级别、段落级、句子级、短语级甚至词语级对语料库进行对齐。典型的并行语料库是国家或者组织的法律性文档，这些文档要求至少使用两种语言发布。总的来说，并行语料库用于跨语言检索。如果检索的文档与语料库的领域接近，并行语料库将是最有效的。最新的研究工作表明：如果领域选择正确，并行语料库效果显著[Rogati 和 Yang，2004]。我们在“并行语料”一节讨论 CLIR 中并行语料库的应用。

我们还注意到，即使在一个单一的语言，如阿拉伯语，仍然有很多不同的字符集（在阿拉

伯语中经常使用 4 种字符集)。语言处理的相关系统不仅要检测语言的不同，而且也要检测字符集的差别。跨语言系统经常要在单一语言中针对错综复杂的字符集进行处理，过程非常费时费力。人们提出 Unicode（www.unicode.org）编码集就是为了将众多字符集映射为单一的字符集，但并非所有的电子文档都使用 Unicode 存储。

4.1.2 评测

CLIR 系统有很多评测标准。最直接明了的方法是：简单地计算跨语言查询的平均准确度。

另一种方法是计算跨语言检索性能与单语检索性能的百分比。这有时会引起误解，因为实现单语言检索的技术方法往往和跨语言检索的技术方法大相径庭。简单技术的性能一般来说大致是单语言性能的 50%，但 CLIR 相关文献中有些结果超出了 100%，因为在跨语言翻译的过程中，其本身就对查询进行了扩展[Levow 等人，2004]。 150

我们注意到阿拉伯语、汉语、荷兰语、芬兰语、法语、德语、意大利语、日语、韩语、瑞典语和西班牙语都有相关的相关度评测方法。这些方法都已经广泛地应用于各类信息检索评测之中，如 TREC（文本检索会议，见 trec.nist.gov）、CLEF（跨语言检索评测论坛，参见 clef.iei.pi.cnr.it）和 NTCIR（见 research.nii.ac.jp/ntcir）。

4.2 跨越语言障碍

为了跨越语言障碍，我们必须回答 4 个核心的问题[Oard，2004]。

- 应该翻译什么？是翻译查询语句或者是文档，还是把查询语句与文档都翻译为某种内部的中间表示形式？图 4-1 给出了查询翻译的流程图。4.2.1 节描述了查询翻译的方法。图 4-2 给出了文档翻译流程图。4.2.2 节阐述了文档翻译的过程。最后，我们将阐述查询语句与文档的内部的中间表示形式。

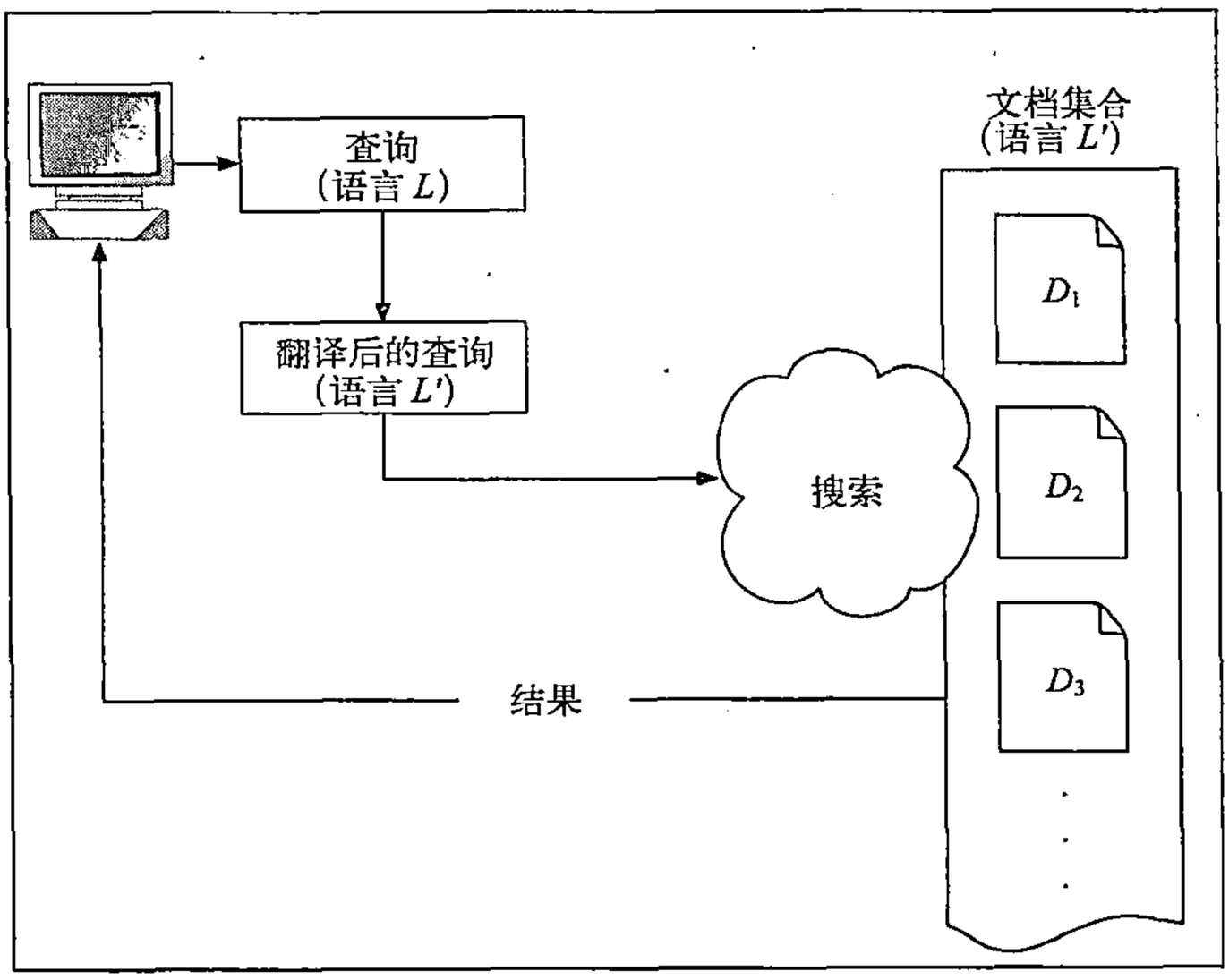


图 4-1 查询翻译

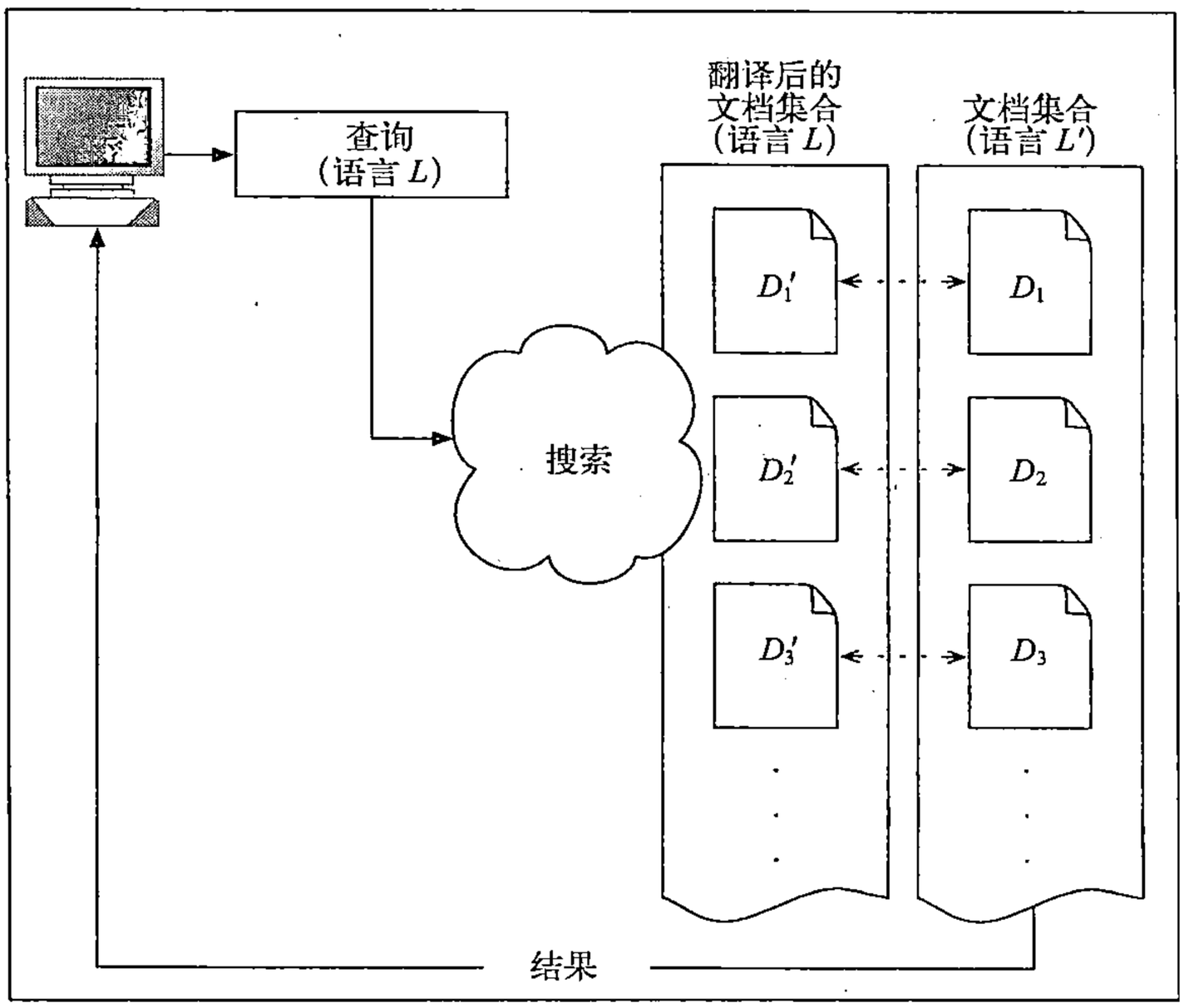


图 4-2　文档翻译

- 应该翻译哪种级别的符号形式（例如，词干、词语、短语等）？4.2.3 节介绍了短语的翻译。
- 我们该如何利用翻译？换句话说，语言 L 中的一个词可能对应语言 L' 中的多个词。我们应该选择一个词、部分词还是全部的词。另外，如果我们更信任其中的一个翻译结果而不是另外一个，就会给它赋予更高的权重。4.2.4 节了介绍了多种不同的选择方法。
- 我们如何去除不当的翻译？一般来讲，不当翻译会导致检索效果很差。目前已经有一些不当翻译移除技术的相关研究。4.2.5 节介绍了翻译的删减策略。

4.2.1 节介绍了几种查询的翻译方法。如果使用复杂的机器翻译系统，引入完整的 NLP 技术，将所有的文档翻译成与查询语句相同的语言是一个特别费力的任务（浅层翻译方法可行，但是效果差一些）。

与单语言检索一样，跨语言检索也有很多的算法模型与实用策略。同样，与单语言检索一样，我们也按照算法与策略来组织本章的内容。类似地，检索模型与算法需要处理 L 语言表达
151 的查询，并且要确定查询与 L' 语言表达文档之间的相似度计算方法。实用策略可以提升任何一个算法的性能。4.2.1 节、4.2.2 节与 4.3.2 节分别介绍查询翻译、文档翻译和内部中间表示形式的核心算法。4.4 节介绍 n 元语法模型、词干还原、命名实体标记等实用策略。

4.2.1　查询翻译

查询翻译最早开始于 20 世纪 70 年代早期，该项工作采用指定的关键词来表示文档，然后应用词典将英文关键词翻译为德文关键词[Salton，1970a]。查询翻译方法使用机器翻译、特定语言的词干还原器、词典、知识库以及自动生成的双语对照的术语词典，采用这些技术和资源进行必要的翻译，最终将用语言 L 写的查询语句翻译为用语言 L' 写的。

关于基于词典的查询翻译技术，文献[Levow 等人，2004]给出了精彩的研讨。在该文献中，对不同的方法进行了系统的研究，并且对具体方法的效果进行了评测。

4.2.2 文档翻译

文档翻译的一种简单方法就是使用我们已经讨论过的任何一种查询翻译方法。例如，将文档从语言 L 翻译到语言 L' 的过程中，我们很可能要使用一种或者多种双语对齐词典。 152

因为文档一般都包含自然语言文本，所以我们完全可以使用机器翻译算法将文档由语言 L 翻译为语言 L'。虽然机器翻译算法并不完美，但是它们早已成为 CLIR 的奠基所在。机器翻译算法的优点是：我们在选择潜在翻译时，使用完整的自然语言文本会有更多的信息可以作为选择判别依据。

4.2.3 短语翻译

与简单使用词翻译不同，基于短语的翻译方法可以显著提高性能。文献[Hull 和 Grefenstette，1996]表明：仅仅使用人工翻译短语来代替词语翻译，即可提升性能。文献[Davis 和 Ogden，1997]从并行语料库中抽取出了双语对照的短语列表。我们将在 4.3.2 节讨论如何抽取双语对照词表。文献[Ballesteros 和 Croft，1997]介绍了使用柯林斯电子词典实现短语翻译的方法。任何短语翻译方法的一个关键问题是：翻译的时候，短语在双语短语对照表中不存在的概率远远大于单个词语不在双语词表中的概率。 153

4.2.4 译文的选择

当双语词表或者单一词表准备就绪后，下一个挑战是如何选择合适的译文。我们已经讨论过查询翻译时词典覆盖率的问题。现在，我们将重点讨论如何选择译文以及如何将译文组合为采用目标语言表达的新查询。

目前有 4 种不同的方法：不平衡方法、结构化查询方法、平衡方法和中间语言法。

1. 双语词表的质量

与相应的单语搜索一样，大多数跨语言检索系统都会使用多种技术提高搜索精度。这些系统的核心是双语词典。这些方法的综述可参见文献[Pirkola 等人，2001]。词典用来将语言 L 中的一个词映射到语言 L' 中的多个词。使用词典的主要问题有：歧义问题、词语覆盖不全、机器可读资源匮乏。

常见的查询一般长度短，上下文内容信息缺乏，一个词对应其他语言中的多个词，词义排歧非常困难。例如，使用英语单词“duck”进行查询。由于该词有多个解释，在目标语言中该词可以翻译为多个词语。“duck”可以定义为“一种在湖上飘游的鸟”，也可能是“躲开”的命令。查询词翻译后，存在大量与查询无关的词，跨语言系统面临的挑战是要减少查询词翻译后的歧义。

词典的覆盖情况对最终效果有什么具体的影响？这方面的评测情况可参见文献[Demner-Fushman 和 Oard，2003]。在该研究工作中，测试文档集有 113 000 篇文档，共计 33 个查询语句。采用 35 个不同的双语词典进行测试。在每个实验中，原始的英文查询基于双语词典进行过滤，只使用词典中已经存在的查询词。针对每个词典，我们要基于词典过滤查询的能力计算出系统的平均准确度。其思想是完美的词典将不会过滤任何词语。实验发现，当词典包含

的词数在 3 000 到 20 000 之间逐渐增长时，系统准确率依次增长。但超过这个范围，平均准确度将不再增长。

154 文献[McNamee 与 Mayfield，2002a]给出了另外一组词典覆盖影响的实验。对于固定的翻译资源，我们随机去除词语的某些翻译结果，逐渐减少翻译结果的数量。这个实验测试了 6 种语言：荷兰语、英语、法语、德语、意大利语和西班牙语。他们发现：和人们的直观感觉一致，随着词典覆盖率的下降，系统性能也随之下降（参见 4.4.1 节）。

2. 不平衡方法

这是使用双语词典最朴素的形式。在 L 语言中，对于一个给定的词语 t，系统使用 t 在语言 L' 中所有的有效翻译结果。这种方法的问题是查询词很可能有无数种翻译结果，我们需要将所有的翻译结果都加入到查询中。与此同时，查询词很可能非常具体，具有专指性，很可能只有一种翻译结果，但是，它的权重很可能小于通用的词语。这与我们的意愿完全相反，我们必须增加专指性词语的权重。

3. 平衡方法

在这种方法里，词语 t 可以翻译为多个词语，这些词语会分别赋予权重，该权重在该词所有翻译结果中进行综合分配计算。因此，通用词语有多种译文，其权重往往会比专指性词语低很多，因为专指性的词往往只有一种翻译结果。计算权重时，往往会综合词频、文档频率与文档长度等多种特征。

更形式化的描述如下：假设一篇文档 D_j 中的词 t_i，其权重为 w_{ij}。跟原来一样，该词语的词频 tf_{ij} 表示词语 i 在文档 j 中出现的次数。用 $|D_j|$ 表示文档 j 的长度。使用 df_i 表示包含词语 i 的文档数量。权重 w_{ij} 使用 tf,df 和文档长度 $|D|$ 组成的相关函数进行计算。我们已经讨论了归一化的余弦方法（参见 2.1.2 节）和概率方法（参见 2.2 节）。这些方法可以理解为词权重的计算函数，这些函数与 tf_{ij} 成正比，与 df_i 以及 $|D_j|$ 均成反比。

为了计算用语言 L 表达的查询词 q_i 的权重。假设查询词 q_i 在语言 L' 中有 k 种翻译结果。对于每一种翻译结果 t_i'，我们可以计算语言 L' 中每一篇文档的 tf_{ij}' 与 $|D_i'|$。类似地，我们可以计算集合的频率，即语言 L' 中的 df_i'。通过这些，我们可以计算语言 L' 中每种相应译文的权重 w_i'。为了计算语言 L 中原始查询词的权重，我们将对应的译文权重做一个简单的算术平均即可，即

$$q_i = \frac{\sum_{i=1}^{k} w_i'}{k}$$

155 文献[Leek 等人，2000；Levow 和 Oard，2000]详细介绍了平衡方法。

4. 结构化查询方法

对于结构化查询来说，文档中词语的总词频是由该词语所有翻译结果的词频总和计算而来的。类似地，查询词的文档频率也是其对应的所有翻译结果的文档频率之和。我们再一次假设查询词 q_i 有 k 个译文，则结果如下：

$$tf_i = \sum_{k=1}^{k} tf_{ij}' \qquad df_i = \sum_{i=1}^{k} df'i$$

我们使用这些修改后的词权重和文档权重计算 w_i。给定词的任何一种译文都会影响到该词的文档频率。文献[Pirkola，1998]更详细地描述了结构化查询方法。最近的一项研究将这些方法进行对比，并发现结构化查询方法的性能要明显优于平衡方法与不平衡方法[Levow 等人，2004]。

5. 中间语言法

有些情况下，我们得不到语言 L 到 L' 的翻译词典。这时系统可以引入中间语言 P，将 L 翻译为 P，再将 P 翻译为 L'。有时，这也称为“传导”翻译。对研究结果最早的探讨可参见文献[Ballesteros，2001]。该研究表明，与直接使用双语翻译相比，使用中间语言会导致性能下降 9%。我们可以采用相关反馈技术克服这种初始阶段的性能下降（参见 4.4.1 节）。

在文献[Gollins 和 Sanderson，2004]中，作者使用了两种不同的中间语言。该工作中，通过两条途径将德语转化为英语。首先，将德语翻译为西班牙语，然后再翻译为英语，最终获得译文。接下来，将德语翻译为荷兰语，再翻译为英语，综合两种翻译结果，获得最终译文。其思想是：在德语翻译为西班牙语过程中的翻译损失能够通过德语翻译为荷兰语的过程进行改进。如果两种翻译方式获得的目标语言词语一致，就采用该译文，这样可以提高最终的检索效果。

例如：假定查询包含查询词 t_1 和 t_2，有一种翻译方式的结果是 e_1、e_2、e_3，另一种翻译方式得到的结果是 e_2 和 e_4，那么，在目标语言中，我们将会采用唯一共同的词 e_2，并判定剩余的词 e_1、e_3、e_4 是错误译文，将之删除。采用中间语言方法的最新研究，可以参见文献[Kishida 和 Kando，2003]。 156

4.2.5 翻译删减技术

一旦确定了译文，我们常常需要针对给定短语的译文列表进行删减处理。在文献[Adriani，2000]中，作者并没有使用所有的翻译结果，只使用可能与给定短语共现的候选翻译结果。

最近，使用英语–阿拉伯语双向词典的研究表明：只有当词语 s 翻译为 t，并且 t 可以翻译为 s 的情况下，才采纳翻译结果 t。这种情况下，系统检索性能才会有所提高[Aljlayl 等人，2002]。

我们可以使用 HMM（Hidden Markov Model，隐马尔科夫模型）来计算最可能的查询翻译，具体可参见文献[Federico 和 Bertoldi，2002]。该方法使用最可能的译文进行查询，并且使用传统的语言模型对检索结果进行合并（就像我们在 4.3.1 节中讨论的那样）。该工作也使用了线性平滑方法，而不是 2.3.2 节中描述的其他平滑方法。作者称其方法优于公式（4-1）中计算相似度的方法，但是我们没有看到直接的对比试验。作者使用不同数量的查询翻译进行试验，他们发现：在许多情况下，一种单一的高质量查询翻译效果要比多个查询好。

4.3 跨语言检索模型与算法

跨语言检索模型与算法阐述了用语言 L 表达的查询与用语言 L' 表达的文档之间的相似度计算方法。4.3.1 节描述了如何在 CLIR 中使用语言模型。

4.3.1 CLIR 中的语言模型

语言模型的方法（见 2.3 节）可能会应用于跨语言检索。下面我们来简要回顾一下，简单的单语言模型将会采用如下公式计算给定文档与查询的相似度。

$$p(q \mid D_i \in R) = \prod_{j=1}^{Q} \alpha P(t_j \mid C) + (1-\alpha) P(t_j \mid D_i)$$

对于给定查询（t_1，t_2，…，t_j，…，$t_{|Q|}$），其中 t_j 是查询词，α 使用 EM（最大期望）算法进行估计[Dempster 等人，1977]，C 是文档集合。当公式按照 HMM 进行组织时，实际上检索方法与结

合了线性插值平滑技术的多项式语言模型非常接近（见 2.3.2 节）。在文献[Rabbiner，1989]中，可以找到关于 HMM 较好的综述。

采用下列公式计算剩余的概率：

157

$$P(t_j \mid C) = \frac{f_j}{|C|}$$

其中 f_j 是词 j 在文档集中出现的次数。

$$P(t_j \mid D_i) = \frac{tf_{ij}}{|D_i|}$$

tf_{ij} 是词 j 在文档 i 中出现的次数。对于跨语言检索来说，我们需要扩展模型，这样采用语言 L 写的文档才可以生成用语言 L' 写的查询[Xu 和 Weischedel，1999]。t_L 表示语言 L 中的一个词，$t_{L'}$ 表示语言 L' 中的一个词。类似地，D_{L_i} 表示用语言 L 写的文档 i，$D_{L_i'}$ 表示用语言 L' 写的文档 i。因此，我们需要计算：

$$P(q_L \mid D_{L_i'} \in R) = \prod_{j=1}^{Q} \alpha P(t_{L_j} \mid C_L) + (1-\alpha) P(t_{L_j} \mid D_{L_i'}) \tag{4-1}$$

现在，我们计算用语言 L' 写的一篇文档生成用语言 L 写的查询的概率。除了 $P(t_{Lj} \mid D_{L'})$之外，其他概率的计算方法并没有变化。这个概率可以通过词典来计算，该词典能够让我们计算翻译概率：$P(t_{L_j} \mid t_{L_k'})$

$$P(t_{L_j} \mid D_{L'_i}) = \prod_{k=1}^{|D_{L'}|} P(t_{L_k'} \mid D_{L_i'}) P(t_{L_j} \mid t_{L_k'}) \tag{4-2}$$

其中，第一项指的是翻译后的查询词词频占检索文档大小的比例。

$$P(t_{L_k'} \mid D_{L'_i}) = \frac{tf_{L_k'}}{|D_{L_i'}|}$$

下一项需要使用词典中实际的翻译结果。概率 $P(t_{Lj}|t_{Lk})$是翻译概率，即对于一个用语言 L' 表示的词语，对应语言 L 的词语的概率。为了简单起见，可能会给这些翻译结果分配相同的概率。比如，如果有一个词语在语言 L' 中有 5 种译文，我们可能会给每一个分配 0.2 的概率。不过，该框架允许采用其他算法估计翻译概率[Brown 等人，1993]。如果有并行语料库（见“并行语料库”一节），就可以用该语料库来估计翻译概率。[Xu 等人，2001]采用了 3 种不同数据源，并对其进行线性组合来估计这个概率（例如，文献[Xu 等人，2001]中使用两部词典和一部从并行语

158 料库中生成的双语词典）。

例子

回到第 2 章的例子，我们演示怎么使用这个方法计算英语查询“gold silver truck”的相关性，并且使用 3 篇西班牙语的测试文档。所有的翻译概率来自于两个在线的英语–西班牙语双语词典。

文档集合如下：

D_{e_1}：“Shipment of gold damaged in a fire.”

D_{e_2}：“Delivery of silver arrived in a silver truck.”

D_{e_3}：“Shipment of gold arrived in a truck.”

D_{s_1}：El envío del oro dañó en un fuego

D_{s_2}：La entrega de la plata llegó en un camión de plata

D_{s_3}：El envío del oro llegó en un camión

为了从西班牙语文档集中找到与英语查询“gold silver truck”最相关的文档，我们需要找到“gold”、“sliver”和“truck”的翻译概率，如表 4-1 至表 4-6 所示。 159

表 4-1 英语集合中所有词的原始词频

$tf_{t,d}$	a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
D_{e_1}	1	0	1	0	1	1	1	1	1	0	0
D_{e_2}	1	1	0	1	0	0	1	1	0	2	1
D_{e_3}	1	1	0	0	0	1	1	1	1	0	1

表 4-2 西班牙语集合中所有词的原始词频

$tf_{t,d}$	camión	dañó	de	del	el	en	entrega	envío	fuego	la	llegó	oro	plata	un
D_{s_1}	0	1	0	1	1	1	0	1	1	0	0	1	0	1
D_{s_2}	1	0	2	0	0	1	1	0	0	2	1	0	2	1
D_{s_3}	1	0	0	1	1	1	0	1	0	0	1	1	0	1

表 4-3 英语集合中所有词的 $P(t \mid C_e)$

a	arrived	damaged	delivery	fire	gold	in	of	shipment	silver	truck
$\frac{3}{22}$	$\frac{2}{22}$	$\frac{1}{22}$	$\frac{1}{22}$	$\frac{1}{22}$	$\frac{2}{22}$	$\frac{3}{22}$	$\frac{3}{22}$	$\frac{2}{22}$	$\frac{2}{22}$	$\frac{2}{22}$

表 4-4 西班牙语集合中所有词的概率

camión	dañó	de	del	el	en	entrega	envío	fuego	la	llegó	oro	plata	un
$\frac{2}{27}$	$\frac{1}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{3}{27}$	$\frac{1}{27}$	$\frac{2}{27}$	$\frac{1}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{2}{27}$	$\frac{3}{27}$

表 4-5 西班牙语集合中所有词的词频

$tf_{t,d}$	camión	dañó	de	del	el	en	entrega	envío	fuego	la	llegó	oro	plata	un
$\frac{tf}{D_{s_1}}$	0	$\frac{1}{8}$	0	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	0	$\frac{1}{8}$	$\frac{1}{8}$	0	0	$\frac{1}{8}$	0	$\frac{1}{8}$
$\frac{tf}{D_{s_2}}$	$\frac{1}{11}$	0	$\frac{2}{11}$	0	0	$\frac{1}{11}$	$\frac{1}{11}$	0	0	$\frac{2}{11}$	$\frac{1}{11}$	0	$\frac{2}{11}$	$\frac{1}{11}$
$\frac{tf}{D_{s_3}}$	$\frac{1}{8}$	0	0	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	0	$\frac{1}{8}$	0	0	$\frac{1}{8}$	$\frac{1}{8}$	0	$\frac{1}{8}$

表 4-6 翻译概率 $P(t_{L_j} \mid t_{L_R})$

	camión	dañó	de	del	el	en	entrega	envío	fuego	la	llegó	oro	plata	un
gold	$\frac{0}{2}$	0	0	0	0	0	0	0	0	0	0	1	$\frac{0}{1}$	0
silver	$\frac{0}{2}$	0	0	0	0	0	0	0	0	0	0	$\frac{0}{1}$	1	0
truck	$\frac{1}{2}$	0	0	0	0	0	0	0	0	0	0	$\frac{0}{1}$	$\frac{0}{1}$	0

采用公式（4-2），可以得到查询词在西班牙语文档集合的文档中出现的概率，如下所示：

$$P(\text{gold}\mid D_{s_1})=$$

$$P(\text{El}\mid D_{s_1})\cdot P(\text{gold}\mid \text{El})+P(\text{envío}\mid D_{s_1})\cdot P(\text{gold}\mid \text{envío})+P(\text{del}\mid D_{s_1})\cdot P(\text{gold}\mid \text{del})+$$

$$P(\text{oro}\mid D_{s_1})\cdot P(\text{gold}\mid \text{oro})+P(\text{daño}\mid D_{s_1})\cdot P(\text{gold}\mid \text{daño})+P(\text{en}\mid D_{s_1})\cdot P(\text{gold}\mid \text{en})+$$

$$P(\text{un}\mid D_{s_1})\cdot P(\text{gold}\mid \text{un})+P(\text{fuego}\mid D_{s_1})\cdot P(\text{gold}\mid \text{fuego})=\frac{1}{8}\times\frac{1}{1}=\frac{1}{8}$$

$$P(\text{gold}\mid D_{s_2})=$$

$$P(\text{La}\mid D_{s_2})\cdot P(\text{gold}\mid \text{La})+P(\text{entrega}\mid D_{s_2})\cdot P(\text{gold}\mid \text{entrega})+P(\text{de}\mid D_{s_2})\cdot P(\text{gold}\mid \text{de})+$$

$$P(\text{la}\mid D_{s_2})\cdot P(\text{gold}\mid \text{la})+P(\text{plata}\mid D_{s_2})\cdot P(\text{gold}\mid \text{plata})+P(\text{llegó}\mid D_{s_2})\cdot P(\text{gold}\mid \text{llegó})+$$

$$P(\text{en}\mid D_{s_2})\cdot P(\text{gold}\mid \text{en})+P(\text{un}\mid D_{s_2})\cdot P(\text{gold}\mid \text{un})+P(\text{camión}\mid D_{s_2})\cdot P(\text{gold}\mid \text{camión})$$

160

$$=0$$

$$P(\text{gold}\mid D_{s_3})=$$

$$P(\text{El}\mid D_{s_3})\cdot P(\text{gold}\mid \text{El})+P(\text{envío}\mid D_{s_3})\cdot P(\text{gold}\mid \text{envío})+P(\text{del}\mid D_{s_3})\cdot P(\text{gold}\mid \text{del})+$$

$$P(\text{oro}\mid D_{s_3})\cdot P(\text{gold}\mid \text{oro})+P(\text{llegó}\mid D_{s_3})\cdot P(\text{gold}\mid \text{llegó})+P(\text{en}\mid D_{s_3})\cdot P(\text{gold}\mid \text{en})+$$

$$P(\text{un}\mid D_{s_3})\cdot P(\text{gold}\mid \text{un})+P(\text{camión}\mid D_{s_3})\cdot P(\text{gold}\mid \text{camión})=\frac{1}{8}\times\frac{1}{1}=\frac{1}{8}$$

$$P(\text{silver}\mid D_{s_1})=$$

$$P(\text{El}\mid D_{s_1})\cdot P(\text{silver}\mid \text{El})+P(\text{envío}\mid D_{s_1})\cdot P(\text{silver}\mid \text{envío})+P(\text{del}\mid D_{s_1})\cdot P(\text{silver}\mid \text{del})+$$

$$P(\text{oro}\mid D_{s_1})\cdot P(\text{silver}\mid \text{oro})+P(\text{daño}\mid D_{s_1})\cdot P(\text{silver}\mid \text{daño})+P(\text{en}\mid D_{s_1})\cdot P(\text{silver}\mid \text{en})+$$

$$P(\text{un}\mid D_{s_1})\cdot P(\text{silver}\mid \text{un})+P(\text{fuego}\mid D_{s_1})\cdot P(\text{silver}\mid \text{fuego})$$

$$=0$$

$$P(\text{silver}\mid D_{s_2})=$$

$$P(\text{La}\mid D_{s_2})\cdot P(\text{silver}\mid \text{La})+P(\text{entrega}\mid D_{s_2})\cdot P(\text{silver}\mid \text{entrega})+P(\text{de}\mid D_{s_2})\cdot P(\text{silver}\mid \text{de})+$$

$$P(\text{la}\mid D_{s_2})\cdot P(\text{silver}\mid \text{la})+P(\text{plata}\mid D_{s_2})\cdot P(\text{silver}\mid \text{plata})+P(\text{llegó}\mid D_{s_2})\cdot P(\text{silver}\mid \text{llegó})+$$

$$P(\text{en}\mid D_{s_2})\cdot P(\text{silver}\mid \text{en})+P(\text{un}\mid D_{s_2})\cdot P(\text{silver}\mid \text{un})+P(\text{camión}\mid D_{s_2})\cdot P(\text{silver}\mid \text{camión})$$

$$=\frac{2}{11}\times\frac{1}{1}=\frac{2}{11}$$

$$P(\text{silver}\mid D_{s_3})=$$

$$P(\text{El}\mid D_{s_3})\cdot P(\text{silver}\mid \text{El})+P(\text{envío}\mid D_{s_3})\cdot P(\text{silver}\mid \text{envío})+P(\text{del}\mid D_{s_3})\cdot P(\text{silver}\mid \text{del})+$$

$$P(\text{oro}\mid D_{s_3})\cdot P(\text{silver}\mid \text{oro})+P(\text{llegó}\mid D_{s_3})\cdot P(\text{silver}\mid \text{llegó})+P(\text{en}\mid D_{s_3})\cdot P(\text{silver}\mid \text{en})+$$

$$P(\text{un}\mid D_{s_3})\cdot P(\text{silver}\mid \text{un})+P(\text{camión}\mid D_{s_3})\cdot P(\text{silver}\mid \text{camión})$$

$$=0$$

$$P(\text{truck}\mid D_{s_1})=$$

$$P(\text{El}\mid D_{s_1})\cdot P(\text{truck}\mid \text{El})+P(\text{envío}\mid D_{s_1})\cdot P(\text{truck}\mid \text{envío})+P(\text{del}\mid D_{s_1})\cdot P(\text{truck}\mid \text{del})+$$

$$P(\text{oro}\mid D_{s_1})\cdot P(\text{truck}\mid \text{oro})+P(\text{daño}\mid D_{s_1})\cdot P(\text{truck}\mid \text{daño})+P(\text{en}\mid D_{s_1})\cdot P(\text{truck}\mid \text{en})+$$

161

$$P(\text{un}\mid D_{s_1})\cdot P(\text{truck}\mid \text{un})+P(\text{fuego}\mid D_{s_1})\cdot P(\text{truck}\mid \text{fuego})$$

$$=0$$

$$P(\text{truck}\mid D_{s_2})=$$
$$P(\text{La}\mid D_{s_2})\cdot P(\text{truck}\mid \text{La})+P(\text{entrega}\mid D_{s_2})\cdot P(\text{truck}\mid \text{entrega})+P(\text{de}\mid D_{s_2})\cdot P(\text{truck}\mid \text{de})+$$
$$P(\text{la}\mid D_{s_2})\cdot P(\text{truck}\mid \text{la})+P(\text{plata}\mid D_{s_2})\cdot P(\text{truck}\mid \text{plata})+P(\text{llegó}\mid D_{s_2})\cdot P(\text{truck}\mid \text{llegó})+$$
$$P(\text{en}\mid D_{s_2})\cdot P(\text{truck}\mid \text{en})+P(\text{un}\mid D_{s_2})\cdot P(\text{truck}\mid \text{un})+P(\text{camión}\mid D_{s_2})\cdot P(\text{truck}\mid \text{camión})$$
$$=\frac{1}{11}\times\frac{1}{2}=\frac{1}{22}$$
$$P(\text{truck}\mid D_{s_3})=$$
$$P(\text{El}\mid D_{s_3})\cdot P(\text{truck}\mid \text{El})+P(\text{envío}\mid D_{s_3})\cdot P(\text{truck}\mid \text{envío})+P(\text{del}\mid D_{s_3})\cdot P(\text{truck}\mid \text{del})+$$
$$P(\text{oro}\mid D_{s_3})\cdot P(\text{truck}\mid \text{oro})+P(\text{llegó}\mid D_{s_3})\cdot P(\text{truck}\mid \text{llegó})+P(\text{en}\mid D_{s_3})\cdot P(\text{truck}\mid \text{en})+$$
$$P(\text{un}\mid D_{s_3})\cdot P(\text{truck}\mid \text{un})+P(\text{camión}\mid D_{s_3})\cdot P(\text{truck}\mid \text{camión})$$
$$=\frac{1}{8}\times\frac{1}{2}=\frac{1}{16}$$

现在我们使用公式（4-1）找到查询在每一个西班牙语文档中出现的概率，这里 α=0.3。

$$P(\text{gold,silver,truck}\mid D_{s_1})=$$
$$\left[(0.3)P(\text{gold}\mid C_e)+(1-0.3)P(\text{gold}\mid D_{s_1})\right]\times$$
$$\left[(0.3)P(\text{silver}\mid C_e)+(1-0.3)P(\text{silver}\mid D_{s_1})\right]\times$$
$$\left[(0.3)P(\text{truck}\mid C_e)+(1-0.3)P(\text{truck}\mid D_{s_1})\right]$$
$$=\left(0.3\times\frac{2}{22}+0.7\times\frac{1}{8}\right)\left(0.3\times\frac{2}{22}+0.7\times 0\right)\left(0.3\times\frac{2}{22}+0.7\times 0\right)$$
$$=0.0000854$$
$$P(\text{gold,silver,truck}\mid D_{s_2})=$$
$$\left[(0.3)P(\text{gold}\mid C_e)+(1-0.3)P(\text{gold}\mid D_{s_2})\right]\times$$
$$\left[(0.3)P(\text{silver}\mid C_e)+(1-0.3)P(\text{silver}\mid D_{s_2})\right]\times$$
$$\left[(0.3)P(\text{truck}\mid C_e)+(1-0.3)P(\text{truck}\mid D_{s_2})\right]$$
$$=\left(0.3\times\frac{2}{22}+0.7\times 0\right)\left(0.3\times\frac{2}{22}+0.7\times\frac{2}{11}\right)\left(0.3\times\frac{2}{22}+0.7\times\frac{1}{22}\right)$$
$$=0.0002491$$
$$P(\text{gold,silver,truck}\mid D_{s_3})=$$
$$\left[(0.3)P(\text{gold}\mid C_e)+(1-0.3)P(\text{gold}\mid D_{s_3})\right]\times$$
$$\left[(0.3)P(\text{silver}\mid C_e)+(1-0.3)P(\text{silver}\mid D_{s_3})\right]\times$$
$$\left[(0.3)P(\text{truck}\mid C_e)+(1-0.3)P(\text{truck}\mid D_{s_3})\right]$$
$$=\left(0.3\times\frac{2}{22}+0.7\times\frac{1}{8}\right)\left(0.3\times\frac{2}{22}+0.7\times 0\right)\left(0.3\times\frac{2}{22}+0.7\times\frac{1}{16}\right)$$
$$=0.0002223$$

D_{s_2}的概率最高，因此，它可能是与查询最相关的文档。 162

4.3.2　双语语料库方法

从逻辑上讲，语言 L 与语言 L' 相似的文档集应当对 CLIR 系统有帮助。找到 L 与 L' 中的相同文档是很烦琐的，但是，为了使 CLIR 技术更好地运行，找到这样的文档是必不可少的，至少也要找到内容相近的文档。一本同时翻译为语言 L 和 L' 的书是很好的双语语料库文档。其他情况下，我们可能有一本用语言 L 写成的书，同时还有一本主题完全不同而采用语言 L' 写成的书。在某种程度上，绝大多数 CLIR 技术的研制过程中都假设存在双语语料库。如今，即使是一些不是太出名的语言，往往都会有一些文本存在双语平行翻译（例如宗教文本），这足够建立一个有用的双语术语表。另外，法律上要求欧盟和专利相关的决议都必须使用多种语言出版。

1. 并行语料库

从 20 世纪 90 年代初期开始，机器翻译领域就使用并行语料库来确定给定词的潜在译文。第一步是匹配两个语料库中的句子。这就是人们所熟知的句子对齐。一般采用双语语料库进行句子对齐并生成翻译概率，文献[Brown 等人，1990]是最早研究这个问题的论文之一。其主要思想是：获取一篇用语言 L 写的文档以及另外一篇用语言 L' 写的文档，其中后者是前者的译文，然后对齐这两个文档。虽然用语言 L 写的一个句子可能映射到用语言 L' 写的多个句子，但是这个问题并不复杂。对齐可以在句子级、段落级或者是文档级。最初的语料库对齐方法采用基于句子字数[Brown 等人，1991]或者句子字符数[Gale 和 Church，1991]的动态规划方法。

在对齐过程中，双语词典可以起辅助作用（例如，如果看到英语句子中包含单词“peace”，我们可能会把它对齐到希伯来语中的“shalom”）。我们注意到，生成双语词典是语料库对齐的一个用途，这样的双语词典在一些不太出名的语言中可能并不存在。

该问题可以表示为一种估计翻译概率 $P(f \mid e)$ 的方法，其中 e 是一个英文单词，f 是一个法语单词。文献[Brown 等人，1993]给出了估计该概率的 5 种模型。这些模型常称为 IBM 模型 1、模型 2、模型 3、模型 4 和模型 5。在模型 1 中，没有考虑词语顺序。模型 2 使用了词语顺序。
163 模型 3 引入了匹配的词语长度。模型 4 和模型 5 中考虑了英语单词与法语单词之间给定连接的概率。后来，文献[Vogel 等人，1996]给出了使用 HMM 的方法。简便实现这些算法的软件工具并不多见，直到一个会议举行为止，该会议的目的就是构建并且传播这样的工具[AlOnaizan 等人，1999]。在这个会议上，人们开发出了程序 GIZA，它包含了 IBM 模型 1、模型 2 和模型 3。后来，人们实现了一个新的版本 GIZA++，该程序包括所有的 5 种 IBM 模型、Vogel 的基于 HMM 的模型以及其他一些改进内容。如今，GIZA++在 CLIR 应用中广为使用，主要用来针对两种不同的语言，从并行语料库中生成双语对照词典。文献[Och 和 Ney，2000b；Och 和 Ney，2000a；Och 和 Ney，2003]详细介绍了 GIZA++，并给出了 5 种不同模型的对比结果。

文献[Hofland，1996]描述了一个使用该技术实现英语与挪威语的对齐算法。文献[Somers，2001；Veronis，2000]综述了对齐技术。文献[Callison-Burch 和 Osborne，2003]描述了一种建立并行语料库的方法，它以较小的集合为初始值，然后使用翻译模型逐渐把文档加入该集合。

2. 可比语料库

可比语料库是两个集合。这两个集合是关于相同内容的文档，但不是相互的译文。在可比语料库部分，我们所讨论的 CLIR 技术都假设用语言 L 写的文档 A 对应用语言 L' 写的文档 B。

4.3.3 可比语料库方法

可比语料库存在的条件是：每种语言写的文档都是关于同一个主题的，但相互之间不是直接翻译得来的。许多 CLIR 技术专注于这种类型的文档集合，因为获取两种语言的可比语料库比获得并行语料库的可能性更大。

我们注意到，当存在用语言 L 写的完整文档，并且存在用语言 L' 写的可比集合时，可以采用一种简单的 CLIR 方法。我们可以采用语言 L 进行查询，并且获得检索结果。结果文档可以映射到其对应语言 L' 的可比文档。然后，将用语言 L' 写的文档列表返给用户。在大多数情况下，可比语料库只是训练文档的一个子集，该集合便于大文档集合的跨语言检索。

1. 双语词典的抽取

关于可比语料库的最早研究可参见文献[Sheridan 和 Ballerini，1996]。在该项研究中，文档 164
按照时间以及独立于语言的描述符对齐（例如："mil" 表示该文档是关于军事的）。在接下来的研究中，作者引入文档中的专有名词来帮助对齐[Sheridan 等人，1997]。这样做的原因是专有名词在某种程度上是与语言无关的。基于这样的对齐，我们可以建立双语词典，并且可以将该词典作为查询翻译的基础。然后，定义词语–词语的相似度函数。给定一个词语，该函数将返回其他相似的词语。这个函数用于查找用语言 L 写的词语在语言 L' 上对应的词语。

Ballesteros 和 Croft 使用共现次数来减少翻译歧义，其假定的前提是：好的翻译结果之间应该在目标语言文档中共同出现，而错误的译文之间则不应该共现[Ballesteros 和 Croft，1997；Ballesteros 和 Croft，1998]。采用按比例适应反复迭代过程，逐步计算给定词语是另外一个词语译文的概率。这个概率用来从候选译文中挑选出最终译文。

另外一种方法使用向量来表示目标词语[Fung，1998]。上下文向量 $\boldsymbol{T}$ 是根据其周围词语得来的。该方法与自动产生单语言词典的方法类似（参见 3.6 节）。然后，我们使用手工建立的双语词表，将上下文词语翻译为源语言表达的词语。我们把新的向量称为 $\boldsymbol{S}'$。其主要思路是：这些词语中一定会有部分词语出现在双语词表中。一旦完成了这一步，就在源语言中使用周围的词语计算源语言的上下文向量，我们称该向量为 $\boldsymbol{S}$。现在，我们可以通过 $\boldsymbol{S}$ 与 $\boldsymbol{T}$ 之间的余弦相似度，最终来确定一个或者多个译文。最近 CLEF（Cross-Language Evaluation Forum，跨语言检索评价会议）提交的论文中已经应用了该技术[Cancedda 等人，2003]。

2. HMM

在 4.2.5 节中，我们介绍了如何使用 HMM 将双语词典融入到跨语言检索中。通过使用可比语料库，该方法可以进行优化，使用更精确的翻译概率。我们可以使用可比语料库中事件发生的频率，而不是统一的翻译概率。应用该概率，基于双语词典，联合统一的均匀概率，文献[Federico 和 Bertoldi，2002；Bertoldi 和 Federico，2003]中提高了系统的准确率。这样的组合可以避免系统的检索词只出现在词典中或者只出现在并行语料库中，从而提高系统的覆盖率。

3. 跨语言相关性模型

2.3 节介绍了单语言模型。一般来讲，这些模型的区别在于：如何估计给定词在相关文档中 165
出现的概率。简单回顾一下，如果存在相关文档，这样的概率估计则是一个烦琐的过程。然而，如果预先定义的查询存在相关文档，就没有必要去执行该查询了。我们知道这是一个进退两难的问题，2.2.1 节介绍了该概率估计的多种不同方法。概率估计可以等同于给定词的相关度计算，我们需要计算该词与查询中每个词的共现概率来确定。其主要思想是，如果文档中出现的词语

包含所有的查询词，其相关度是非常高的。形式化的表述如下：

$$P(w \mid R_q) \approx P(w \mid Q) = \frac{P(w, q_1, q_2, \cdots, q_k)}{P(q_1, q_2, \cdots, q_k)}$$

下一步是计算任意一个词与查询中每一个词共现的概率。估计公式为

$$P(w \mid q_1, q_2, \cdots, q_k) = \sum_{M \in \mu} P(M) \left(P(w \mid M) \prod_{i=1}^{k} P(q_i \mid M) \right)$$

其中 $P(w \mid M)$是在模型 M 中找到 w 的概率。通用模型 μ 可以是文档集中的所有文档。如果 $P(M)$是均匀分布的，则 $P(M) = 1/\mu$。不同的估计仿佛都是基于 tf 的，本质上 idf 可以用来估计 $P(w \mid M)$。

这个模型采用可比语料库的跨语言版本是：需要简单修改计算词概率的估计公式，主要是计算语言 L' 中的词共现概率，要求该词必须与语言 L 所有查询词共现在一篇，可比文档中。可比语料库用来将用 L 语言写的共现文档映射到用 L' 语言写的文档，而不是简单地计算共现频率。

形式化描述如下：

$$P(w, q_1, \cdots, q_k) = \sum_{M_L, M_{L'} \in \mu} P(M_L, M_{L'}) \left(P(w \mid M_{L'}) \prod_{i=1}^{k} P(q_i \mid M_L) \right)$$

我们需要通过对每一个词除以 $P(w, q_1, \cdots, q_k)$的和来实现 $P(w, q_1, \cdots, q_k)$的归一化。现在，我们使用（这是使用相关性模型以来的一次重大飞跃）$P(w|Q)$来估算 $P(w|R_q)$。

我们采用 KL 距离相对熵矩阵计算文档相关度。这些矩阵需要对文档的 $P(w|D)$分布与估计到的 $P(w|R)$分布结果进行比较。

形式化的表述为

166

$$KL(R \mid D) = \sum_{w} P(w \mid R) \lg \frac{P(w \mid R)}{P(w \mid D)}$$

其中 $P(w|D)$的估计公式为

$$p(w \mid D) = \lambda \left(\frac{tf_{w,D}}{\sum_{v} tf_{v,D}} \right) + (1 - \lambda) P(w)$$

其中，v 表示文档集合上全部词汇表中的一个词。$P(w|D)$包含两部分：第一部分表示的是在文档 D 中出现词 w 的概率；第二部分表示的是词 w 在文档集上出现的概率。参数 λ 用来调节这两个部分的权重。

显然，这个模型的可扩展性不强，因为该模型在查询时需要计算每一个词在一种语言中的概率。为了加快计算速度，我们需要针对查询使用简单的相似度计算方法，检索返回排名靠前的文档，再采用此模型对这些文档进行重新排序。文献[Lavrenko 等人，2002]报道了系统的平均准确率。

4. GVSM

传统的向量空间模型将文档与查询表示为多维空间中的向量。一旦完成向量表示，系统就可以通过计算文档与查询向量之间的距离得到两者的相似度。另一种方法是使用词语-文档矩阵，其中每一行代表该词在每篇文档中出现的次数[Wong 等人，1985]。每一列代表一个传统的文档向量。如果计算 $\boldsymbol{Q}' = (\boldsymbol{A}^{\mathrm{T}})(\boldsymbol{Q})$，就可以获得一个表示查询的新向量 $\boldsymbol{Q}'$，它的每一个元素代表

文档集中的一篇文档。$\boldsymbol{Q}'$的每个元素是其对应文档与查询词的内积。如果不包含查询词，则表示该文档的元素值为 0；如果包含全部的查询词，则该元素会有一个较大的值。在单语言检索中，我们可以简单地计算：

$$SC(\boldsymbol{Q},\boldsymbol{D}_i)=\cos\left(\boldsymbol{Q}',\boldsymbol{A}^{\mathrm{T}}\boldsymbol{D}_i\right)$$

其中 $\boldsymbol{D}_i$是一篇文档向量。

跨语言检索可以使用可比语料库为语言 L 建立 $\boldsymbol{A}^{\mathrm{T}}$ 矩阵，为语言 L'建立另一个矩阵 $\boldsymbol{B}^{\mathrm{T}}$[Carbonell 等人，1997]。为了对用语言 L'表示的文档 D_i进行排序，需要计算 $\boldsymbol{B}^{\mathrm{T}}\boldsymbol{D}_i$，这和在语言 L 上计算查询向量的过程一样。此时，我们可以得到一个向量，其每个元素代表用语言 L 写的一篇文档，并且还有一个类似的向量，该向量中的每个元素代表用语言 L'写的一篇文档。通过计算两个向量之间夹角的余弦，可以计算跨语言的相似度。下面给出了跨语言相似度计算的公式：

$$SC(\boldsymbol{Q},\boldsymbol{D}_i)=\cos\left(\boldsymbol{A}^{\mathrm{T}}\boldsymbol{Q},\boldsymbol{B}^{\mathrm{T}}\boldsymbol{D}_i\right)$$

167

5. LSI

我们在 2.6 节讨论了单语言的 LSI。回顾一下，LSI 以一个词文档矩阵 $\boldsymbol{A}$ 为起始，然后使用奇异值分解进行计算[Dumais，1994]：

$$\boldsymbol{A}=\boldsymbol{U}\sigma\boldsymbol{V}^{\mathrm{T}}$$

相似度计算的定义如下：

$$SC(\boldsymbol{Q},\boldsymbol{D}_i)=\cos\left(\boldsymbol{U}^{\mathrm{T}}\boldsymbol{Q},\boldsymbol{U}^{\mathrm{T}}\boldsymbol{D}_i\right)$$

在跨语言检索中，需要对一个新的词-文档矩阵应用奇异值分解方法，该矩阵融合了语言 L 中的词-文档矩阵 $\boldsymbol{A}$ 以及语言 L'中的词-文档矩阵 $\boldsymbol{B}$[Dumais 等人，1997]。即

$$\begin{bmatrix}\boldsymbol{A}\\ \boldsymbol{B}\end{bmatrix}=\boldsymbol{U}_2\sigma_2\boldsymbol{V}_2^{\mathrm{T}}$$

完成这个工作后，相似度可以通过如下公式进行计算：

$$SC(\boldsymbol{Q},\boldsymbol{D}_i)=\cos(\boldsymbol{U}_2^{\mathrm{T}}\boldsymbol{Q},\boldsymbol{U}_2^{\mathrm{T}}\boldsymbol{D}_i)$$

我们注意到有些研究者对跨语言检索的 LSI 与 GVSM 方法进行有针对性的比较 [Cabonell 等人，1997；Littman 和 Jiang，1998]。此外，实质上 VSM、LSI 与 GVSM 是同一个算法的变种，即这些方法都使用奇异向量将文档与查询映射到向量空间，它们只是在给向量元素权重赋值上存在一些差异而已。

文献[Littma 和 Jiang，1998]在小的克兰菲尔德文档集以及大的 TREC 集合上，分别比较了 VSM、LSI 和 GVSM 单语言检索方法。对于单语言检索来说，LSI 在克兰菲尔德集合上性能最佳，但是 VSM 在 TREC 上效果最好。

使用 TREC 法语与德语文档集进行跨语言检索，LSI 性能优于 GVSM。LSI 采用 ADE 算法（Approximate Dimension Equalization，维度近似均匀化），使得整体性能得到了提高。ADE 算法是 LSI 与 GVSM 之间一定权重比例的融合。

GVSM 例子

现在，我们给出一个简单的 GVSM 例子，采用 4.3.1 节例子中相同的英语和西班牙语文档。首先，我们计算原始查询的向量。这个通过查询向量与文档矩阵的乘积得到。

168

$$\boldsymbol{Q}' = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{Q} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}^{\mathrm{T}} \times \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} = [1 \quad 3 \quad 2] \tag{4-3}$$

接着，我们获得西班牙语文档集的向量。该向量采用$\boldsymbol{Q}'$表示，通过使用西班牙语文档矩阵与西班牙语文档集合向量计算而得到。然后，我们计算集合内每篇文档的向量。

$$\boldsymbol{Q}' = \boldsymbol{B}^{\mathrm{T}}\boldsymbol{D}_{s_1} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{\mathrm{T}} \times \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix} = [8 \quad 2 \quad 6] \tag{4-4}$$

169

$$\boldsymbol{Q}' = \boldsymbol{B}^{\mathrm{T}}\boldsymbol{D}_{s_2} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{\mathrm{T}} \times \begin{bmatrix} 1 \\ 0 \\ 2 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 0 \\ 2 \\ 1 \\ 0 \\ 2 \\ 1 \end{bmatrix} = [2 \quad 17 \quad 4] \tag{4-5}$$

$$\boldsymbol{Q}' = \boldsymbol{B}^{\mathrm{T}}\boldsymbol{D}_{s_3} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 2 & 0 \\ 1 & 1 & 1 \end{bmatrix}^{\mathrm{T}} \times \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 6 & 4 & 8 \end{bmatrix} \tag{4-6}$$

最后，我们计算英语查询向量与每一个西班牙语文档向量的相似度：

SC(Gold, Silver, Truck, Ds_1) = cos (<1,3,2>,<8,2,6>) = 0.6814

SC(Gold, Silver, Truck, Ds_2) = cos (<1,3,2>,<2,17,14>) = 0.9274

SC(Gold, Silver, Truck, Ds_3) = cos (<1,3,2>,<6,4,8>) = 0.8437

4.4 跨语言检索实用策略

第 3 章介绍了单语言检索实用策略，现在我们要讨论跨语言检索实用策略，其基本思想是实用策略要不受给定检索模型的影响。 170

4.4.1 跨语言检索的相关反馈

假定给定 L 语言上一个的查询，我们将其翻译成 L' 语言，然后再提交这个查询，那么就有两次不同的时机应用相关反馈（见 3.1 节）。在查询翻译前和翻译后都可以对查询进行扩展。Ballesteros 和 Croft 第一次提出了这种想法，参见文献[Ballesteros 和 Croft，1997]。文献表明在英语–西班牙语翻译前和翻译后进行查询扩展，检索准确率会有所提高。

文献[Larkey 等人，2002b]在英语–阿拉伯语上重现了这种处理过程。在翻译前，使用英语查询，我们得到了每个查询的前 10 篇文档。这些文档中的词按照它们在 10 篇文档中的权重进行排序。我们在查询中加入排名靠前的 5 个词，但是与原始查询中词项的重要度相比，其权重减半。对于翻译成阿拉伯文的词语，取检索到的前 10 篇文档，在查询中加入前 50 个词。

另外一种查询扩展的方法可以参见文献[Adriani，2000]。在这里，作者假定真正的问题是查询翻译，所以只使用翻译后的查询扩展。我们在初始的结果集中可以得到前 20 篇文档，并且对这些结果构建词项–词项共现矩阵。接着，对这些词的相似度求和，计算每一个翻译后的查询词的权重。我们将这些文档中前 10 个词项加入到查询中。翻译后处理可以直接应用 Rocchio 相关反馈[Rocchio，1971]（见 3.1.1 节），具体可以参见文献[Ruiz，2003]。

文献[McNamee 和 Mayfield，2002a]给出了查询扩展的相关研究。在这项研究中，作者引证

了不同的研究，其中给出了大相径庭的研究结论（例如，有些说查询扩展起作用，而另外一些则说查询扩展不起作用）。作者在足够大的数据集上进行了新的研究。他们发现，综合使用翻译前查询扩展和翻译后查询扩展，效果可以提高 15%。进一步研究表明：即使在翻译资源不好的情况下使用翻译前和翻译后的扩展，系统效果仍然得到提高。在这些实验中，他们测试了在有限翻译资源情况下的影响。他们得到的主要结论是：如果翻译资源词汇的覆盖率相当高，翻译前扩展就不会有太多提高。如果翻译资源词汇覆盖率较低，这种扩展的提升效果非常显著。

最近，在中文、日文和韩文的多语言检索中，相关反馈常与逻辑回归一起使用。结果显示：这种方法能使检索效果提高多达 66%[Chen 和 Gey，2003]。

171 有趣的是，最近的研究结果表明：当使用英文查询一个法文的文档集时，翻译前查询扩展确实可以提高效果，但是在中文的文档集上查询时，性能却会下降[Levow 等人，2004]。

4.4.2　词干还原

我们在 3.8 节中讨论了英文的词干还原。对于跨语言检索，词干还原对各种架构模块都有重要的影响。首先，源语言的查询处理器必须能够识别出词项。接下来，用于翻译的精确双语词表可能决定词干还原技术的类别。如果使用词干还原器 Lovins[Lovins，1968]处理查询词，而使用词干还原器 Porter[Porter，1980]建立双语词表，可能并不是很有效。

我们可以使用许多不同的词干还原器，其中网上就有许多可免费使用的版本。广义地讲，欧洲语言的词干还原器遵循 Porter 和 Lovins 词干还原器的通用模式，去除前缀和后缀即可。我们注意到有些语言，比如德语，一些极长的词很难进行词干还原，这是由于它们对应于英语中的多个词。文献[Kamps 等人，2003]给出了较长德语词的切分方法。

1. 回退翻译

当使用双语词表或者词典对词干还原后的文档或者查询进行翻译时，逐步试着将查询词匹配翻译词典是非常有效的。词“jumping”可能并不能匹配任何词汇，但是词根形式的“jump”就可以实现匹配。当对一个未进行词干还原或者表层形式的词与词干进行匹配时，有 4 种组合形式。在下面的例子中，我们使用 stem(*t*)来表示词 *t* 的词干。

- 查询或者文档词的表层形式直接匹配词典中词的表层形式（例如“jumping”匹配词典中的“jumping”）。
- 查询或文档词的词干可能匹配词典中词的表层形式（例如 stem(jumping)匹配 jump）。
- 查询或文档词的表层形式可能匹配词典中词的词干（例如 jump 匹配 stem(jumping)）。
- 查询或者文档词的词干可能匹配词典中的词干（例如 stem(jumping)匹配 stem(jumped)）。

我们可以按照上面的顺序依次进行匹配。使用这种逐渐放宽约束的方式来将查询词或者文档词与翻译词典进行匹配，我们常称为回退翻译。相关研究已经表明该技术可以提升检索效果，

172 文献[Oard 等人，2000]在 CLEF 数据上进行了测试实验。

目前我们需要考虑的是：人类有数百种语言，并且其中的一些语言并不为人们广为流传，以至于我们没有充分的理由来为这些语言耗费资源，专门创建一个依赖该语言的词干还原器。其他一些工作致力于词干自动还原的研究（通常称为统计词干还原法）。

2. 自动构建词干还原器

最初在西班牙语上的研究表明：按词典顺序，我们从相似词中识别出通用的后缀，从而自动构建词干还原器[Buckley 等人，1994]。Linguistica 软件包中就包括了类似的词干还原器的自

动构建，这个程序仅需检测原始文本，就可以自动学习任何人类语言的结构。其主要思想体现为：该软件可以检测英语文本，并且识别出有一类带有后缀“ing”、“ed”与“s”的词。系统仅需要 500 000 个候选词，就可以合理地标识出后缀。Linguistica 软件包的详细内容可以参见文献[Goldsmith，2001]。我们可以使用带权重的互信息找到第一批候选的基本后缀。这样，我们可以识别出每类词规则的标签，然后使用 MDL（Minimum Description Length，最小描述长度）来纠正错误。MDL 确保分隔点可以找到通用词干和通用后缀。对于 CLIR 来说，这样做的唯一问题是不能发现一些不常用的词。因此，我们需要做一些额外的工作，使用规则归纳法来扩充 Linguistica 词干集。这项工作将所有多于 3 个字母的词，按词频对词干进行分组。我们按照词频保存所有两字母、3 个字母和 4 个字母的词（长词干中重叠的词干部分予以删除）。从本质上说，我们使用最频繁的后缀进行词干还原。更详细的内容可以参考文献[Oard 等人，2000]。

这种方法的核心思想是：词干还原器其实就是一种较好标识出词分隔点的工具。分隔点前面的部分就是词干，剩余部分就是不需要作为词干的部分。采用了统计词干还原和回退翻译的系统，其结果显然要比没有进行词干还原的语料库效果更好 [Oard 等人，2000]。有趣的是，人们发现基于规则归纳法的词干还原比基于 Linguistica 软件的词干还原的性能还要好一些。

最近，HMM 常用来构建词干还原器[Nunzio 等人，2003]。这种方法并不去除前缀，但是从初始状态开始，最终转换到大小分别为 1、2、3 或 4 个字符的后缀状态（目前，这种方法主要集中于欧洲语言）。将新语言的词表简单提交到 HMM 上，通过频率就可以标识出哪些词是另外一些词的词干。一旦模型训练完成，就可以对一个输入词进行处理，并且输出给定词最可能的词干。

研究这种方法的出发点是：与“jumping”相比，“jump”在训练语言中可能有更多的实例，并且模型将会学习发现，当输入“jumping”时，产生词“jump”的概率会比较高。在荷兰语、法语、德语、意大利语和西班牙语上，基于 HMM 自动生成词干与词干还原器 Porter 的一系列对比实验表明：在效果上，基于 HMM 自动生成的词干还原器比人工编写的 Porter 词干还原器仅仅低一点点（少于 5%）。 173

3. 阿拉伯语的词干还原器

研究人员也详细研究了中东语言。阿拉伯语信息检索的最初研究见文献[Al-Kharashi 和 Evens，1994]。这项工作使用人工整理的词干还原器，每个词汇表中的词使用词典来映射到对应的词干或者词根。Khoja 去除常见的前辍和后辍，并且设法标识出给定词的词根[Khoja 和 Garside，1999]。这种词干还原器的增强版本可以参见文献[Larkey 和 Connell，2000]。这种方法的问题是词根通常过于宽泛。例如，阿拉伯词“办公室”和“课本”有相同的词根。随后，有些方法仅仅是简单地去除复数形式和其他“危害”低的词干后，系统的性能提升显著。这些阿拉伯词干还原器常称为浅层词干还原器。关于阿拉伯语的浅层词干还原器，可以参见文献[Larkey 等人，2002b；Darwish 和 Oard，2003；Aljlayl 等人，2002；Aljlayl 和 Frieder，2002]。研究者直接比较了 3 种词干还原器，其中包括：早期的一种叫“Al-Stem”的浅层词干还原器、文献[Larkey 等人，2002b]中提出的浅层词干还原器以及 Al-Stem 的增强版本[Darwish 和 Oard，2003]。此外，文献[Chen 和 Gey，2002]做出了部分改进。我们注意到：这些词干还原器对于阿拉伯语的单语言检索很有用，对于 CLIR 检索，用途更加显著，并且它们同样支持查询从 L 语言翻译为阿拉伯语，翻译后的新查询接着在阿拉伯语文档集上运行，然后检索出阿拉伯语文档。

4. 亚洲语言

中文和其他亚洲语言出现了一个有趣的问题：这些语言不用空格来表示词的边界。文献[Dai

等人，1999]在由 300 个句子构成的训练集上，使用分阶段逻辑回归的统计方法来标识出中文词边界。在这项工作中，单个字或者被称作“二元”的两个字对的相关频率使用文档频率、加权文档频率、局部频率、上下文信息（周围字的频率）和位置信息来计算。文献[Xue 和 Converse，2002]给出了一种综合的方法，它综合了基于频率的统计算法和基于词典的算法，而基于词典的
174 算法从机读词典中简单地匹配出集合中最长的字符串。有趣的是，中文和日文的跨语言检索可以忽略词的切分问题，并且可以直接通过重叠的 n 元语法得到合理的结果（参见 4.4.3 节）。

4.4.3　n 元语法模型

n 元语法模型是一种与语言无关的方法，使用的是字符序列[Damashek，1995]。在 3.4 节中，我们在英语上使用了 n 元语法。对于欧洲语言，n 元语法模型的大小 n=4 或者 n=5 的效果比较好。有趣的是，n 元语法模型与词干还原效果相当。与 Porter 词干还原器相比，使用最不常用的 n 元语法来表示一个词，其结果稍有提高[Mayfield 和 McNamee，2003]。其研究思路是：针对每个词仔细选择的 n 元语法，足够可以替代词干。

在文献[McNamee，2001]中，使用 n 元语法模型可以避免亚洲语言的切分问题。有趣的是，其效果与更复杂的切分算法相当。此外，使用六元语法模型的跨语言检索实验可以参见文献[Qu 等人，2003b]。

4.4.4　音译名

最近的研究重点是：使用音译名的形式来辅助解决跨语言的人名匹配问题。Pirkola 等人使用音译名规则识别如何将 L 语言的名字音译成 L' 语言的名字[Pirkola 等人，2003]。核心出发点是：在翻译查询或文档词所使用的机读双语词表中，我们找不到这些专业术语和人名。在这个研究工作中，从源语言和目标语言的成对词表中，我们可以使用机器学习的方法学习到音译名转换规则。首先通过定义编辑距离，引入合理的近似阈值，就可以识别出音译词项。Levenshtein 编辑距离是计算将源词修改成目标词，需要进行替换、插入和删除操作的最小操作数[Levenshtein，1966]。例如，“fuzzy”到“fuzy”的编辑距离是 1。

一旦找到词的候选集，所有的音译转换就变成了考查最小编辑距离了。每个字到字的转换都赋予一个错误值。赋值包括：未改变时赋值为 0；没有明显改变时赋值为 1，如辅音替换辅音，元音替换元音，或者插入和删除一个字符；较明显情况下赋值为 2（如元音替换辅音，或者辅音替换元音）。接着，标识出错误值最小的转换。一旦完成，就使用一个阈值来确保给定的变换是经常出现的。然后，我们将这些变换规则用于源语言到目标语言的翻译。根据已知规则置信
175 度的不同，我们可以产生更多的规则而使召回率更高，或者是规则更少而准确率更高。在多种语言上的测试结果表明：结全使用这些转换和 n 元语法模型，系统效果会有显著的提高[Pirkola 等人，2003]。

语音方法

对于英语–日语检索，文献[Qu 等人，2003a]研究了一种使用语音的方法进行音译。对于英语–日语检索，一般使用词的语音。音译方法如下：

英语词→英语语音→日语语音→日语词

给定一个英语词，我们要找到其对应的语音形式。词的第一个语音表示使用了 CMU 语音发音词典。该词典包含了 39 种不同的音素。一旦识别出日语语音，标准的映射程序将日语语音转换

为目标的片假名字符集。

对于日语-英语翻译，问题会更难一些，这是因为日语中没有单词边界。首先，我们使用一部词典，采用词切分算法来匹配出词典中最长的词。我们使用日英词典 EDICT。使用与英-日翻译同样的语音处理过程，我们得到日-英语音翻译。这种音译可以直接看做是词典 EDICT 的附录。由于词切分有可能是错误的，所以音译检查是必需的。例如，词“Venice”在词典 EDICT 中没有对应项，被音译成“bunny”和“cheer”。这是因为这个日语词的子串部分在词典 EDICT 中没有对应项。因此，我们需要对音译结果进行检查，确定是否像在源语言中一样，以合理的互信息值在目标语言中共现。我们发现：如果查询中包含了 EDICT 中不出现的词，查询效果会有明显提升。

4.4.5 命名实体识别

3.8 节介绍了英文的命名实体识别。对这种技术明显的扩展是：我们还可以识别其他语言文档中出现的人名、地名和位置词。如果可以识别出这些实体，就为提高 CLIR 中这些命名实体的音译与语音技术打开了大门（详见 4.4.4 节和“语音方法”一节）。同样，如果知道一个词是人名，就可以不用在普通翻译词典中查找，而避免不正确的翻译。

机器学习技术可以很好地解决这个问题，这是因为我们可以使用已经标记了命名实体的训练样本集，然后使用机器学习算法来识别实体[Carreras 等人，2002]。早期工作使用语言无关的 bootstrapping 算法[①]，这种算法对上下文和词型进行反复迭代学习并重新估值[Cucerzan 和 Yarowsky，1999]。这种算法在未标注的文本集合上进行机器学习，当在很短人名列表上进行训练后，不需要其他语言的具体信息，就可以达到较好的性能。 176

对于每一个需要识别和标记的命名实体类来说，一般都假定用户可以提供一个无歧义的短（大约 100 个）样本列表。此外，我们还需要知道语言的一些具体信息（如大写字母的出现和词语分隔符的使用）。这种算法依赖于词内部及其上下文的线索，并将其作为独立的证据。内部线索是指词的构词结构。这些线索使用前辍或者后辍作为某些命名实体的特征标记。例如，如果知道“Maria”、“Marinela”和“Maricica”是罗马尼亚语中女性名字的话，就可以很容易猜测“Mariana”也是同样的类别，这是因为它们有共同的前辍“Mari”。后辍的信息量通常更大。例如，在罗马尼亚语中，“-escu”几乎是姓氏最好的特征标记，这同样可以应用到波兰语中的“-wski”、塞尔维亚克罗埃西亚语中的“-ivic”以及英语中的“-son”。这种构词上的信息可以在 bootstrapping 过程中学习到。我们使用算法来识别命名实体，包括英语、希腊语、印地语、罗马尼亚语和土耳其语中的命名实体。

文献[Maloney 和 Niv，1998]给出了一种阿拉伯语的命名实体识别方法，它依赖于构词分析中的特定模式。文献[McNamee 和 Mayfield，2002b]给出了一种不依赖于语言资源的阿拉伯语命名实体标记方法。这些识别算法与英语中的识别算法类似。

研究结果表明：双语词表一般对命名实体的覆盖率都很小。对于一些跨语言的任务（通常使用英语或者欧洲语言），我们可以识别出实体，并且希望在目标语言中使用同样的字母拼写方式。最近的实验表明：当在双语词典中删除专有名词时，会导致性能降低 50%以上。

自然语言学习计算会议（CoNLL-2003）关注与语言无关的命名实体识别方法。我们可以得

① 可以译为“自举算法”，学术界一般采用原文。它是一种反复迭代、直到收敛的机器学习算法。——译者注

到英语和德语的结果，排名靠前的系统在准确率和召回率上均可以达到 85%以上。在德语中，准确率在 80%～85%之间[Sang 和 Meulder，2003]。英语中的前 3 个结果和德语中的前两个结果都使用了最大熵模型[Guiasu 和 Shenitzer，1985]。

总的来说，外语的命名实体识别以及在跨语言检索中利用这些命名实体是未来需要研究的
177 课题之一。

4.4.6　检索融合

我们在 8.3 节中详细介绍了单语言检索中不同检索算法的融合。直接合并不同模型的检索结果（例如，向量空间模型、语言模型、LSI 等）并不能够提高准确率，但是合并不同形式的结果（例如，仅查询标题、标题+描述等）却可以达到提高精确率的目的。在阿拉伯语的跨语言检索中，融合不同查询表示形式的方法可以参见文献[Larkey 等人，2002a]。实际情况是：用户可以向多个文档集提交一个查询，而每个文档集采用不同的语言表达。这可以看做 MLIR (MultiLingual Information Retrieval，多语言信息检索)。对于多语言检索，目前有两种架构：集中式和分布式。分布式架构分别在每种语言上对文档创建索引。集中式文档集则综合所有的文档，只创建一个索引。

为了在 MLIR 上实现文档排序，最近研究人员测试了 3 种方法。

- 将查询翻译为每种语言，并且在每种语言上都创建一个单独的索引。将所有的相关性得分进行归一化，并且给出单一的文档排序列表。这是一种分布式方法。
- 将查询翻译为每种语言，并且在每种语言上都创建一个单独的索引。但是不使用得分来进行合并，而是使用结果列表中的相对位置来合并。这也是一种分布式方法。
- 将查询翻译成每一种语言，并且将所有的查询结合在一起，转换为一个长查询。为每一种语言的所有文档创建一个大索引。在这个大索引上运行这个长查询。这就是集中式方法。

集中式方法的性能优于前两种方法。这种方法成功的原因是：文档中存在专有名词，该方法能在整个文档集中追溯它们的实际分布[Martinez 等人，2003]。我们还可以通过权重化文档集来实现结果融合，这种权重基于对检索效果的预测。使用这种方法，如果翻译词典的质量比较差，那么源文档集的权重可能会很低。

此外，在一个给定的文档集上，查询中歧义词的数量可以用来对文档集的权重进行计算。结果表明：这种方法可以使准确率得到少许提高[Lin 和 Chen，2003]。

4.5　本章小结

在某些情况下，跨语言检索系统实际上超过了单语言检索。因此，有些人已经认为我们已经解决了跨语言检索问题。实际上，换一个角度来看，用户对这些系统的反应不佳，这些系统
178 还没有被广泛应用，因此，我们还有许多工作要做。未来跨语言检索的研究方向可以参见文献[Oard，2003]。

许多商用搜索引擎关注的重点还是单语言检索。但是，CLIR 系统并没有被广泛接受。其中一个关键问题是：有多少用户真地希望使用他们已知的语言去查询文档，而返回的结果却是他们不知道的语言构成的？我们需要做更多工作来调研有多少项目存在这种需求，并且需要收集

这些系统没有采集 CLIR 算法的准确理由。

4.6 练习题

(1) 假定一个系统中的文档由多种语言写的。请描述哪些因素会让你决定这些文档应该在建索引前进行翻译，或者仅仅是将查询进行翻译。

(2) CLIR 的结果可能会超过相应单语言系统性能的 100%，请给出一个详细的例子。

(3) 考虑由语言 A 和语言 B 组成的两个文档集。语言 A 的文档在语言 B 中都有对应的文档。请给出一种可以充分利用这种可比语料库的 CLIR 算法。 179

第5章

检索效率优化

到目前为止，我们讨论的算法关注的都是如何提高查询处理的效果，而查询效果采用准确率和召回率两个指标来衡量。前几章所介绍的信息检索模型以及实用策略的重点都是如何从文档集中找出与查询相关的文档，但是并没有考虑查找到这些文档需要花费的时间。

然而，实用系统的用户肯定都要关注运行时性能，速度较快的检索系统比慢的系统更受用户欢迎。大部分信息检索研究都集中在如何提高准确率和召回率上，这是因为大家寄希望于计算机硬件速度将会持续不断地增长。而且，如果一个算法不能检索到相关文档，那么提高它的运行时性能意义不大。

我们很难具体分析串行信息检索算法的性能，因为检索性能往往与查询词的搜索选择过程相关。大多数算法的时间复杂度为 $O(q(tf_{\max}))$，这里的 q 是查询语句中词的个数，$tf_{\max}$ 指的是查询词选择范围的最大值。实际上，这是查询响应时间的最大估计值，因为大多数词都很少出现（根据齐普夫定律[Zipf，1949]，大约有一半的词只出现一次）。

目前，我们还没有一个标准的分析模型用来有效地评测查询性能。因此，串行信息检索算法的性能都是通过大规模的数据实验来评价，这往往费时费力。

值得庆幸的是，随着文档集规模不断增大，已经有越来越多的研究工作开始关注如何改善运行时性能。在本章中我们将介绍相关工作。提高信息检索算法性能的另外一种方法是利用多处理器系统，第 7 章将介绍这一方面的内容。然而在研究并行算法时，我们需要与最好的串行算法进行比较，这一点极其重要。因此，本章将介绍我们所知道的用于信息检索的最佳串行算法。

我们也需要注意的是，本章所介绍的大多数算法都是直接应用于向量空间模型的，当然这些算法也同样适用于概率模型。显而易见，对于其他的检索模型和实用策略，同样需要类似的研究以提高性能。

早期的信息检索系统只处理非常小的文档集，之后人们引入了倒排索引来提高查询的处理性能，其代价是存储索引需要空间，构建索引时需要时间。后来，人们又提出了签名文件，它的优点是速度很快，而且占用空间较少，缺点就是与倒排索引相比，它支持的检索功能很少。

倒排索引压缩通常可用来提高查询处理性能。另外，为了降低查询处理的开销，我们可以仅对部分文档排序，这比对所有文档进行排序要快很多。而且，在某些情况下，这种方式对准确率和召回率并不会产生什么实质性影响[Lee 和 Ren，1996]。

在 5.1 节中，我们首先综合介绍倒排索引技术，然后介绍一些倒排索引压缩方法。5.2 节主要介绍提高查询处理性能的算法，5.3 节将全面介绍签名文件。

5.1 倒排索引

由于大多数文档集都是静态的，因此，快速构建倒排索引并从中检索出文档集中的词是完

全可行的。早在 20 世纪 60 年代，倒排索引就已经被用于信息检索系统和数据库管理系统中[Bleir，1967]。倒排索引并不需要扫描整个文档集，而是对文档进行预处理，识别出文档集中每一个词。这些词统称为*索引*。对于索引中的每一个词，需要采用一个*倒排表*来存储记录哪些文档中包含这个词。图 5-1 展示了一个倒排索引。

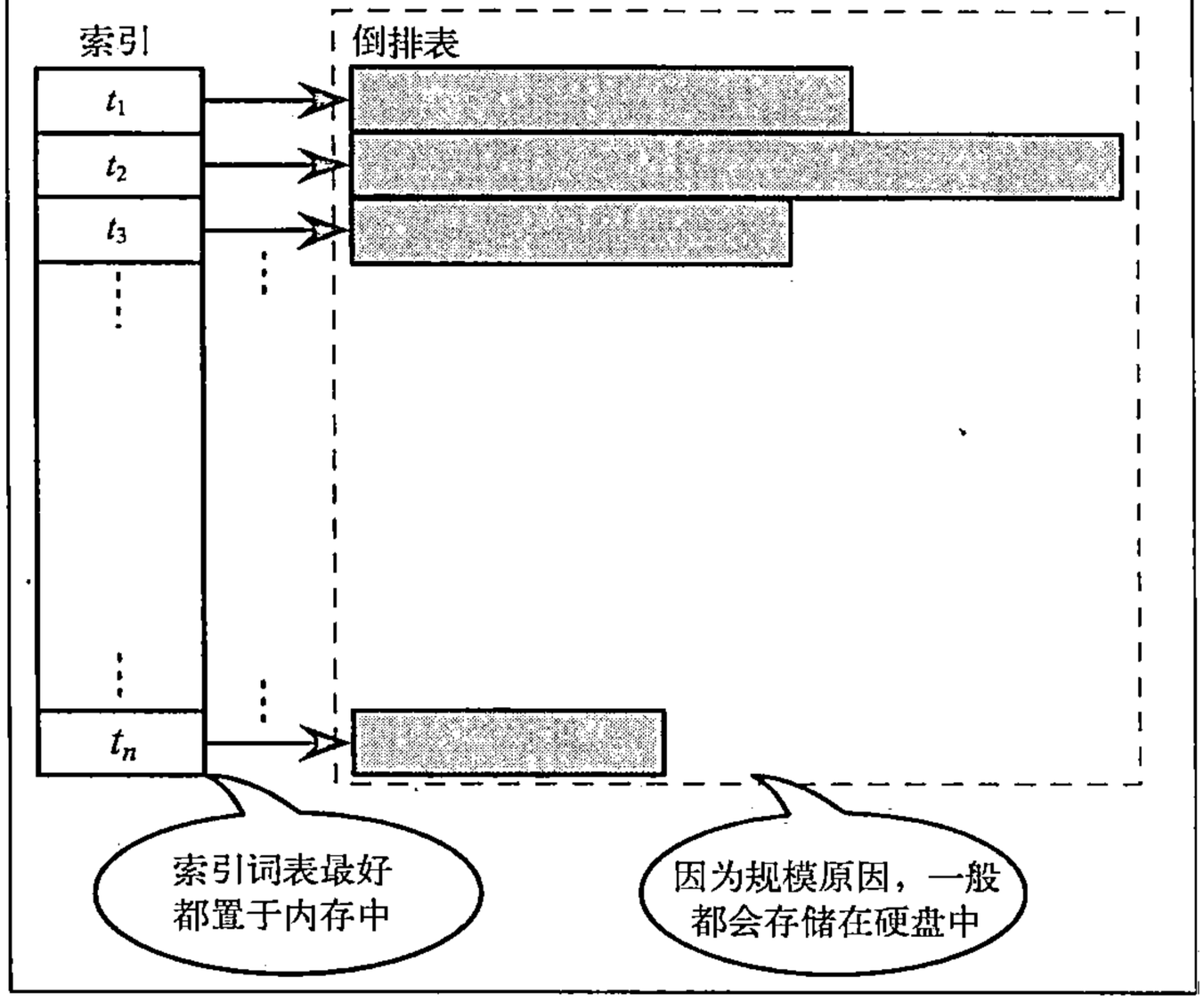


图 5-1 倒排索引

倒排表中的每一个条目也可以包含词在文档中的位置信息（如词位置、句子、段落），这样的结构有利于实现邻近搜索。另外，每一个条目还可以包含词在该文档中的权重值，这个值既可以人工分配，也可以自动生成。该权重经常用于文档的相关性计算。当所有文档的相关度计算完成后，我们可以根据相关度对文档排序，然后识别出所有与给定查询相关的文档，最后呈现给用户。

构建索引时需要额外的开销，因为它需要扫描整个文档集，将高效的倒排索引存储到辅助存储器上也需要大量的 I/O 操作。但是，倒排索引能大大减少 ad hoc 查询时所需的 I/O 量[Stone，1987]。每当接收到一个查询，先查询索引，检索到相应的倒排表，最后检索算法根据倒排表的内容对文档进行排序。 182

索引的大小是另一个值得关注的问题。大多数索引的大小和原始文档差不多。这意味着所需要的存储空间将是原始文档大小的两倍。然而，索引压缩技术可以将索引大小压缩到原始文档大小的 10%以下[Witten 等人，1999]。索引中所包含的词或短语依赖于所使用的语言解析算法。

倒排索引中倒排表的大小可以通过齐普夫分布来估计。齐普夫分布是齐普夫提出的，它描述了自然语言中每个词的词频分布情况：如果把词出现的频率按由大到小排列，那么每个词的词频与它排序序号的乘积是一个常量[Zipf，1949]。表 5-1 展示这个常量取 1 时的词频分布。

表 5-1　齐普夫分布中的前 5 个词

排序序号	频率	常量
1	1.00	1
2	0.50	1
3	0.33	1
4	0.25	1
5	0.20	1

使用公式$\frac{C}{r}$（其中 r 是词的排序序号，C 为常量值），我们可以估计出给定词的出现次数。

183 常量 C 与词所处领域相关，通常等于词频最高词的出现次数。

5.1.1　构建倒排索引

倒排索引由两部分组成，所有独立的词列表称为*索引*，词对应的一系列表统称为*倒排表*。为了对文档进行排序，我们还需要保留词频和权重信息。因此，对于集合中的每个词，都有一个包含多元组的倒排表与之关联。集合每个词的元组均采用<doc_id,tf>的形式来表示。未压缩的索引通常需要 4 个字节来存储文档 ID；长文档中有些词的出现次数有可能超过 255 次，因此我们需要采用 2 个字节来存储词频。

假设文档集由文档 1 和文档 2 共计两篇文档构成，文档 1 中词“sales”出现两次，“vehicle”出现一次，文档 2 中“vehicle”出现一次。那么，索引将包含两个条目：“sales”和“vehicle”。倒排表仅仅是一个与每个索引词相关联的链接表。在本例中，倒排表如下所示：

sales→(1,2)

vehicle→(1,1)(2,1)

倒排表中的条目按照文档标识符升序排列。显而易见，构建倒排索引的成本较高，但是一旦构建完成，便能有效地进行查询。现在我们便开始介绍倒排索引的构建算法以及查询处理的算法。

构建倒排索引的一种可行算法如下：按顺序扫描整个文档集，一次处理一个词。该算法的输出结果是一些存储在磁盘上的文件。有如下几种文件。

- **索引文件**：该文件包含了集合中每个词的倒排表。如果词 t 在 n 个不同的文档中均出现

184 了，那么 t 的倒排表如下所示：

$$t \rightarrow (d_1,tf_{1j}),(d_2,tf_{2j}),\cdots,(d_i,tf_{ij})$$

其中 d_i 表示文档 i 的标识符，tf_{ij} 表示词 j 在文档 i 中的出现次数。

- **文档文件**：该文件含有每篇文档的各种信息，比如文档 ID、文档名、发表日期等。
- **权重文件**：该文件存储的是文档集中每篇文档的权重。这个权重是余弦相似度计算中的分母，余弦相似度指的是查询向量与文档向量夹角的余弦值（可参见 2.1 节）。

构建倒排索引的具体流程是这样的：扫描整个文档集，每次只处理一个词。当遇到一个词的时候，首先检查该词是否是一个停用词（如果需要去除停用词的话），或者检查该词是否已经判别过。我们可以用一个散列函数来快速定位该词在数组中的位置。散列函数的冲突可以通过一个线性链表来解决。文献[McKenzie，1990]给出了不同散列函数的性能分析。一旦定位了该词的倒排表，首先检查倒排表中第一个条目的文档 ID 是否与待处理词的文档 ID 一致。如果一致，则将倒排表中第一个条目的词频加 1。如果不一致，表明该词是在文档中第一次出现，因

此需要在倒排表的开始处添加一个条目。

倒排表通常存储在内存中。每当新加入一个倒排表条目时，都需要为其动态分配内存。每次分配内存时，先检查为构建索引所预留的内存是否已经用完。如果用完，则暂停处理，将内存中的倒排表全部写到磁盘，然后再继续处理。向磁盘写入的倒排表中，每个词的倒排表都是连续存储的。

当所有词都处理完毕后，索引就构建完成了。此时，我们就可以扫描每个词的倒排表来计算其逆文档频率。一旦计算完逆文档频率，接着就可以计算文档权重了（即余弦相似度中的分母）。文档权重的计算也与逆文档频率一样，需扫描每个词的倒排表。

5.1.2 压缩倒排索引

倒排索引文件研究的一个关键目标是研究一些算法来减少 I/O 带宽和存储开销。索引文件的大小决定其所占用的存储开销。此外，由于大索引文件需要更大的 I/O 带宽来读取，因此，其大小也直接影响了处理时间。

尽管文本压缩已被广泛研究[Bell 等人，1990；Gutmann 和 Bell，1994；Gupte 和 Frieder，1995；Trotman，2003]，但是在压缩倒排索引文件方面的研究工作相对来讲还是比较少的。然而，在 Moffat 和 Zobel 的研究中，他们使用了一种相对比较容易解压缩的索引生成方式。压缩索引后的大小不到原始文档集的 10%，而且更让人印象深刻的是，这里面还包含了停用词。 185

同义词表和倒排表是压缩倒排索引的两个主要方面。尽管有些研究表明，假设内存代价相对较低，我们可以不必关注于索引压缩，但仍然需要做些工作，参见[Witten 等人，1999]。《国王詹姆斯圣经》（大约 5MB），包含了 9 020 个词项，而传统的 TREC 文档集（稍多于 2GB）包含 538 244 个词项[Witten 等人，1999]。当遇到新的领域时，新词的数量总会有一些增长，但是认为词表数量大约稳定在 100 万或 200 万个还是合理的。对于每一个索引词，索引词的平均长度为 6，文档频率计数器需要 4 个字节，倒排表入口的指针需要 4 个字节，所以总共需要 14 个字节。我们保守估计有 200 万个索引词，未经压缩的索引很可能在 32MB 以内。甚至，我们可以上升一个数量级，存储索引所需的内存空间保守地讲也会在 1GB 以下。

假如索引相对较小，而且可以很容易加载到内存中，我们就不用过多地讨论索引压缩的技术细节。我们注意到，词干还原降低了压缩的需求，相对直接的方法就是使用哈夫曼编码[Witten 等人，1999]。同样，使用短语可以提高召回率和准确率（见 3.8.2 节）。按照短语存储索引也可以很好地压缩索引。这还取决于如何识别和限制短语。大多数系统往往会去除出现频率较低的短语。

在介绍索引压缩算法之前，我们首先介绍一种相对比较简单的算法，也就是称作 BA（Byte Aligned，字节对齐）的索引压缩算法[Grossman，1995]。通过使用字节边界来实现 BA 压缩，运行时会付出些许代价，但可以获得一定的压缩比。BA 算法易于实现，而且压缩比比较高（使用停用词后，压缩文件大小占未压缩索引文件大小的比例大约是 15%）。变长编码压缩的具体介绍可以参见文献[Witten 等人，1999]。尽管这些工作可以获得比较好的压缩比，但其代价往往会是实现更加复杂。

1. 固定长度的索引压缩

正如前一节中所讨论过的，倒排表的每个条目都是按照文档 ID 升序排列的。当倒排索引经过删除变动操作后（参见 5.1.5 节），这种文档的顺序就会出现例外。因此，RLE（Run-Length 186

Encoding，行程长度编码）方法适用于文档 ID 的编码。对于任意一个文档 ID，仅仅需要计算当前 ID 以及待处理 ID 之间的偏移量。对于没有其他文档 ID 存在的情况下，我们就存储文档 ID 的压缩形式。采用这种技术，相对较小的数值可以保证在整个数据中占较大比重。

这种方案可以很有效减小 ID 的范围，允许它们以更精小的形式存储。随后，采用接下来的方法用于数据压缩。对于一个给定的输入值，保留最左面的两个比特，用来存储整个存储值占用的字节数。两个比特位的表现形式有 4 种可能的组合，因此，对于所有的文档 ID，可以使用一个两比特位的指示器。整型数据的存储长度可以为 6 比特、14 比特、22 比特或者 30 比特。最佳的情况下，所有的值都小于 $2^6=64$，通过上述方法，每个单独的数据记录大小使用一个字节就可以存储。而在没有压缩的情况下，所有的文档 ID 都需要使用 4 字节存储。

对于待压缩的每一个值，需要计算存储值所需的最小字节数（注意：这种说法并不是特别精确。因为我们并不需要做零点位移，因此，一位移的存储空间可能会比实际需要的空间大小少一些。因为这种存储方法往往需要额外的增量操作，而且它仅适合于数据边界的情况，因此该方法极少使用）。表 5-2 给出了我们可以存储的值范围，同样长度指示器分别为 1 字节、2 字节、3 字节以及 4 字节。如果文档集中的文档数目超过 2^{30}，这种方法可以将指示器长度扩展为 3 个比特，这样表示的范围就达到了 $2^{61}-1$。

表 5-2　整型数据的存储范围及其字节数

长　　度	所需字节数
$0 \leqslant x < 64$	1
$64 \leqslant x < 16\,384$	2
$16\,384 \leqslant x < 4\,194\,304$	3
$4\,194\,304 \leqslant x < 1\,073\,741\,824$	4

对于词频，因为每一个频率值均独立于先前的词频值，因此，我们并不需要使用值间偏移量的概念。然而，我们可以使用相同的编码方式。因为，我们并不期望一篇文档中包含的词超过 $2^{15}=32\,768$ 次，使用一个或两个字节就可以来存储这个数值，其中只需要 1 比特作为长度指
187 示器。

2. 示例：固定长度压缩

给定任意一个索引词 t_1，我们考虑其索引的条目，假定 t_1 在文档 1、文档 3、文档 7、文档 70 和文档 250 中出现。BA 压缩使用最高的两个比特位来表明存储该值需要的字节数。对于前 4 个值，只需要一个字节就可以表示；对于最后一个值，180，需要两个字节来表示。需要注意的是：我们只需要计算倒排表中每一项的差。最后的差值为：250−70＝180。我们需要计算所有的最终数值。这些数值和压缩后的相应比特字符串见表 5-3。

表 5-3　字节对齐方式压缩

值	压缩的比特串
1	00 000001
2	00 000010
4	00 000100
63	00 111111
180	01 000000 10110100

在没有压缩之前，倒排表中的每条记录需要 4 字节，5 条记录总共需要 20 个字节。这些数值和它们对应压缩后的比特字符串详见表 5-4。

表 5-4 基准线：压缩前

值	未压缩的比特串
1	00000000 00000000 00000000 00000001
3	00000000 00000000 00000000 00000011
7	00000000 00000000 00000000 00000111
70	00000000 00000000 00000000 01000110
250	00000000 00000000 00000000 11111010

在这个例子中，未压缩的数据需要 160 比特，使用 BA 压缩后仅需要 48 比特。

5.1.3 变长索引压缩

Moffat 和 Zobel 也使用了倒排表中的差值。他们充分利用这样一个事实：对于大多数长的 188
倒排表，两个记录项的差值相对比较小。他们首先提到：可以在这些差值中发现一些共同的方法，其中哈夫曼编码就可以提供最好的压缩手段。采用这种方法，初始时扫描一遍文档就可以得到所有偏移量的频率分布规律。基于这些频率分布，就可以确立一种压缩模式。然后，再使用新的扫描算法进行第二遍扫描。例如，如果我们在整个索引中发现偏移量为 1 的频率最高，该算法将会使用 1 比特（bit）来表示偏移量 1。

Moffat 和 Zobel 使用了一系列通用编码，详见文献[Elias, 1975]。这种编码使用 $2\lfloor \log_2 x \rfloor + 1$ 比特来表示整数 x。前 $\lfloor \log_2 x \rfloor$ 比特是 $\lfloor \log_2 x \rfloor$ 的一元表示（一元表示是一种基于 1 的整数表示方式，这种表示方式仅使用数字 1。如数字 5_{10} 表示为 11111_1）。在前几位的一元表示后，下面一位是一个停止位 0 。这样，我们就可以表示出不超过数值 x 的 2 的最高幂对应数。接下来的 $\lfloor \log_2 x \rfloor$ 比特用来表示剩下的 $x - 2^{\lfloor \log_2 x \rfloor}$ 。

举例来说，比如十进制数 14 的压缩。首先，$\lfloor \log_2 x \rfloor = 3$ 表示需要使用一元表示法的 111 来表示。接下来一位是停止位。随后，剩下的部分 $x - 2^{\lfloor \log_2 x \rfloor} = 14 - 8 = 6$，由 $\lfloor \log_2 14 \rfloor = 3$ 得到，所以使用 3 个比特位，即二进制 110 即可存储。因此，14_{10}（表示的是十进制的数 14，下同）压缩后的编码为 1110110。

由于我们知道数字停止位前的 n 位，这样解压只需要扫描一遍，停止位后也有 n 位。前 8 位使用表 5-5 中给出的 Elias γ 编码。 189

表 5-5 γ 编码：前 8 个整数

x	γ
1	0
2	100
3	101
4	11000
5	11001
6	11010
7	11011
8	1110000

变长压缩举例

我们使用与上节中相同的例子，在表 5-6 中给出了差值为 1、2、4、63 和 180 的编码。这样编码仅需要 35 比特。与简单的 BA 压缩相比，它少用了 13 比特。同样，我们的例子中相对较大的偏移与较小的偏移的分布相同。实际收益是显而易见的，一些很小的偏移量存储只需要一个 1 或者一个 0。Moffat 和 Zobel 使用 γ 编码来压缩倒排表中的词频，但是使用一种更加复杂的编码方式来压缩倒排表中的记录项。

表 5-6　γ 编码

值	压缩的比特串
1	0
2	100
4	11000
63	11111011111
180	111111100110100

5.1.4　基于倒排表大小的变长压缩

我们可以使用 γ 方式生成基于向量 V 的一种编码范式，其中向量 V 由 i 个正整数组成，且 $\sum v_i \geqslant N$。为了对向量 V 中的整数 x（$x \geqslant 1$）进行编码，我们需要找到满足下式的 k 值：

$$\sum_{j=1}^{k-1} v_j < x \leqslant \sum_{j=1}^{k} v_j$$

换句话说，找出向量 V 的第一维分量，这样所有分量的和大于或等于数值 x 的进行编码。拿数值 7 为例，使用向量 V<1,2,4,8,16,32>，我们发现前 3 个分量(1,2,4)的和等于或者超过 7，所以 k=3。现在 k 就可以用一些形式进行编码（通常使用一元表示方法），后面跟着差值：

$$d = x - \sum_{j=1}^{k-1} v_j - 1$$

190　通过计算前几项的和，我们得到：$d = 7 - (1+2) - 1 = 3$，现在可以进行 $\lceil \log_2 v_k \rceil = \lceil \log_2 4 \rceil = 2$ 比特的编码。然后，我们进行抽象泛化，γ 方案可以看做是使用由 2 的幂组成的向量 V<1,2,4,8,…,>，并且以二进制对 k 进行编码。

显然，V 可以根据所给的不同压缩特性进行改变。对于较小的数，使用 v 中较小的值来优化压缩，而 v 中同样提供了较大的值对较大的数据进行压缩。文献[Zobel 等人，1992]给出了一种聪明的解决方案。对于每一个倒排表使用不同的 V，这样 V=<b,2b,4b,8b,16b,32b,64b, …,>，其中 b 是倒排表中已知偏移量的中位数。

1. 基于倒排表大小的压缩举例

还是使用我们的例子——1, 2, 4, 63, 180，中位数 b，在向量 V=<4, 8, 16, 32, 64, 128, 256> 中有 4 个结果。表 5-7 给出了本方案中 5 个倒排表的一个例子。

表 5-7　基于倒排表的变长压缩

值	压缩的比特串
1	000
2	001
4	011
63	11110000010
180	1111100110111

这种方案也需要 33 比特，并且我们可以看到，在本例中使用的中位数并不是一个好选择，因为数字的改变较大。在比较典型的倒排表中，由于数字均匀地靠近中位数，所以可以得到更好的压缩效果。

2. 基于吞吐量优化的压缩

Ahh 和 Moffat 研究出一种保证在有效的查询处理时快速解压，而且压缩率相当高的索引压缩方法[Anh 和 Moffat，2004]。他们研究出一种变长编码方法，可以充分利用每一个倒排表中文档标识符偏移量的分布规律。这种方法综合了位对齐压缩和字节对齐压缩；每个 32 位的字包含一个可变整数的编码，但是这个字内的每一个整数使用相同的比特数进行编码。这样，一个字可以分为两部分：用来作为“选择器”的比特位以及用作存储数据的比特位。

选择器部分包含一个字间划分策略表的索引，该表基于存储数据的比特数对数据进行划分， 191
以确保字中的整型数据编码使用相同的比特数。基于倒排表中最大的文档标识符偏移量来选择适当的划分策略。Anh 和 Moffat 提出 3 种基于这种策略的不同方案，它们的差异主要在于一个字中比特的划分。

- Simple-9：使用 28 比特来存储数据，4 比特来存储选择器域。选择表有 9 行，也就有 9 种不同的方式来相等地划分 28 比特。
- Relative-10：与 Simple-9 类似，但是仅使用 2 比特来存储选择器域，剩下 30 比特有 10 种划分方法。主要不同在于：仅使用 2 选择器比特，每个词仅能从 10 种可用划分中选择 4 种——与先前词的选择器的值有关联。与 Simple-9 相比，这种算法有一定的提高。
- Carryover-12：这种方法是对 Relative-10 的一种改进。由于划分浪费的一些空间通过将未用完的比特来存储下一个词的选择器值进行重新利用，这样就可以使用所有的比特来存储下一个字。这种方法是 3 种方法中压缩效率最高的，当然也是最复杂的，需要更多的解压时间。

3. Simple-9 举例

我们继续使用先前所用的例子，差值分别为 1、2、4、63 和 180，表 5-8 给出了选择表。为了找出合适的编码方式，我们对表中的每一行进行检查。由于在第一个偏移量为 28 处有一数值大于 2^1，所以不能使用 *a* 行。我们也不能使用 *b* 行，这是因为在偏移量为 14 处有一数值大于 2^2。继续向下检索这张表，我们发现同样不能使用 *e* 行，这是因为在第一个偏移量为 5 处有值大于 2^5。我们必须使用行 *f*，这是由于前四位表示偏移量的最大值（63）小于 2^7=128。对于前四个偏移量，就得到了 4 个 7 比特的编码（见表 5-9），其中行 *f* 的编码为 4 位（例如 0101）。

表 5-8 划分 28 比特数据的 9 种不同方式（使用 Simple-9）

选择器	编码	长度（比特）	未使用的比特数
a	28	1	0
b	14	2	0
c	9	3	1
d	7	4	0
e	5	5	3
f	4	7	0
g	3	9	1
h	2	14	0
i	1	28	0

表 5-9　Simple-9 示例

值	压缩的比特串
1	0000000
2	0000001
4	0000011
63	0111110

这个例子中的最后一个偏移量 180，在前 32 位的词的表示范围内不能表示。因此，我们需要增加一个 32 比特来进行编码。这样，使用 Simple-9 来对我们的例子进行压缩，总共需要 64 位。这个例子表明：这些压缩方法对于较长的倒排表是最有效的。此外，由于每个词的每个数值编码为固定长度，可以明确的是这些方法考虑到了非常快的解压时间。

4. 块寻址的索引压缩

另外一种减少索引大小的方法是创建一个可以对固定大小的文本块进行寻址的索引。这种
192 文本块中可能包含不止一篇文档。通过线性扫描这些块，可以对特定的索引项计数。这可以均衡考虑创建索引的时间和用于提高索引与查询处理速度的存储方式[Navarro 等人，2000]。

5.1.5　索引剪枝

到目前为止，我们已经讨论完了倒排索引的无损压缩方法。一种有损压缩方法称作静态索引剪枝，其基本思想可以参见文献[Carmel 等人，2001]。本质上，倒排表的记录项在不显著降低准确率的情况下可以删除或者剪枝。科研人员进行了关于索引项的特定剪枝和统一剪枝的实
193 验。使用特定的索引项剪枝，每个索引项有不同的剪枝级别。静态剪枝使用统一的方式简单地去除倒排表中的记录项——不考虑索引项。实验表明，在 70%的倒排索引上进行分层次的剪枝不会显著影响平均准确率。如何采用硬件实现这种基本方法，可以参见文献 [Agun 和 Frieder，2003]。

5.1.6　在构建索引前对文档重新排序

如果我们使用一种算法在压缩倒排索引前对文档重新排序，索引压缩的效率同样可以得到提升[Blandford 和 Blelloch，2002；Silverstri 等人，2004]。由于许多编码方式的压缩效果很大一部分取决于文档标识符间的间隔，其基本思想是：如果可以恰到好处地将文档作为算法的输入，那么，我们就能够减小平均间隔，由此实现最大化的压缩。以文档 d_1, d_{99} 和 d_{1000} 为例，这些文档都包含相同的索引词 t。对于这些文档，我们得到索引词 t 对应的倒排表记录项——d_1, d_{51}, d_{101}。

每个倒排表记录项间的文档间隙是 50。无论采取什么样的方式，我们在提交文档进行索引之前可以对文档编号进行重新整理。我们可以按照 d_1,d_2,d_3 的顺序提交这些文档，这样就完全消除文档的间隔。我们注意到，对于文档 D，有 2^D 种可能的次序，所以任何一种尝试排序文档的方法都要面对显著的扩展性问题。

该算法在提交文档进行索引前将文档与其他文档进行比较，其基本思想为相似的文档将包含相似的索引词，并且如果基于文档间的相似度进行排序，就能削减文档间隙。每一种算法使用 Jaccard 相似系数（见 2.1.2 节）来计算文档相似度。我们提出了两种基本方法：自顶向下（从作为整体的文档集开始）或者自底向上（从每个单独的文档开始）。

1. 自顶向下

一般来说，这两个自顶向下算法由 4 个主要阶段组成。在第一阶段（称作中心点选择），从文档集中选择出两组文档并且用作下一阶段的分割。在重新分配阶段，所有剩下的文档在选择的中心点中通过相似系数来切分。在递归阶段，先前的阶段以递归的方式在这两个结果集划分中重复执行，直到每一个都变成单例。最终为合并阶段，从每一个递归调用中自底向上地合并形成每一个划分，从而创建一个顺序。

第一种自顶向下的算法被称作事务型 B&B，这是由于该算法是 Blelloch 和 Blandford 算法的一种实现，详细可以参见文献[Blandford 和 Blelloch，2002]。这种重新排序的算法在 4 种压缩算法中压缩比最佳，但不具备可扩展性。

第二种自顶向下的算法被称作二分法，这样命名是因为它的中心选择阶段由随机选择的两篇文档作为中心，因此大大降低了这一阶段的开销。由于其中心选择如此简单，二分法的压缩效果较低，但是效率较高。

194

2. 自底向上

自底向上算法开始认为文档集中的每篇文档是独立的，并且它们基于相似度逐渐聚成组。第一个自底向上的算法受到了文档聚类中非常流行的 K-Means 方法（见“K-Means 算法”一节）的启发。第二个算法使用 k-scan 算法———一种基于中心向量查找算法的简单版本的 K-Means 算法。

K-Means 算法最初选择 k 篇文档作为类的表示，并且依据相似度将其他的文档赋值给这些类。在第一遍的最后，重新计算类的中心，依据与新中心的相似度，各个文档重新归到新的类别中。这种迭代一直进行，直到类的中心趋于稳定。这种算法的 single-pass（单遍法，指的是不回溯处理，一遍完成）版本仅执行该算法的第一回合，并且作者使用 Buckshot 聚类技术选择 k 个初始中心（见“Buckshot 聚类算法”一节）。

k-scan 算法是 single-pass 的 K-Means 算法的一种简化版本，仅需要 k 步就可以完成。在每一步中适当地形成各类，开始的时候选择一篇文档作为一个类的中心，然后将一部分未赋类别的文档赋值给相似度最高的类别。

5.2 查询处理

最近的研究集中于提高查询的运行时性能。Moffat 和 Zobel 已经证明，通过修改倒排索引来支持快速倒排表扫描，可以提升查询性能[Moffat 和 Zobel，1996；Moffat 和 Zobel，1994]。其他工作已经表明，只需要检索较少的查询词，就可以得到较好的准确率和召回率[Grossman 等人，1997]。通过降低向量空间模型中的复杂度，可以进一步消减计算量[Lee 和 Ren，1996]。

5.2.1 倒排索引的修订

Moffat 和 Zobel 提出如何切分倒排表，以便快速搜索倒排表来定位到特定的文档[Witten 等人，1999]。对于查询中的每一个词，经典的排序算法要扫描整个倒排表。对于倒排表中的每一条记录，更新一系列文档的得分。Moffat 和 Zobel 建议应该首先处理词频最小的词。

这样做的前提是：低频词承载了大部分的含义，并且很可能使所在文档排名靠前。我们要处理这些词的全部倒排表。一些算法建议在 d 篇文档的得分赋值非 0 后停止处理。这样做的前

195

提是：此时，查询中的高频索引词将直接产生文档的得分，而且这些文档不会在最终的前 t 篇文档中出现，其中 t 是需要显示给用户的文档数。

一种建议的改进方案是继续处理查询中所有的词，但是仅仅更新 d 篇文档中可以找到的词的权重。换句话说，在达到 d 分的某些阈值之后，剩余查询将变为 AND 结果的一部分（它们仅仅增加包含另外一个查询词的文档的得分），而不是向量空间中常用的 OR 操作。此时，我们可以很容易地改变嵌套循环的顺序，该嵌套循环用来给相应文档增加得分。在得分达到 d 之前，基本的算法如下：

```
对于查询 Q 中的每一个词 t
    获取词 t 对应的倒排表记录项
    对于每一个记录 t，它在文档 i 中的倒排表记录项
        更新文档 i 的得分
```

对于查询词对应的倒排表较小时，内层循环还是比较小的，然而当非常频繁地处理索引词时，特别长的倒排表就非常普遍了。同样，在访问 d 篇文档后，就没有必要更新每篇文档的得分，而只需要更新那些得分为非 0 值的文档。

为了避免扫描非常长的倒排表，修正后的算法如下：

```
对于查询 Q 中的每一个词 t
    获取包含词 t 的文档的倒排表 p
        对于在倒排表中 d 篇文档中的每一篇文档 x
            扫描倒排表 p，找到文档 x
            如果 x 存在
                更新文档 x 的得分
```

这里的关键是倒排索引必须转换成允许快速访问倒排表记录项的形式。我们假定倒排表中的每条记录是按文档标识符排序的。当遇到一篇新文档时，这篇文档的记录可以追加到现有倒排表后。Moffat 和 Zobel 提出，通过分割倒排表和在每一个分割块中增加指针来改变倒排表。我们可以通过检查第一个分割块的指针（指针包含本分块中最大的文档标识符和下一分块的指针）快速扫描倒排表。这种检查指是否需要跳转到下一分块，或者当前分块是否需要扫描。这种处理一直继续，直到找到分块，我们要查找的文档与在这个分块中的元素相匹配。在 TREC 数据中运行对应的 TREC 查询集合，每一块的大小 d 设置为 1 000 时，可以得到最好的 CPU 时
196 间[Moffat 和 Zobel，1996]。

5.2.2　部分结果集检索

另外一种提高运行时性能的方法是在计算资源的开销达到一些阈值后停止处理。一种方法是计算磁盘的 I/O 操作，并且在 I/O 操作达到一定阈值后停止计算[Yee 等人，1993]。这种方法的关键是对查询中的词进行排序并且以这种顺序进行处理，排序依据是索引词优良的一些指标。通过这样做，在处理完重要的索引词之后，查询处理就停止。对这些索引词排序类似于对它们的倒排表进行排序。度量倒排表特性的方法有 3 种。

1. 基于文档频率的裁剪

最简单的索引词质量的度量方法就是依赖于其文档频率。这种方法可以参见文献[Grossman 等人，1997；Grossman 等人，1994]。文献表明：在 TREC-4 文档集上，根据文档频率排序后，

使用 25%到 75%之间的查询词的结果几乎不会引起准确率和召回率的降低。在一些情况下，使用少数索引词，准确率和召回率还会提高，这是因为排序靠后的词有时就是噪音词，比如 good、nice、useful 等。这些词的倒排表特别长，结果是对成千上万个文档计算得分，而且对于查询结果的效果提高甚微。使用词频是一种实现动态停用词表的方法，而且不必使用静态的停用词表，高频词就可以排除出去。

2. 基于最大估计权重的裁剪

另外两种对查询词进行排序的方法可以参见文献[Yee 等人，1993]。第一种是计算给定查询词的最大词频 $tf_{\max}$，并且使用下面的公式作为排序的度量：

$$tf_{\max} \times idf$$

其思想为：在所有文档（仅指所有出现了该索引词的文档）中频率高的词比频率低的词应该更重要一些。我们假定最大词频是索引词在一篇文档中出现频率的较好的指标。

3. 基于倒排表中磁盘页权重的裁剪

基于索引词权重的裁剪可以用来度量倒排表的特性并选择出首先处理哪些倒排表。问题是倒排表可能特别长，并且偏差比较大。为了避免这个问题，一种新的度量方法是按倒排表使用的磁盘页进行排序，而不是按整个倒排表排序。在创建索引时，倒排表按索引词频降序存储，并且索引包含倒排表中每一页的记录，而不是仅包含一个指向第一条记录的指针。记录项记录 197
了一个给定页中的最大词频。然后，倒排表按如下公式进行排序：

$$tf_{\max} \times idf \times f(I)$$

其中函数 $f(I)$表示一页中的记录数。由于一些页不满，并且需要进行归一化，这样它们就不会和满页一样精确地排序，因此这是有必要的。$f(I)$使用的值为 i^e，其中 $0 < e < 1$。

遗憾的是，这种度量在索引中需要为倒排表中每一页添加一条记录项。然而，结果表明（对于不同长度的查询），如果访问所有 100%的磁盘页，仅检索大约 4 0%的磁盘页就可以获得 80%的文档。所有的这些实验使用的都是较小的文档集。

5.2.3 简化向量空间

最近的研究表明，在许多情况下，仅有限下降准确率和召回率即可简化向量空间[Lee 等人，1997]。在这项工作中，使用基于余弦方法（见 2.1 节）的 5 种方法进行了测试，测试文档集为 5 个小的文档集和 TREC 文档集中的《华尔街日报》部分中的 10 000 篇文章。首先，我们回顾一下基本的余弦系数公式：

$$SC(Q, D_i) = \frac{\sum_{j=1}^{t} w_{qj} d_{ij}}{\sqrt{\sum_{j=1}^{t} (d_{ij})^2 \sum_{j=1}^{t} (w_{qj})^2}}$$

第一种修改版本是替代文档长度归一化，不使用基于文档 D_i 中词数的平方根的权重计算方式。第二种变化是直接去掉文档长度归一化（直接使用点积），如下所示：

$$SC(Q, D_i) = \sum_{j=1}^{t} w_{qj} d_{ij}$$

第三种方法是去掉 *idf*。这将在倒排表中少记录一项。

198

$$SC(Q, D_i) = \sum_{j=1}^{t} tf_{qj} tf_{ij}$$

第四种方法是去掉 *tf* 但是仍然保留 *idf*。这将使倒排表中的每一项不需要存储 *tf*。这将显著消减计算量、存储量和 I/O 开销。

$$SC(Q,D_i)=\sum_{j=1}^{t} w_{qj}w_{ij}$$

如果词 j 在查询中出现，则权重 w_{ij} 为 1，否则为 0。如果词 j 在文档中出现，则 $w_{ij}=idf_j$，否则为 0。

第五种也就是最后一种方法，直接计算查询和索引项的匹配数目。

$$SC(Q,D_i)=\sum_{j=1}^{t} w_{qj}w_{ij}$$

其中如果词 j 在查询中出现，则 w_{qj} 为 1，否则为 0。如果词 j 在文档中出现，w_{ij} 等于 1，否则为 0。

在 TREC 的子集上，我们进行了两组实验。第一组是使用 TREC 描述信息（长查询），第二组使用 TREC 概念（短查询）。在长查询中，基准余弦方法性能最好，使用了文档长度取平方根的归一化方式的效果较好。短查询的结果比较有意思，第四种和第五种（没有使用 *idf*，直接匹配词计数）方法比基准方法的准确率更高一些。对于这些有些奇怪的结果，唯一的解释是，短查询实际上比较明确，所以额外的权重没有多少效果。

5.3　签名文件

签名文件在顺序扫描原始文本和构建倒排索引中起着重要的作用。签名是文档的一种编码。基本思想是使用相对较小的签名来对所有文档进行编码（通常的目标是只需要几个比特来表示签名）。一旦完成签名，只需要扫描签名，而不需要扫描原始文档。通常，一个签名不能唯一表示一篇文档（即一个签名表示多篇文档），所以通常有必要分两个阶段来实现检索。第一个阶段扫描所有的签名并且标识所有的命中结果，第二个阶段是在所有命中结果中扫描所有文档的原始文本，保证这些命中结果是正确匹配项。因此，签名文件要与模式匹配综合使用。图 5-2 表明了文档到签名的映射。

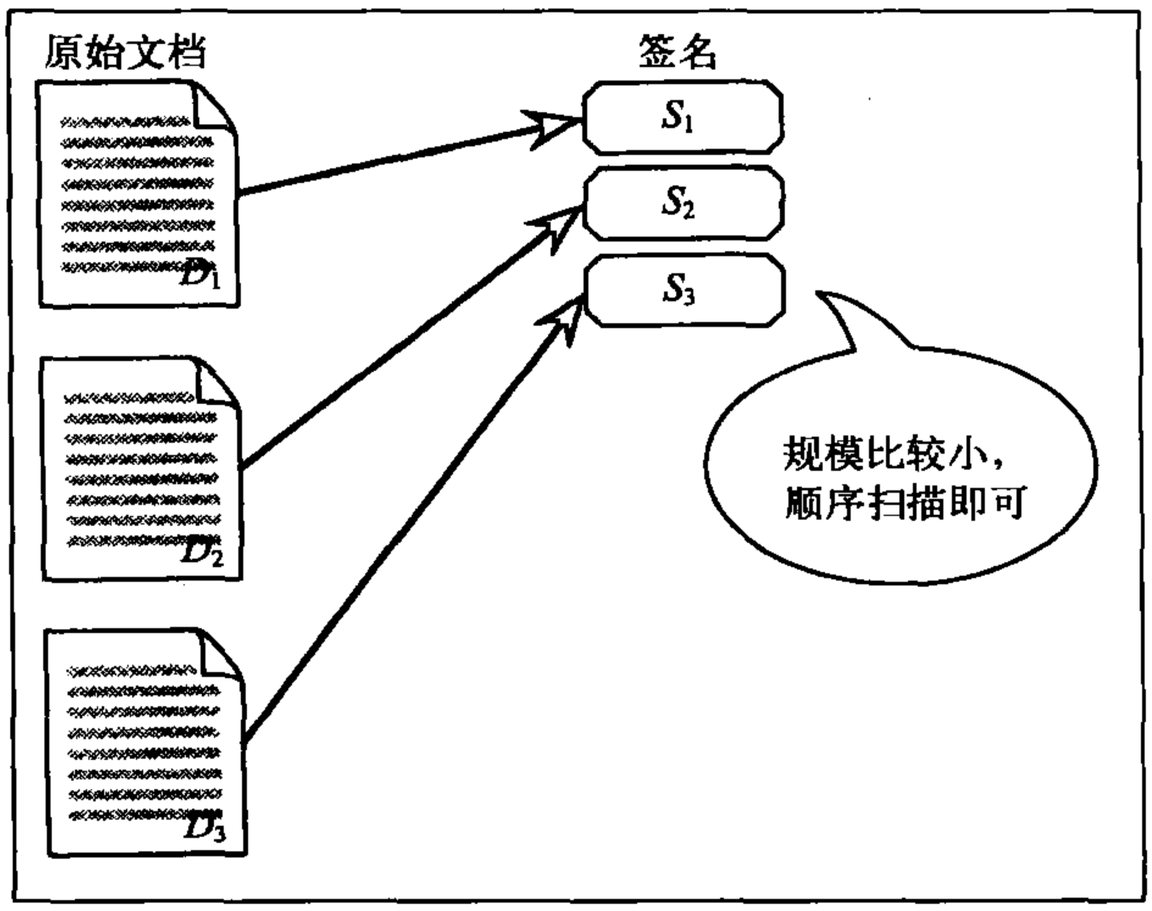


图 5-2　文档到签名的映射

签名通常使用不同的散列函数来构建。我们对文档中的每个词应用一个或者多个散列函数。 199
通常，散列函数用来设置签名中的一个比特。例如，如果文档中包含词“信息”和“检索”，并且 h（“信息”）和 h（“检索”）分别对应到第一个和第四个比特，那么这篇文档对应的一个四比特的签名就是 1001。

当一个词不在 w 签名列表中，而该词与列表中的某个签名有着相同的位图，就会发生错误的匹配。例如，词 t_1 在签名中将第一位和第三位置为 1，词 t_2 在签名中将第二位和第四位置为 1。第三个词 t_3 可能对应第一位和第二位比特，因此就认为与签名文件匹配，即使 t_3 与 t_1 和 t_2 并不相等。表 5-10 给出了刚刚讨论的 3 个词和它们对应的散列值。

表 5-10 构造签名

词	*h(term)*
t_1	0101
t_2	1010
t_3	0011

下面我们考虑一个例子，文档 d_1 包含词 t_1，文档 d_2 包含 t_1 和 t_3，文档 d_3 包含 t_1 和 t_2。表 5-11 给出了这 3 篇文档的签名。

表 5-11 文档签名

文档	签名
d_1	0101
d_2	0111
d_3	1111

因此，查找词 t_3 的查询将会在文档 d_3 中得到一个错误的匹配，尽管文档 d_3 中并不包含词 t_3。 200

通过将签名长度扩展到 1 024 比特，并且在一个签名中存储较少的词数，错误匹配的几率将会降低到 3%以下[Stanfill 和 Thau，1991]。

为了实现文档检索，对于每一个查询，构建一个签名。在查询签名和每篇文档签名中执行布尔“与”操作。如果布尔“与”返回 `true`，文档就被添加到可能的命中列表中。类似地，如果仅需要找到查询词在哪些文档中出现，可以使用布尔或操作。为了最小化地减少正例误判的情况，我们对同一个词应用了多个散列函数[Stanfill 和 Kahle，1986]。

布尔签名不能存储近似的信息或者文档中出现的词的权重信息。大多数相关性度量方法都认为包含查询词多次的文档将比仅包含这个词一次的文档的排序位置靠前。使用布尔签名，不能表示出一篇文档中词出现的次数，因此这些相关性度量方法也就不能实现。

如果将数据加载到内存，那么数据签名就非常有用了。同样，在签名文件中增加或者删除文档都比在倒排索引中容易得多，并且签名文件中记录项的顺序无关紧要。这种有些无序的处理非常适合并行处理（见 7.1.2 节）。然而，我们有必要一直对错误的匹配进行检查，并且基本定义并不支持查询排序。一种支持文档排序的修正方案是将签名文件划分成组，每个词频的文件划分成一组[Lee 和 Ren，1996]。

通过扫描来删除错误的正例

一旦签名找到匹配项，就要使用扫描算法来验证这种匹配是不是因为冲突引起的错误的正

201 例。我们不详细介绍这些方法，许多文本扫描算法的详细实现可以参见文献[Lecroq，1994]。我们可以使用不需要错误验证的签名算法（如果使用足够长的签名），并且准确率和召回率不会有明显降低[Lee 和 Ren，1996]。然而，为了完整起见，这里简要介绍文本扫描算法。

信息检索中使用的扫描涉及模式匹配算法，这是因为扫描是为了从一个文本字符串中找出一种模式。一般情况下，模式匹配定义为查找包含给定模式起始位置的输入文本的所有位置。如果模式的大小为 p，文本的大小为 s，简单的嵌套循环模式匹配的时间复杂度为 $O(ps)$。

Aho 和 Corasick 的算法实现了确定性有限状态自动机，用来标识文本中的匹配项[Aho 和 Corasick，1975]。Knuth、Morris 和 Pratt(KMP)同样给出了一种时间复杂度为 $O(s)$的算法，它从前向后扫描文本，但是对匹配模式进行了预处理，以保证系统可以安全地跳转到适当的位置[Knuth 等人，1997]。

Boyer-Moore 算法是使用另外一种预处理模式的方法，但是它首先处理模式的最后一个字符，然后回退到字符串的起始位置。人们研究出两种预处理函数，当模式中发生重复时，确定跳过模式中的部分，并且跳过的文本直接不能匹配模式。这些函数使用从当前查找点收集到的知识[Boyer 和 Moore，1997]。算法将时间复杂度提高到了线性时间，甚至在当前串有多处模式出现[Galil，1979]。

后来，在 20 世纪 80 年代，人们提出一种通过在模式及其后边的 p 个字符应用散列函数进行模式匹配的算法，详细可以参见文献[Karp 和 Rabin，1987]。如果散列函数中发生了匹配（即 h(pattern)和 h(text)发生冲突），就需要检查文本和模式的内容。目的是减少错误的冲突。使用大素数会极少发生冲突，如果有的话。另外一种模式匹配算法可以参见文献[Frakes 和 Baeza-Yates，1993]。这种算法使用比特字符串集合，该集合表示当模式流经文本时不断更新的布尔状态值。

这些算法中最好的是线性运行时间 αs，其中 α 为常量，取值范围为 $0 \leqslant \alpha \leqslant 1.0$，并且 s 是字符串的大小。我们的目标是减小这个常量。在最差的情况下，必须进行 s 次比较，但是这些算法的平均情况经常是亚线性的。这些算法尽力避免在文本中回溯。通过每一次比较，扫描可以继续向前移动，有利于磁盘在物理上进行连续扫描。KMP 算法为许多模式构造了一个有限状态自动机，所以这属于直接应用。Uratani 和 Takeda 提出了一种算法，综合了 Aho 和 Corasick 提出的有限状态自动机方法和 Boyer-Moore 避免大量查询空间的思想。本质上，有限状态自动
202 机使用 Boyer-Moore 提出的降低查找空间的一些方法来构造。有限状态自动机自右向左扫描文本，这与 Boyer-Moore 方法相同。需要注意的是，这适合查询中包含多个词的情况[Uratani 和 Takeda，1993]。与不断使用 Boyer-Moore 算法进行直接比较，对于一个包含 100 个模式的查询，Uratani-Takeda 算法探测次数减少了 10 倍。当只有两个模式时，Boyer-Moore 算法的平均探测率（文本引用探测的次数与文本长度的比率）为 0.236，而 Uratani-Takeda 算法的平均试探率为 0.178。

5.4　重复文档检测

一种能够同时提高信息检索系统效果和效率的方法是去除重复和近似重复的文档。重复文档可以在添加到倒排索引前删除或者在检索结果中去除。这样做的难点是我们不是只希望删除严格重复的文档，而是同样能够比较好地去除近似重复的文档。然而，我们不希望将近似重复的阈值设置得太宽，而将事实上用户更偏向将非常不同的两个文档视为重复的。

对于 Web 搜索，文档重复问题尤为严重。对于查询词“apache”的搜索可能会产生许多关

于 Web 服务器产品的网页副本和许多关于印第安部落的副本。其实结果应该只显示给用户两个超链接，但是现在却显示了成千上万个超链接。此外，这些冗长的网页影响到了词项和文档的权重计算方式。而且，增加了构建索引的时间，并且影响了检索效率[Chowdhury 等人，2002；Cho 等人，1999]。

5.4.1 精确重复检测

重复检测经常通过为每篇文档计算一个唯一的散列值来实现。然后通过查找散列值检查每一篇文档是否重复，散列值或者直接在内存中存储或者使用持续化存储查找系统。几种常用的散列函数有 MD2、MD5 和 SHA[SHA1，1995]。之所以使用这些函数，是因为它们可以达到我们期望的 3 个目的，即这些函数可以计算任意长度的文档，计算容易，并且冲突率低。

虽然这种方法速度快而且实现简单，但是问题是只能找出严格重复的文档。稍微改变一下（比如多一个空格）就会将两篇相似文档视为不同的文档。例如，一个显示访客数量和其他内容 203
的网页，即使文档内容一样也会产生不同的签名。唯一不同的是计数器，而这足够为每篇文档生成不同的散列值。

5.4.2 近似重复检测

尽管不可能精确定义一篇文档是不是另外一篇文档的副本，研究者还是提出许多确定文档是否与另外一篇文档重复的度量方式。第一种是*形似度*[Broder 等人，1997]。这个方式认为：如果一篇文档包含的语义内容大体相同，不管在句法上是不是严格匹配，我们都认定其是重复的。

$$r\left(D_i,D_j\right)=\frac{\left|S(D_i)\cap S(D_j)\right|}{\left|S(D_i)\cup S(D_j)\right|} \tag{5-1}$$

如公式（5-1）所定义，文档 D_i 与 D_j 的形似度 r 是两篇文档特征的交集与二者并集的比值。这种方式可以用来计算两篇文档的模糊相似度。例如，假定 D_i 和 D_j 共享 50%的词，每篇文档有 10 个词，那么它们的相似度就是 $\frac{5}{15}=0.33$。许多研究者已经探索使用形似度来为文档重复查找提供阈值 t[Brin 等人，1995；Garcia-Molina 等人；1996; Shivakumar 和 Garcia-Molina，1998；Broder 等人，1997；Shivakumar 和 Garcia-Molina，1998；Fetterly 等人，2003]，如果超过阈值 t，就认为文档是重复的。

当使用形似度时，需要探讨两个普遍的问题。第一个是应该使用哪些特征，以及阈值 t 为多少。第二个是当使用非常大的文档集时会引发效率问题，以及如何进行优化[Broder，1998]。普遍采用余弦值（见 2.1.2 节）来标明两篇文档之间的相似度。

对于重复检测，二进制的特征表现形式将两篇文档的相似度计算转化为基于词项的形似度计算。因此，随着两篇文档的距离靠近 1.0，待比较的特性就变得更相似。

1. shingle

我们所讨论的第一种近似重复检测算法就是使用 shingle（搭叠技术）。shingle 就是文档中连续词的集合。shingling 技术（例如 COPS[Brin 等人，1995]、KOALA[Heintze，1996]和 DSC[Broder，1998]）本质上都是比较匹配的 shingle 的数目。

文档子集的比较允许算法使用公式（5-1）给出的形似度计算出两篇文章的重叠率。这依赖于每篇文档子集或特征的散列值，并且过滤那些散列值来减少算法必须进行比较的数量。因此， 204

这种特征的过滤提高了运行时性能。需要注意的是，最简单的过滤是一种在句法上基于简单句法规则的的严格过滤，并且琐碎的子集就是整个文档集。在 shingle 方法中，不是比较文档集，而是比较子文档，这样每篇文档都可能产生许多可能的重复。返回许多潜在的匹配项需要大量用户来标识可能的重复，这样就降低了这类方法潜在的有效性。为了解决其本身客观存在的效率问题，研究人员提出了许多优化技术来减少比较的次数。

DSC 算法减少了使用的 shingle 的数目。在文献[Heintze，1996]中去除了高频的 shingle。在文献[Broder 等人，1997]中，每隔 25 个 shingle 就保存一个 shingle。然而这种消减降低了效果。更糟糕的是，即使移除相对低频的 shingle（只取那些在超过 1 000 篇文档中出现的词），并且每隔 25 个保留一个 shingle，DSC 算法处理 3 000 万篇文档需要 CPU 运行 10 天。

DSC 算法有一种效率更高的改进算法，DSC-SS，使用超级 shingle。算法将多个 shingle 合并成一个超 shingle。这样算法就使用比较少的超级 shingle，而不是使用许多 shingle。形似度定义为两篇文档中超级 shingle 是否匹配。这种方法效率很高，因为算法不需要统计所有重叠的 shingle 数。然而，作者注意到 DSC-SS 算法“对短文档效果并不是很好”，所以并没有给出运行时结果[Broder，1998]。这是合情合理的，因为超级 shingle 可能比较大，很可能会将短文档全部包含到超级 shingle。

2. 通过相似度进行重复检测

另外一种方法是直接计算两篇文档的相似度。如果相似度超过一定的阈值，就可以认为文档之间是相互重复的[Sanderson，1997；Buckley 等人，2000；Hoad 和 Zobel，2002]。这些方法类似于文档聚类（见 3.2 节）。遗憾的是，在这个方法中，所有的文档两两之间都要进行比较，即每篇文档都要与其他的文档进行比较，并且计算相似度。因此文档与文档的相似度计算方法的时间复杂度是相当高的，理论上时间复杂度为 $O(d^2)$，其中 d 是文档数。在实际中，这个方法
205 只用于计算具有重叠词的文档。这样，实际的运行时间根据数据的不同而不同，很难精确预测。

3. 将文档看做查询

另外一种方法是将每篇结果文档看做一个新的查询来检索与这篇文档匹配的其他文档。由于要在整个文档集上查询每篇文档，对于大的文档集或者动态文档集，这种方法在计算上并不可行。Sanderson 和 Cooper 在这方面使用了一种改进方法，使用 Rocchio 相关反馈选择词项（见 3.1.1 节[Sanderson，1997；Cooper 等人，2002]）。对于大的文档集，其中一个普通的词可能在数百万篇文档中出现，这就不具备可扩展性。使用词项选择方法明显减少了这种开销，但是仍然需要至少与通过相关反馈算法选择的词数同样多的 I/O 操作。Kolca 等人提出一种使用经典的余弦相似度的优化方法。在这项研究中，假定在超过 5%的文档集中出现的词实际上在每篇文档中都出现了。这种优化的效果能够达到在全部文档中使用余弦相似度效果的 90%，但是执行速度却提高了一个数量级[Kolcz 等人，2004]。

4. I-Match

I-Match 仅使用文档中的若干词进行散列。算法的关键是采用哪些词进行散列。I-Match 使用文档集的统计数据对一篇文档进行散列。例如，使用 idf 来标识哪些词应该用来作为比较的基准。我们可以使用文档统计数据来判断词项在进行文档重复检测中的有效性。先前研究表明，在文档集中频率比较高的词往往不会加入到文档的语义内容中[Grossman 等人，1995；Smeaton 等人，1997]。I-Match 认为删除频率较低的词或者频率非常高的词，可以在重复检测上更好地表示文档。算法的伪代码如下所示。

- 获取文档。
- 将文档分解为标记流，去除格式化标签。
- 使用词项阈值（idf）过滤，仅保留有用的词项。
- 将相关词项插入到词项 unicode 码降序树。
- 循环遍历词项树，将每个词添加到 SHA1[4]摘要。完成循环时，得到一个元组(doc_id, SHA1 摘要)。
- 使用主键将元组(doc_id,SHA1 摘要)插入到存储数据。
- 如果摘要值有冲突，那么文档就是相似的。

I-Match 方法的整个运行时间，在最差的情况下（即所有文档两两之间都是重复的），时间复杂度为 $O(d \lg d)$，否则时间复杂度为 $O(d)$，其中 d 是文档集中文档的数量。所有相似的文档必须进行分组。也就是说，对应的文档标识符要存在一个组中。在最极端的情况下，所有的散列值都一样（所有的文档两两之间都相似）。在这种情况下，将所有的文档标识符在一种数据结构（树）中分组存储的时间复杂度为 $O(d \lg d)$。然而，I-Match 方法通常的时间复杂度为 $O(d)$。*idf* 值的计算可以通过以下两种方式之一：第一种是在去重工作之间使用训练文档集生成词 idf 的元组集合。当文档集的大小增长时，结果表明词的 *idf* 权重改变比较小，所以这是一种可以接受的解决方案[Frieder 等人，2000a]。

另外一种方法是分两个阶段遍历文档集，其中第一个阶段计算词项的 idf 权重，第二个阶段使用 I-Match 算法找出重复项。这种方法实际上增加了算法的复杂度，但是理论上并没有改变。Conrad 等人检验了这些方法，发现当使用动态文档集和仅使用 idf 较高的词时，如果使用动态词表，结果是不稳定的[Conrad 等人，2003]。最近，他们开发出一种新的非严格重复文档检测的测试文档集[Conrad 和 Schriber, 2004]。他们的建议是第一种方法更适用于解决这个问题。

I-Match 生成单个超级 shingle，如果 shingle 的大小足以涵盖整篇文档，那么 I-Match 的时间复杂度与 DSC-SS 算法相当。否则，DSC-SS 算法产生 k 个超级 shingle，而 I-Match 只需要产生一个。由于 DSC-SS 算法中的 k 是一个常量，所以两个算法的时间复杂度是一样的。

在实际应用中，I-Match 算法效率更高。然而，I-Match 算法比 DSC-SS 算法的真正优越性体现在其不会忽略掉小文档。使用 DSC-SS 算法，对于足够小的文档，无法检测出用于重复性检测的 shingle。因此，那些短文档不在考虑范围之内，即使这些短文档可能是重复的。尽管 I-Match 效率很高，但是当对一些较小的改变进行散列时，I-Match 同样存在原始散列技术的脆弱问题。I-Match 算法最近的一项算法改进是使用随机词汇表，根据 idf 范围而各不相同。这些不同的词汇表可以生成一篇文档的多个签名。所有的这些签名都认为是有效的签名，这种改进降低了 I-Match 的脆弱性。Kolcz 等人证明这种随机化的方法在不损失 I-Match 准确率的情况下，使用 10 个随机词汇表时，召回率提高了 40%～60%[Kolca 等人，2004]。

5.5 本章小结

信息检索系统的性能评价同时涉及效果（精确度）和效率（运行时间和存储开销）这两个方面。在本章中，我们主要考虑了检索效率。

一开始，我们讲述了使用倒排索引的概念和动机。倒排索引是一种索引词到文档集的多对多的映射。使用倒排索引，仅需要访问包含特定查询词的文档，因此与其他查询处理结构相比，

明显降低了 I/O 需求。在介绍完倒排索引的概念后，我们继续举例阐述了倒排索引的具体实现方法，而且还说明了剪枝的一种修订版本。我们同样概括讲述了不同的索引压缩技术，主要回顾了两种压缩技术。第一种（固定长度的索引压缩）与第二种（变长索引压缩）方法相比，更加简单易行，而且在查询处理时间上更有效。然而，变长索引压缩的压缩率确实要好一些。

最后，概述了签名文件。签名文件包含文档签名的集合，每篇文档对应一个签名。一篇文档签名是每一篇文档的编码。文档中的关键索引词被散列到一个向量，文档 i 中存在词 j 表明在文档签名 i 的第 j 位记作 1。为了确定哪篇文档与特定查询相关，只需要检查签名文件。由于词的散列可能导致错误的正例指示，二阶段查找的策略是必需的。在第一个阶段中，涉及签名文件的检查，所有可能的候选项都需要确定。在第二个阶段，对第一个阶段确定的所有可能的候选项进行全文本扫描。

尽管传统上更多的精力都放在了信息检索的效果上，但是效率问题也是非常重要的。信息检索系统在效率优化上的失败可能导致一个精确率很高的系统在运行和存储性能上开销极高。随着存储技术继续提高，而且价格一直下降，存储限制变得不那么紧迫。然而，随着在线数据的指数级增长，存储仍然是一大关注热点，而且运行时性能被认为是最重要的。人们使用并行处理技术来提高整体的运行时性能，详细请参见第 7 章。

208

5.6　练习题

(1) 编写一个索引工具，为《爱丽丝梦游仙境》建立倒排索引。假定输入的 10 行作为一篇单独的文档。假定在创建倒排索引时，你有足够的内存来存储所有的倒排表。记录索引一共需要的空间和构建索引的时间。在索引中存储每一个词的 idf。每一个倒排表记录项应该包含文档中的词频。使用 100 个频率最高的词作为停用词。通过向量空间模型的 *tf-idf* 相似度计算方法，使用下面的 5 个查询测试你的索引。

- rabbit watch
- looking glass
- tea party
- cheshire cat
- queen of hearts

(2) 使用倒排索引压缩技术调整你刚刚写的代码。选用本章中的一种技术。度量 5 个同样查询的性能、存储负荷和创建索引的时间开销。接着使用剪枝索引重复上面的查询。

(3) 选择一个包含 10 个词的查询。执行这个查询并且检索出相关文档，从中选择出前 10 个相关文档。现在，按照查询词在整个文档集中的词频进行排序。仅使用一个查询词重新执行查询——整个文档集中出现次数最小的查询词。标出仅使用这一个查询词检索出的相关文档数。每次在查询中新增加一个词，重复这个过程。与原始查询一样，是不是需要所有的 10 个查询词才能检索出相关文档？讨论一下通过这个练习你可以得到什么结论，并且如何利用这项技术在不降低精确率的情况下提高运行时间。

(4) 开发一个基于签名的索引，你可以为书中的每篇“文档”构建一个签名。为每篇文档使用 24 位的签名。现在使用前面练习使用的 10 个查询作为简单的布尔或操作。与倒排索引相比，比较使用签名的运行时性能。描述一下使用签名所固有的功能损失。使用一种启发式规则，对于大的数据集，签名可以作为“第一遍过滤器”，然后叙述如何使用倒排索引进行具体的分析。

209

第6章

结构化数据与文本的融合

在前面的章节里，我们已经讨论了搜索和检索相关文档的许多主要问题。但是，仅仅对海量非结构化数据进行搜索查询，还是远远不够的。

当前，TB级的结构化数据已然存在。NCR公司最近展示了其支持300 TB数据的数据库系统[Holmes, 2004]。不难想象，数据库规模很快将增长至PB级。而DBMS（数据库管理系统）的研究重点是支持成千上万个并发用户对结构化数据进行增、删、改、查操作的算法。

我们很难对结构化数据的特征进行形式化定义。一般而论，结构化数据具备一定重复性，其数据往往可以规约为某个易于识别的数据类型，比如姓名、地址、电话号码和薪资。每个结构化数据项必须易于识别，有时甚至可以列出少量有效的项值列表（即性别仅有两个有效值——男性或女性）。

诸如我们日常生活当中的航空订票系统、自动取款机与信用卡验证设备等系统，都充斥着结构化数据。大型的结构化数据库产品有可能占用 300TB 存储器，使用 1 016 个处理器[Holmes，2004]。

结构化数据与文本的融合需求是显而易见的。大多数基于RDBMS（关系数据库管理系统）的商用系统，往往都包含文本类型的字段，如注释字段，这可以让用户在一定规则下随意输入一段文字注释。商用数据库系统允许用户在关系中以 BLOB（二进制大对象）或 CLOB（字符大对象）数据类型存储这些非结构化字段。实际问题在于我们不能高效地对它们进行访问。这里缺乏信息检索中类似倒排索引的检索方法，即便有类似的方法，往往也是采用一种非标准的方式实现的。

同样，信息检索系统通常也有大量的结构化信息（即文档作者、出版日期等），这些数据也可以按不同的域进行存储。我们往往采用专门的起止分隔符来标识这些数据域。这里的问题仍然是访问效率低下。在信息检索系统当中，往往缺乏一些检索结构化数据的最优访问方法（如B树）以及相应的查询优化技巧。

将 DBMS 与信息检索系统比作武术比赛，各自只在自己的领域当中进行比较与提高。就像跆拳道高手可以与其他跆拳道高手过招，信息检索系统能够有效地处理非结构化数据。而柔道高手可以对抗其他柔道高手，DBMS 亦能有效地处理结构化数据。当跆拳道高手碰上柔道高手时，问题便出现了，这与在结构化数据库系统内访问非结构化数据类似。

本章接下来介绍的方法是在已有 RDBMS 之上构建非结构化数据的处理技巧。与教柔道高手一些跆拳道的技术类似，但仍然要基于柔道的格斗模式。

我们可以在数据库系统的基础上进行扩展以处理非结构化数据，也可以在非结构化系统的基础上进行扩展以处理结构化数据。在这一章中，我们所采取的方法是对数据库系统进行扩展，即将信息检索看成数据库系统的一个应用（见图 6-1）。这样做的理由是，与信息检索系统相比，关系数据库系统经历了多年的发展，其理论基础与架构更加坚固。因此，要实现两者的融合，

较为直接的方法就是以数据库系统为基础，在其之上添加信息检索的功能组件。除了实现结构化数据与文本的融合之外，这种做法还可以给我们带来两个好处：并行处理与动态更新。并行处理能够利用多处理器的优势，提高系统的运行时性能。

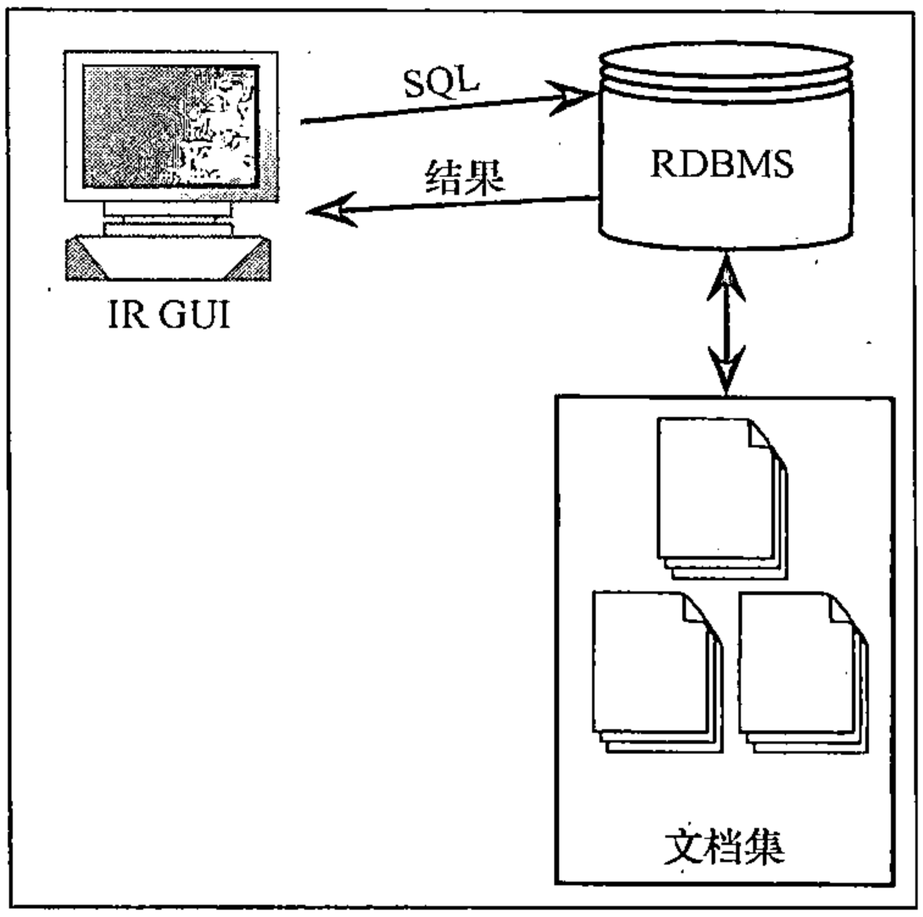


图 6-1　信息检索作为 RDBMS 的应用示意图

第 7 章将会介绍一些并行信息检索算法。虽然这些算法确实可以提升性能，但仍没有哪个算法在使用中取得特别明显的加速比。也就是说，我们添加的额外处理器并没有得到充分利用。但是，大多数主流的数据库商家（即 IBM、Sybase、Oracle 和 Microsoft）都提供了并行处理的
212 解决方案。还有一些数据库供应商专注于开发专用的并行处理硬件来定制数据库系统（NCR Teradata）。关系型基于集合论的运算符本质上具备无序性，这种无序性使得并行操作易于实现。我们将信息检索作为数据库的应用，因为底层的 DBMS 可以并行化，其上的应用自然而然就是一个并行信息检索算法。

把信息检索当做 RDBMS 的一个应用的另一个好处是：文档数据可以更加容易地进行更新。大多数信息检索系统在构建倒排索引时，都需要一个相当长的预处理过程。对于已存在文档进行的增删改操作，会导致倒排索引需要进行相应的修改。大多数信息检索系统并不支持在线文档更新。RDBMS 有一个较强的基础架构（并发控制和恢复管理）支持实时更新，并且更新过程中若出现异常，整个更新可以回滚，不会有不完整作业遗留在数据库系统当中。

仍然存在一个关键的问题：到底应该选择使用哪个数据库模型，以及如何添加信息检索功能组件？数据库系统模型分为倒排表型、层次型、网状型、关系型和面向对象型[Date，1994]。目前，大多数商用系统都是基于关系模型的。研究其他模型是否能更好地支持非结构化数据这
213 个课题虽然很有意思，但使用关系模型更加实际。过去的一年中，RDBMS 软件的销售已达 71 亿美元。假如我们使用其他数据模型来实现文本数据的融合，这将意味着无数场所实际使用的 DBMS 模型都需要转换更新，这样的代价将是天文数字。

要判断关系系统能够主导市场多长时间，比较有效的方法是看看其之前的系统。层次系统 IMS 以及网状系统 IDMS 曾经在 20 世纪 70 年代风靡整个市场。至 1980 年，两者发展仍然良好。此时，第一个关系系统供应商 Oracle 成立。20 世纪 70 年代中期，关系系统在学术界呼声

颇高，人们研制了相对完备的原型系统 System R，夯实大量工作的基础。

IBM 公司于 1984 年推出了它的第一个商用关系系统 SQL/DS，并于 1986 年推出后续版本 DB2。关系系统初期的市场占有率并不显著，直到 20 世纪 90 年代初，即 Oracle 公司成立 10 周年的时候，整个状况大为改观。我们首先要学会使用关系模型，随后将讨论如何使用最近研发的多维数据库系统。

最后一个问题：如何去运用关系模型？存在两种选择：扩展关系引擎，或是将信息检索作为 RDBMS 的应用。

6.2 节主要回顾早期扩展关系模型的相关工作。这些工作的主要问题是缺乏标准。每种扩展多少都有不同之处，用户不能将应用程序从一个系统放到另一个系统之上，不具备可移植性。另一个问题是，为了支持在数据库引擎上的每一个附加扩展，我们每次都要修改查询优化。甚至，每增加一个新功能，都要使原本就很复杂的引擎愈加复杂。一些接口允许用户在引擎上扩展功能，但却使恶意或是粗心的用户有意无意间给数据库引擎引入新的漏洞。最后，我们不得不为每一个附加扩展开发相应的并行算法。

如果将信息检索作为 RDBMS 的应用，以上问题均可避免。研究重点变为，为每类信息检索功能研制高效统一的 SQL（结构化查询语言）算法，其中 SQL 算法遵循 ANSI SQL-2003 标准[SQL，2003]。本章介绍了以下信息检索方法：

- 布尔型关键字搜索；
- 邻近搜索；
- 基于词项的相关度排序；
- 相关反馈。

214

按照本章后面所述方法稍作修改，我们在标准 SQL 关系 DBMS 之上实现了基于词语级、短语级、段落级、*n* 元语法或是相关反馈进行的相关度排序。详情可参见文献[Lundquist 等人，1997；Grossman 等人，1997]。

6.1 节简要回顾一下关系模型与 SQL。6.2 节给出了数据库技术与信息检索融合方面的工作进展。6.3 节描述的一些算法仅使用标准 SQL，将前面章节所述的信息检索模型与策略作为关系系统的应用。

接下来，6.4 节描述了一个支持半结构化数据查询的方法。这里，我们主要介绍如何建立一个功能齐全的 XML 搜索引擎，当然仍然将其作为关系数据库系统的附加应用。总而言之，我们将通过标准的关系数据库技术，全面介绍结构化、半结构化及文本数据的融合方法。

紧接着，6.5 节描述了一个多维数据模型，支持层次化结构数据与文本的融合。使用这种方法，直接通过整合的组织架构，就可以支持客观存在的层次关系。

最后，在 6.6 节中，我们介绍了一种将不同地理位置上分散存储的跨域数据进行融合的方法，从而支持问答范式。跟以前一样，最后我们会总结本章内容，并对未来工作进行展望。

6.1 关系模型回顾

关系模型方面的文献最初出于 Codd 之手 [Codd，1970]。之前的数据模型都是导航型的，应用程序开发人员不得不记录数据库的遍历方式，他们需要明确地告诉系统如何定位所需数据。关系模型将数据存储在关系中，开发人员只需简单地描述需要哪些数据，而不用考虑如何

获取这些数据。在 20 世纪 70 年代初期，关系系统并未得到长足发展，主要源于其额外的计算与存储开销。经过数年努力，人们研制出新的算法来提高查询优化的性能。这些算法应用于关系系统之后，大大消减了这些额外的开销。

随着时间的推移，其带来的好处逐渐超过其开销，现在已成为大多数商用数据库系统产品的核心。只在一些性能要求极高的应用上，人们仍然会使用导航型系统，不过，关系系统依然
215 占据主导地位。

关系数据库的基本概念及相关术语

关系数据库系统将数据存储在关系当中，这里关系是以集合论为基础的概念。其中，关系中的属性是指在有限集$\mathcal{L}=\{A_0, A_1, A_2, \cdots, A_n\}$里面的任意符号。集合$\mathcal{L}$上的关系$\mathcal{R}$是笛卡儿积$\mathrm{dom}(A_0)\times\mathrm{dom}(A_1)\times\mathrm{dom}(A_2)\times\cdots\times\mathrm{dom}(A_n)$的一个子集，其中 $\mathrm{dom}(A_i)$表示 A_i 域值集合。$R[A_0\ A_1\ A_2\cdots A_n]$表示集合$\{A_0, A_1, A_2,\cdots, A_n\}$上的关系$\mathcal{R}$，称为$\mathcal{R}$的模式。在 $R[A_0 A_1 A_2\cdots A_n]$中，列 A_i称为 R 的一个*属性*，记为 $R.A_i$。

简单来说，每个属性里面都包含数据值，它往往是从给定值域中选择出的单一值。比如，颜色*属性*对应的值域为*红*、*绿*、*黑*等。关系是属性的集合。相对每个属性，关系中的行或称为元组都有一个值，这个值来源于属性的域。

R 元组由$<a_0, a_1, a_2,\cdots, a_n>$指明，其中 $a_i\in\mathrm{dom}(A_i)$。任取 $x\in R$,其属性 A_i的值记为 $x[A_i]$。同样，任取 $x\in R$，$\mathbf{W}$ 为 x 元组的属性集时，$x[\mathbf{W}]$代表属性集合值。

考虑一下关系 EMP 和 EMP-PROJ（见表 6-1 和表 6-2）。EMP 关系有 4 个属性（emp_no、emp_name、age 和 salary），而 EMP-PORJ 关系有两个属性（emp_no 和 project）。组织中的每个员工对应于关系 EMP 的一个元组，以确定员工的唯一 ID、姓名、年龄和工资。当然，每
216 个员工可能会被分配给任意数量的工程，只在 EMP 关系中加入 project 属性是起不到效果的，因为它只能存储一个单值。即便换一种方案——加入 project1、project2 和 project3 三个属性，仍然不足以解决问题，因为员工可能会分到更多数量的工程。这时，便没有地方对第 4 个乃至第 n 个工程进行存储。

表 6-1　员工表（EMP）

emp_no	emp_name	age	salary
100	Hank	35	$10 000
200	Fred	40	$20 000
300	Mary	25	$30 000
400	Sue	23	$40 000
500	Mike	30	$50 000

表 6-2　员工-工程表（EMP_PROJ）

emp_no	project
100	A
100	B
100	C
200	B
300	A
300	C
400	A

不同数据模型间的差别在于它们处理*多值关系*的方法。一个员工可以被分配到许多工程，而一个工程又可分配给许多员工，这就是多对多（MANY-MANY）关系。在导航模型中，EMP 表中的主记录存储了各个单值属性的值，同时存储指针指向包含多值数据的列表。用户若想看到某个员工被分配到的所有工程，则需发出一条请求，遍历这条由主记录指向多值列表的链接。

在关系模型的解决方案中，我们需要另外增加关系表。在我们当前的例子中，我们仅需要增加单一关系 EMP-PROJ 去存储这些多值信息。我们可观察到，EMP-PROJ 有一个 emp_no 属性，

用来匹配 EMP 当中的值对应关系。因而，当有 100 个员工参与工程 A、B 和 C 时，不再需要预先设立 EMP 到 EMP_PROJ 的链接。查询时，用户请求获取这两个关系匹配的所有元组。在这种方式下，用户只用声明需要何种数据，不用描述如何获取数据。

这一点很重要，基于 ad hoc[①]的特点，可能会在数据库建立并填充数据后很长时间才发生数据请求。关系模型非常适合处理这类 ad hoc 的请求，因为无需重新定义数据之间的关系。此外，数据独立性可以降低应用程序的开发时间，因为开发商无需学习在多值关系上烦琐的数据查找方式，改由数据库优化器做出决定并选择最佳访问路径。

这里，我们仍存在一个问题：当有必要跟踪工程的单值信息时，比如已完结工程的交付日期或是新工程的预算，若修改 EMP-PROJ 关系，使之包含这些额外属性，这时就会出现不必要的存储冗余（见表 6-3）。

注意到 delivery_date 属性和 budget 属性对于工程都是单值描述的（假定所有日期都用 Julian[②]日历表示，因此也是单值描述符）。对于每一个分配到该工程的员工而言，这些是完全重复的。员工 100、300 和 400 在工程 A 上的 delivery_date 和 budget 只是该元组的不同副本而已。当需要对数据进行更新操作时（即增加预算），就必须更新每个出现点的数据。为了避免这些问题，我们需要增加第三种关系，主要用于存储 PROJECT 的单值数据。结果如表 6-4 所示。 217

表 6-3 员工-工程表（EMP_PROJ）

emp_no	project	delivery_date	budget
100	A	06/30/1997	$90 000 000
100	B	09/15/1997	$25 000 000
100	C	03/31/1998	$60 000 000
200	B	09/15/1997	$25 000 000
300	A	06/30/1997	$90 000 000
300	C	03/31/1998	$60 000 000
400	A	06/30/1997	$90 000 000

表 6-4 工程表

project	delivery_date	Budget
A	06/30/1997	$90 000 000
B	09/15/1997	$25 000 000
C	03/31/1998	$60 000 000

此时，数据库存在 3 个关系，一个表示 EMP 实体，一个表示 PROJECT 实体，还有一个表示这两个实体间的关系 EMP_PROJECT。显而易见，当需要更新工程的单值信息时，只需要涉及工程中相应的单个元组。

Peter Chen 在一篇有重要影响的论文中描述了 ER（实体-联系）图，其中首先对实体和联系进行了定义，随后定义了它们潜在的实际联系[Chen，1976]。通常，在大型关系系统中，ER 图用来帮助开发人员理解数据与数据之间的联系。一旦完成，规范化的数据库设计便部署完毕。

---

① ad hoc 指的是数据集相对稳定查询多变的数据访问请求。——译者注

② Julian 日历：指的是儒略日，计算机系统中的日历表示法，采用年和一年中经过的天数。例如，87.5 是 1987 年 1 月 5 日，即 1987 年的第 5 天。——译者注

规范化是一个过程，用以确保数据库设计满足特定的规则，以加强数据的一致性与完整性。第一范式（1NF）只是简单约定数据要存储于单值属性[①]。例如员工表 EMP 显然是 1NF。但是当 name 属性扩大到允许将员工的整个名与姓存储至同一属性中，这个实体将不再是 1NF，因为 name 属性允许名和姓同时并存在一个单一的数据元素里。

2NF（第二范式）约定所有属性在关系主键上要完全独立。修改后的 EMP_PROJ 关系不是 2NF，因为 delivery_date 和 budget 在复合主键 emp_number-project 上不完全独立。不
218 过，仅对于 project 属性，这两个属性是独立的。3NF（第三范式）约定实体只在唯一主键上独立。主键是一个或多个属性的唯一标识元组。数据库至少应满足 3NF。

应当明确一点：任何联系之间不存在预先定义好的连接，我们必须在查询处理的时候完成关系的连接操作，而不是在数据定义的时候。

由于关系是基于集合论的，所有典型的集合论操作——笛卡儿积、并、交、差集都要在关系模型中实施。还有一些操作包括以下几点。

- **选择操作**——在 $R[XYZ]$上做选择，记为$\sigma_{A=a}(R)$，定义为：

$$\sigma_{A=a}(R) = \{x | x[A] = a, x \in R\}$$

其中 A 是 R 的一个属性。

- **投影操作**——在 $R[XYZ]$上做投影，记为 $\pi_A(R)$，定义为：

$$\pi_A(R) = \{x[A] | x \in R\}$$

其中 A 是 R 的一个属性集。

- **连接操作**——在 $R[XYZ]$与 $S[VWX]$上做连接（共享公共属性 X）记为:

$$R[XYZ] \bowtie S[VWX] = \{x | x[VWX] \in S \text{和} x[XYZ] \in R\}$$

其中 V、W、X、Y和 Z是属性的不相交集。如果没有公共属性存在，R 与 S的连接就是 R 与 S 的笛卡儿积。

大约 35 年前首次提出关系模型的时候，人们引进了关系代数和关系演算。选择、投影和连接操作构成了关系代数的一部分。由于这些对于用户而言并不友好，所以开发出两种不同的查询语言 QUEL 和 SQL（SQL 起源于 SEQUEL）。SQL 被 IBM 公司采纳并用于其商用数据库系统。1982 年开发出的 SQL/DS 在 1985 年被 ANSI 认定为第一个 SQL 标准。此后，SQL 开始流行。今天，SQL 是为数不多的被产业界、学术界和不同国际标准组织共同认定的标准之一。最新通过的版本是 SQL-2003[SQL，2003]。

219 文献[Date，1994]详细介绍了 SQL。一条 SQL 查询语句的结构如下：

```
SELECT <属性列表>
    FROM <关系列表>
    [WHERE <条件列表>]
    [ORDER BY <属性列表>]
    [GROUP BY <属性列表>]
    [HAVING <条件列表>]
```

在 SELECT 关键字后面列出请求得到返回的属性。FROM 子句描述将要涉及哪些关系。WHERE 子句描述返回元组应满足的条件。因此，实际上整个查询语句是一个对预期结果的规格

① 数据库表的每一列都是不可分割的基本数据项。——译者注

定义。在下面的查询中，只应当检索 EMP 表中的员工号码。不管用什么表达方式，查询都不会去描述检索员工号码的方法。另一种形式包含了 WHERE 子句以给出约束条件。

```
SELECT emp_no
   FROM EMP
```

下面的查询只检索 emp_no 为 400 的元组。没有任何信息说明如何去寻找该元组。如果系统在 emp_no 属性上有一个 B 树索引，则会有一个时间复杂度为 $O(\lg n)$的算法去遍历这棵树，并找到匹配上的元组。否则，系统只能采用线性扫描法。无论任何情况，查询人都不指定要使用何种算法去检索数据。

```
SELECT emp_no
  FROM EMP
  WHERE emp_no = 400
```

GROUP BY 用来对结果集进行分组，并对分组应用聚合函数。聚合函数在 SQL 标准中包括 COUNT（划分的大小）、SUM（划分内属性值总和）、MIN（划分内最小值）、MAX（划分内的最大值）和 AVG（划分内所有值的平均值）。如果没有使用 GROUP BY，这些操作符会作用于整个结果集。

下面我们考虑这么一种需求：最终要产生一个报表，其中包括所有的员工号码以及为每个员工分配的工程数量。下面的查询可以获取这种信息：

220

```
SELECT emp_no, COUNT(*)
  FROM EMP_PROJ
  GROUP BY emp_no
```

按员工号码分组划分 EMP_PROJ 关系。若无匹配元组，COUNT 返回 0。如果存在 WHERE 子句，则要求每个划分中的元组满足 WHERE 条件。

HAVING 约束通常要基于聚合进行。下面的查询查找至少参与了 4 个工程的员工：

```
SELECT emp_no, COUNT(*)
  FROM EMP_PROJ
  GROUP BY emp_no
  HAVING COUNT(*) > 3
```

ORDER BY 用来依据其子句指定的属性对元组排序。由于集合天生就是无序的，所以如果没有 ORDER BY 子句，查询结果集是随机排列的。下面的查询在执行时会将查询结果按员工号码升序排列（若需要被降序序列，则要增加 DESC 选项）。

```
SELECT emp_no
  FROM EMP_PROJ
  ORDER BY emp_no
```

如果需要进行 JOIN 操作，首先在 FROM 子句中指定多个关系，之后在 WHERE 子句中加入 JOIN 条件。下面的语句查询所有参与了工程 A 的员工的年龄：

```
SELECT a.emp_no, a.age
  FROM EMP a, EMP_PROJ b
  WHERE a.emp_no = b.emp_no AND
        b.project = 'A'
```

该查询只表示将两个关系进行了连接，未指出实际的连接顺序或者 WHERE 子句的执行顺序。

221

6.2　相关工作进展

前人的工作可以分为两类：一类是对信息检索与 DBMS 进行融合，另外一类是对 RDBMS 进行扩展，添加信息检索功能。我们现在详细介绍一下这两类方法。

6.2.1　独立系统的融合

有几个研究者给出的融合方案是编写软件中间层，用来在基础 DBMS 和信息检索系统间进行通信[Schek 和 Pistor，1982]。我们需要对查询请求进行解析，将结构化部分提交至 DBMS，文本搜索部分交给信息检索系统，最后需要结合这两类结果，并提交给用户。开发这种软件不需要太长时间，因为信息检索系统和 DBMS 很容易使用，这一点颇具吸引力。

这种方法的主要优点是，DBMS 和信息检索系统作为商业产品，由供应商负责持续的升级改进，软件开发成本便可达到最低。缺点在于其数据完整性、可移植性及运行时性能都比较差。

1. 数据完整性

DBMS 的事务日志与信息检索的事务日志并不能协同，因此将会丧失数据完整性。更新事务中如果出现错误，则 DBMS 只会停留在两种状态：整个事务完成，或是整个事务完全未进行。更新不可能只完成一半。

信息检索日志（如果存在的话）不会理解 DBMS 日志。于是，用于协调这两个系统的保护程序需要处理所有的还原操作。我们一般在 DBMS 应用程序内部检测到错误时才实现还原操作，而且在很多情况下应用程序直接忽略了这类代码。因此，如果在信息检索系统中出现了故障，DBMS 不会发现。如果更新操作需要同时在两个不同系统中进行，很可能在 DBMS 上操作成功，却在信息检索系统中操作失败。部分更新显然是有可能出现的，但是这会导致逻辑上的缺陷。

2. 可移植性

可移植性的丧失源于查询语言不标准。目前，标准的信息检索查询语言并不存在，一些工作还在进行当中。退一步讲，就算存在标准的信息检索查询语言，将其推广给商家并得到认可
222 尚需时日。这样的话，每当引进新的 DBMS 和信息检索系统时，开发人员都需要重新接受培训。另外，由于同时兼顾多个系统，系统管理工作也会困难很多。

3. 性能

运行性能低下源于缺乏并行处理与查询优化。虽然大多数商用 DBMS 已经实现了并行处理，但大多数信息检索系统仍未做到这一步。

每个关系 DBMS 都包含查询优化。优化器的目标是在请求数据时选择一条合适的访问路径。基于规则的查询优化器预先定义规则，而基于成本代价的查询优化器在对不同访问路径进行成本估计之后，选择代价最低的那一条。这两种情况既不包含对查询中非结构化部分的规则，也没有对其成本进行估计，因为查询优化器并不了解信息检索系统所选择的访问路径。因此，查询优化若想同时作用于结构化和非结构化两部分的数据，这种优化就只能由保护程序进行。这将是一个复杂的过程。提出该解决方案的作者对优化难度已做了阐述[Lynch 和 Stonebraker，1988]。因此，运行时性能会受到并行算法与查询优化不足的限制。

4. SQL 扩展

Blair 在 1974 年的一篇未发表的论文中提出，SQL（这里实际上是指早期的 SEQUEL 版本）可以加以修改以支持文本查询[Blair, 1974]。其后，在 1978 年到 1981 年之间出版的一系列论文中，学者们描述了若干种对 SQL 的扩展方式[Macleod, 1978；Macleod, 1979；Crawford, 1981]。早在 1980 年研制的信息检索原型系统 SMART 就采用了关系数据库系统 INGRES 来存储检索相关的数据 [Fox, 1983b]。

这些论文中描述的扩展，既支持相关度排序又支持布尔检索。论文作者更关注在 RDBMS 中的文本检索效率。他们接着指出，在 RDBMS 中可借助额外一张表存储倒排索引，从而使我们能够轻松查看这些索引的内容。信息检索系统通常会隐藏倒排索引，而仅仅表现为一个用于获取数据的访问结构。通过将索引存储为关系的形式，作者指出用户可以方便地查看索引的内容，并在必要时对索引进行更改。作者提到的扩展，形如 RELEVANCE(*)，能够使用一个预先定义的相关度函数计算文档与查询的相关度。

最近，一种叫做 SQLX 的新语言可用于在多媒体数据库中访问文档[Ozkarahan, 1995]。SQLX 假定我们已经实现了基于关键字的初始类别搜索（关于文档聚类的描述，可参见 3.2 节）。 223
SQLX 扩展允许使用特殊的连接符属性搜索结果，以避免某些特定的连接操作。

6.2.2 自定义运算符

自定义运算符允许用户通过修改 SQL 将自己的功能函数添加至 DBMS 引擎，该思想最早见于文献[Stonebraker 等人, 1983]。其商业应用已衍生出数款产品，包括 Teradata 多媒体对象管理器、Oracle 特定的一些功能组件、IBM DB2 文本扩展器以及 Microsoft SQL Server 的一些功能特性[Connell 等人, 1996；Loney, 1997]。下面是一个在域上自定义运算符功能的查询例子，域必须定义为函数形式，并只接受单一参数。现在，我们给定参数的数据类型为 rectangle。因此这个例子同时用到了自定义运算符与自定义数据类型。

```
EX: 1    SELECT MAX(AREA(Rectangle))
           FROM SHAPE
```

在信息检索领域中，类似 PROXIMITY()的运算符可以定义为：用邻近搜索方式计算结果集。采用这种方式，“SQL 的简约风格”得以保存延续，而用户也可以随意添加所需的功能。几年后，人们已经定义了一些自定义运算符来实现信息检索[Lynch 和 Stonebraker, 1988]。下面的查询语句可以获取同时包含 term1、term2 与 term3 的全部文档：

```
EX: 2    SELECT Doc_Id
           FROM DOC
          WHERE SEARCH-TERM(Text, Term1, Term2, Term3)
```

此查询可以利用一个倒排索引快速定位词项。要做到这一点，优化器需要理解新的访问方法。因此，自定义运算符也有可能对自定义访问方法做出要求。

下面的查询使用 PROXIMITY 函数确保 3 个词项要同时出现在大小为 5 的窗口当中：

```
EX: 3    SELECT Doc_Id
           FROM DOC
          WHERE PROXIMITY(Text, 5, Term1, Term2, Term3)
```
224

自定义运算符的优点是，它们不仅可以处理文本问题，也支持空间数据以及图像的处理

等。用户可以任意添加他们所需要的功能。当然，自定义运算符面临的主要问题仍然是数据完整性、可移植性和运行时性能。

1. 数据完整性

自定义运算符允许应用程序开发人员向 DBMS 添加新功能，而不是增加使用 DBMS 的新应用程序。这很不幸地为应用程序的开发人员打开一扇破坏DBMS数据完整性的大门。为了使自定义运算符更加高效，它们会链接到整个 DBMS 一样的模块中，并访问 DBMS 的整个地址空间。内存或是磁盘上打开的文件可被自定义运算符访问，这也使得自定义运算符随时有可能破坏这些数据。

为了保护 DBMS 不被错误的自定义运算符破坏，就要使用 RPC（远程过程调用）来调用自定义运算符。这确保了自定义运算符只能访问自己的地址空间，不能再访问DBMS的整个地址空间。遗憾的是，RPC 的消耗非常大，不适合作为高性能应用程序的解决方案。

2. 可移植性

在A处实现的自定义运算符不一定会提交给B处。更糟的是，运算符可能会重名，但实际功能却完全不同。不采用自定义运算符，所有 RDBMS 开发人员可自由书写实际应用程序，且期望可以运行在任意 RDBMS 之上。而使用自定义运算符时，这种期望往往落空，因为应用程序只能运行在具备该算符的地方。

3. 性能

默认的查询优化不会了解特定的自定义运算符。优化通常基于查询的实质信息进行工作。EQUAL 查询可以预测为比 LESS THAN 函数检索到更少的结果。这样的知识指导着优化器去选择访问路径。

如果不明白自定义运算符的实际语义，优化器就不能高效地使用它。一些自定义运算符可能会需要一个完全不同的访问结构，如倒排索引。除非优化器知道倒排索引与其对应访问路径存在，否则这条路径永远不会被选中。

Lynch 讨论了各个自定义运算符的存储信息能否帮助查询优化。在自定义运算符得到广泛
225 接受后，优化器便需要一些收集其信息的手段。

另外，自定义运算符的并行处理必须定义在其自身之内。DBMS 不会了解自定义运算符，更不会知道如何对其并行执行。

6.2.3　NFN 方法

之后，NFN（非第一范式）方法被提出[Desai 等人, 1987；Schek 和 Pistor, 1982；Niemi 和 Jarvelin, 1995]。其中写到，在使用 3NF（第三范式）时，存储多对多关系的方式异常笨拙。一般情况下，先用两个关系分别存储实体，再用另一个独立的关系存储它们之间的联系。

在倒排索引中，存在文档与词的多对多联系。一个词可能会出现在很多文档中，而一篇文档又可能包含很多词。后面将会提到，这里建立一个 DOC 关系来存储文档，建立一个 TERM 关系来存储不同的词，并建立一个 INDEX 关系记录词在文档中的出现情况。

若不使用这3个关系建模，则需使用一个NFN关系存储文档信息，使用一个嵌套关系指明词在文档中的出现情况。

虽然这样可以提高运行时性能，但是会引发可移植性问题。目前还不存在规范 NFN 的标准。另外，NFN 使得 ad hoc 查询难度加大。

既然自定义运算符与 NFN 方法都各有欠缺，我们介绍一种方法，它使用未加修改的标准关系模型来实施各种信息检索功能。这种方法具备数据一致性与可移植性，运行时性能也在可接受范围以内[Grossman 等人, 1997]。

诸如图像处理及 CAD/CAM 这样的一些应用，可能仍会用到自定义运算符，这是由于它们的处理过程不是面向集合论的，也很难使用标准的 SQL 实施执行。

6.2.4 使用标准 SQL 进行文献搜索

Blair 对关系系统做进一步开发，以支持典型的信息检索功能[Blair, 1988]。 Blair 的工作包括包含非结构化数据（比如在文档标题中查询文本）的结构化数据（比如作者信息扩展）查询。下面的关系对此做了建模。 226

- DIRECTORY(*name, institution*)——表示的是作者的姓名及其所属的研究机构名。
- AUTHOR(*name, DocId*)——表示的是某篇具体文档的作者。
- INDEX(*term, DocId*)——表示的是用来索引某篇具体文档的索引词项。

下面的查询按照出版物的数量对研究机构进行排序，要求文档中必须包含 input_term。

```
Ex: 4   SELECT UNIQUE institution, COUNT(UNIQUE name)
          FROM DIRECTORY
          WHERE name IN
             (SELECT name
                FROM AUTHOR
                WHERE DocId IN
                SELECT DocId
                  FROM INDEX
                 WHERE term = input_term
                 ORDER BY 2 DESCENDING)
```

Blair 列举了使用关系模型进行文档检索的几个好处。这些好处是在关系模型中融入信息检索功能的基础，因此我们将其中一些列出，具体如下。

- 还原例程
- 性能估计工具
- 数据库重组例程
- 数据迁移例程
- 并发控制
- 复杂权限控制机制
- 逻辑与物理数据独立
- 数据压缩和编码例程
- 自动强化完整性约束
- 灵活定义事务边界（比如提交与回滚）
- 在序列化应用程序语言中嵌入查询语言的能力 227

6.3 信息检索作为关系应用

SQL 扩展的工作始于一份未公开的文章[Blair, 1974]，并由 Macleod 与 Crawford 在 1978 年

至 1981 年发表的数篇文章中延续下来[Macleod, 1978；Crawford, 1981]。

Macleod 对扩展的初始描述，基于一个存储词的 QUERY(term)关系和一个关联词在文档中出现状况的 INDEX(DocId, term)关系。下面查询返回的每篇文档结果至少要包含 QUERY 中的一个词：

```
EX: 5    SELECT DISTINCT(i.DocId)
           FROM INDEX i, QUERY q
          WHERE i.term = q.term
```

文档集中的高频词或者停用词通常会被忽略。因此，建立一个 STOP_TERM 关系来存储这些高频词。该 STOP_TERM 关系仅包含一个属性（term)。下面的查询确定的文档包含查询中列出的没有出现在 STOP_TERM 关系中的那些词：

```
Ex: 6    SELECT DISTINCT(i.DocId)
           FROM INDEX i, QUERY q, STOP_TERM s
          WHERE i.term = q.term AND
                i.term ≠ s.term
```

最后，为了实现输入词 InputTerm1、InputTerm2 和 InputTerm3 的逻辑 AND 操作，Macleod 和 Crawford 推荐的查询如下：

```
Ex: 7    SELECT DocId
           FROM INDEX
          WHERE term = InputTerm1
         INTERSECT
          SELECT DocId
           FROM INDEX
          WHERE term = InputTerm2
        INTERSECT
         SELECT DocId
          FROM INDEX
         WHERE term = InputTerm3
```

该查询包含 3 个组成部分，每部分的结果对应于包含查询中的一个词项的文档集。

228 INTERSECT 关键字用来计算 3 个结果集的交集。这种处理便实现了 AND 操作。

Macleod 和 Crawford 继续扩展相关度排序功能。主要加入了 corr()函数——一个决定文档与查询相关度的内置函数。

一个 SEQUEL (SQL 的先驱) 的例子如下：

```
EX: 8    SELECT DocId
           FROM INDEX i, QUERY q
          WHERE i.term = q.term
         GROUP BY DocId
         HAVING CORR() > 60
```

其他的扩展定义随后也会给出，诸如在返回集中限定只获取前 n 个元组的操作。Macleod 和 Crawford 给出了详细的设计示例，描述一篇文档检索系统如何设计成为数据库的子应用。

我们现在介绍的工作都依赖于在固有标准的关系模型上应用标准 SQL 实施信息检索功能[Grossman 等人, 1997]。首先要介绍的是，在将文本载入关系 DBMS 之前的预处理工作。

6.3.1 预处理

输入文本原本存储在源文件中，可能是远程站点或是本地 CD-ROM 上。现在假定数据文件为 ASCII 编码，或是可以很容易地使用 SGML 标签转换为 ASCII 编码。SGML 标签将文档的不同部分进行标准化定义[Goldfarb, 1990]。本书后面例子中的所有标签可在 TIPSTER 集中找到出处，它是 TREC 在前些年的一个标准数据集。标签以符号'<'开始，以符号'>'结束（例如 <TAG>）。

预处理器读取输入文件，并输出到不同的无格式文件中。每个词被读入后，在SGML标签列表中做一次检查。预处理器的主要算法就是只解析词，然后使用一个散列函数将它们散列到一个小的散列表中。如果词项未曾在文档中出现过，则向散列表添加一个新的条目。冲突处理借助于散列表上关联的单链表。如果词项已存在，则更新其词频。当遇到文档结束标签时，扫描散列表。对散列表中的每个条目，生成一个记录。该记录包含当前文档 ID、词项和其词频。散列表输出后，内容复位成 NULL，继续处理下一篇文档。一系列实验表明，实现预处理器最有效的方式见文献[Pulley, 1994]。 229

处理后，两个输出文件会存储至磁盘，然后批量加载至关系数据库。每个文件对应一个关系。第一个关系——DOC——包含每篇文档的信息。

第二个关系——INDEX——建模倒排索引，并指示词项在文档中的出现情况及其出现频率。

两个关系如下：

```
INDEX(DocId, Term, TermFrequency)
DOC(DocId, DocName, PubDate, Dateline)
```

这两个关系由预处理器生成。另一个关系 TERM 是通过统计每个词在已知文档集上的出现次数而得来。至少，这种关系包含了文档频率（df）和逆文档频率（idf）。这些已在 2.1 节中描述过。词项关系的结构为：TERM(Term, Idf)。

亦可使用 API（应用程序接口）让预处理器直接将数据存入数据库。但是，对于某些应用，INDEX 关系包含的元组数量达到亿级或者更多，这就需要调用上亿次 DBMS 的 INSERT 操作，每进行一次插入，事务日志都会更新。所有的关系 DBMS 都支持某种形式的批量装载功能，可使一个大型无格式文件能够快速迁移至关系当中，而不涉及重大开销。此时，日志会被关闭（这通常不可能在在线 API 中出现），大多数厂商均提供该高效率负载实现。在并行实施中，对于无格式文件，可使用多个处理器加载。这种速度远超任何的 API 实现。

本章中所有的例子均假定关系最初通过一个预处理器执行，之后通过批量加载而来。注意到 DOC 和 INDEX 表由预处理器输出。而 TERM 关系不是某个输出。在最初的预处理器测试中，发现该表用 DBMS 建立比在预处理器内建立更加容易。当 INDEX 关系创建完后，计算 TERM 关系的 SQL 语句如下：

```
EX: 9    INSERT INTO TERM
                SELECT Term, log(N/COUNT(*))
                 FROM INDEX
                GROUP BY Term
```
230

N 是集合中的文档总数，通常在执行此查询之前确定。当该值未知时，可通过 SELECT COUNT(*) FROM DOC 获取。该语句将 INDEX 关系按照词项进行分区，由 COUNT(*)获取每个分区中的文档数（即文档频率）。idf 由文档频率除以 N 得来。

在下面的实施案例中，提供输入文本，且预处理器建立好两个文件以准备载入关系 DBMS 中生成 DOC 与 INDEX。之后，借助 SQL 填充 TERM 关系。

6.3.2　实施案例

在这一章中都要使用下面的案例。两篇文档从 TIPSTER 集合中获取并使用关系建模。文档包含结构化与非结构化两种数据。

```
<DOC>
<DOCNO> WSJ870323-0180 </DOCNO>
<HL> Italy's Commercial Vehicle Sales </HL>
<DD> 03/23/87 </DD>
<DATELINE> TURIN, Italy </DATELINE>
<TEXT>
Commercial-vehicle sales in Italy rose 11.4% in February from a year earlier, to 8,848
units, according to provisional figures from the Italian Association of Auto Makers.
</TEXT>
</DOC>


<DOC>
<DOCNO> WSJ870323-0161 </DOCNO>
<HL> Who's News: Du Pont Co. </HL>
<DD> 03/23/87 </DD>
<DATELINE> Du Pont Company, Wilmington.DE </DATELINE>
<TEXT>
John A. Krol was named group vice president, Agriculture Products department, of this
diversified chemicals company, succeeding Dale E. Wolf, who will retire May 1. Mr.
Krol was formerly vice president in the Agricultural Products department.
</TEXT>
</DOC>
```

231

预处理器接收两篇文档作为输入，创建出两个文件以载入关系 DBMS。生成的相应 DOC 和 INDEX 关系在表 6-5 和表 6-6 中给出。

表 6-5　DOC

DocId	DocName	PubDate	Dateline
1	WSJ870323-0180	3/23/87	TURIN, Italy
2	WSJ870323-0161	3/23/87	Du Pont Company, Wilmington，DE

表 6-6　INDEX

DocId	Term	TermFrequency	DocId	Term	TermFrequency
1	commercial	1	2	krol	2
1	vehicle	1	2	president	2
1	sales	1	2	diversified	1
1	italy	1	2	company	1
1	february	1	2	succeeding	1
1	year	1	2	dale	1
1	according	1	2	products	2
…	…	…	…	…	…

INDEX 建模了一个倒排索引来存储词项在文档中的出现状态。如果没有这个关系，就不可

能在关系模型上的文本检索中获得高性能。只把整个文档存储在 BLOB 虽然能够避免存储问题，但大多数在 BLOB 上的搜索操作是受限制的，因为 BLOB 通常不能被索引。因此，任何一个 BLOB 搜索都会牵扯到线性扫描，这将大大慢于倒排索引的时间复杂度 $O(\lg n)$。

典型的信息检索系统会经过一系列漫长的预处理阶段，其中会完成解析，并识别出所有系统中已经存储的词。每个词项关联的倒排表代表有哪些文档包含这个词（见 5.1 节）。词项上会设置指针指向该倒排表。在这种方式下，使用散列函数快速定位到词项，继而定位到倒排表。倒排文件技术非常有效，曾经还在 20 世纪 60 年代中期被用于最早的结构化系统，如 TDBMS [Bleir，1967]。

232

事实上，一个词项可能会出现在很多文档中，且一个文档也会包含很多词项，因此词项与文档之间是多对多的关系。为之建模时，文档与词项应被视为实体（类似员工与工程），并且要建立连接 EMP_PROJ，INDEX 关系随之建立。INDEX 关系中的每个元组指明词项在文档中的出现情况。

注意到词频（tf）或者说每个词项在文档中出现的次数，也是其在 APPEARS-IN 关系中的出现特征，于是将其存储在表中。关系的主键为（DocId，Term），观察到词频完全独立于该主键。

对于邻近搜索，诸如“查找所有出现了短语 vice president 的文档”，就又会需要一个额外的 offset 属性。没有该属性的话，通过 INDEX 关系只能得出“vice”和“president”两词共现的文档，而不保证它们出现在一起。为表示“vice”与“president”相邻，offset 属性记录词项在文档中的偏移量。第一个词项偏移量赋值为 0，第二个赋值为 1，依次类推，第 n 个赋值为 n-1。表 6-7 中的 INDEX_PROX 关系记录了这个必需的属性 offset，以实现邻近搜索。

对于 INDEX_PROX 关系，还有几点需要指出。由于停用词是要被忽略的，所以偏移量序号是不连续的。词项的每次出现都对应一个偏移量，所以词项在表中不具唯一性，这与 INDEX 关系是不同的。

表 6-7 INDEX_PROX

DocId	Term	Offset	DocId	Term	Offset
1	commercial	0	1	year	11
1	vehicle	1	…	…	…
1	sales	2	1	makers	26
1	italy	4	…	…	…
1	rose	5	2	krol	2
1	february	8	…	…	…

233

从 INDEX_PROX 中获取 INDEX 关系的语句是：

```
EX: 10    INSERT INTO INDEX
            SELECT DocId, Term, COUNT(*)
              FROM INDEX_PROX
             GROUP BY DocId, Term
```

最后，有关词项的单值信息也是必需的。TERM 关系（见表 6-8）包含词项的逆文档频率。当词项出现频繁且逆文档频率较低时，表示相关程度不高；当词项出现频率不高但逆文档频率较高时，表示相关程度高。一个词项只对应一个逆文档频率，因此它可以存至 TERM 关系中。

表 6-8　TERM

Term	Idf	Term	Idf	Term	Idf
according	0.9031	italy	1.9231	sales	1.0000
commercial	1.3802	krol	4.2768	succeeding	2.6107
company	0.6021	president	0.6990	vehicle	1.8808
dale	2.3856	products	0.9542	year	0.4771
diversified	2.5798	…	…	…	…
february	1.4472	…	…		

为了在信息检索处理时，保持 SQL 查询的固定句法结构，也为减少查询的复杂性，这里使用一个 QUERY 关系。QUERY 关系（见表 6-9）对每个查询词项包含一个简单元组。查询过程是简化的，源于 QUERY 关系可以和 INDEX 关系合并来确定 QUERY 中的词项是否在 INDEX 中出现。
234 如果没有 QUERY 关系，将会需要一个很长的 WHERE 子句匹配过程。

最后，STOP_TERM 关系（见表 6-10）用来标示在解析阶段要忽略的词项。本章不使用该关系，只是指明该关系模型能存储在数据定义和填充中用到的内部结构。

表 6-9　QUERY

Term	tf
vehicle	1
sales	1

表 6-10　STOP_TERM

Term
a
an
and
…
the
…

下面的查询体现了这种方法的潜力。该 SQL 请求“找到所有描述 vehicle 并且 sales 为 3/23/87 的文档”。其中，关键字检索对应非结构化数据，而发表日期属于结构化数据元素。

这个例子快速体现出融合结构化数据与文本的方式。大多数信息检索系统支持这类将 DATE 作为一个“域”的搜索方式，它们是在文档中被标记的或是总出现在特定区域的文本片段。对这些域做整体解析，并存储于关系结构中。例 6.1.1 给出的查询序列涉及复杂的非结构化数据，无法被信息检索系统轻易解释识别。

```
EX: 11    SELECT d.DocId
            FROM DOC d, INDEX i
           WHERE i.Term IN ( "vehicle ", "sales ") AND
                 d.PubDate = "3/23/87" AND
235              d.DocId = i.DocId
```

6.3.3　布尔检索

布尔查询一般包含操算符 AND、OR 和 NOT。返回的结果集要满足这些布尔条件。

在小型文献检索系统（例如卡片目录系统）中，布尔查询是很有用的。它们允许用户便捷地描述信息需求，并返回所有的匹配结果。而对于大型文档集，它们的作用偏小，这源于其返回的结果集无序，并且匹配数量过大，使得用户不得不一次次调整布尔条件，直到看到满意的结果。相关度排序可解决这个问题，通过计算文档与查询的相关度，然后基于此对结果集排序，而用户只需浏览最前面的结果即可判断结果集是否满足自己的信息需求。

首先，我们使用 SQL 实现布尔检索，然后介绍如何不改变 SQL 而实现邻近搜索，最后阐述如何使用 SQL 实现相关度排序。

给定任意输入的词“InputTerm”，下面的 SQL 查询将返回所有包含该词的结果。

```
Ex: 12  SELECT DISTINCT(i.DocId)
          FROM INDEX i
         WHERE i.Term = InputTerm
```

我们需要采用专门的应用程序方式来获取文档中的实际文本内容。一般来说，文本都会采用大属性进行存储，比如 BLOB 或者 CLOB。当然，文本也很有可能划分为独立的部分（即段落、行、句子、短语等）。如果文本就存储在一个独立的大属性字段中（这里例子中我们称之为“Text”），那么，我们就可以扩展查询，执行一个嵌套的子查询来获取文档 ID。然后，利用这些 ID 即可准确找到 DOC 当中合适的文本内容。

```
Ex: 13  SELECT d.Text
          FROM DOC d
         WHERE d.DocId IN
               (SELECT DISTINCT(i.DocId)
                  FROM INDEX i
                 WHERE i.Term = InputTerm)
```

在本节剩下的内容中，我们只关心是否能在结果集中获取到文档 ID。我们要么在特定的应用模式下执行一条单独的查询，得到相应的文档 ID，要么将查询扩展为例子 13 所示的形式。 236

实际应用中，通常会将例 12 中的查询扩展至匹配 n 个词项。如果该布尔请求是 OR 查询，扩展会很简单，不会增加连接操作的数量。

```
EX: 14   SELECT DISTINCT(i.DocId)
          FROM INDEX i
         WHERE i.Term = InputTerm1 OR
               i.Term = InputTerm2 OR
               i.Term = InputTerm3 OR
                        ...
               i.Term = InputTermN
```

遗憾的是，更多复杂的查询包含布尔 AND。比如一个查询有 n 个输入项，则 INDEX 关系就要连接 n 次，查询语句如下所示。

```
Ex: 15  SELECT a.DocId
          FROM INDEX a, INDEX b, INDEX c, … INDEX n-1, INDEX n
         WHERE a.Term = InputTerm1 AND
               b.Term = InputTerm2 AND
               c.Term = InputTerm3 AND
                        ...
               n.Term = InputTermn AND
               a.DocId = b.DocId AND
               b.DocId = c.DocId AND
                         ...
               n-1.DocId = n.DocId
```

多重连接代价昂贵。连接操作的执行顺序会影响到性能，因此需要一个基于成本的优化器来计算不同执行顺序的代价[Elmasri 和 Navathe，1994]。对执行顺序列表的剪枝方法见文献[Selinger，1979]，即使这样，代价仍然不堪忍受。

除了关注性能之外，实际情况是商用系统均有一个连接操作的上限。虽然理论上可以对 n

个词项做连接，但是大多数的实现都限制在这个上限值之下（通常在 16 个左右）[White 和 Date，1989；McNally，1997]。布尔 AND 的复杂性让很多研究人员扩展 SQL，或是开发自定义运算符以便使用更简单的 SQL 查询。

文献[Grossman 等人，1997]提出了一种新方法：查询输入中无论查询词数量有多少，我们只需要进行固定数量的连接。该方法减少了查询条件的数量。但是，该方法需要在查询过程中
237 增加额外的排序操作（该方法使用了 GROUP BY），而之前的方法并没有这样的操作。

下面的查询使用标准 SQL 语法计算布尔 AND：

```
Ex: 16  SELECT i.DocId
          FROM INDEX i, QUERY q
         WHERE i.Term = q.Term
         GROUP BY i.DocId
         HAVING COUNT(i.Term) =
                (SELECT COUNT(*) FROM QUERY)
```

WHERE 子句确保结果集包含的只是与 INDEX 匹配的查询项。GROUP BY 将结果集按照文档分组。HAVING 子句从这些分组中筛选出成员数量与查询词数量一致的结果。

对于包含 k 项$(t_1, t_2, \cdots, t_k)$的查询，结果集元组如表 6-11 所示，其中文档 d_i 包含全部这 k 个项。

表6-11　结果集

DocId	term
d_i	t_1
d_i	t_2
…	…
d_i	t_k

GROUP BY 子句计算文档基数 k。在计算当中，HAVING 子句过滤出基数 k 等于查询词个数的文档。匹配后，生成元组 d_i 保存至结果集。

到此，我们均假定：INDEX 关系中，给定词在每篇文档中最多只出现一次。这与我们给出的例子一致——额外加入词频来记录词项在文档中出现的总次数。在邻近搜索中，对于一篇文档，一个词项可能会在 INDEX 关系中记录多次，这样的话，查询语句便需要作修改，因为仅一个词项都有可能使文档基数达到 k，使得结果集中的 d_i 甚至不包含剩下的全部 k–1 个词项。

下面的查询使用 DISTINCT 关键字，确保接受计算的文档中的词项各异。这样的查询作用
238 于 INDEX 关系中，在这个关系中，文档中的词项重复导致 INDEX 关系中的词项重复。

```
EX: 17   SELECT i.DocId
           FROM INDEX i, QUERY q
          WHERE i.Term = q.Term
          GROUP BY i.DocId
          HAVING COUNT( DISTINCT( i.Term))
                 = (SELECT COUNT(*) FROM QUERY)
```

这条查询在执行时会判断是否存在重复现象。但是如果知道文档中没有出现重复项，那么与之前相比，该查询显然降低了执行效率。DISTINCT 关键字通常要触发排序操作。

应用面向集合的方法处理布尔关键字查询，会幸运地引发一个附带作用，即阈值 AND（TAND）可以很容易实现。偏 AND 表示有 k 个子条件为真即为真，而这些子条件实际上都不是必需的。下面的查询返回所有包含至少 k 个查询词的文档。

```
EX: 18   SELECT i.DocId
           FROM INDEX i, QUERY q
          WHERE i.Term = q.Term
          GROUP BY i.DocId
          HAVING COUNT(DISTINCT(i.Term)) > k
```

6.3.4 邻近搜索

简要回顾一下，邻近搜索在信息检索系统中负责确保查询词以特定序列出现，或者至少是出现在文档中特定窗口大小的片段中。大多数用户在搜索“vice president”时，都不希望包含这样的结果：“His primary vice was yearning to be president of the company.”

欲实现邻近搜索，就要用到我们案例中的 INDEX_PROX 关系。offset 属性可计算出文档中各词项的相对位置。

下面的查询乍看有点复杂，实际上是应用标准 SQL 找出那些包含 QUERY 中的词项，并且全部词项要出现在固定大小窗口之中的文档。以给出的案例来说，“vice”与“president”分别出现在位置 7 与位置 8，若窗口大小为 2 或者更大，将返回文档 2。 239

```
Ex: 19  SELECT a.DocId
          FROM INDEX_PROX a, INDEX_PROX b
         WHERE a.Term IN (SELECT q.Term FROM QUERY q) AND
           b.Term IN (SELECT q.Term FROM QUERY q) AND
           a.DocId = b.DocId AND
           (b.Offset - a.Offset) BETWEEN 0 AND (width - 1 )
         GROUP BY a.DocId, a.Term, a.Offset
         HAVING COUNT(DISTINCT(b.Term)) =
         (SELECT COUNT(*) FROM QUERY)
```

INDEX_PROX 表要与自身相连接，直到计算出文档中所有词项之间的距离。比如说，文档 d_i 包含 k 个词项 $(t_1, t_2, \cdots, t_k)$，对应偏移量为 $(o_1, o_2, \cdots, o_k)$，最开始的两个条件确保我们只需讨论匹配词项的偏移量即可，第三个条件确保对比的偏移量不会跨越两个或更多的文档。下面的元组满足前三个条件均为 TRUE。

在表 6-12 中，我们举例说明了查询的逻辑过程。在第一步中，对任取的文档 d_i，INDEX_PROX 与自身做连接操作，最终可以生成符合要求的元组，其中 INDEX_TERM 中的每个词会匹配上其他词。这张表仅显示那些无论自身还是自身的匹配都属于文档 d_i 的词项，因为加入条件 a.DocId = b.DocId 后，只有这些元组取值为 TRUE。我们还假定表 6-12 中的词项与查询中那些词项匹配，这样就能满足条件 b.term IN (SELECT q.term FROM QUERY)。

表6-12 INDEX_PROX自连接结果

a.DocId	a.Term	a.Offset	b.DocId	b.Term	b.Offset
d_i	t_1	o_1	d_i	t_1	o_1
d_i	t_1	o_1	d_i	t_2	o_2
d_i	t_1	o_1	d_i	t_k	o_k
d_i	t_2	o_2	d_i	t_1	o_1
d_i	t_2	o_2	d_i	t_2	o_2
d_i	t_2	o_2	d_i	t_k	o_k
d_i	t_k	o_k	d_i	t_1	o_1
d_i	t_k	o_k	d_i	t_2	o_2
d_i	t_k	o_k	d_i	t_k	o_k

第四个条件检查偏移量，仅当存在的各词项在指定的窗口时才返回 TRUE。GROUP BY 子句按照文档中的特定偏移量进行分组，HAVING 子句确定分区大小是否与查询项数量一致。当满足
240 以上条件时，认定文档包含 QUERY 中所有的词项，并在 offset 窗口大小范围之内。于是，文档 d_i 即被放入最终结果集当中。

给出一个例子，查询包含“vehicle”与“sales”，要求被双词窗口覆盖，表 6-13 中的元组在 WHERE 子句的 4 个条件全为 TRUE。前 3 个条件负责过滤掉不符合查询的词项，第 4 个条件过滤掉超出窗口的词项组。GROUP BY 子句将中间结果进行划分，致使第一偏移词“vehicle”位于一个分区，第二偏移词“sales”位于另一个分区。第一部分有两个词与查询的大小匹配，因此，文档 1 属于最终结果集（见表 6-13）。

表6-13　经由WHERE子句四重条件筛选之后的结果

a.DocId	a.Term	a.Offset	b.DocId	b.Term	b.Offset
1	vehicle	1	1	vehicle	1
1	vehicle	1	1	sales	2
1	sales	2	1	sales	2

6.3.5　使用标准 SQL 计算相关度

对于大型文档集合，相关度排序相当重要，因为布尔查询经常会返回数千篇文档。如今，无论是万维网上的搜索引擎（如 Google 和 Yahoo!），还是商用信息检索系统（如 Convera's RetrievalWare 和 Verity's Topic），全部都实现了相关度排序。目前，存在大量算法可以计算查询与文档间的相关度。第 2 章已经详细介绍了这些算法及其衍生算法。

正如我们在 2.1 节所提到的，向量空间模型非常常用。在 TREC（文本检索会议）中，基于此模型的系统都展现出良好的性能。回想一下，在向量空间模型中，文档与查询被表示为 t 维向量，其中 t 是文档集中不同词项的数目（见 2.1 节）。用查询向量 $\boldsymbol{Q}$ 与文档向量 $\boldsymbol{D}_i$ 之间的距离来对文档进行排序。该距离的计算见下面的点积表达式：

$$SC(\boldsymbol{Q},\boldsymbol{D}_i)=\sum_{j=1}^{t} w_{qj}\times d_{ij}$$

其中，w_{qj} 是查询 q 中第 j 个词项的权重，d_{ij} 是第 i 篇文档中第 j 个词项的权重。

在最简单的情况下，每个向量元素被赋予权重 0 或 1（1 表示其对应的词项存在）。现在有
241 大量的权重计算模式，*tf-idf* 便是其中之一。这里，组合使用词频与逆文档频率（见 2.2.1 节）。下面的 SQL 语句便使用 *tf-idf* 权重进行了点积查询的计算。

```
Ex: 20   SELECT i.DocId, SUM(q.tf * t.idf * i.tf * t.idf)
          FROM QUERY q, INDEX i, TERM t
         WHERE q.Term = t.Term AND
                 i.Term = t.Term
         GROUP BY i.DocId
         ORDER BY 2 DESC
```

WHERE 子句确保只有 QUERY 中的项参与计算。这样，那些查询之中不包含的项在查询向量中会被赋予零权重，对最终求和不起作用。逆文档频率从 TERM 关系中获取，在选择列表中用于计算 *tf-idf* 权重。ORDER BY 子句确保结果按相关度排序。

这里，我们已经使用了一个简单的相关度计算方法，另有许多衍生算法可参见文献[Salton, 1989]。也可以使用标准 SQL 实现这种相关度计算。通常，余弦相似度计算及其变体最为常用，其定义如下：

$$SC(\boldsymbol{Q}, \boldsymbol{D}_i) = \frac{\sum_{j=1}^{t} w_{qj} d_{ij}}{\sqrt{\sum_{j=1}^{t} (d_{ij})^2 \sum_{j=1}^{t} (w_{qj})^2}}$$

分子即是刚刚讨论的点积，但分母会通过文档向量与查询向量的大小进行归一化。归一化因子在查询时计算得到，但查询语法由此变得过于复杂。为了简化 SQL，另外建立两个独立的关系：DOC_WT (DocId, Weight)和QUERY_WT (Weight)。DOC_WT存储每篇文档的向量大小,QUERY_WT仅包含一个元组，用于指出查询向量的大小。

这两个关系在下面的 SQL 中受到重用：

```
Ex: 21    INSERT INTO DOC_WT
             SELECT DocId, SQRT(SUM(i.tf * t.idf * i.tf * t.idf))
               FROM INDEX i, TERM t
             WHERE i.Term = t.Term
             GROUP BY DocId
EX: 22    INSERT INTO QRY_WT
             SELECT SQRT(SUM(q.tf * t.idf * q.tf * t.idf))
               FROM QUERY q, TERM t
             WHERE q.Term = t.Term
```

242

在每个 INSERT-SELECT 语句中，都会计算向量的权重，求其平方根，并最终求和以得到总向量权重。下面的查询用于计算余弦相似度。

```
EX: 23    SELECT i.DocId, SUM(q.tf * t.idf * i.tf * t.idf) /
                          (dw. Weight * qw. Weight)
           FROM QUERY q, INDEX i, TERM t, DOC_WT dw; QRY_WT qw
          WHERE q.Term = t.Term AND
                i.Term = t.Term AND
                i. DocId = dw.DocId
          GROUP BY i DocId, dw.Weight, qw.Weight
          ORDER BY 2 DESC
```

对内积的计算做了修改，加入了两个新关系，DOC_WT 与 QRY_WT 以使用归一化的权重。而 WHERE 子句中因此也增加附加条件用来获取每篇文档的权重。

在该运算的实现当中，需要用到内置的平方根函数，目前很多 SQL 都支持该函数。我们注意到实现查询时，其实无需非标准的平方根函数，只用将整体系数做简单开方即可。这个改动不会影响文档的排序，因为 $a \leqslant b \Rightarrow a^2 \leqslant b^2 (a,b \geqslant 0)$。为了实现简便，我们使用内置的 sqrt 函数（很多商业 SQL 实现都支持该函数）来计算参数的平方根。

类似这样的针对 SUM()元素的修改，也可用于其他相似度计算的实现。例如，在利用额外的计算和存储进行的一些文档统计（平均词频日志）、一些集合统计（平均文档长度与文档数量）以及一些词项统计（文档频率）时，这种修改可以支持临界点归一化与概率测量的实现。

针对 2.1.2 节提到的临界点归一化测量与 2.2.3 节提到的概率测量的 SQL 在文献[McCabe 等人, 1999]中有所记载。从本质上讲，这里唯一改变的就是 SUM 函数的作用域包含了新的权重。最终结果需要对多个相关度测量做融合处理，这点可由 SQL 轻易实现。我们将在 8.3 节对相关

243 度合并的应用做更详细的介绍。

6.3.6 相关反馈在关系模型中的实现

应用关系模型来支持相关反馈，可参见文献[Lundquist 等人，1997]。回顾一下，相关反馈的定义是指：基于首次查询返回的那些假定相关的文档，而向查询中增加新词项的过程（见 3.1 节）。针对这种工作，下面的 3 个步骤分别使用了 3 个独立的 SQL。

步骤 1：执行初始查询。使用之前刚刚描述过的 SQL。

步骤 2：从前 n 个文档中获取词项。应用在 INDEX 关系上的查询获得文档 ID 列表（可在步骤 1 执行时存储在一个临时关系里），可以得到前 n 篇文档中的非重复词项。如果 DBMS 可以限制单一查询返回的元组数量（很多商业系统都具备这种能力），该查询的执行非常高效。之后，使用一个 INSERT-SELECT 语句将新词项插入至 QUERY 关系中。

步骤 3：执行修改后的查询，使用的 SQL 同步骤 1。

6.3.7 关系信息检索系统

结构化与非结构化数据的融合需求使得一个基于可扩展、标准 SQL 的信息检索引擎原型 SIRE 得以发展，相关描述见文献[Frieder 等人，2000b；Frieder 等人，2003]。SIRE 的目的最初是为美国国立卫生研究院（NIH）建立一个国立互补与替代医学研究中心（NCCAM），通过实际应用 Oracle DBMS 来促进对商业关系数据库的投资。SIRE 当然也包含对传统个性化检索的全部支持。此外，针对并发控制、数据恢复、安全性、可移植性、可扩展性和鲁棒性这种关系数据库的常用附加功能，SIRE 无需额外的努力就可以提供。而像那些在传统信息检索市场上不常出现的功能，既然数据库供应商总是持续不断地增强其特色，使其看起来就像达到了软硬件双方面的升级，基于 SIRE 的解决方案一边要借助较少的投资来跟上技术发展，一边又要被用户
244 拿去和更加传统的（惯用的）信息检索系统方案做比较。

为了演示 SIRE 的适用性及多功能性，如相关度测量、邻近搜索、n 元语法、段落、短语索引及相关反馈这样的主要信息检索模型、算法及实用策略，全部在标准 SQL 的基础上给出实现。将 SIRE 架设至主机上的关系数据库平台包括 NCR DBC-1012、Microsoft SQL Server、Sybase、Oracle、IBM DB2 和 SQL/DS, 甚至包括 mySQL，由此可见系统的可移植性。在提高效率方面使用的优化方法，有一些我们在之前的章节中已经给出（见第 5 章），还有一些关系数据库所特有的技术，比如索引剪枝、查询阈值和聚类索引。所有这些优化方法都是通过减少 I/O 来有效降低查询执行时间的。其他的一些实现细节（例如可通过增加查询处理框架以实现查询与结果的缓存）见文献[Frieder 等人，2000b]。

最近的相关工作主要把精力集中于在基于 SIRE 的方法上增加并行技术以及增加高效的文档更新操作之上。随着文本不断改变，特别是网络环境中的文本，文档更新开始变尤为必要。而在传统定义上，信息检索只是一个“只读”环境。早期的并行处理工作使用 NCR 机配置 24 个处理器，并达到了 22 倍的速度提升[Lundquist 等人，1999]。随后的工作在东京大学的文章[Goda 等人，2001]中有所描述，将 SIRE 方法进一步扩展至一个具有 100 多个节点的 PC 集群之上。在瑞士苏黎世联邦理工学院，研发人员展示出 SIRE 方法可像简单检索那样，改善文档插入与更新的吞吐量[Grabs 等人，2001]。

6.4 使用关系模式进行半结构化搜索

目前，已存在大量方法能够解决 XML（可扩展标记语言）文档的搜索问题，但是它们都缺乏融合结构化与非结构化数据的能力。使用关系系统处理 XML 时，会建立一个独立的关系模式，用于映射具体的 XML 模式或 DTD（文档类型定义）[Schmidt 等人，2000；Shanmugasundaram 等人，1999]。文献[Florescu 和 Kossman，1999]中描述了一个基于静态关系模式的方法，文献[Deutsch 等人，1999]在其之上，补充描述了一个完整的 XML 查询语言实现——XML-QL。更近一点，文献[DeHaan 等人，2003]介绍了一些算法，可用于将 XQuery 表达式转译成 SQL。

6.4.1 背景

当前，XML 已成为平台无关的数据交换标准[Buneman 等人，1996；Goldman 等人，1999]。现有大量方法可用于存储 XML 数据并高效地访问它们[Abiteboul，1997]。其中一种方法是自定 245
义树型文件结构，但这种方法缺乏可移植性，也无法利用已有的数据库技术[Kanne 和 Moerkotte，2000]。而在其他的方法当中，都要从头开始建立一个存储半结构化数据的数据库系统[McHugh 等人，1997；Quass 等人，1996]，或是使用一整套倒排索引[Shin，1998]。

文献[Luk 等人，2002；Deutsch 等人，1999；Bonifati 和 Ceri，2000]给出了几种流行的 XML 查询语言。在 1997 年 8 月，一个被吹捧为基于路径的基本查询语言——XPath——的初步工作被提交至 W3C（万维网联盟）。在 1998 年，AT&T 研制了查询语言 XML-QL [Deutsch 等人，1999]，能够全面满足 XML 查询语言集合的全部特性需求该要求由 W3C 设置。描述 XPath 的规范发布于 1999 年。随后，在 2001 年 12 月，XPath2.0 发布。最新的半结构化查询语言之一 XQuery，也是十分强大的。该语言吸收了很多好点子，这些点子既源于较早的半结构化查询语言（如 XML-QL 和 XPath），又源于一些传统的关系查询语言，如 SQL。XQuery 1.0 的第一个公开草案在 2001 年 6 月得以发表，并于 2003 年 5 月进行一次更新[Boag 等人，2003]。

6.4.2 使用静态关系模式支持 XML-QL

现在，我们先简单介绍一下一种支持任意 XML schema 的静态关系模式。这种方案首次由文献[Florescu 和 Kossman，1999]提出，以支持 XML 查询处理。随后，在伊利诺伊理工大学信息检索实验室（www.ir.iit.edu）的研究指出，完整的 XML-QL 查询语言可完全由基本结构构建而成，这是通过将半结构化 XML-QL 转译至 SQL 来完成的。静态模式可容纳任何 XML schema 的数据，无需文档类型定义或 XSchema。

静态关系存储模式负责存储所有不同的 XML 路径，其中每篇文档对应关系中相互独立的行，这与文献[Florescu 和 Kossman，1999]中的边表概念很像。事实如其名，每一行对应 XML 结构图中的一条边。这种静态关系模式可以存储任意的 XML 文档。

XML 文档层次结构的保存方式十分变通，在每篇文档索引至数据库时，仅需表中的信息便可重建。该关系如下：

TAG_NAME(*TagId*, *tag*)　　ATTRIBUTE(*AttributeId*, *attribute*)

TAG_PATH(*TagId*, *path*)　　DOCUMENT(*DocId*, *fileName*)

INDEX(*Id*, *parent*, *path*, *type*, *tagId*, *attrId*, *pos*, *value*)

246 下面再次给出一个实际案例。

```
<DOC>
<DOCNO> WSJ870323-0180 </DOCNO>
<HL> Italy's Commercial Vehicle Sales </HL>
<DD> 03/23/87 </DD>
<DATELINE> TURIN, Italy </DATELINE>
<TEXT>
Commercial-vehicle sales in Italy rose I 1.4% in February from a year earlier,
to 8,848 units, according to provisional figures from the Italian Association of Auto
Makers,
</TEXT>
</DOC>
<DOC>
<DOCNO> WSJ870323-0161 </DOCNO>
<HL> Who's News: Du Pont Co. </HL>
<DI)> 03/23/87 </DD>
<DATELINE> Du Pont Company, Wilmington, DE </DATELINE>
<TEXT>
John A. Krol was named group vice president, Agriculture Products department,
of this diversified chemicals company, succeeding Dale E. Wolf, who will retire
May 1. Mr. Krol was formerly vice president in the Agricultural Products department.
</TEXT>
</DOC>
```

6.4.3 存储 XML 元数据

下面几张表存储 XML 文件的元数据(描述数据的数据)。TAG_NAME(见表 6-14)和 TAG_PATH(见表 6-15) 共同存储了 XML 文件中的标签与访问路径信息。TAG_NAME 存储了 XML 集合中所有唯一标签的命名，TAG_PATH 存储那些在 XML 文档中出现的不同访问路径。ATTRIBUTE 关系(见表 6-16) 存储所有属性的命名。在我们的例子中，我们已经加入了 LANGUAGE 属性，该属性属于标签 TEXT。在 TAG_NAME 和 TAG_PATH 关系中，tagId 是在预处理阶段分配的唯一主键。同样，attributeId 也是唯一的。对于我们在本章早些时候给出的那些例子，这几张表每次都是在为新的 XML 文件建立索引时填充的。该过程由解析 XML 文件，抽取信息并保存至表构成。

表6-14 TAG_NAME

tagId	tag
10	DOC
11	DOCNO
12	HL
13	DD
14	DATELINE
15	TEXT

表6-15 TAG_PATH

tagId	path
10	[DOC]
11	[DOC,DOCNO]
12	[DOC,HL]
13	[DOC,DD]
14	[DOC,DATELINE]
15	[DOC,TEXT]

表6-16 ATTRIBUTE

AttributeId	attribute
7	LANGUAGE

6.4.4 跟踪 XML 文档

因为 XML-QL 允许用户指定需要被查询的文件，所以很多时候我们不想浏览数据库中的所有记录，而只需要浏览与相应文件所对应的一个记录子集。每当为新文件建立索引时，该文件得到一个唯一的标识符作为固定值，每个值对应一个 XML 文件。对于这个例子，我们只存储了包含这个文档的文件名，此关系见表 6-17。其他的相关属性应该包含文档长度或是归一化长度[Kamps 等人，2004]。

表6-17 DOCUMENT

docId	fileName
2	doc_0.xml
3	doc_1.xml

247 ~ 248

6.4.5 INDEX 关系

INDEX 表（见表 6-18）建模了一个 XML 索引。针对文档所包含的全部标签、属性或值，该索引包含它们的所有映射。当然，因为属性及标签的顺序对于 XML 极其重要（例如，存在首次出现、二次出现等概念），INDEX 关系对于这些标签的位置或顺序亦会进行存储。

id 列是分配给文档中所有元素及属性的唯一整数，parent 属性存储当前标签的父标签 ID，这些属性是为保存 XML 文档固有的层级性质而设。

path 对应 TagPath 中的主键键值，type 指示访问路径是否由元素或属性所终结（值为 E 或 A）。TagId 和 AttrId 分别是 TagName 与 Attribute 表的外键，DocId 属性标记 XML 文档。pos 记录标签的出现顺序，用于那些使用 XML-QL 索引表达式特征的查询，它指出了在同一父节点下各元素的位置关系（从 0 开始计数）。这一列存储输入 XML 的原始顺序，可为用户的详细高级查询服务。最后，value 包含了 XML 最基本的单元——最底层标签内的值。当我们达到位置 value 时，之前所有这些应用关系来建模倒排索引的定义都会被用到。

表6-18 INDEX

id	parent	path	type	tagId	AttrId	docId	pos	value
41	0	10	E	10	1	6	0	NULL
42	41	11	E	11	1	6	0	WSJ870323-0180
43	41	12	E	12	1	6	0	Italy's Commercial...
44	41	13	E	13	1	6	0	03/23/87
45	41	14	E	14	1	6	0	TURIN, Italy
46	41	15	E	15	1	6	0	Commercial-vehicle...
47	46	15	A	15	7	6	0	English
48	0	10	E	10	1	7	0	NULL
49	48	11	E	11	1	7	0	WSJ870323-0161
50	48	12	E	12	1	7	0	Who's News···
51	48	13	E	13	1	7	0	03/23/87
52	48	14	E	14	1	7	0	Du Pont Co···DE
53	48	15	E	15	1	7	0	John A. Krol···
54	53	15	A	15	7	7	0	English

249

6.5　多维数据模型

在之前的两节中，我们介绍了为信息检索系统或 XML-IR 系统应用关系模型作为核心存储及检索组件的方法。当然，应用多维数据模型，也可以提供类似功能。这种模型天生就支持层次维，当然也就适用于某些文档中的自然层级。DOCUMENT 维可以包含文档的层级（比如文档、章、节、段落、句子）。另外两个附加维分别是 LOCATION 和 TIME。对于文档的位置信息与发布日期，在信息检索中经常接到相应的数据请求。相应的 LOCATION 的层级（比如国家、地区、州、市）与 TIME 的层级（比如年、月、日）非常适用于多维建模。

多维数据模型中的数据处理一般会涉及 OLAP（联机分析处理）。ROLAP（关系 OLAP）指的是应用关系数据库系统实现的多维模型。MOLAP 指的是应用多维数据库系统实现的多维模型。我们讨论的重点不在于使用 ROLAP 或是 MOLAP，而在于我们当前能够更加容易地描绘文档中自然体现的层次维度。

通常情况下，OLAP 应用程序一般使用星型模型。我们用到的维——DOCUMENT、TIME 和 LOCATION 围绕 fact 表排列成星星的形状，其中 fact 表类似于 6.3 节中提到的 INDEX 关系。该表映射至一个给定项以关联相应的 DOCUMENT、TIME 和 LOCATION 维。

一旦数据使用星型模式进行描述，便可以整体移植进 ROLAP 或 MOLAP 系统。该多维系统还有其他好处，它们可以更加自然地表示文档集中的层级数据。文献 [McCabe 等人，2000] 描述了应用 MOLAP 系统实现相关度排序。

6.6　协同器

本章我们已经描述了通过关系模型实现结构化数据、半结构化数据与文本的融合。这种方法能够利用现有关系系统的强大功能，并且可以提供当数据实际存储于一个集中资源库时的简单集成方式。

在现实生活中，一些应用程序需要将数据存储在不同的位置。对于一个数十亿字节的文本
250 集，并且已经架构了相应的搜索引擎访问方式，将其强行冗余复制到关系系统中是很不现实的。相反，可以在用户与数据之间设置一个协同器，用来判断哪些数据源与查询最相关，提交查询给这些数据源相应的搜索引擎处，随后再对结果进行合并处理。如果所有数据源都是文本，就变成一个相对简单的元搜索（即对一系列搜索引擎的简单搜索实现）。如果数据源包含文本、XML 甚至是关系数据，那么协同器需要运行在所有数据源的顶层，并在不同数据源与用户之间进行协同。

尽管已经进行了大量研究，最近的一份数据融合调查仍然指出，还有一个关键的研究性问题遗留在结构化、半结构化与非结构化数据融合之中[Raghavan 和 Garcia-Molina，2002]。协同器方面相关的所有研究工作都是要直接解决这些问题的。现存的协同器有两种：因特网协同器与内联网协同器。

6.6.1　因特网协同器

因特网协同器响应用户查询时，发送多个查询至每个远程因特网站点搜索引擎，整合返回

结果，最终展现给用户。例如，用户可能需要查询在因特网上出售的图书，他可以选择同时在 Waldenbooks 和 Daltonbooks 上查询，因特网协同器会给两个站点同时发送请求，并展示整合后的结果。一般情况下，所有站点需要遵循一个通用的模式，但互联网协同器处理的是一个更富挑战性的问题，即对同一个查询需要处理不同的使用模式。

目前，学术界存在一些协同器研究工程，所有工程都将重点放在不同区域的数据调解问题上。MIX 工程来自加州大学圣地亚哥分校，该工程关注于模式融合[Baru 等人，1998]，将多个大规模数据集合视为一个大型的分布式数据库，其中全部的数据以标准模式的 XML 文档表示，应用 XML 查询语言进行查询。这种方式最大的缺点是所有的遗留数据必须符合规定模式，这对于一些小型应用是合理的，但是对于数十亿字节的应用并不是一种可行的处理方式。Tukwila 数据融合系统来自华盛顿大学，该系统采用了一种类似的方法[Ives 等人，1999]，不过更加单单重视模式的融合，最终目标没有放在提供回答至用户问题上面。斯坦福大学的 TSIMMIS 工程把重点放在给大规模数据提供接口，并以典型的网络搜索方式对其搜索[Garcia-Molina 等人，1997]。

Melnik 等人介绍了一种针对数字图书馆的中介基础架构[Melnik 等人，2000]，该架构允许用户围绕各种数据源自行开发外包，使用一个通用语言查询所有数据源。高等查询语言 SDLIP（Simple Digital Library Interoperability Protocol，简单数字图书馆互操作协议）因而被用于查询 251
这些数据源。这要求用户定义针对给定查询确定正确数据源的方法。

Melnik 等人描述的中介基础架构避免了很多非结构化数据源的细节问题。查询语言假定用户是对一个单一文档集进行查询，查询语言没有在结构化或半结构化数据查询方式上做任何简化。在许多方面，这种中介基础架构类似于 InfoBus，InfoBus 使用 CORBA 在各种网络非结构化服务中隐藏细节[Paepcke 等人，2000]。在所有工作当中，核心焦点只有一个，就是对多个站点返回结果进行融合。数据源的选择在这里被忽视，虽然它是在大量数据源顶端架设协同器结构的主要挑战。

总的来讲，因特网协同器结构在查询执行之前想要完成模式调解统一是非常困难的。如果一个数据源缺乏类似 `publisher_date` 这样的属性，而另一个数据源却包含这个属性，那么在查询时再去确定这件事便毫无意义。因为数据源完全不受协同器结构掌控，在查询前确定详细元数据是不现实的。在内联网协同器中，情况完全不同。元数据可在查询之前定义好，模式调解统一工作也可以完成。

6.6.2 内联网协同器

内联网协同器结构的关键是在查询时刻以前，所有数据源的模式长时间有效。图 6-2 给出了一个高等协同器结构样本。

从本质上讲，该样本包含了以下关键元素。

- **查询处理器**。查询处理器接受自然语言查询，并将其解析成核心语法结构集合，如主语、谓语和宾语。此外，查询分析器在查询之上执行词性标注操作，确定查询里每个词项最有可能的词性。最后，使用实体标注器对顶层语义概念进行识别，如查询中的场所、人物、地点、组织等。这些工具经常用于各种问答系统，只不过它们被用于大规模文档集合时效率较低。因此，协同器结构只把这类工具用在解析查询上。我们假设有充足的时间在查询上进行复杂的自然语言处理，而不是在文档集合上。

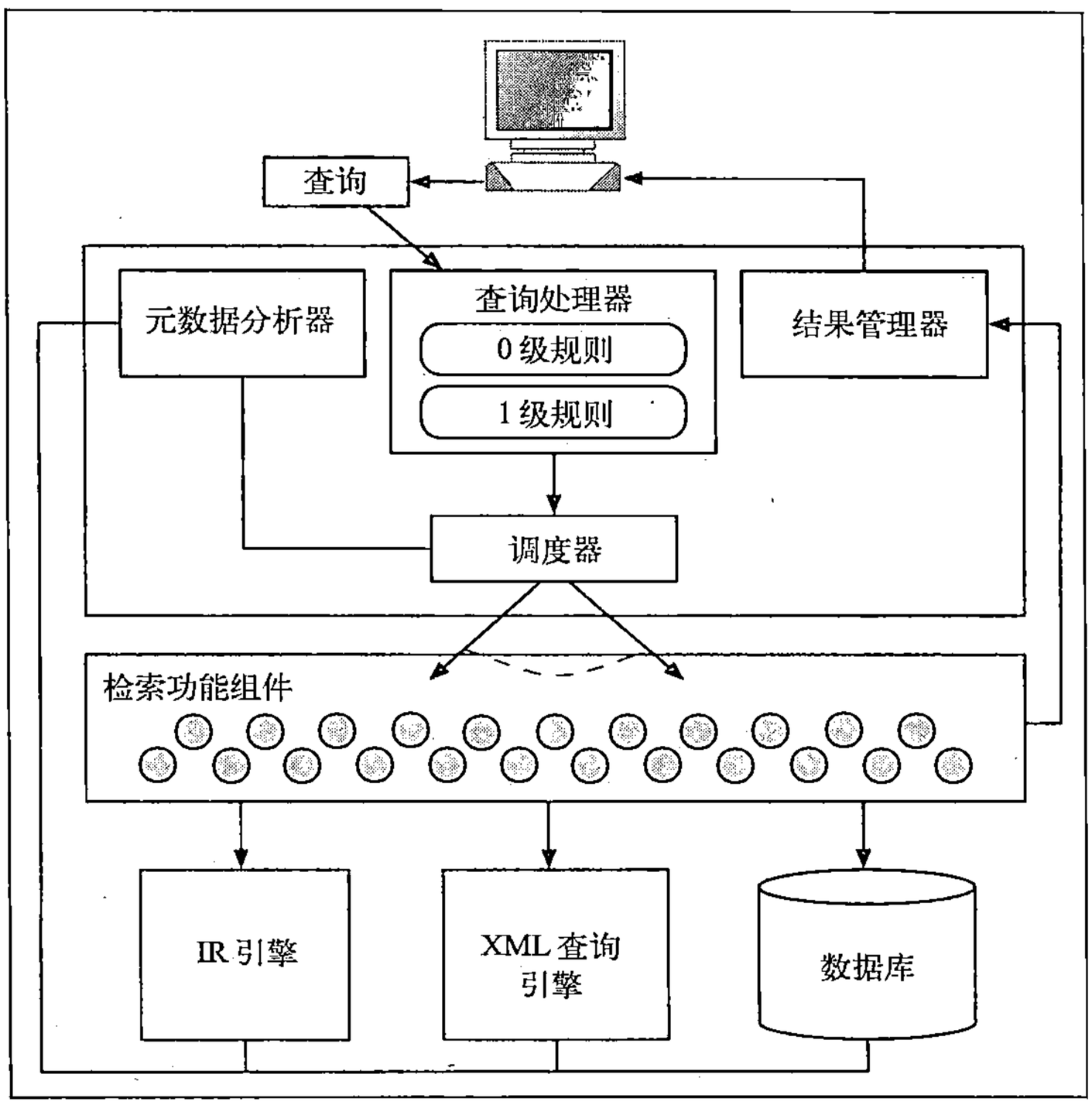


图 6-2　内联网协同器结构

- **0 级规则**。第一套规则定义为“0 级”。这些规则在查询和已有的元数据列表中提取语法元素，并确定查询的高层语义概念。试想一下，课程编号“CS 522”可能会被“二字母前缀后跟三数字”这一规则所识别。

252

  一条 0 级规则的构成模式如下：

  当 主语或宾语 = [课程前缀列表] [3 个数字]
  　则 主语或宾语 = [COURSE_NUMBER]

- **1 级规则**。“1 级”规则在查询处理器与 0 级规则识别的语义概念之上提取输出，并映射至一个或多个检索功能组件上，这些检索功能组件之后会获取实际数据，作为对查询的回答。
- **检索功能组件**。这些小型的功能组件是协同器结构的核心，包含对数据源进行实际检索的代码。它们使用数据源对应的语言与之对话。比如，对于关系数据源使用 SQL 脚本进行交流，对于 XML 数据源发送 XML-QL 或一些其他的 XML 查询语言。这种灵活性是这些功能组件的核心特征，它允许协同器方便地连接几乎所有类型的数据源。简单地说，如果一个数据源是可查询的，那么协同器便能够通过一个检索功能组件对其访问。有人可能会注意到，整套检索系统虽然来自于多个异构数据源，而实质上只采用了英文查询，并从中选择对应的检索功能组件。该想法本质上就是对规则框架（既包含 0 级又包含 1 级）进行联合，以支持对检索功能组件进行正确的选择。

- **调度器**。各种面向数据源类型的调度器负责异步调用恰当的检索功能组件给数据源，并视为满足查询要求。
- **结果管理器**。结果管理器负责将各个不同处理源生成的结果进行归并，其中某些数据权重可能会高于其他数据源。
- **元数据分析器**。中介器亦包含一些分析器，这些分析器审查新输入源，并判定其核心问题，认为输入源即真实数据。分析器存在于结构化、非结构化、半结构化的数据之上。

6.7 本章小结

我们讨论了几种聚焦于融合结构化数据与文本数据的方法。为帮助读者理解，最初我们简要回顾了关系数据库模型，之后罗列出数据融合领域已有的工作进展。其中，我们分析讨论了数据融合中涉及的核心问题：数据完整性、可移植性及性能，并指出维护协调两个独立系统十分艰难且代价昂贵。

我们需要将传统的关系数据库管理特性与传统的信息检索功能进行融合，基于这个出发点，我们描述了早期对关系数据库管理系统扩展自定义运算符的相关努力。这类扩展能够提供信息检索的功能，但仍然在性能方面有所隐患，可移植性也遭舍弃。之后，我们提供了应用标准的SQL 实现信息检索与关系数据库融合的细节说明，并给出了实际执行案例——SIRE。再后来，我们发现了将 XML 检索视为关系模型子应用的优势。而本章最后以多维数据模型数据融合与协同器结构的简要介绍作为终结。 253

6.8 练习题

(1) 设计一个程序，将《爱丽丝梦游仙境》转化成适合装进本章描述的 INDEX 关系中。

(2) 将上一题的输出加载至你选中的关系 DBMS。 254

(3) 实现一个简单的点积 SQL 查询方式，查询你刚刚加载的数据。实现 10 种不同的查询。

(4) 注意到 Alice 被多次复制，实现一个霍夫曼编码压缩算法，以减少存储空间。重新加载 INDEX 关系，计算存储开销。

(5) 叙述 2.2.1 节中 Robertson 与 Sparck Jones 开发的概率方法如何被当做关系数据库系统的应用程序实施。对 2.2.4 节中 Kwok 开发的方法重复该练习。 255

第7章 并行信息检索

人们常基于指令流与数据流的数目来描述并行结构，即单个和多个指令流与数据流。指令流与数据流不同组合所组成的完整分类体系，可参见文献[Flynn，1972]。针对一个计算任务，并行结构的性能一般采用加速比（speedup）来评价，加速比定义为 T_s/T_p，其中 T_s 是最佳串行算法的处理时间，而 T_p 为待评估并行算法的处理时间。加速比越高，并行算法的性能越好。测量加速比是用来评估一个算法是否具备可扩展性。如果一个算法在 16 个处理器上能取得近线性加速比，那么它在成百上千个处理器上未必会取得类似的加速比。但是，如果一个算法在两个处理器上能达到的加速比就很小或者几乎没有加速比，那么该算法肯定不能扩展到大规模的处理器集群上。

MIMD（Multiple Instruction Multiple Data，多指令流多数据流）的含义是每个处理器单元本身都可以执行不同的指令流。与 SIMD（Single Instruction Multiple Data，单指令流多数据流）系统相比，这种方法更难实现同步操作，因为 SIMD 系统每个处理器上执行的是相同的代码，而 MIMD 系统需要等待消息的传递。

在 SIMD 结构中，所有处理器同时并发执行同样的指令代码。作为控制指挥中心的主处理器向它的从处理器集合发布指令，然后所有的从处理器同时针对不同的数据序列进行计算。如果所有处理器执行相同的指令代码来处理不同的数据片段，那么 SIMD 系统将会非常高效。在这种情况下，我们使用 SIMD 引擎完全有可能达到比较大的加速比。例如，采用这种 SIMD 的方案，可以让一些图像处理的应用程序将每个独立像素点或者是像素集合传给一个独立的处理器。

在本章中，我们将只引入针对并行处理器编写的算法。这些算法与分布式算法从根本上就不同，我们要将它们严格区分开来。设计分布式信息检索算法，是为了满足在多个物理分散的位置上存储数据的需求。分布式信息检索系统的最广为人知的例子就是万维网。我们将在第 8 章中集中讨论万维网以及其他的分布式信息检索系统。而并行系统则与分布式系统大相径庭，处理单元是彼此相连的，它们往往就在同一块集成电路板上。

7.1 并行文本扫描搜索

假定文档集包含 n 篇文档，在并行模式匹配中，我们会将该文档集划分为 p 份，其中 p 通常为处理器的个数[Evans 和 Ghanmemi，1988]。每个处理器接受一份文档集与一份查询的副本。针对每个处理器上的 $\left\lceil \frac{n}{p} \right\rceil$ 个文档，我们采用串行算法进行扫描处理。一旦完成了文档集的搜索，所有命中的相关文档结果都会反馈到控制处理器。因为一个待匹配的模式串可能会横跨两个或者多个划分，所以我们需要额外增加一个步骤，需要专门检查划分之间的空白部分。这种额外

的检查会导致并行算法附加的开销。

并行字符串匹配不过是并行文本扫描搜索的简单特例罢了，因为字符串匹配假定待搜索的文本完全存储在内存当中。并行字符串匹配算法的综述，可参见文献[Breslauer 和 Galil，1991]。在长度为 1 的文本串中，如何搜索长度为 k 的模式串？文献阐述了几个不同的并行字符串搜索匹配算法。

并行模式匹配算法的一个重大缺陷是划分之间的空白部分的搜索会导致颇为可观的开销。甚至在一些情况下，并行算法居然没有产生任何加速效应。并行签名文件方法针对顺序存储的文件也仅仅能做到线性速度，不过其运行时间并不比实现串行索引算法更快。Salton 在连接机和 SUN3 上实现这个算法的时候，发现了这个现象[Salton，1988]。另外，Stone 采用分析模型计算发现：与多个 32KB 处理器组成的并行机相比较，串行机性能更好。这种现象发生的前提是，串行机采用的是倒排索引，而在 32KB 处理器的并行机采用的是文件扫描方式[Stone，1987]。文献[Cockshott，1989]中也再现了这一结论，该文指出：信息检索其实并不需要太多的计算来研究出较有效的并行处理算法。

一般来说，并行算法会从最佳的串行算法起步。将并行扫描程序与串行扫描相比，加速比并不是个准确的做法，因为众所周知，最好的串行算法都会采用倒排索引方法。 258

7.1.1 文本硬件扫描

人们研制了很多种专用的硬件机器来扫描文本，具体介绍可参见文献[Hurson 等人，1990]。这里简要介绍其中的两种系统，以完善本书这部分的内容。

1. Utah 检索系统

Utah 检索系统（URS，Utah Retrieval System）是按照非确定性有限状态自动机（FSA，Finite State Automata）实现的，采用了一系列专门的比较运算符[Hollaar 和 Haskins，1984；Hollaar 和 Haskins，1991]。构建有限状态自动机旨在一次性探测多条不确定性路径，因此，有限状态自动机并不需要回溯处理。从根本上来说 Utah 检索系统是一个智能磁盘控制器，一般都将该硬件放置在磁盘控制器附近，以便只有那些匹配上了检索标准的数据才会返回到调用的处理器。专用算法背后的动机其实都是为了提高系统的运行时性能。Utah 检索系统可以实现近似搜索，不过，我们目前还不清楚 Utah 检索系统是否能存储文档的词项权重信息。因此，这个方法存在与基于签名方法同样的一些问题。

Utah 检索系统后续的研究工作中使用了索引和简化的倒排表。这个倒排表并没有包含邻近信息，可以使用索引来判别应该扫描哪些文档。该系统采用有限状态自动机来扫描文档，从而获得查询相应的信息。我们需要利用这个扫描步骤来判别文档中查询词的具体位置。

2. DPPM

为了避免有限状态自动机方法需要预先的大量计算以及同时搜索大块文本，研究者提出了 DPPM（Data Parallel Pattern Matching，数据并行模式匹配算法）[Mak 等人，1991]。该算法将数据块与模式串的一个顺序连续的序列子串进行比较。随后，搜索模式串的这些字符分别独自与整个文本块进行比较。假定模式串与文本块不匹配的概率很高，那么一旦发现与子串不匹配，就可以尽早做出淘汰出局的判断，从而可以快速扫描整个文本块的内容。一旦模式不可能匹配上，就可以执行此种策略。这种提前发现不匹配的机制避免了很多不必要的比较，因此可以大大减少整个搜索处理的时间。

采用多个独立 DPPM 搜索引擎来识别匹配模式串在文档中的偏移量，我们大致可以给出其架构。基于简单的计算以及预先定义好的偏移量信息，系统可以支持特定的信息检索处理单元，该处理单元可以是在 Utah 检索系统中提出来的。接着，我们可以采用超大规模集成电路实现 DPPM 引擎，并对全局结构进行相应的分析。该分析过程理论上可以达到每秒 1GB 文本的搜索
259 速度。

Bailey 等人在 PADRE 系统（它是一个分布式的内存模式匹配系统）中深入地研究了并行搜索方法，并探索了将该系统扩展到 1TB 的相关课题[Bailey 和 Hawking，1996]。另外，他们所提出的扩展方法将数据划分到一台虚拟机的多个 CPU 上——就目前所显示的情况来看，该方法不完全合理[Chowdhury 和 Pass，2003]。

7.1.2　并行签名文件

我们可以采用几种方法来并行扫描签名文件。因为签名文件本身就可以采用任意顺序进行扫描，因此这种文件结构本身就支持并行化处理。

1. 连接机

早期采用签名文件来表示文档的算法是针对连接机（CM，Connection Machine）研制的[Stanfill 和 Kahle，1986]。5.3 节中详细描述了文本签名的串行算法。

每个处理单元都会存储多个签名文件，不过，指派给每个处理单元的签名文件都是针对一篇文档的。这样做的原因是我们假定一篇文档不需要扩展到多个处理单元上进行计算。用户输入的查询会为每个查询词生成一个位图，然后将该位图广播到所有的处理器。每个处理器会将该位图与签名列表进行比对，一旦匹配，处理单元对应文档的信箱区（mailbox，存储器中的公用单元）将会根据查询词的权重进行更新。因为签名文件中并没有记录文档中词的出现次数，所以文档对应词重复次数的权重信息最后获取不到。但是，我们可以使用在文档集基础上计算的全局权重信息。

一旦扫描了所有的签名文件，就可以对所有信箱区取全局最大值，通过这种方式来实现文档排序。随后，将信箱区为全局最大值的处理单元一律重置为零，然后再重复计算最大值，寻找相应文档。循环往复，直到检索到了所需数量的文档为止。

该算法的商用版本在实现上采用了一些优化策略[Sherman，1994]。在连接机中，签名算法的实际参数如下：

$w=30$ 词/签名

$s=1\,024$bi/签名

$i=$用来表示一个词的 10 个散列函数

260 单个处理单元中放置了 55 个签名。这里，我们并不遵循一个处理单元对应一个相应签名的假设。而且，权重也不是按照文档而是按照签名对来计算的。基本思想是：信息检索排序的时候，采用 60 个词为粒度对文档进行切割比采用整个文档来排序更优。因此，我们会为每个签名设置一个权重。

为了解析查询，我们会使用排名在前 100 个的查询权重。查询的位图跟以前一样生成，然后我们采用快速微代码来快速校验签名中对应的 i 个位。无论什么时候查询词匹配上了签名，则相应地更新对应的权重。当完成签名匹配过程后，我们会采用一定的评分机制来综合计算签名对的二者平均得分。采用签名对可以将近邻信息纳入相关度排序中。为了计算跨越文档边界

（在当前处理单元之上与之后皆可）上的签名部分，我们还必须增加一些处理器之间的通信。不过，这两个处理器传输开销实际很小，毕竟仅仅是相邻处理器间需要进行通信。

该算法完全存储于内存中。以新闻报道的查询服务为例，我们在检索的时候仅仅需要使用最近的新闻文章。随着时间的流逝，过期的文档会从连接机中导出，从而为新文档腾出处理空间。通过这种方式，我们可以将几百万个文档全部存储在内存中，其运行时性能一般都控制在1~3 秒以内。

2. 数字阵列处理器

数字阵列处理器（DAP，Digital Array Processor）实现签名采用的是两阶段搜索方法。大多数并行算法都是基于串行算法位序列方式实现的，详细信息参见文献[Mohan 和 Willett，1985]。该算法采用一部词项词典计算签名，并设置每个词项对应的位单元。1 024 位长的签名分布在每个处理器（4 096 个处理器）上。因此，我们可以并行搜索 4 096 篇文档。查询条件被广播到每个处理器上。每个词只能设置一个位，因此位匹配的个数可以用来计算相似度。这里我们利用的假设是：查询匹配上的一个位意味着查询与该位对应的词匹配上了。文档中的多个词可能会映射到签名中的同一个位，所以该假设并不总是对的。

为了证实是否真的匹配上了，我们需要开始第二阶段。该阶段需要实现模式匹配算法，并对文档进行检查，从而计算真正匹配上的词语数量。这一阶段仅仅针对第一阶段排名较高的文档。该算法的性能号称很好，但到目前为止还没有给出实际的测试结果[Pogue 和 Willett，1987]。 261

3. HYTERM

HYTERM（HYbrid TExt Retrieval Machine，文本混合检索机）是 Lee 研制的另一个方法。该方法是专用搜索硬件与划分签名文件的混合方法。划分签名文件既可以在硬件上也可以在软件上实现[Lee，1995]。HYTERM 在架构上采用多层签名机制来实现对布尔查询的匹配。一旦完成这一步，就可以通过软件扫描或者专用的模式匹配设备将没有命中的结果排除掉。

HYTERM 将签名进行划分，以便每一个划分都有特定的键或者是签名文件的特定部分。给定的划分并不需要存储划分键，这样就可以节省内存。我们可以快速检测键，以判别是否需要搜索整个划分部分。划分存储在内存中，在填充过程中将它们分布到各个处理器上， Lee 将这些处理器称为签名模块。开始的时候，仅仅使用一个签名模块。一旦完成后，仅仅使用一个比特位的键，随后将这些签名分配到两个处理器上。这个过程一直持续直到所有空闲处理器都被占用为止，随后新处理器加入，处理过程无限期地持续。

实际文本都是存储在许多个小文本文件中的。如果签名模块识别出了候选文档并检测是否能够命中查询，文本处理模块则从硬盘中检索出文档。值得注意的是，将文档分布在多个硬盘上，查询失败的概率就越大。假如宕掉一块硬盘，也仅仅意味着访问不到一小部分文档，但总体查询依然可以工作，仅仅是准确率与召回率比之前稍低一些罢了。我们一般都采用散列分配算法或者循环分配算法将文档均匀分布在磁盘之上。

4. Transputer

20 世纪 90 年代早期，研究者利用 Transputer（计算机集成块）研制了两个算法[Cringean 等人，1990；Cringean 等人，1991]。第一个是基于词项的算法，其中签名中仅仅存储所有的词项，而第二个算法采用三元组（即交叉三字符序列，具体参见 3.4 节）来实现通配符搜索。

另一个基于签名的算法是在 Transputer 网络上实现的。在任意一个并行互联网络中，Transputer 负责构建单元。Transputer 通常都作为一个芯片分布，并可以与其他 Transputer 相互

连接。采用这个方法，研究人员测试了各种不同的互联网络，最终还是选择使用三元链接网络。在三元链接网络中，主处理器发送消息给 3 个独立的线性处理器阵列。只采用一个线性阵列的情况下，数据传输需要 p 步，其中 p 为处理器的数目。

262 我们需要再次使用二阶段算法。在第一个阶段中，主处理器顺序扫描每篇文档的签名，随后与签名匹配的相应文档分发到 p 个处理器上，然后在每个处理器上实现顺序字符串匹配算法。在这个工作中，顺序字符串匹配采用的是 Horspool 改进的 Boyer-Moore 算法[Horspool，1983]。

在性能测试中，我们去掉签名，只进行字符串匹配。采用 15 个处理器进行测试实验，最终获得了 12.6 倍的加速比。对不同长度的签名作进一步的测试，结果发现：签名越长，错误命中的情况越少。因此，需要并行字符串匹配的情况就越少。采用 512 位签名，15 个处理器仅仅获得了 1.4 倍的加速比，因为只需要搜索原始文档集中 13%的文档。

进一步的测试采用基于三元组的签名，而不采用词项。每个三元组散列映射为单一比特位。在每个处理器上，修改模式匹配算法以实现通配符搜索，其中包括一个或多个字符的通配符号（例如：搜索“st*”将找到所有以“st”开头的字符串）。在 15 个处理器上，基于三元组签名的初始加速比仅有 2.1。算法第一个阶段的串行签名匹配是导致加速比偏低的主要原因。为了降低这种影响，增加额外的 Transputer 元件，以保证 2 到 4 个处理器可以用来并行扫描签名文件。采用 4 个 Transputer 进行并行签名匹配，可以在 12 个处理器上将加速比提高到 4.5。

另外一个基于词项的 Transputer 算法可参见文献[Walden 和 Sere，1988]。在这个工作中，主处理器将查询请求发送到每个处理器上，从中判别出“相关”文档，并返回到主处理器上进行最终排序。“相关”定义为已经匹配上查询中的一个词项。文档签名主要用来节省存储空间，并没有做什么工作来避免错误命中的情况。这项工作很有意思的一个方面是调研了 3 种不同的互联网络：环型、线性阵列和树型。在 10MB 文档集上，采用环型互联网络，利用 15 个处理单元的加速比接近于线性，而利用 63 个处理单元，加速比仅有 6.14。实质上，测试文档集太小而不能充分利用处理单元的效率。在树型结构中，63 个处理单元可以产生 7.33 的加速比。

7.2　并行索引

另外一个方法就是并行处理倒排索引，其主要思路是划分索引，以便可以在不同处理器上处理索引的不同部分。图 7-1 给出了将倒排索引划分到两个处理器的示意图。实际上，这样做比较困难，因为我们需要将索引进行简单划分，并在 p 个处理器上将相同数目的词平均分配，但结
263 果往往很难保证任务的平均划分。倒排表规模大小的不对称性造成了这个困难。不管怎么说，我们可以针对连接机、DAP 以及其他架构实现并行索引算法。在本节中，我们将讨论这些算法。

7.2.1　在连接机上实现并行索引

相对于最好的串行算法来说，基于签名的算法并没有提高性能。因此，人们研制了基于倒排索引的新方法，该方法同时实现了两种算法：全内存存储的索引与基于磁盘的索引[Stanfill 等人，1989]。倒排表是文档 ID 的序列，采用二维数组的形式存储。该二维数组定义的是倒排列表入口与倒排表位置之间的映射关系。倒排表入口一次性分配到位，从第一行开始，只有当第一行填满之后，才会移到第二行。这样，我们就可以按照循环分配方式将数据分配到不同的处理器上（每列可以采用单一的处理器进行处理）。

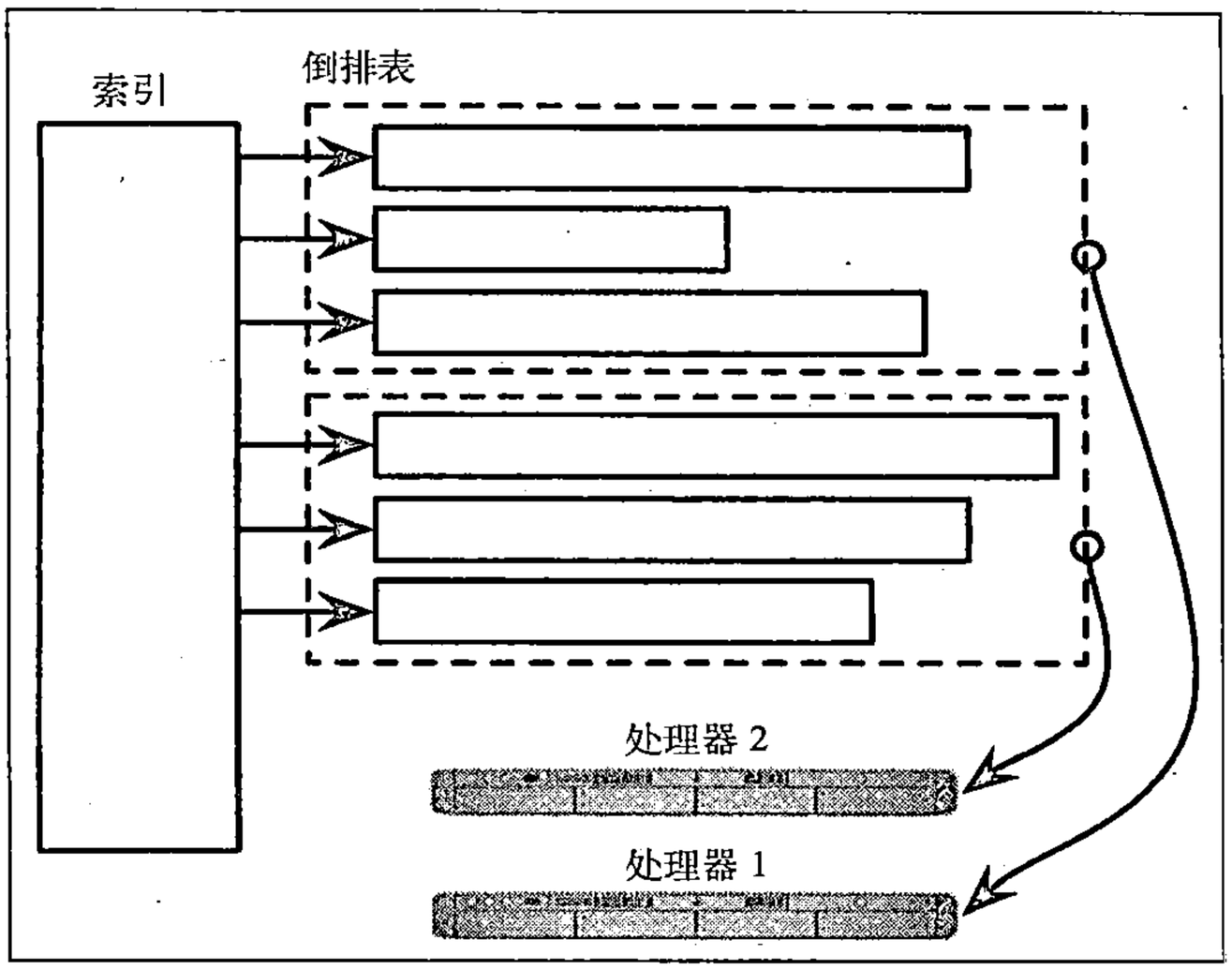


图 7-1 划分倒排索引

现在，我们以词 t_1、t_2 与 t_3 的倒排表为例。假定词 t_1 在文档 d_1 和 d_2 中出现。该词的倒排表存储在数组第零行的前两个位置上，采用数组的形式保存在内存之中。假定词 t_2 在文档 d_1 和 d_3 中出现，而词 t_3 在文档 d_1、d_2 和 d_3 中出现。针对这 3 个词，可以构建 2×4 维的数组，如表 7-1 所示。

264

表 7-1 倒排表的并行存储

1	2	1	3
1	2	3	

如果使用所有的处理器，采用这种方法就可以单步处理 1 024 个倒排结果的数据行。一个完整的数据行称为数据条。因为各个不同的词散布在给定的数组中，因此，我们有必要跟踪记录某个词映射到哪个倒排表。为了达到这个目的，我们需要使用第二个数组，其中每个元素记录的是词项对应的起始行号、起始列号以及倒排表的长度。起始行与起始列可以唯一表示出该词在倒排表中对应的起始位置。继续沿用我们的例子，词 0、词 1 与词 2 的索引表可参见表 7-2。

表的第一行表示的是词 t_1 从数组的位置[0,0]开始，直到位置[0,1]，二维倒排表数组如表 7-2 所示。因为我们采用的是每次分配一行的模式，因此可以推导出这种位置信息。

n 个词项组成的查询可以在 n 步中处理。从本质上来说，该算法可以归结如下：

```
do i = 1 to n
    curr_row=index(i)
    for j = 1 to row_length do in parallel
        curr_doc_id = doc_id(curr_row)
        score(curr_doc_id) = score(curr_doc_id) + weight(curr_row)
    end
end
```

这仅仅是算法的概要。当取得的不是一个整行的时候，我们还需要做一些其他工作来使部分处理器无效（可以用位掩码来使处理器无效）。而且，当遇到长倒排表，或者是倒排表从一个

数据条的末端开始的时候，在内循环的单次迭代处理过程中，我们往往需要处理更多行的数据。

表7-2　索引词与倒排表项的映射

词	起始行	起始列	长度
t_1	0	0	2
t_2	0	2	2
t_3	1	0	3

265

对于每个查询词，我们需要在索引中进行查找，最终需要决定处理倒排表的哪个数据条。每个处理器定位到相应数据块中 doc_id 的地址。该 doc_id 对应的相关度评分必须更新，因为映射到该倒排表本身就意味着该文档中的一个词与查询中的某个词匹配上了。采用这种方式，最终可以更新获取文档评分的一维数组（即原算法中的“mailbox”结构）。我们将该一维数组的每个元素派发到每个处理器上，因此，每个处理器对应一篇相关文档。如果文档集的文档数目超过处理器的个数，那么会使用一个“虚拟处理器”。（注意：文档评分元素绝不会更新多个，因为对应每一个词-文档对来说，倒排表只会包含一个词。）采用全局最大搜索策略，我们在评分数组中找到得分排名最高的元素，最终实现搜索排序。随后标记出该元素。该过程循环往复，直到检索出所有文档为止。

当倒排表使用一个完整数据条的时候，该算法能取得很好的加速比。不过，问题是一个数据条往往只能存储一到两个词的倒排结果。只包含一个词的倒排表会导致一个处理器来更新得分数组，而其余 1 023 个处理器则无所事事。倒排表也可以按照节点来划分[Stanfill, 1990；Stanfill 和 Thau，1991]以适用于处理器（或者节点）集群。这种处理更便于数据从磁盘上迁移到倒排数组中。

一个节点往往包含 32 个处理器。问题是：如果倒排表中的项按照文档范围任意分配给节点（即节点 1 接收文档 0 和文档 2 的倒排结果，而节点 2 接收文档 1 和文档 3 的倒排结果），那么，可想而知，其中一个节点的倒排结果可能会大大多于另外一个。为了避免此类问题，节点在划分的时候，每个划分包含词 ID 的范围。在部分划分中偶尔会有些空的空间，因为很可能在特定的文档范围内，给定范围的词确实没有实际数据。实际结果表明：对于常规的文档集合来说，存储空间的使用率往往在 80%到 90%之间。这样划分后的倒排表减少了空闲处理器的数量，从而最终提高了系统的加速比。

7.2.2　连接机的倒排索引

前面的算法每次只能得出一个查询词。采用并行处理可以更新排序性能，但是即使采用优化的处理器技术（全部使用数据条），t 个词的查询处理的时间复杂度依然是 $O(t)$。[Asokan 等人，1990]给出了一个方法，可以达到对数级别的时间性能。该算法包括如下步骤。

266

步骤 1：将处理器集合分组为不同的集群。每个集群仅处理一个查询词。

步骤 2：每个集群同步访问索引，以访问该集群所对应的倒排表，因此，集群 1 获得词 1 的倒排表，而集群 2 获得词 2 的倒排表，依次类推。

步骤 3：采用全部 p 个处理器来合并 $\left\lceil \frac{n}{p} \right\rceil$ 个倒排表，其中 n 为文档数目。这种方法可以有效地产生查询词所对应的有序文档表。因为倒排表中已经包含了权重信息，所以可以同样获得

文档的权重。因此，合并后的倒排表结果可能如下所示：

$$<D1, 0.5><D1, 0.3><D2, 0.5><D2, 0.9>$$

该倒排表出现的前提是：文档 1 包含了两个查询词，其权重分别为 0.5 与 0.3；同样，文档 2 包含了这两个查询词，其权重分别为 0.5 与 0.9。

步骤 4：利用所有处理器从该列表中去重，并生成总分。经过这个步骤之后，我们得到的倒排表如下所示：

$$<D1, 0.8><D2, 1.4>$$

步骤 5：根据每篇文档所赋予的得分，对倒排表进行排序。我们的倒排表现在就是：

$$<D2, 1.4><D1, 0.8>$$

我们采用改进的双调排序来合并表，因此该算法的时间复杂度为 $O(\log_2 t)$。这看上去要优于 $O(t)$ 的时间复杂度。但是，我们需要注意到的是：给定一个搜索词，该算法需要 $O(\left\lceil \frac{n}{p} \right\rceil$ 个处理器组成的集群来处理该词对应的单个倒排表。如果存储特别多的词，很可能需要重复进行这类操作。

7.2.3 在 DAP 上实现并行索引

和连接机一样，使用了 DAP（数字阵列处理器）上的词签名的早期扫描算法，后来也被索引算法替代了[Reddaway，1991；Bond 和 Reddaway，1993]。

DAP 算法与 CM 算法之间关键的不同点在于它是否使用了压缩倒排表。而且，当 DAP 采用更简单的互联网络（采用网状结构而不是超立方体结构），运行在 DAP 上的算法据说效率更高，而求全局最大值之类的全局性操作速度相当快。因为整个过程中并没有真正用到远程“传送”操作，DAP 方法的作者宣称没有必要采用超立方体结构。

压缩方法基于这个发现：大的文档命中列表往往会有相同的起始比特位。假设命中结果列 267
表中包含文档 8、文档 9、文档 10、文档 11、文档 12、文档 13、文档 14 与文档 15。它们的二进制值都是以 1 开头的，即为 1000、1001、1010、1011、1100、1101、1110 与 1111。我们可以将这一个比特位作为块的标记，那么该块的数据可以只采用 3 个比特位来存储。因此，块 1 可以包含的索引信息为 000、001、010、011、100、101、110 与 111，信息的表示所占比特位总数则从 $8\times4=32$ 变为 $1+8\times3=25$。显然，这种表示方式的关键在于命中的文档号需要恰好在一个数据块之内。对于 DAP 来说，我们采用 24 位编码（无压缩）来表示极少出现的词（指的是哪些每 50 000 篇文档中只出现一次的词）；而对那些出现很频繁的词，我们采用 8 位的块编码，在块内再使用 16 位的偏移量（数据块最大容量为 64KB）来表示倒排表中的项。最终，对于最高频的词，我们将一个 64KB 的文档块数据看成是 256 个独立的子块。DAP 与其他并行算法关键的不同之处在于倒排表需要并行地扩展为非压缩形式。据说，基于 DAP 的系统性能要比以前的串行系统快 200 倍。采用 4 096 个处理器进行的其他实验表明 DAP 610 比 VAX 6000 的性能有显著的提高（要快 100 多倍）[Manning，1989；Reddaway，1991]。

7.2.4 并行索引划分

在一个没有任何共享信息的环境下，如何判定哪个倒排索引划分方法最优？文献[Tomasic

和 Garcia Molina, 1993]给出了一个分析模型。该文献具体研究了 3 种方法。第一种为系统方法，即将索引按照索引词进行划分。词 *a* 的整个倒排表放在磁盘 1 上，而词 *b* 的倒排表则存储在磁盘 2 中，依次类推。我们采用循环调度的方式将倒排表分配到磁盘之上。

基于文档的划分方法常称为磁盘方法。在这种方法中，文档 1 索引的倒排表都放在磁盘 1 上，文档 2 索引的倒排表都放在磁盘 2 上，等等。同样，我们采用循环调度方式将文档分配到磁盘上。因此，为了检索词 *a* 完整的倒排表，需要从每个磁盘上将词 *a* 对应的部分倒排表检索出来，并将它们归并。尽管归并会比系统方法条目花费的时间多得多，但是检索可以并行实现。

主机方法将每篇文档的倒排表进行划分，并将它们交给独立的处理器。因此，文档 1 发送到处理器 1，文档 2 发送到处理器 2 上，依次类推。

人们通过对一些文本应用频率分布，研制了相应的分析模型（比盲目地遵从齐普夫定律更加接近真实情况的方法）。分析模拟的结果是：主机方法和磁盘方法的性能相当，而系统方法的性能并不乐观。这是因为系统方法需要串行读取很长的倒排表并传输这些列表。如果传输的成
268 本大大降低，系统方法也会很有竞争力。

7.2.5　在 CM-5 机上实现并行倒排索引算法

文献[Masand 和 Stanfill，1994]描述了另外一种在 CM-5 机上实现的算法。在这个算法里，文档分配到 64 个不同的处理器上，每个处理器分别建立压缩的倒排索引。倒排索引的构建速度相当快。20 分钟内可以索引 2.1GB 的文档集，而索引文件大小仅为原始文本大小的 24%。

查询的处理过程为：将用户查询分别发送到每个处理器上，并为每个处理器获取相关度排序结果。一旦获得这些数据后，在所有处理器上计算全局最大值，找出最高排名的文档。该文档首先排列出来，循环执行全局最大值计算，直到检索出的文档数目达到要求为止。

7.2.6　在倒排表上执行布尔操作

在两个倒排表 A 和 B 上执行布尔操作，是倒排索引相关计算中需要使用并行处理的另一个领域 [Bataineh 等人，1989；Bataineh 等人，1991]。我们需要划分倒排表，以确保每块划分等长。通过计算每一个划分数据的布尔操作结果，可以得到最终的布尔计算结果。我们没有必要将一个划分中的值与另外一个划分的值进行比较，因为在具体划分的时候，我们已经确保每个划分所包含的值都在一定的范围之内，而其他划分的值不会和该范围交叉重叠。这个划分过程可以并行完成。

一旦确定好划分，每个划分的数据都会发送到一个独立的处理单元上。随后，每个处理单元会单独计算其值的布尔操作结果。最后，我们会获得最终结果并将其存储在磁盘上。该算法最早是在 NCUBE/4 和英特尔 iPSC/2 机器上实现的。词“human”与“English”对应的倒排表长度分别为 520 316 与 115 831（使用的是 MEDLINE 数据库），采用 8 个 NCUBE 处理器，我们可以观测到系统能取得 5 倍的加速比；而采用 16 个 IPSC 处理器，可以获得 7 倍的加速比。值得注意的是，随着处理器数目的增加，并行算法的性能提升速度开始下降。导致这种现象发生的原因是每个处理器处理的工作量在下降，而通信耗费在增加。

7.2.7　作为 RDBMS 应用的并行检索

人们将信息检索引擎作为 RDBMS 模型的应用（参见第 6 章），这种研发背后的动机之一就

是商业数据库的并行实现能力。在并行关系数据库引擎之上采用 SQL 脚本实现信息检索的应用，可以并行实现信息检索系统。 269

文献[Grossman 等人，1997]具体描述了如何作为并行关系数据库的应用来轻松实现并行信息检索应用。根据这些发现，我们不难假定：完全有可能采用并行关系数据库技术研发一套可扩展的并行信息检索系统。

为了验证这个假定，我们采用 24 个处理器的数据库引擎来开展可扩展性实验[Lundquist 等人，1999]。然而，最终的实验结果却令人失望。采用与文献[Grossman 等人，1997]一样的关系表结构定义，我们只能达到 40%的处理器性能。

我们在实验中使用的并行硬件设备（NCR DBC/1012）可以支持自动负载均衡。用来实现负载均衡的散列方法主要基于索引结构，该结构已经采用关系模式定义好了。在用来平均分配负载的 DBC/1012 的架构基础上，我们必须将均匀分配的属性集作为散列函数的输入。而在刚开始的实现过程中，散列函数是基于词的，因此，数据难免不均匀。我们改变散列函数的输入，在词的基础上引入文档 ID 信息，最终可以确保处理器的负载是平均分配的。在后续的实验中，我们发现均衡处理器的利用率高于 92%。和单个处理器的实验进行直接对比，我们采用 24 个节点，可以取得几乎 22 倍的加速比。

7.2.8 并行索引小结

随着并行 I/O 成本的降低，信息检索中的并行处理越来越实用化。以前的一些算法存在的问题主要是内存有限制，处理器间的通信代价过高。签名文件曾经盛行一时，但最近并没有多少人使用，原因在于它存在不必要的过高 I/O 请求，而且不能计算比较复杂的相关度。并行倒排索引算法越来越为人们所接受，采用改进的压缩技术后，该方法变得更加经济实用。

7.3 聚类与分类

文献[Ruocco 和 Frieder，1997]在英特尔 Paragon 上研制了并行聚类分类算法。采用商用机器，作者实现了单遍聚类法以及单链接聚类算法（参见 3.2 节）的并行处理过程。采用 TREC 文档集中的《华尔街日报》数据，作者测试了他们方法的性能，在 16 个节点上能取得近线性的扩展性能。 270

Buckshot 聚类算法是常用的高效算法 [Cutting 等人，1992]，该算法最近也实现了并行化[Jensen 等人，2002]。该并行算法仅仅使用了低成本的 PC 集群，采用 MPI 进行通信，并且不需要采用专门的并行硬件设备。文章作者采用了比较新的 TREC 文档集测试该算法，结果表明节点数与文档集合大小呈现近线性的比例关系。

为了准确地比较所研制方法的性能，我们须保证并行与串行实现该算法的结果都是一样的。否则，算法性能的提高（通过并行化处理）会以牺牲准确度为代价。

单遍聚类算法和数据、呈现以及处理顺序是密切相关的。也就是说，提交作为输入的数据顺序会直接影响所产生的输出结果。因此，我们需要在并行实现过程中提供一种机制模拟文档呈现的顺序，以此来作为算法的输入。只要文档呈现的顺序一致，无论是采用串行方法还是并行方法来实现，其最终的聚类结果形式都是一样的。作者还注意到类别大小的巨大差异，因此建议采用评测方法的时候尽可能降低类别大小的不一致性。类别大小不一致是单遍算法的必然

后果，因此，目前还没有可改进的地方。

7.4　大型的并行信息检索系统

在研制大型通用并行信息检索系统方面，目前已经做了一些工作。本节详细介绍其中最突出的一些工作。文献[MacFarlane 等人，1997]概述了并行信息检索系统的一些工作。

7.4.1　PADRE

PADRE（并行文档检索引擎）系统最早是在富士通 AP 1000 系统上实现的文本检索系统。多年以来，PADRE 已经发展成为现代搜索系统，现在已经形成了 CSIRO 全景式企业搜索引擎的内核。作为内核，PADRE 是一个分布式常驻内存的模式匹配系统，但也支持相关度排序。多年以来，其架构已经经过了多次重大革新。开始在 AP1000 系统上的架构可以参见文献[Hawking，1991；Hawking，1994b；Hawking，1994a；Bailey 和 Hawking，1996]。后来又在最近的 INEX 竞赛中应用于一些复杂的任务，比如 XML 检索[Hawking 等人，2000；Craswell 等人，2002；
271 Vercoustre 等人，2002]。

Bailey 等人研究了 PADRE 系统的并行搜索，并探索了如何将该系统扩展到 1TB 的数据之上[Bailey 与 Hawking, 1996]。而且，他们的扩展方法是将数据分发到虚拟机的多个 CPU 上——该方法现在看来未必总是合理的。[Chowdbury 与 Pass，2003]。

7.4.2　并行信息检索框架

在对称多处理器上，什么样的框架最适合运行并行信息检索？一些工作专门研究了这一课题 [Lu 等人，1997] 。作者构建了一个多线程多任务的搜索引擎来调研执行并行查询最有效的方法。此外，模拟器可以支持不同的系统参数，如 CPU 数量、线程和磁盘，并且可以比较不同实现下的计算结果。采用这种方法，我们可以定位出瓶颈所在。从这种角度来说，我们发现如果没有额外的资源，如线程、CPU 和磁盘，往往很难得到进一步的扩展能力。总的来说，不同的系统配置可以得到不同的可扩展检索性能。

7.4.3　PLIERS

微软剑桥研究院的研究人员也研制了一个叫做 PLIERS（采用 MPI 的可移植的并行信息检索）的并行信息检索系统[MacFarlane 等人，1999]。该系统使用了 MPI 并行应用框架来实现可移植的并行信息检索系统[Gropp 和 Lusk，1998]。PLIERS 系统在 Okapi 单处理器信息系统上，实现了索引、文档搜索、文档更新与信息过滤模块的并行版本，通过标准 MPI 通信过程实现了并行能力[Robertson，1997]。而且，该系统支持多种搜索模式，其中包括布尔搜索、邻近搜索、信息块搜索以及相关性搜索。同时，几种不同的 MPI 实现方式发现：尽管在具体实现上存在一些不同之处，但是并不妨碍系统的可移植性。最终，该系统移植到了多个不同架构的环境上，如工作站网络（NOW，Network of Workstation）、富士通 AP 1000 和 AP 3000 并行机、PC 集群以及阿尔法机（Alpha farm[①]）。作者的研究结论表明：采用这种方法实现的并行信息检索系统可

① DEC 出品的一种型号的并行机。——译者注

以获得性能上的提升，而且还进一步发现 MPI 指令集可以提高信息检索查询事务以及信息过滤中词选择的性能。

7.5 本章小结

随着在线数据规模的持续增长，信息检索解决方法需要进一步地研究，以便能出来日益膨胀的数据集。为了解决数据爆炸式增长的问题，我们需要研究并行解决方案。刚开始，并行信息检索解决方案主要聚焦于基于硬件的全文过滤。最终，这些硬件解决方案逐步让位于软件实现方案，这些软件方案可以将硬件方法进行粗略映射。最近的并行研究大多数独立于具体的算法以及硬件架构。 272

我们首先综合阐述了并行文本扫描技术。我们介绍了两种全文本扫描硬件解决方案，其中包括 Utah 检索系统以及数据并行模式匹配系统。这两个系统都支持硬件级别的过滤，以达到减少检索文档集的目的。与软件全文本扫描相比，它们确实取得了客观的性能提升，但是，一般来说，全文本扫描会带来过多的 I/O 请求，因为所有的文档都必须扫描。基于 Utah 检索系统后续的工作依靠索引技术，但是为了能够快速融入最新研究成果，最新的架构往往会采用通用处理器。这就大大降低了专用解决方案的通用性。

后来的工作研制了软件支持的文本扫描算法。为了降低全文本扫描时的 I/O 请求，更多的工作偏重于签名分析。采用 SIMD 架构的早期研究主要包括：DAP 架构以及连接机。最终结果表明在性能方面的扩展能力有限。后续的签名分析工作是在 MIMD 系统上进行评测，比如 Inmos Transputer，结果多少有一些提高。

全文本扫描方法，无论是全文本还是签名分析，对 I/O 请求都存在限制。这种限制最终促进了并行索引方法的研究。文献[Stone，1987]清晰地分析了基于索引方法的需求所在，其中发现：与使用签名分析方法的并行引擎相比，使用索引技术的串行计算机的检索速度要快得多。在 SIMD 和 MIMD 结构上都研制了并行索引方法，其中有些工作能取得近线性的加速比。

然后，我们介绍了聚类算法与分类算法的并行实现方法。这些方法都是在英特尔 Paragon 处理器上实现的，该处理器已经作为产品在销售使用。对于所研究的所有算法，都能取得近线性的加速比。

最后，我们简要讨论了并行搜索系统。目前，并行信息检索还是一个亟需进一步探索的领域。我们还需要进一步研究可扩展的并行算法以支持第 2 章所讨论的各种检索模型与算法，同时也可以支持第 3 章所列出的各种实用策略。当前，已知的这类算法很少，已实现并在生产环境下进行评测的算法更是少之又少。

如何采用不同的方法来实现并行信息检索系统，可参见文献 [Lundquist 等人，1999]。在这些工作中，有一种方法是将信息检索运算符定义为并行数据库的操作原语。我们不需要研制新的并行算法即可实现并行化操作。采用 24 个节点，我们大致可以达到 22 倍的加速比。这种加速比确实是鼓舞人心的，毕竟我们并不需要实现新的软件。 273

尽管现在可用的商用并行系统非常多，各种不同类型的应用程序构成了信息检索的王国，但是有一点非常明确：目前，如何更好地支持并行信息检索领域的应用仍然是有待解决的问题。

7.6　练习题

(1) 在 *tf-idf* 向量空间中，针对 t 个词的查询，请进行串行倒排索引的常态算法分析。如果采用 p 个处理器对文档集进行并行线性扫描，请与刚才的情况进行比较。

(2) 请在 MIMD 机上设计一个倒排索引并行搜索算法，要求其性能等同或者超过串行算法。分析你所设计的算法，明确给出相关的分析过程。

274

(3) 请设计一个简单的文档聚类并行算法，分析其性能。将其与文档聚类串行算法进行比较。

第8章 分布式信息检索

严格来说，本书迄今为止关注的都还是如何使用单台机器来提供信息检索服务的。在第 7 章中，我们讨论了如何在多处理器的单台机器上提高检索性能的应用。虽然高效的性能对于用户接受系统非常关键，但如今，文档集往往是分散在不同地理区域内。因此，相对于高效处理数据的能力来说，更重要的是如何处理不同分布地点的数据，这点完全可以证实。限制数据集中化的因素有很多，包括数据的安全性、难以大量传输数据、数据的更新速度、政治和法律的各种规定以及其他可能的所有权问题等。从数据工程的角度来看，如何在分布式环境下完成数据处理系统工程的全面讨论，可参见文献[Shuey 等人，1997]。

网格（Grid）是最近广为人知的系统处理架构[Foster 等人，2001；Foster，2002；Alliance，2004]。网格是以全球电力输送网而命名的。在电网中，电器（在我们的领域中，对应的则是系统）只需要接上电源就能够立即工作，并且能够快速方便地为社会大众所使用。于是，在现代信息检索领域内，我们有一个类似的想法，那就是采用分布式信息检索系统（DIRS，Distributed Information Retrieval System），即可访问位于不同物理区域内不同机器上的数据（参见图 8-1）。

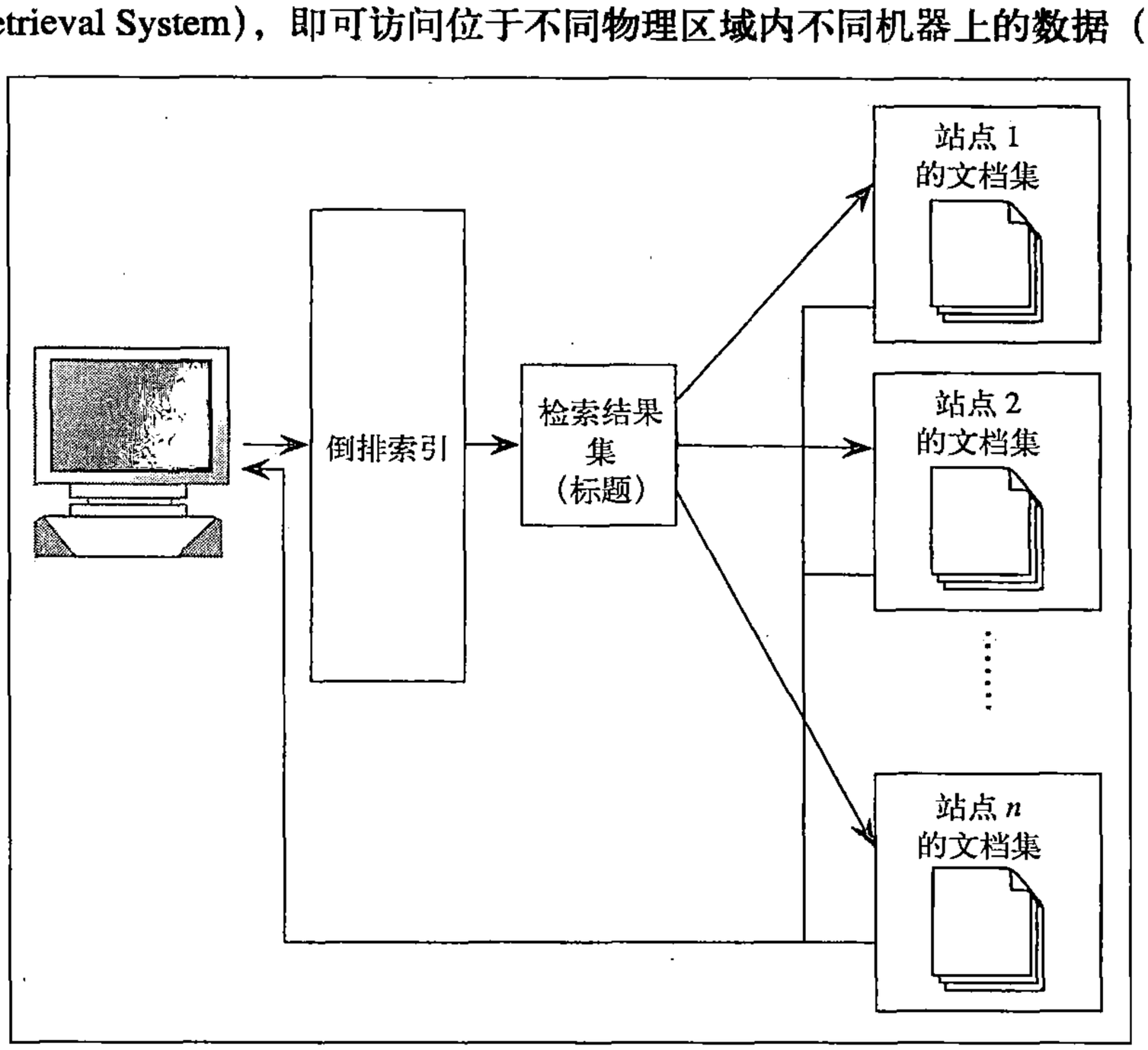


图 8-1　分布式文档检索

早在 20 世纪 80 年代初期，人们就已经意识到分布式信息检索系统是不可或缺的。最初，

科研人员提出了一个理论模型，这个模型描述了分布式信息检索系统的一些关键组成部分，这将在 8.1 节详细介绍。

在 8.2 节中，我们将简要介绍一下 Web 搜索引擎。我们注意到，即使是最流行的搜索引擎，
275 事实上也仅仅是实现了本书中已经讨论的一些算法和效率优化技术。这里我们不再讨论特定搜索引擎的实现细节，因为这些细节不仅涉及所有权问题，而且一直也在改变。即使本书选择介绍这些细节，那么在本书出版时，这些技术也早已过时。此外，如果包含了这些细节的介绍，本章也就变成我们对某个特定搜索引擎的宣传，这就违背了本书的初衷，也超出了本书的介绍范围。不过，我们注意到 www.searchenginewatch.com 上提供了这些搜索引擎的完整列表，包括很多诸如 Google、Yahoo!、MSN Search 以及 AOL Search 等著名的搜索引擎。

最后，我们将在 8.4 节介绍了 P2P（点对点）方面的工作。目前，这些 P2P 方面的各种工作都还处于初期阶段。在本书的后续版本中，我们希望能更好地介绍这些新技术的各种最新进展。

8.1　分布式检索的理论模型

276 我们首先定义集中式信息检索系统的模型，然后将其扩展为分布式信息检索系统的模型。

8.1.1　集中式信息检索系统模型

我们将信息检索系统定义为一个三元组，即 $I=(D,R,\delta)$，其中 D 表示文档集，R 表示查询集，而 $\delta_j : R_j \rightarrow 2^{D_j}$ 则表示第 j 个查询与其相关文档集的映射。

许多信息检索系统都依赖于知识库。知识库用于扩展用户查询，以便引入查询关键词的同义词来匹配文档中的同义词。因此，一个包含词“curtain”（门帘）的查询，也可以匹配包含有词“drapery”（帏帐）的文档。

为了在模型中引入知识库，文献[Turski，1971]提出将上述的三元组扩展为四元组，即：

$$I=(T,D,R,\delta)$$

其中 T 表示非重复词项的集合以及关系 $\rho \subset T \times T$，即 $\rho(t_1,t_2)$ 表示 t_1 是 t_2 的同义词。利用这种同义词关系，文档就可以表示为描述符（descriptor）集合与属性符（ascriptor）集合。以文档 D_1 为例，描述符集合 d 由文档 D_1 中所有符合下述条件的词项组成。

- 每个描述符都是唯一的。
- 任何两个描述符之间的语义都是不同的。

我们将描述符的同义词定义为属性符。每个属性符必须只能与一个描述符同义。因此，描述符是表达文档含义的最小集合。

除了知识库之外，我们还定义了描述符集合上的泛化关系为 $\gamma \subset d \times d$，其中 $\gamma(t_1,t_2)$ 表示 t_1 比 t_2 更具概括性。因此，γ(动物, 狗)就是一个有效泛化关系的例子。

泛化关系假定所有成对出现的描述符都可以构建层次化的知识库。知识库的构建方面，自动构建和人工构建的方法都有所尝试[Lenat 和 Guha，1989]，但是很多词还是很难定义。我们很难用这种模式来表示涉及时空、思想、信仰等概念的关系。但是，该模型并没有讨论如何构建这类知识库，只介绍了这种层次性的知识库一旦构建后会具备什么样的属性。

使用知识库的动机其实就是简化文档描述，仅仅用那些两两完全不同义的词来表示。也就

是说不要将额外的同义词加入到文档的语义值中。使用泛化关系就可以处理“列出所有动物”之类的提交查询，最终能够返回包含关于狗、猫等动物信息的文档，即使这些文档中并没有出现“猫”或者“狗”之类的词。 277

然后，还可以使用这种泛化关系定义文档的偏序关系。我们使用 $\preceq$ 来表示偏序，用 $t(d_i)$ 来表示文档 d_i 的描述符列表。则偏序关系 $\prec$ 定义如下：

$$t(d_1) \preceq t(d_2) \Leftrightarrow (\forall t' \in t(d_1))(\exists t'' \in t(d_2))(\gamma(t', t''))$$

这样一来，如果文档 d_1 的描述符都是文档 d_2 的描述符的泛化，那么我们记作 $d_1 \preceq d_2$。例如，包含词“动物”和“人”的文档先于包含词“狗”和“约翰”的文档。需要注意的是，这是一种偏序关系，因为如果两篇文档中的词项没有关系，则这两篇文档不存在先后次序关系。

因此，可以推论如下：对于通用查询 q_1 以及比 q_1 更具体的查询 q_2，其中 $q_1 \preceq q_2$，则 q_1 对应的文档必须包含（是后者的超集）q_2 对应的所有文档。其形式化的定义如下：

$$(q_1, q_2 \in Q) \wedge (q_1 \preceq q_2) \rightarrow (\delta(q_1) \supset \delta(q_2))$$

此处所述模型的证明可参见文献[Turski，1971]。这就意味着，当提交到系统的两个查询 q_1 和 q_2 具有泛化关系，并且 q_1 比 q_2 更具通用性时，就不必在所有文档集中执行每个查询，只需查找与 q_1 相对应的结果文档集 $\delta(q_1)$，然后在 $\delta(q_1)$ 中迭代查找，从而得到 $\delta(q_2)$。

8.1.2 分布式信息检索系统模型

集中式信息检索模型能够划分成 n 个本地的信息检索系统 $S_1, S_2, \cdots, S_n$ [Mazur，1984]。每个系统 S_j 的形式化定义为：$S_j = (T_j, D_j, R_j, \delta_j)$，其中 T_j 代表知识库，D_j 代表文档集，R_j 代表查询集，而 $\delta_j : R_j \rightarrow 2^{D_j}$ 则表示查询到文档集的映射。

合并各个本地节点后，就可以这样定义分布式检索系统：

$$S = (T, D, R, \delta)$$

其中：

$$T = \bigcup_{j=1}^{n} T_j$$

$$s_j = s \cap (T_j \times T_j), \quad R_j = R \cap (d_j \times d_j)$$

这表明全局的知识库可以根据本地的知识库重建，而且在节点 j 上的查询只包含节点 j 上的描述符。这样的话，查询中的词不是描述符的话，就不会检索到任何文档。 278

$$D = \bigcup_{j=1}^{n} D_j$$

文档集 D 是通过综合所有节点上的文档集构成的。

$$R \supset \bigcup_{j=1}^{n} R_j, \quad \preceq_j = \preceq \cap (R_j \times R_j)$$

而查询则通过综合每个本地节点上的查询构成。在节点 j 上定义的偏序关系仅仅只适用于节点 j 的查询。

$$(\forall r \in R)(\delta(r) = d : d \in D \wedge r \preceq t(d))$$

对于系统中的每个查询，检索获得的文档集肯定包含了那些比查询更详细的文档，也就是

说，被检索到的文档与查询语句是泛化关系。

γ 表示在不同节点上划分的层次结构。发送到源节点的查询将会被发送到每个本地节点上，然后在每个本地节点上执行本地查询。本地节点的结果将会返回到源节点，融合之后得到最终的结果集。只有每个本地节点符合成为信息检索系统子系统的标准，该模型才允许使用此方法。

如果满足如下条件：

$$(T_1 \supset T_2) \wedge (R_1 = R_2) \bigcap (d_1 \times d_2) \wedge (s_1 = s_2) \bigcap (T_1 \times T_2)$$

则 $S_1 = (T_1, D_1, R_1, \delta_1)$ 才是 $S_2 = (T_2, D_2, R_2, \delta_2)$ 的子系统。

知识库 T_1 是 T_2 的超集。

$$D_1 \supset D_2$$

节点 S_1 的文档集包含文档集 D_2。

$$R_1 \in R_2 \wedge \preceq_1 = \preceq_2 \bigcap (R_1 \times R_2)$$

节点 S_1 的查询包含节点 S_2 的查询。

$$\delta_1(r) = \delta_2(r) \bigcap D_1 \text{，如果 } r \in R$$

节点 S_1 的查询返回的文档集包含所有节点 S_2 查询返回的文档集。下面的例子表明，对层次
279 结构的任意分割，可能产生一个无效的子系统。

下面我们考虑一个关于人的层次结构：

γ(人, 哈瑞得)，γ(人, 赫伯特)，γ(人, 玛丽)

再看第二个关于动物的层次结构：

γ(动物, 猫)，γ(动物, 狗)，γ(猫, 黑猫)，γ(猫, 柴郡猫)，γ(狗, 杜宾犬)，γ(狗, 贵妇犬)

假设这个层次结构划分到节点 S_1 和节点 S_2 上。在 S_1 上的层次结构是：

γ(人, 哈瑞得)，γ(人, 玛丽)

γ(动物, 猫)，γ(动物, 狗)，γ(狗, 杜宾犬)，γ(狗, 贵妇犬)

S_2 上的层次结构是：

γ(人, 赫伯特)，γ(人, 哈瑞得)

γ(动物, 猫)，γ(动物, 杜宾犬)，γ(猫, 柴郡猫)，γ(猫, 黑猫)

假设文档集包含以下描述符：

D_1 = (玛丽, 哈瑞得，赫伯特)

D_2 = (赫伯特, 狗)

D_3 = (人, 狗)

D_4 = (玛丽, 柴郡猫)

D_5 = (玛丽, 狗)

D_6 = (赫伯特, 黑猫，杜宾犬)

D_7 = (赫伯特, 杜宾犬)

由最通用的查询词（人，动物）组成的查询，结果应该包括从文档 2 到文档 7 的文档（因为文档 1 的标签不包括动物，而这个查询需要满足布尔与操作）。然而，根据 S_1 给出的层次结构，我们只能检索出文档 D_3 和 D_5，而在 S_2 上只能检索文档 D_6 和 D_7。那么，如果只是简单地合并两个节点的结果，就会遗漏文档 D_2 和 D_4。由于 S_1 和 S_2 上返回的文档集不能通过简单地合并而得到最终结果，在 S_1 和 S_2 上的信息检索系统并没有达到构建子系统的必要标准。

在实际应用中，使用泛化层次关系还会带来另一个问题。层次关系不仅难以构建，而且对层次进行划分也非常重要。这种分布式模型还需要进行扩展，以便为相关性排序引入带权重的关键词[Mazur，1988]。 280

8.2 Web 搜索

任何讲解分布式信息检索的章节，如果不提 Web 搜索引擎，那么它肯定是不完整的。通过因特网访问网页的搜索工具，其实就是实现本书所讨论的许多算法和启发式规则的最佳实例。实际上，这些系统本身就是分布式系统，因为它们访问的数据都存储在分布于世界各地的网络服务器上。大部分系统有一个集中式索引，但是，所有系统都会以超链接的形式存储指向各种网络服务器的指针。

这些系统每天要为上千万用户查询提供检索服务，并且所有系统都索引了 TB 级的网页。搜索引擎技术更新频繁（一些供应商会在一周内制作新版本或者发布补丁），所以我们并不会详细介绍每个搜索引擎。我们注意到 www.searchenginewatch.com 上列出了 16 种不同的 Web 搜索引擎，而在 www.searchengineguide.com 中列出了 2 500 多种专业搜索引擎。

8.2.1 Web 搜索引擎评测

在传统的信息检索环境中，使用标准的查询语句和数据评测单个系统。而在网络环境中，并没有这样的评测条件。而且，由于网络中存在着海量数据并且是动态更新的，任何大规模的人工评测实际上也是不可能的。为了能够自动评测 Web 搜索引擎，文献[Beitzel 等人，2003b]中提出了一种使用在线目录分类的方法，在线分类目录法是开放式目录项目（ODP）的一部分。我们将在线目录作为查询对应的已知相关文档。如果查询语句匹配了已存储文档的标题或者包含一个已知词的目录文件名，就认为查询匹配成功。作者比较了两种系统的排序结果，一种是使用这种自动方法，另外一种是在有限的规模上，由许多用户人工标注系统排序的结果。这两种排序结果在统计学上是相同的。

8.2.2 高准确率检索

与传统的搜索引擎环境相比，Web 搜索引擎评价的另外一个关注点是对成功的不同测量方式。在传统方式下，我们把准确率和召回率作为主要的衡量指标，而响应时间和空间要求都很容易解决。然而，在网络环境中，对响应时间的要求非常苛刻。而且，召回率的估计也非常困难。而且，对于很多用户来讲，如果第一页（前 10 个可能的参考链接）没有他们需要的链接，他们都不会关注第二页，因此，对准确率的关注也很有限。因此，Web 搜索引擎开发者把重点放在如何保证迅速生成第一页的检索结果，获得较高的精度，而且确保第一页没有明显不匹配的结果产生。例如，在文献[Ma 等人，2003]中，根据网页模板可以有效地从 Web 文档中抽取出 281
有效文本，并丢弃框架的部分内容或者整个框架内容，以屏蔽那些有可能匹配查询的广告框架。在文献[Beitzel 等人，2004a]中，对于当前的新闻列表服务，后台使用高效、高准确率的方法，保留需要显示的头条新闻，并快速筛选并丢弃那些没有太大相关性的内容。

8.2.3　查询日志分析

总之，虽然网络搜索使用与传统信息检索系统领域几乎相同的技术，但是网络环境中提供了许多新的机会来重新解决这些老问题，特别是性能和精度优化以及查找精度的评测方面。鉴于此，文献[Beitzel 等人，2004b]公开发表了大型查询日志每小时的数据分析结果，而查询日志事先按照话题进行了分类。利用这些结果，我们可以对系统进行优化。例如，正如这些日志分析结果所显示的那样，用户在一天中的某些时间和一周中的某些天的查询请求模式，存在着有一定规律的重复现象。因此，我们可以在一天中的某些时间和一周中的某些天提前预测用户查询请求，并提前缓存一些可能的网页来满足部分用户的查询请求。这样，可以减少页面访问延时，从而提高系统吞吐量。而且，从精度优化方面来讲，我们同样可以调整排序策略来更好地预测某些用户的请求。简而言之，有许多优化策略是可行的。你可以使用这些日志做些什么优化呢？你会通过什么衡量方式来论证检索是成功的呢？我们相信，在这本书的下一版中，我们将介绍更多这样的衡量和优化方法。

8.2.4　PageRank 算法

我们将会用这个最著名的算法来结束 8.2 节。该算法大大改进了 Web 搜索效果。PageRank 算法（以 Page 的名字命名）最早见于文献[Brin 和 Page，2000]。PageRank 算法拓展了 Hub（中心型节点）和 Authority（权威型节点）的概念，这些概念最早见于文献[Kleinberg，1999]。PageRank 算法是现在 Google 这个最流行搜索引擎的核心所在。从本质上讲，PageRank 算法就是利用指向网页和网页指出的链接来调整该网页流行度的权重，而与用户的查询请求无关。因此，传统的检索模型可能对两篇文档的排序相同，而 PageRank 算法将会大大提高更“流行”（popular）文档的相似度。这里，“流行度”定义为在其他网页中存在大量链接指向该网页。这种算法对于网页检索效果极佳，但对于没有任何超链接的文档不起作用。关于页面 A 的 PageRank 计算方法
282 如下所示，其中 $D_1, \cdots, D_n$ 为所有链接到页面 A 的页面：

$$\text{PageRank}(A) = (1-d) + d \sum_{D_1, \cdots, D_n} \frac{\text{PageRank}(D_i)}{C(D_i)}$$

其中 $C(D_i)$是页面 D_i 中指向其他页面的链接的数目，d 为抑制因子，取值范围为 0~1。抑制因子的作用是使那些没有链接页面的值为非 0，同样可以在确定一个给定网页的 PageRank 值时，起到平滑其他链接权重的作用。这样明显减少了 PageRank 收敛所需要的计算时间。该公式采用迭代的方式进行计算。最初，所有的页面赋值为一个任意的 PageRank 权重。算法使用前一次的计算得分迭代进行，直到新计算出的得分与原得分变化不是很大。例如，对于图 8-2 中给出的例子，我们通常使用的抑制因子为 0.85，并且每个网页的初始权重为 1.0，迭代进行了 8 次后结果收敛。

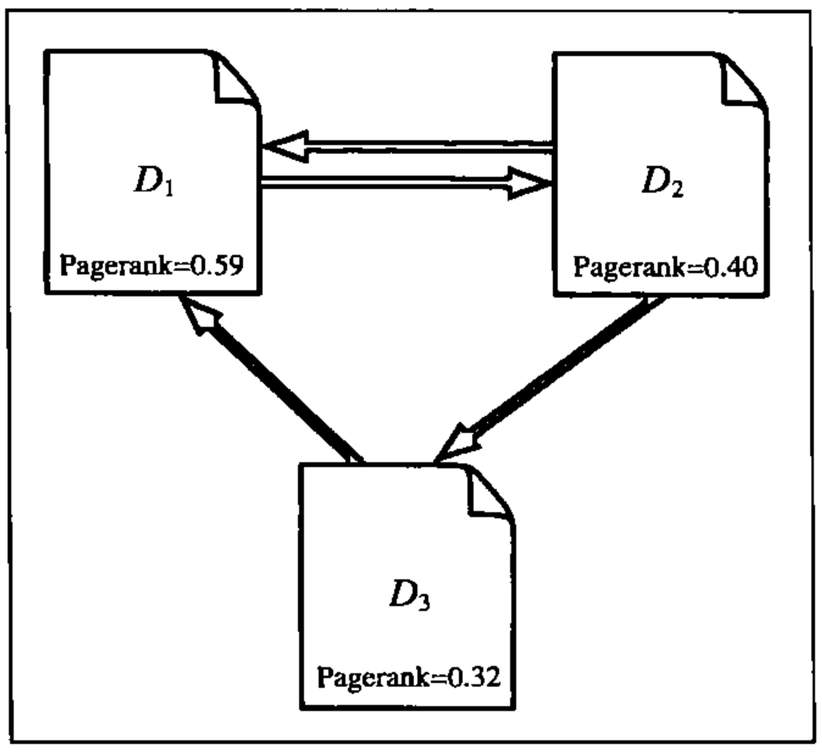


图 8-2　简单的 PageRank 计算的例子

8.2.5 Web 搜索引擎的效果提升

使用网络服务器实现信息检索系统，其所用的算法并没有太大不同。对于每一台机器，第5章中介绍的所有算法都是适用的。倒排索引的压缩也是一样，部分相关序排序的方法也相同，等等。不过，在专门提高基于 Web 的信息检索系统的性能方面，过去和现在都取得一定成效。

在提高精度上，我们有理由相信，将检索请求发给不同的搜索引擎，并将返回结果融合为一个结果，可以大大提高 Web 搜索引擎的准确性。早在 TREC-4 时，就有人提出过这种想法。后来，原型系统 CYBERosetta 中[Desantis 等人，1996]就将相同的检索请求副本同时发送到多个检索服务器，该系统由软件生产力协会（SPC，Software Productivity Consortium）为 DARPA 所研发。在达到超时限制后，系统将已经获得的结果融合为一个结果呈现给用户。

8.3 结果融合

Lee 在文献[Lee，1997]中提出了结果成功融合的条件。之后不久，如何采用融合结果来提高搜索精度，开始引起了广泛的学术研究兴趣。也就是说，Lee 采用几种常见的融合启发式规则来合并从多个独立的搜索引擎返回的检索结果，表明了相关文档集合的重叠程度比不相关文档集合大得多。他还发现：使用 CombMNZ 融合启发式规则的检索精度高于所使用的任何一个单一搜索引擎。Alaoui 等人扩展了 Lee 的工作，并得出结论：为了最佳地利用结果融合技术，所使用单一搜索引擎的模型算法以及实用策略应该大大不同[Alaoui 等人，1998]。

最近，在文献[Chowdhury 等人，2001；Beitzel 等人，2003a]中重新研究了 Lee 的结果融合。和 Lee 原来的研究有所不同，该研究中所有的系统参数（例如词干还原器、语言解析器、停用词表等）都保持不变，只有使用的相似度计算方法不同。另外，也不同于 Lee 的研究的是，该研究只使用最大的相似度计算方法。研究结果表明：采用新的方法，Lee 所述的相关与非相关文档的交叠程度要有限得多。令人惊讶的是，其他的系统参数对融合的整体效果影响要远远大于最初假定的相似度计算方法。

这种实验观测结果依赖于最新的高精度计算方法。从这些研究发现中，我们可以看出：这些年来，虽然相似度计算的精度不断提高，但是不管使用任何所谓更好的方法，结果集之间的变化都不是很显著。既然检索可以获得的结果集大同小异，因此，结果融合至少是这种形式的融合，检索的精度就算没有实际减少的话，也只能实现非常小的提高。其中的奥秘可能是：一种检索模型与算法会随着时间的推移而不断演化，在演化过程中，人们会逐渐引入一些实用性的特色功能，而这些特色功能往往是另一种算法中首创的。比如，几乎在所有的检索模型中，都会引入词频和文档长度。因此，随着这些算法模型的演化，最终返回非常相似的文档结果也就不足为怪了。这必然就降低了检索算法融合策略提高效果的概率了。

在效率方面，[Liu 等人，1996]阐述了一种提高 Web 服务器性能的方法。他们讨论使用预启动进程或者称为“cliettes”，以避免常见 CGI（Common Gateway Interface，公共网关接口）开始进程中的启动开销。该方法用于实现一个可以搜索 8 个图书馆数据集的原型系统。

当前，大多数 Web 服务器都使用非常详尽的全文索引，但是如果网络信息持续增长，那么使用单一索引将变得不切实际了。基于 Web 的分布式查询处理领域的早期工作，可参见文献

[Duda 和 Sheldon，1994]。在这个文献中，使用 WAIS（Wide Area Information Service，广域信息服务系统）的系统基于这些服务器上内容的初始检索结果，只将查询发送到特定的服务器上。每个服务器的文档内容按照多个专门的字段来描述，比如一篇新闻文章的标题或者一封邮件的主题。这里的内容索引其实是个折中的方法，居于两种情况之间，其一是将查询请求发往所有的服务器，其二是提供一个非常详细的全文索引，并将查询请求只发往那些能够匹配索引的服务器中。

基于向量空间模型，人们在利用 GlOSS（Glossary-of-Servers Server）搭建服务器方面做了大量工作，可以为给定的查询估计最好的服务器[Gravano 和 Garcia-Molina，1995]。查询向量将匹配每个独立服务器的特征表示向量，并由前 n 个服务器来提供排序与检索服务。人们研究了几种描述服务器特征的方法。最简单的方法是将给定服务器上每个词的 *tf-idf* 值进行累加，并且基于服务器上的文档数对该值进行归一化，这样就可以产生每个服务器的中心向量。给定一个查询，我们可以利用这个 *tf-idf* 向量空间系统计算相似度（如 2.1 节所述），然后基于相似度即可对服务器进行排序。哪一个服务器是可能的查询数据源？哪一个数据库可能包含所有查询中的词？为了解决这些问题，我们引入不同的相似度阈值。据估计，查询中所有的词语可能包含在哪个数据库的假设。据估计，GlOSS 服务器上的索引大小大概只有全文索引的 2%。

任意组合第 2 章的模型算法与第 3 章的实用策略，我们即可在 Web 服务器上使用全文索引完成查询处理。不过，基于网页中的超文本链接，人们还实现了另一种检索算法，具体可参见文献[Yuwono 和 Lee，1996]。这项工作研究了一种称为向量激活扩散的算法。在该算法中，基于简单查询中的词匹配来对文档排序。此外，那些链接到原始结果集的文档也会加入到结果集中。与原始结果集中的文档权重相比，新链入的文档权重会小一些。以比例因子在 0 到 0.5 之间进行实验，以 0.1 为步长进行递增，结果表明 0.2 是最好的比例因子。在 2 393 个网页的小测
285 试集上进行试验，测量其平均精度，实验结果显示向量激活扩散算法略好于 *tf-idf*。此外，该系统并没有使用全文索引。索引的对象仅仅限于：HTML 标记为粗体或斜体的词语、每个列表的第一句、标题、各级标题以及锚超文本。

总之，我们有必要说明，基于 Web 的搜索引擎领域与传统信息检索系统环境的不同还在于每个系统的评测方法。在传统的环境中，人们使用标准的基准查询和数据混合来评估不同的系统。而在 Web 领域中，每个搜索引擎只索引现有数据的一部分，而必须索引的部分和不必索引的部分未必是重叠，就算确实是重叠的，也未必对同一版本（不同时间段）的数据建立了索引。

8.4　P2P 信息系统

现在，我们来关注一个新兴的领域，它是互联网领域和信息检索的交叉领域，即 P2P 体系结构。P2P 体系结构是分布式环境，根据其定义，认为网络上的每个节点都是潜在的信息源（服务器），也是需求信息的客户端（客户端），同时也是信息传播的中间路由器（路由器）。每个节点都是独立的，而且系统以纯粹无中心的方式运行。而在信息检索系统领域，所提供的资源都是以可检索数据的形式呈现的。

P2P 系统最主要的特点就在于其天然的随机性以及耐久性。P2P 系统可以从容地处理系统中节点的加入与离开。这些节点提供的资源都可以根据需要在系统中动态添加或者删除。此外，

单一节点的故障不会导致整个系统崩溃。

P2P运动的起源通常要归功于Napster（它是一个音乐文件共享系统），尽管Napster实际上依靠的还是依据中心集中式方式而实现的。也就是说，Napster并不是以完全无中心的形式存在的，因此，这并不是一个纯粹意义上的P2P体系。但是，Napster确实为用户提供了P2P的功能，因为用户可以与他人动态地共享文件。Napster从性能和可靠性的角度看存在一些争议，除此之外，Napster的集中式实现模式最终也注定了它会遇到法律问题。现在，Napster再也不能以其原来的形式存在了。Napster的灭亡给P2P技术爱好者带来了教训。作为回应，他们创建了Gnutella协议[V0.4，2004]，这是真正的P2P，是许多当今P2P研究的基础。（Gnutella协议[V0.6，2004]的后续版本也存在，并扩展了P2P的体系结构，引入了层次结构。该协议及其应用稍后讨论。）

基于Gnutella（版本0.4）协议的系统一般只提供了最原始的搜索能力。也就是说，它们一般依赖于名称精确搜索，而名称精确搜索往往通过子串匹配来实现。具体来说，如果查询中的 286
所有词是某个文件元数据的子串，那么查询就匹配该文件。匹配的文件按照其元数据与查询的相似度进行分组，最终返回到客户端。目前，我们还没有可为大家接受的方法来对这些分组进行排序。用户选择其中的一组结果，从相应的服务器下载相关文件[Rohrs，2000；Rohrs，2001]。

大多数基于Gnutella协议（版本0.4）建立的P2P信息检索系统依然存在其他问题，其中包括：网络全局泛滥问题、搜索结果不确定而且精度差。因为从定义上看，P2P信息检索系统是无中心的，所以对于每一个检索请求，信息将被发送到所有有可能含有相关文档的节点上。因为每个节点搜索与文档排序的计算能力是有限的，所以任何潜在的相关文档都会发送到请求的节点上。给定潜在相关文档的数目，结果信息的网络总流量一般会大大高于已有的网络资源。这种情况就称为网络全局泛滥。为了减少信息传输量，文献[Yu 等人，2003]研究了一种结果过滤和合并的技术，而且节点一般都会收集邻居节点的信息。因此，查询请求节点所需的潜在结果到达中间节点时，只将合并和过滤过的结果返回到请求节点。层次P2P网络的最新合并技术可参见文献[Lu 和 Callan，2004]。

另一种减少网络流量的方法是，有选择性地将查询请求发送到有限的目标节点中。这种方法的不足是很可能导致结果的不确定性，或者是生成很差的结果。由于只选择性地访问有限的节点，所以不同网络条件下，同一个查询请求每次查询返回的结果可能都是不同的，从而导致结果的不确定性。此外，选择的目标节点不恰当的话，也会导致搜索精度很差。惠普公司的PeerSearch[Tang 等人，2002；Tang 等人，2003]通过索引映射和查询路由选择的方法来降低目标节点选择不当的风险，这个方法依据网络上向量空间模型不同维度的逻辑映射。因此，语义上接近的节点应该映射在一起，相互接近，从而减少网络流量。

最近，Gnutella协议的更新版本发布了[V0.6，2004]。这个版本可以支持逻辑层次关系。即在最初的P2P体系结构模型中，从逻辑上讲，所有的节点在功能上都是对等的，而且所有的信息是分散的。而在这个新版协议中，一些节点作为叶子节点（客户端和服务器节点），而另一些节点作为目录节点，目录节点并不提供具体的内容，仅仅为其关联的叶子节点提供全局的目录服务。这些由分散无中心的存储与集中式存储组成的混合网络，为检索提供了更多的机会。使用目录节点，就可以更准确地获取所需内容，同时更明确地选择适当的目标节点进行检索。在文献[Liu 和 Callan，2003]中，作者在这种网络中提出并验证了一套P2P信息检索系统。结果表 287
明：依靠目录节点中的信息，网络流量明显减少，而且检索精度所受影响控制在最小的范畴。

目前，一些学术性 P2P 系统包括 Edutella [Nejdl 等人，2002]和 PeerDB [Ng 等人，2003]。但是这些系统主要关注的还是标准化接口以及因特网访问服务的查询语言。它们的目标是创建一个 P2P 超级计算机或联邦数据库系统，而不是一个信息检索系统。

P2P 系统的另一个研究重点是带宽效率问题。人们不依赖于 Gnutella 协议，而是提出了更智能的路由协议[Ratnasamy 等人，2001；Sripanidkulchai 等人，2003；Stoica 等人，2001]。这些研究工作着力于提高响应速度，而不是用来完善查询的检索返回方式，也不是解决结果如何排序的问题。

举例：P2P 信息检索系统

PIRS[Yee 和 Frieder，2004]是基于 Gnutella 建立的一个 P2P 信息检索系统，最主要的目标是通过增加系统支持的查询模式数目，并改进查询排序机制来提高检索系统的效果。建立一个能反映各种不同用户查询模式的元数据语料库，可以最终增加系统能支持的查询模式数目。目前，典型 Gnutella 网络中找到的歌曲都包含元数据，这些元数据是由机器自动产生的[例如，使用网站，（如 freedb.org）中的 ID3 数据[Nilsson，2004]。如果不知道元数据命名规则或者有意给出了模糊查询，用户将无法找到所需的文件。例如，在 Gnutella 网络中，检索“芝加哥本地乐队”将无法返回任何有意义的结果，而查询“The Off-key Singing Trio”[①]则能得到有效的结果。

PIRS 解决了这个问题，主要方式是：从文件被选取的那组服务器中，强迫客户端随机复制其中的元数据。元数据复制的模式显著提高了查询的有效结果数目。

PIRS 使用文件组的集合元数据对文件组进行排序，而不是针对单个文件进行排序。它考虑 4 个排序要素。

- **词频**——衡量查询词在文件组元数据中出现的总次数。
- **逆词频**——词语频率的一种度量方法，将所有组频率的总和除以该词词频，通过这种方式实现归一化，最后能计算出每个词语的贡献度。
- **准确度**——衡量查询词词频在组中词总数的比值。
- **文件组大小**——衡量每组中所有文件的总数。

288

实验结果表明：组大小优于其他排序要素。该要素的效果主要基于以下两个原因：大的文件组意味着一个文件更可能匹配查询；大的文件组一般来说是比较受欢迎的，因此这必然也是很好的下载候选种子。总的来说，使用随机抽样测试方法，结果表明：与元数据复制和结果排序等其他综合策略相比，文件组大小排序方法在检索性能方面可以提高 30%。

8.5　其他的体系结构

除了 P2P 体系结构外，还存在一些核心的分布式体系结构，包括共享磁盘体系结构和分布式磁盘体系结构。在每种情况中，只要搜索是由多台检索服务器完成的，都可称为分布式体系结构。如果搜索是由多个处理器组成的一台大机器完成的，那么它就是平行信息检索搜索（详见第 7 章）。

① 芝加哥当地一个乐队组合的名称。——译者注

8.5.1 共享磁盘体系结构

即使有许多分布式处理器，我们仍然可以将它们连接到单个共享磁盘上。输入文件可能会分配到不同的服务器上建立索引，但所有的最终索引都会存储在同一个存储阵列中。我们可以在不同的服务器上处理查询，但最后都要访问这个存储阵列中的索引。因此，存储区域成为各个服务器争夺的单点资源，它会导致效率降低。它的优点是，即使在存储区域发生单点故障，仍可以引入额外的存储阵列作为备份；在索引的时候，我们可以对两个存储区域进行同步，一旦一个磁盘发生故障，则可以立即启用备份存储。

如果一个处理器崩溃，它仅仅是处理这个处理器上的查询请求，因此用户根本不会受到任何影响，整个索引仍然可以提供给所有用户。因此，这种体系结构的可靠性是相当不错的。

8.5.2 分布式磁盘体系结构

在这个体系结构里，每个服务器都有自己的本地磁盘，而索引在各个本地磁盘上建立。在建立索引时，文档集可以划分到不同的服务器上，它们并行地创建索引，并将自己的索引部分存储在本地磁盘上。查询也分布在各个服务器上，查询请求发送到与包含相关倒排表的各个处理器上。常用的词会收录在多个不同的处理器上，但不常见的词可能只会收录在少量的处理器上。

这种方法不存在单点资源竞争问题，因此它的效率特别高。如果一个处理器崩溃，则认为该处理器所维护的索引不可用。对于 Web 搜索应用程序，这可能是一个合理的折中做法，并且 289
给定一个查询，用户并不需要知道可能遗失了一些文档。对于重要任务的搜索引擎来说，必须找到所有相关文件。因为服务器挂掉而忽略部分文件是不能被接受的。而且，这个体系结构要求处理器是冗余的，因此，需要的机器数量是翻倍的，那样的代价也会很昂贵。

因此，相对于共享磁盘体系结构来说，通常认为分布式磁盘体系结构更有效率，但是可靠性不够。

8.6 本章小结

本章重点介绍了如何去搜索地理上分散的文档集，并介绍如何使用独立机器集合来搜索文档集。Web 搜索引擎本质上是分布式信息检索系统的示例，因为实际的文档都是存储在世界各地的服务器上。

首先，我们讨论了分布式信息检索系统的理论模型，然后对分布式检索算法的近期研究做了简要讨论，并介绍了最近 P2P 系统在信息检索方面的应用。最后，我们讨论了其他可行的分布式体系结构。

我们可以很合理地期望 CLIR 可以持续发展，期望越来越多的国家增加其所存储的电子文档集，那么高精度、高效率的分布式搜索系统的需求就会持续增长。

8.7 练习题

(1) 写一个分布式信息检索算法，将倒排索引存储在不同的机器上，每台机器中存储的索引

大小大致相当。计算出你设计的方法所需要的通信开销。

(2) 文档更新对本章介绍的分布式信息检索算法将产生什么影响？请予以描述。

(3) 最近网络搜索引擎面临了一个问题：网页设计人员往往会在他们的网页上加上经常被检索的词，从而使搜索引擎更关注他们的网页。用户可能添加迪斯尼乐园到他的页面上，虽然这个页面介绍的是厨房的水管装置。请提出一个启发式规则来解决这个问题，并谈一谈如何来避免降低“正常”文档或未添词文档集的检索效果。

290

第9章

总结与下一步研究方向

本书中，我们介绍了一系列信息查找与检索方法，这些方法中大部分都着重于提高信息检索引擎的准确率。与传统关系数据库等其他搜索与检索领域不同的是，信息检索的准确率并不是一成不变的。也就是说，在传统的关系数据库领域，所有的技术方法都会产生百分之百的精度。因此，从性能评测的角度来看，其主要评估的是系统整个吞吐量以及单个查询的性能。

在信息检索领域，系统的精度会随着搜索引擎的准确率与召回率的调整而变化，并且与具体采用的搜索方法和检索的数据集密切相关。因此，所有信息检索的性能评测都必须综合考虑搜索结果的准确度以及相应的处理时间。对数据库系统与信息检索系统的性能进行评测，即所谓的系统基准测试，必须综合考虑系统的存储开销。随着存储技术的发展，硬件成本相应下降，因此，人们更关注削减计算时间并提高系统的准确度，而很少关注削减系统存储开销。

为了综合评估数据库系统的性能，人们也设立了许多不同的基准测试，其中大部分的基准都已经在商业中得到了应用。文献[Kohler，1993]里面包括了 TPC 系列基准的案例。20 世纪 90 年代初，人们才开始重视研发信息检索方法或者引擎的统一评测基准。这些在信息检索系统评测中使用的数据集一般规模都很小，通常也就是兆级别的数据，研究的各类查询往往都受限于专业领域、数据量和复杂度。

在 1985 年，Blair 和 Maron[Blair 和 Maron，1985]发表了一篇开创性的文章，该文章质疑以 291
前基于小规模数据集的性能测试结果在大规模文档集上并不一定有效。20 世纪 90 年代初，美国国家标准技术研究院（NIST）利用美国国防高级研究计划署创建的文档集，发起了一个国际会议，推动文本检索领域内学术界、产业界与政府直接的合作与技术转移。这个国际会议就是今天的 TREC 会议，其主旨就是：在各类数据集的基础上，建立相对较大规模的测试集来提高信息检索领域的评测方法，最终促进信息检索领域的科学研究。

TREC 每年 11 月份在 NIST 举办。多年以来，参赛队伍的数目在稳定增长，而比赛任务的类型也发生了很大的变化。在最近的 2003 年的第十二届会议中，TREC 总共包含了 6 个比赛任务，即 Genomics（基因搜索）、HARD（高精度文档检索任务）、Novelty（新颖性监测）、QA（问答）、Robust（健壮性检索）和 Web 检索。各个任务之间没有任何关联。任务每年都会持续修订，因此各个任务的具体规格要求没有关联。我们有足够的理由来说比赛所采用的数据类型、查询、评测标准以及交互模式（在整个流程中用户是否参与）每年都会有很大的变化。所有任务共同的主题都是来构建搜索系统评测的测试语料库。

参与 TREC 比赛的流程如下：首先，主办方发布参赛报名通知，任务设立者最终会定义好每个任务的规格要求，每个参赛队会得到比赛的文档集与主题集（即查询），在此数据基础上，开展一系列实验，随后提交实验的结果作为评判依据。主办方可以拿到相似性评测程序，并对提交的结果进行评测。在年度会议上，人们会评估、总结并向与会者展示比赛过程中的科学发现。会后，所有的参赛者都会提交总结论文，并出版 TREC 会议论文集。

早期的 TREC 论坛采用的数据规模也就是几个吉字节，表 9-1 给出了文档集样本规模的统计数据。图 9-1 给出了该文档集中的文档样例，图 9-2 给出了查询主题样本。

表9-1　TREC数据的规模

硬盘编号	数据集	数据大小（MB）	文档数	每篇文档的词数的中间值	每篇文档的平均词数
1	华尔街时报（Wall Street Journal），1987~1989	267	98 732	245	434.0
1	美联社（Associated Press），1989	254	84 678	446	473.9
1	计算机精选（Computer Select），Ziff-Davis出版	242	75 180	200	473.0
1	联邦纪事（Federal Register），1989	260	25 960	391	1 315.9
1	美国能源部文摘	184	226 087	111	120.4
2	华尔街时报（Wall Street Journal），1990~1992	242	74 520	301	508.4
2	美联社（Associated Press），1988	237	79 919	438	468.7
2	计算机精选（Computer Select），Ziff-Davis出版	175	56 920	182	451.9
2	联邦纪事（Federal Register），1988	209	19 860	396	1 378.1
3	圣约瑟信报(San Jose Mercury News)，1990~1992	287	90 257	379	453.0
3	美联社（Associated Press），1990	237	78 321	451	478.4
3	计算机精选（Computer Select），Ziff-Davis出版	345	161 021	122	295.4
3	美国专利，1993	243	6 711	4,425	5 391
4	财经时报，1991~1994	564	210 158	316	412.7
4	联邦纪事（Federal Register），1994	395	55 630	588	644.7
4	国会纪要，1993	235	27 922	288	1 373

<DOC>
<DOCNO>WSJ880406-0090 </DOCNO>
<HL> AT&T Unveils Services to Upgrade Phone Networks Under Global Plan </HL>
<AUTHOR> Janet Guyon (WSJ Staff) </AUTHOR>
<DATELINE> NEW YORK </DATELINE>
<TEXT>
American Telephone & Telegraph Co. introduced the first of a new generation of phone services with broad implications for computer and communications equipment markets. AT&T said it is the first national long-distance carrier to announce prices for specific services under world-wide standardization plan to upgrade phone networks. By announcing commercial services under the plan, which the industry calls the Integrated Services Digital Network, AT&T will influence evolving communications standards to its advantage, consultants said, just as International Business Machines Corp. has created de facto computer standards favoring its products. . . .
</TEXT>
</DOC>

图 9-1　TREC 文档样例

<top>
<num> Number: 168
<title> Topic: Financing AMTRAK
<desc> Description:
A document will address the role of the Federal Government in financing the operation of the National Railroad Transportation Corporation (AMTRAK)
<narr> Narrative:
A relevant document must provide information on the government's responsibility to make AMTRAK an economically viable entity. It could also discuss the privatization of AMTRAK as an alternative to continuing government subsidies given to air and bus transportation with those provided to AMTRAK would also be relevant.
</top>

图 9-2　TREC 查询样本

今天，随着参赛特定任务着重点的变化，数据的类型也大相径庭，数据规模也有所不同。在本书写作之际，2005 年度的 TREC 的部分任务已经准备了 1TB 的数据，而在 2004 年，其初始的文档集规模仅仅是 400GB 多一点。因此，在大约十年间，数据规模已经从几个 GB 增为 1TB，增加了 3 个数量级。数据的增长或许也促使人们必须研制评测的新标准与新方法。

从 TREC 诞生以来，人们对 TREC 各项活动的兴趣逐步递增。随着各类以 WWW 搜索引擎为代表的分布式信息检索引擎的不断认识与普及，参与 TREC 比赛的学术团体与商业机构逐年 292
增多。随着参与者的增加，人们研发并评测了越来越多的技术方法。从 TREC 比赛参与者中获取的各种思路以及原型实验，不断地应用于商业产品与系统中，这一点也证明了 TREC 比赛的成功。

多年来，在不同的 TREC 会议论文集中，我们可以发现原始的平均准确率起初提升了，而后又下降了。表面上看起来，参与评测的系统在过去的这些年里准确度似乎在下降。实际上，查询与比赛任务这些年来的难度在逐渐增加。当经过修正的新系统采用往年的数据与查询来测试的话，往往都可以取得比以前更高的准确度。性能表面的下滑往往都是因为查询与任务本身难度相对增加造成的。

我们并不去具体评价这些年来某个引擎的性能优劣，毕竟本书的核心在于算法，而且单一的算法模型和实用策略的效果如何，也不一定都有文献报道出来。每届 TREC 会议的详细情况 293
可以参见出版的论文集，也可以通过 trec.nist.gov 在线访问。

尽管非常成功，但 TREC 也有其不足之处。前面已经提过，信息检索系统的性能评测包括准确度与性能的其他辅助要素。但是，TREC 比赛仅仅评价准确度，而很少关注系统处理时间与存储消耗。从相关度（准确率）的角度来看，对 TREC 常见的批评主要集中在其仅仅判断文 294
档-查询相关度的平均值。

目前，NIST 会给出一部分人工判别文档结果，但数量有限。相关性文档的判定采用 Pooling 系统来实现。当前在 TREC 中使用的 Pooling 方法 [Harman，1995]的主要过程是：从多个引擎中获取并选择排名靠前的文档集合，对它们进行归并并排序，然后将文档 ID 保留并形成相关文档集合（具体来说，需要去除重复的文档 ID）。Pooling 尽管很高效，但是往往也会产生一些错误的所谓不相关文档。为了避免这类问题，最近有些研究者介绍了一些方法来避免 Pooling 的使用，如文献[Sanderson 和 Joho，2004]。

针对 Pooling 方法不完备的问题，我们也注意到一些研究者已经提出了更有效、更通用的新评测方法。Pooling 方法假定能找到最相关的文档，如果实际情况不是这样的话，平均准确率往往会很难保证稳定[Buckley 和 Voorhees，2004]。而且，平均准确率并没有考虑用户的偏好。没有找到的文档则视为非相关文档。基于用户偏好的评测方法重点关注在命中相关文档之前越过的非相关文档数目。在命中一篇相关文档之前，只计算非相关文档数往往不够，因为不同查询对应的相关文档实际上差别非常大。与只有一两篇相关文档的查询相比较，有 100 篇相关文档的查询命中一篇相关文档的几率要大得多。为了平衡相关文档数的差别，我们提出了下面的计算公式：

$$bpref = \frac{1}{r}\sum_{r}\left(1-\frac{|\text{排序序号高于}r\text{的结果数目}n|}{R}\right)$$

最近有另外一种评测方法采用词项相关度而不是文档相关度。这种算法的过程为：针对给定的查询，检索出一组相关词列表，针对该列表来评测系统检索相关词的能力。对于一个事先给定的查询来说，不需要采用测试文档集相关的详尽知识，就可以定义好一组相关词。因此，这种方法具有良好的可扩展性，可以扩展到非常大规模的文档集[Amitay 等人，2004]。

我们注意到 CLEF（Cross-Language Evaluation Forum，跨语言检索评测论坛）仍然采用的是 TREC 的基本风格，不过重点是跨语言的评测（参见 4.1 节）。CLEF 的网址为 clef.iei.pi.cnr.it:2002。值得一提的是，最近已经完成了一个人机对话的新测试集[Orad 等人，2004]。

尽管过去的一些研究工作已经取得了不小的成绩，但是，我们仍然要清晰地认识到信息检索仍处在初期。20 年前，信息检索相关的模型算法屈指可数，大部分的研究文献基本上都是围绕这么 4 个检索模型的：向量空间模型、概率模型、布尔模型与模糊集模型。我们写作本书第
295 一版之后，语言模型才开始应用于信息检索问题，最后才成为第 9 种检索方法。

直到最近，分布式信息检索也仅仅是理论原型阶段。随着因特网用户爆炸式的增长与 WWW 的到来，分布式信息检索尤其是 WWW 上的信息检索才开始成为日常的应用。

当我们将本书第 1 版交付出版社的时候，跨语言检索才刚刚起步。现在，这已经是一个相对成熟的领域，它让我们更进了一步，让用户可以去检索世界范围内的文档，并且采用单一语言就可以阅读结果。

我们也应该注意到当前的 HTML 网页搜索是目前唯一的受限搜索类型。绝大部分大公司在搜索方面都会遇到大量无超链接的文本形式（如 Word 文档、PowerPoint 演示文稿、PDF 文件）。对 Web 搜索很有效的技术手段对这些数据未必管用。

从信息检索学术界的角度来看，NIST TREC 活动的人数不断增加，人们对信息检索的兴趣日趋高涨。最初几年，TREC 比赛大多数任务的参与者不足 30 个。而在第六届 NIST TREC 会议上，参与者超过了 50 个。而在第十二届会议上，参与者超过了 100 个。TREC 会议起初的测试数据仅有 2GB 的文本（这个数据量当初对很多研究者已经是非常困难了），现如今，测试集已经增长到了 1TB 文本了。

随着人们对信息检索领域兴趣的日益增加，我们也可以很清楚地看到未来的走势。问题关键所在就是哪些领域仍然需要做进一步的深入研究。我们还是采用贯穿本书的同一个脉络来展望未来吧，顺序依次为：先探讨模型与算法，然后介绍我们所关注的实用策略与运行效率问题。并行处理与分布式处理相关的探讨可以作为我们对未来走向展望的总结。

我们仍然需要进一步研究信息检索的算法模型。在过去的 TREC 任务中，任何一个任务的平均准确率都极少达到 40%。测试指标的显著提升需要我们更新颖的洞察力以及更有潜力的新方法。在过去的几年中，尽管我们在检索精度方面取得了持续而又稳定的进步，但是当前的结果仍然是不可接受的。而且，这种长足持续的进步也不可能导致检索准确率的实质性提升，这一点在处理特别大的数据集时更加显著。我们完全有理由质疑：当前简单的模式匹配方法仍然将持续成为以后的主流方向，就像我们在过去两到三年在 TREC 中观察到的一样。要超越目前这种状况，我们可能要融入更多复杂的 NLP 技术。如今，最近的研究表明信息抽取与语言浅层分析在计算上是可行的。

人们还需要其他的算法来处理网络在线数据的多样性。本书通篇只是阐述了文本数据的处理。众所周知，一图胜千言，我们还必须找到从图像中抽取并整合其中隐含的千言万语的方法。 296
但是，当前的信息检索模型并不支持这种模式。目前，在图像整合方面已有一些研究工作，比如 IRIS（在 IBM 公司 ImageMiner 项目中使用）[Alshuth 等人，1998]。但是，他们仍然没有将结构化数据、文本与图像数据整合为一个紧密结合的环境。因此，一个很合理的期望就是，未来需要一个图像与文本融合的扩展语料库。这个语料库才可能真正评测出文本与图像检索新算法的进展。

信息检索处理的各类实用策略可以采用一个连续分布的过程来表示，其两个端点分别对应的是简单模式匹配与完全自然语言文本处理。如今，绝大多数信息检索策略都是更接近于该过程的起始点，即简单模式匹配。例如，基于片段与 n 元语法这两种技术很显然都是偏重于纯粹模式匹配分析的。语义网络以及语言解析技术则更接近于 NLP 过程，但是它肯定不能像 NLP 那样支持完整内容的分析。信息检索更多的是利用文本的表层表示信息，甚至更糟糕的是仅仅采用字符表示信息，相比较而言，我们必须抽取文本的语义信息，坚信这样会大大提高信息检索的准确率。当我们出版本书第一版的时候，文本信息抽取还是一个相对很新的领域。如今，人们已经研制了各类成熟的商用文本抽取器。但问题仍然是：如何利用这些工具来提高信息检索的性能？

现在并行处理架构已经随处可见，也已经成为日常的应用了。它们已经不再是研究意义上的实验原型，即使我们的个人电脑大部分都是采用并行处理引擎的。因此，信息检索应用研发的时候必须充分利用并行处理能力。在第 7 章中，我们阐述了一些正在研制过程中的并行处理研究工作。然而，目前尚没有哪一个研究可以表明其可以扩展布局到成千上万个节点上，也没有哪一个技术可以处理各种格式的数据文件，支持多语言检索，有效地支持所有讲到的这些检索算法与策略，支持在线恢复时的多用户并发访问，支持检索方法与策略“即插即用”组合。此外，因为并行构架中底层模型的多样性，即使有一些解决方案考虑到了上述问题，但是它们也很难在多种并行结构中无缝地移植。很显然，在信息检索并行化方向上，我们还需要进行更深入的研究探索。

随着无线技术持续发展，数据已经不仅仅存储在计算机主机上，而且还会分布在全球范围内的移动计算设备上。这种分布式特性给我们带来了几个新的问题，而信息检索领域以前并不太关注这些问题。比如说，因为这些存储设备可以移动，其中的大多数数据只能在不确定的时间间隔内来获取。此外，每个搜索站点只能访问到有限的信息，而且这些信息会很快发生变化。 297
因此，分布式信息检索算法必须考虑到这些限制条件。当前，分布式操作系统与分布式数据库领域内一些正在进行的工作往往会研究这些问题，我们对这些研究工作所得的结论进行相应的

调整会更合适。不过，到今天为止，在信息检索领域内，还没有开展这方面的研究工作。

在全书中，我们一再倡导为信息检索构建一个即插即用的系统架构。我们分别阐述了信息检索的模型与算法、实用策略、系统性能、异构数据的融合范式以及拓扑结构等方面的内容。未来信息检索领域需要进一步研究与探索的主要问题是：如何全面综合所有这些信息检索的因
298 素以取得真正有机协同的效果？

参考文献

[Abiteboul, 1997] Abiteboul, S. (1997). Querying semistructured data. In *Proceedings of the International Conference on Database Theory*.

[Adams, 1991] Adams, E. (1991). *A Study of Trigrams and Their Feasibility as Index Terms in a Full Text Information Retrieval System*. PhD thesis, George Washington University, Department of Computer Science.

[Adriani, 2000] Adriani, M. (2000). Dictionary-based CLIR for the CLEF multilingual track. In *Working Notes of the Workshop in Cross-Language Evaluation Forum (CLEF)*, Lisbon.

[Agun and Frieder, 2003] Agun, S. K. and Frieder, O. (2003). HAT: A hardware assisted topdoc inverted index component. In *ACM Twenty-Sixth Conference on Research and Development in Information Retrieval (SIGIR)*.

[Aho and Corasick, 1975] Aho, A. and Corasick, M. (1975). Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18:333-340.

[A1-Kharashi and Evens, 1994] Al-Kharashi, I. and Evens, M. (1994). Comparing words, stems, and roots as index terms in an arabic information retrieval system. *Journal of the American Society for Information Science*, 45(8):548-560.

[AI-Onaizan et al., 1999] AI-Onaizan, Y., Curin, J., Jahr, M., Knight, K., Lafferty, J., Melamed, I. D., Och, F. J., Purdy, D., Smith, N. A., and Yarowsky, D. (1999). Statistical machine translation, final report, JHU workshop, `http ://www. clsp.jhu.odu/ws99/projocgs/mt/f inal_report/mt-f inal-report. ps.`

[Alaoui et al., 1998] Alaoui, S., Goharian, N., Mahoney, M., Salem, A., and Frieder, O. (1998). Fusion of information retrieval engines (FIRE). *In Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA-98)*.

[Aljlayl and Frieder, 2002] Aljlayl, M. and Frieder, O. (2002). On arabic search: Improving retrieval effectiveness via a light stemming approach. In *Proceedings of the ACM Eleventh Conference on Information and Knowledge Management*, Washington, DC.

[Aljlayl et al., 2002] Aljlayl, M., Frieder, O., and Grossman, D. (2002). On bidirectional English-Arabic search. *Journal of the American Society of information Science and Technology (JASIST)*, 53(13).

[Allan et al., 1995] Allan, J., Ballesteros, L., Callan, J.P., Croft, W.B., and Lu, Z. (1995). Recent experiments with INQUERY. In *Proceedings of the Fourth Text REtrieval Conference (TREC-4)*, pages 49-63.

[Alliance, 2004] Alliance, Globus (2004). The globus alliance.

[Alshuth et al., 1998] Alshuth, P., Hermes, T., Herzog, O., and Voigt, L. (1998). On video retrieval: Content analysis by lmageminer. In *Proceedings of the IS&T/SPIE Symposium on Electronic Imaging 98, Science and Technology, Multimedia Processing and Applications, Storage and Retrieval for hnage and Video Databases*, volume 3312, pages 236-247.

[Amitay et al., 2004] Amitay, E., Carmel, D., Lempel, R., and Softer, A. (2004). Scaling IRsystem evaluation using term relevance sets. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 10-17.

[AnhandMoffat, 2004] Anh, V.N. and Moffat, A. (2004). Inverted index compression using word-aligned binary codes. *Information Retrieval*. Available at http://www.kluweronline.com, pre-publication date April 2, 2004.

[Asokan et al., 1990] Asokan, N., Ranka, S., and Frieder, O. (1990). A parallel free text search system with indexing. In *Proceedings of the International Conference on Databases, Parallel Architectures, and their Applications (PARBASE-90)*, pages 519-534.

[Bailey and Hawking, 1996] Bailey, P. and Hawking, D. (1996). A parallel architecture for query processing over a terabyte of text. Technical report.

[Ballerini et al., 1996] Ballerini, J.P., Buchei, M., Domenig, R., Knaus, D., Mateev, B., Mittendorf, E., Schauble, P., Sheridan, P., and Wechsler, M. (1996). SPIDER retrieval system at TREC-5. In *Proceedings of the Fifth Text REtrieval Conference (TREC-5)*, pages 217-228.

[Ballesteros, 2001] Ballesteros, L. (2001). *Resolving ambiguity, for cross-language information retrieval: A dictionary approach*. PhD thesis, University of Massachusetts Amherst.

[Ballesteros and Croft, 1997] Ballesteros, L. and Croft, B. W. (1997). Phrasal translation and query expansion techniques for cross-language information retrieval. In *Proceedings of the Twentieth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 84-91.

[Ballesteros and Croft, 1998] Ballesteros, L. and Croft, W. B. (1998). Resolving ambiguity for cross-language retrieval. In *Proceedings of the Twenty-First Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 64-71.

[Baru et al., 1998] Baru, C., Gupta, A., Ludascher, B., Marciano, R., Papakonstantinou, Y., Velikhov, P., and Chu, V. (1998). XML-based information mediation with MIX. In *Proceedings of the ACM Special Interest Group on Management of Data (SIGMOD '98)*, pages 597-599.

[Bataineh et al., 1991] Bataineh, A., Ozguner, F., and Sarwal, A. (1991). Parallel boolean operations for information retrieval. *Information Processing Letters*, 39:99-108.

[Bataineh et al., 1989] Bataineh, A., Sarwal, Alok, Ozguner, E, and Dick, R. (1989). Parallel boolean operations for information retrieval. In *Conference on Hypercubes, Concurrent Computers, and Applications*, pages 445-448.

[Beckwith and Miller, 1990] Beckwith, R. and Miller, G. (1990). Implementing a lexical network. *International Journal of Lexicography*, 3(4):302-312.

[Beitzel et al., 2003a] Beitzel, S., Jensen, E., Chowdhury, A., Frieder, O., Grossman, D., and Goharian, N. (2003a). Disproving the fusion hypothesis: An analysis of data fusion via effective information retrieval strategies. In *ACM Eighteenth Symposium on Applied Computing (SAC)*, Melbourne, Florida.

[Beitzel et al., 2003b] Beitzel, S., Jensen, E., Chowdhury, A., Grossman, D., and Frieder, O. (2003b). Using manually-built web directories for automatic evaluation of known-item retrieval. In *Proceedings of the Twenty-Sixth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, Toronto, Canada.

[Beitzel et al., 2004a] Beitzel, S., Jensen, E., Chowdhury, A., Grossman, D., and Frieder, O. (2004a). Evaluation of filtering current news search results. In *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 494-495, Sheffield, United Kingdom.

[Beitzel et al., 2004b] Beitzel, S., Jensen, E., Chowdhury, A., Grossman, D., and Frieder, O. (2004b). Hourly analysis of a very large topically categorized web query log. In *Proceedings, of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 321-328, Sheffield, United Kingdom.

[Belew, 1989] Belew, R. (1989). Adaptive information retrieval. In *Proceedings of the Twelfth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 11-20.

[Bell et al., 1990] Bell, T.C., Cleary, J.G., and Witten, I.H. (1990). *Text Compression*. Prentice Hall, Englewood Cliffs, NJ.

[Berger and Lafferty, 1999] Berger, A. and Lafferty, J. (1999). Information retrieval as statisticcal translation. In *Proceedings of the 22nd Annual International ACM SIGIR conference on Research and Development in Information Retrieval*, pages 222-229.

[Berry, 1992] Berry, M. W. (1992). Large-scale sparse singular computations. *The International Journal of Supercomputer Applications*, 6(1):13-49.

[Bertoldi and Federico, 2003] Bertoldi, N. and Federico, M. (2003). ITC-irst at CLEF 2003: Monolingual, bilingual, and multilingual information retrieval. In *Workshop on CrossLanguage Information Retrieval and Evaluation (CLEF)*, Darmstadt, Germany.

[Blair, 1974] Blair, D. (1974). SQUARE (Specifying QUeries as Relational Expressions) as a document retrieval language. Unpublished working paper, University of California, Berkeley.

[Blair, 1988] Blair, D. (1988). An extended relational document retrieval model. *Information Processing and Management*, 24(3):349-371.

[Blair and Maron, 1985] Blair, D.C. and Maron, M.E. (1985). An evaluation of retrieval effectiveness for a full-text document-retrieval system. *Communications of the ACM*, 28(3):289-299.

[Blandford and Blelloch, 2002] Blandford, D. and Blelloch, G. (2002). Index compression through document reordering. In *Proceedings of the 2002 IEEE Data Compression Conference (DCC'02)*.

[Bleir, 1967] Bleir, R.E (1967). Treating hierarchical data structures in the SDC time-shared data management system (TDMS). In *Proceedings, 22nd ACM National Conference*, pages41-49.

[Boag et al., 2003] Boag, S., Chamberlin, D., Fernandez, M., Florescu, D., Robie, J., Simeon, J., and Stefanescu, M. (2003). XQuery 1.0: A query language for XML. W3C working draft. Available from http://www.w3.org/TR/xquery.

[Bond and Reddaway, 1993] Bond, N. and Reddaway, S. (1993). A massively parallel indexing engine using DAP. *Cambridge Parallel Processing. Technical Report.*

[Bonifati and Ceri, 2000] Bonifati, A. and Ceil, S. (2000). Comparitive analysis of five XML query languages. *SIGMOD Record*, 29(1):68-79.

[Boughanem and Soule-Depuy, 1997] Boughanem, M. and Soule-Depuy, C. (1997). Mercure at TREC-6. In *Proceedings of the Sixth Text REtrieval Conference (TREC-6)*, pages 187-193.

[Boyer and Moore, 1977] Boyer, R.S. and Moore, J.S. (1977). A fast string searching algorithm. *Communications of the ACM*, 20(10):762-772.

[Breslauer and Galil, 1991] Breslauer, D. and Galil, Z. (1991). Parallel string matching algorithms. *Carnegie-Mellon University. Technical Report*: *CUCS-002-92.*

[Brill, 1992] Brill, Eric (1992). A simple rule-based part of speech tagger. *Proceedings of the Third Conference on Applied Computational Linguistics*, pages 152-155.

[Brin et al., 1995] Brin, S., Davis, J., and Garcia-Molina, H. (1995). Copy detection mechanisms for digital documents. In *Proceedings of the ACM Special Interest Group on Management of Data (SIGMOD).*

[Brin and Page, 2000] Brin, S. and Page, L. (2000). The anatomy of a large-scale hypertextual web search engine. Technical report, Stanford University.

[Broder, 1998] Broder, A. (1998). On the resemblance and containment of documents. In *In SEQS: Sequences 91.*

[Broder et al., 1997] Broder, A., Glassman, S., Manasse, M., and Zweig, Geoffrey (1997). Syntactic clustering of

the web. Technical Report 1997-015, Digital Equipment Corporation, Systems Research Center (SRC).

[Broglio et al., 1994] Broglio, J., Callan, J., Croft, W.B., and Nachbar, D. (1994). Document retrieval and routing using the INQUERY system. In *Proceedings of the Third Text Retrieval Conference (TREC-3)*, pages 29-38.

[Brown et al., 1990] Brown, P., Cocke, J., Pietra, S. Della, Pietra, V. Della, Jelinek, F., Lafferty, J., Mercer, R., and Roossin, P. (1990). A statistical approach to machine translation. *Computational Linguistics*, 16(2):79-85.

[Brown et al., 1991] Brown, P., Lai, J., and Mercer, R. (1991). Aligning sentences in parallel corpora. In *Proceedings of the 29th Annual Meeting of the ACL*, pages 169-176, Berkeley, CA.

[Brown et al., 1993] Brown, P., Pietra, S. Della, Pietra, V. Della, and Mercer, R. (1993). The mathematics of statistical machine translation: Parameter estimation. *Computational Linguistics*, 19(2):262-311.

[Buckley et al., 2000] Buckley, C., Cardie, C., Mardis, S., Mitra, M., Pierce, D., Wagstaff, K., and Walz, J. (2000). The smart/empire tipster ir system. In *TIPSTER Phase III Proceedings, Morgan Kaufmann.*

[Buckley et al., 1994] Buckley, C., Salton, G., Allan, J., and Singhal, A. (1994). Automatic query expansion using SMART: TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, pages 69-80.

[Buckley et al., 1995] Buckley, C., Singhal, A., Mitra, M., and (G. Salton) (1995). New retrieval approaches using SMART: TREC-4. In *Proceedings of the Fourth Text Retrieval Conference (TREC-4)*, pages 25-48.

[Buckley and Voorhees, 2004] Buckley, C. and Voorhees, E. (2004). Retrieval evaluation with incomplete information. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR).*

[Buneman et al., 1996] Buneman, P., Davidson, S., Hillebrand, G., and Suciu, D. (1996). A query language and optimization techniques for unstructured data. In *Proceedings of International Conference on Management of Data (ACM-SIGMOD)*, pages 505-516.

[Burgin, 1995] Burgin (1995). The retrieval effectiveness of five clustering algorithms as a function of indexing exhaustivity. *Journal of the American Society for Information Science*, 46(8):562-572.

[Callan, 1994] Callan, J.P. (1994). Passage-level evidence in document retrieval. In *Proceedings of the Seventeenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 302-310.

[Callison-Burch and Osborne, 2003] Callison-Burch, Chris and Osborne, Miles (2003). Building and using parallel texts: Data driven machine translation and beyond. In *Proceedings of the North American Association of Computation Linguistics (NAACL) : Workshop on Building and Using Parallel Texts: Data Driven Machine Translation and Beyond.*

[Cancedda et al., 2003] Cancedda, N., Dejean, H., Gaussier, E., Renders, J., and Vinokourov, A. (2003). Report on CLEF-2003 experiments: Two ways of extracting multilingual resources from corpora. In *Proceedings of. the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign.*

[Carbonell et al., 1997] Carbonell, J., Yang, Y., Frederking, R., Brown, R., Geng, Y., and Lee, D. (1997). Translingual information retrieval: A comparative evaluation. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence (IJCAI-97)*, pages 708-715.

[Carmel et al., 2001] Carmel, D., Cohen, D., Fagin, R., Farchi, E., Herscovici, M., Maarek, Y. S., and Soffer, Aya (2001). Static index pruning for information retrieval systems. In *Proceedings of the 24th annual international ACM SIGIR conference on Research and development in information retrieval*, pages 43-50.

[Carreras et al., 2002] Carreras, X., Marques, L., and Padro, L. (2002). Named entity extraction using AdaBoost. In *Proceedings of CoNLL-2002*, pages 167-170, Taipei, Taiwan.

[Cavnar, 1993] Cavnar, W. (1993). N-gram based text filtering for TREC-2. In *Proceedings of the Second Text REtrieval Conference (TREC-2)*, pages 171-179.

[Cavnar and Vayda, 1993] Cavnar, W. and Vayda, A. (1993). N-gram based matching for multifield database access in postal applications. In *Proceedings of the Second Annual Symposium on Document Analysis and Information Retrieval*, pages 287-297.

[Charniak, 1993] Charniak, E. (1993). *Statistical Language Learning*. The MIT Press, Cambridge, MA.

[Chen and Gey, 2002] Chen, A. and Gey, F. (2002). Building an arabic stemmer for information retrieval. In *Proceedings of the Eleventh Text REtrieval Conference (TREC-2002)*.

[Chen and Gey, 2003] Chen, A. and Gey, F. (2003). Experiments on cross-language and patent retrieval at NTCIR-3 workshop. In *Proceedings of the NTCIR-3 Workshop*.

[Chert, 1995] Chen, H. (1995). Machine learning for intormation retrieval: Neural networks, symbolic learning, and genetic algorithms. *Journal of the American Society for Information Science*, 46(3):194-216.

[Chen and Lynch, 1992] Chen, H. and Lynch, K. J. (1992). Automatic construction of networks of concepts characterizing document databases. *IEEE Transactions on Systems, Man, and Cybernetics*, 22(5):885-902.

[Chen et al., 1993] Chen, H., Lynch, K. J., Basu, K., and Ng, T. (1993). Generating, integrating, and activating thesauri for concept-based document retrieval. *IEEE Expert*, 8(2):25-34.

[Chert and Ng, 1995] Chen, H. and Ng, T. (1995). An algorithmic approach to concept exploration in a large knowledge network (automatic thesaurus consultation): Symbolic branch and bound search versus connectionist hopfield net activation. *Journal of the American Society for Information Science*, 46(5):348-369.

[Chen et al., 1995] Chen, H., Yim, T., and Frye, D. (1995). Automatic thesaurus generation for an electronic community system. *Journal of the American Society. for Information Science*, 46(3)： 175-193.

[Chen, 1976] Chen, P. (1976). The entity relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9-36.

[Chen and Wang, 1995] Chen, S. and Wang, Jeng-Yih (1995). Document retrieval using knowledge-based fuzzy information retrieval techniques. *IEEE Transactions on Systems, Man, and Cybernetics*, 25(5):793-803.

[Chen and Goodman, 1998] Chen, S. E and Goodman, J. (1998). An empirical study of smoothing techniques for language modeling. Technical Report TR-10-98, Harvard University.

[Cho et al., 1999] Cho, J., Shivakumar, N., and Garcia-Molina, H. (1999). Finding replicated web collections. In *Proceedings of the ACM SIGMOD Conference on Management of Data (SIGMOD'99)*, pages 355-366.

[Chowdhury et al., 2001] Chowdhury, A., Frieder, O., Grossman, D., and McCabe, M.C. (2001). Analyses of multiple-evidence combinations for retrieval strategies. In *Proceedings of the Twenty-Fourth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, New Orleans, Louisiana.

[Chowdhury et al., 2002] Chowdhury, A., Frieder, O., Grossman, D., and McCabe, M.C. (2002). Collection statistics for fast duplicate document detection. *ACM Transaction on Information Systems*, 20(2):171-191.

[Chowdhury and Pass, 2003] Chowdhury, Abdur and Pass, Greg (2003). Operational requirements for scalable search systems. In *Proceedings of the 2003 International ACM CIKM Conference on Information and Knowledge Management*, pages 435-442.

[Church, 1988] Church, K. (1988). A stochastic parts program and noun phrase parser for unrestricted text. In *Proceedings of the Second Conference on Applied Natural Language Processing*, pages 136-143.

[Cockshott, 1989] Cockshott, P. (1989). Disadvantages of parallelism in text retrieval. In *Proceedings of the IEE Colloquium on Parallel Techniques for Information Retrieval Digest*, pages 377-387.

[Codd, 1970] Codd, E. (1970). A relational model for large shared data banks. *Communications of the ACM*,

13(6):377-387.

[Cohen, 1995] Cohen, J. (1995). Highlights: Language and domain-independent automatic indexing terms for abstracting. *Journal of American Society for Information Science*, 46(3):162-174.

[Cohen and Kjeldsen, 1987] Cohen, P.R. and Kjeldsen, R. (1987). Information retrieval by constrained spreading activation in semantic networks. *Information Processing and Management*, 23(4):255-268.

[Connell et al., 1996] Connell, W. O., leong, I.T, Schrader, D., Watson, C., Biliris, A., Choo, S., Colin, P., Linderman, G., Panagos, E., Wang, J., and Walter, T. (1996). A teradata contentbased multimedia object manager for massively parallel architectures. In *Proceedings of the 1996 ACM Special Interest Group on the Management of Data (SIGMOD)*, pages 68-78.

[Conrad et al., 2003] Conrad, J., Guo, X., and Schriber, C. (2003). Online duplicate document detection: signature reliability in a dynamic retrieval environment. In *Twelfth International Conference on Information and Knowledge Management (CIKM)*, pages 443-452.

[Conrad and Schriber, 2004] Conrad, J. and Schriber, C. (2004). Constructing a text corpus for inexact duplicate detection. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 582-583.

[Cooper et al., 2002] Cooper, J., Coden, A., and Brown, E. (2002). A novel method for detecting similar documents. In *Proceedings of the 35th Hawaii International Conference on System Sciences (HICSS)*.

[Cooper, 1991] Cooper, W.S. (1991). Some inconsistencies and misnomers in probabilistic information retrieval. In *Proceedings of the Fourteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 57-62.

[Cooper et al., 1992] Cooper, W.S., Gey, F., and Dabney, D. (1992). Probabilistic retrieval based on staged logistic regression. In *Proceedings of the Fifteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 198-210.

[Craswell et al., 2002] Craswell, N., Hawking, D., Krumpholz, A., Mathieson, I., Thom, J.A., Vercoustre, A.-M., Wilkins, P., and Wu, M. (2002). Xml document retrieval with padre. In *Proceedings of the Seventh Australian Document Computing Symposium*.

[Crawford, 1981] Crawford, R. (1981). The relational model in information retrieval. *Journal of the American Society for Information Science*, pages 51-64.

[Crestani, 1994] Crestani, F. (1994). Comparing neural and probabilistic relevance feedback in an interactive information retrieval system. In *Proceedings of the IEEE International Conference on Neural Networks*, pages 3226-3230.

[Cringean et al., 1990] Cringean, J., England, R., Manson, G., and Willett, P. (1990). Parallel text searching in serial files using a processor farm. In *Proceedings of the Thirteenth Annual ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 429-445.

[Cringean et al., 1991] Cringean, J., England, R., Manson, G., and Willett, P. (1991). Nearestneighbour searching in files of text signatures using transputer networks. *Electronic Publishing-Origination, Dissemination, and Design*, 4(1):185-202.

[Croft and Harper, 1979] Croft, W.B. and Harper, D.J. (1979). Using probabilistic models of document retrieval without relevance information. *Journal of Documentation*, 35(4):282-295.

[Croft and Xu, 1994] Croft, W.B. and Xu, Jinxi (1994). Corpus-specific stemming using word form co-occurence. In *Proceedings for the Fourth Annual Symposium on Document Analysis Information Retrieval*, pages 147-159.

[Crouch et al., 1994] Crouch, C., Crouch, D., and Nareddy, K. (1994). Associative and adaptive retrieval in a

connectionist system. *International Journal of Expert Systems*, 7(2):193-202.

[Crouch, 1989] Crouch, C.J. (1989). A cluster-based approach to thesaurus construction. In *Eleventh International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 309-320.

[Crouch, 1990] Crouch, C.J. (1990). An approach to the automatic construction of global thesauri. *Information Processing and Management*, 26(5):629-640.

[Cucerzan and Yarowsky, 1999] Cucerzan, S. and Yarowsky, D. (1999). Language independent named entity recognition: Combining morphological and contextual evidence. In *Joint SIGDAT Confernce on EMNLP and VLC*.

[Cutting et al., 1992] Cutting, D.R., Karger, D.R., Pedersen, J.O., and Tukey, J.W. (1992). Scatter/gather: A cluster-based approach to browsing large document collections. In *Proceedings of the Fifteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 318-329.

[Dai et al., 1999] Dai, Y., Khoo, C., and Loh, T. (1999). A new statistical formula for chinese text segmentation incorporating contextual information. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 82-89.

[Damashek, 1995] Damashek, M. (1995). Gauging similarity via n-grams: Language independent categorization of text. Science, 267(5199):843-848.

[D'Amore and Mah, 1985] D'Amore, R. and Mah, C. (1985). One-time complete indexing of text: Theory and practice. *Eighth Annual ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 155-164.

[Darwish and Oard, 2003] Darwish, K. and Oard, D. (2003). CLIR experiments at Maryland for TREC-2002: Evidence combination for arabic-english retrieval. In *Proceedings of the Text Retrieval and Evaluation Conference (TREC-2003)*.

[Date, 1994] Date, C. (1994). *An Introduction to Database Systems*. Addison-Wesley.

[Davis and Ogden, 1997] Davis, M. W. and Ogden, W.C. (1997). Free reosurces and advanced alignment for cross-language text retrieval. In *Proceedings of the Sixth Text REtrieval Conference (TREC-6)*.

[Deerwester et al., 1990] Deerwester, S., Dumais, S.T., Furnas, G.W., Landauer, T.K., and Harshman, R. (1990). Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, 41 (6):391-407.

[DeHaan et al., 2003] DeHaan, D., Toman, D., Consens, M., and Ozsu, M. (2003). A comprehensive XQuery to SQL translation using dynamic interval encoding. In *Proceedings of the ACM Special Interest Group on Management of Data (SIGMOD)*, San Diego, California.

[Demner-Fushman and Oard, 2003] Demner-Fushman, D. and Oard, D. W. (2003). The effect of bilingual term list size on dictionary-based cross-language information retrieval. In *Proceedings of the 36th Hawaii International Conference on System Sciences*.

[Dempster et al., 1977] Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximumlikelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society*, 39:1-38.

[DeRose, 1988] DeRose, S. (1988). Grammatical category disambiguation by statistical optimization. *Computational Linguistics*, 14(1):31-39.

[Desai et al., 1987] Desai, B.C., Goyal, P., and Sadri, F. (1987). Non-first normal form universal relations: An application to information retrieval systems. *Information Systems*, 12(1):49-55.

[Desantis et al., 1996] Desantis, R., Frieder, O., and Moini, A. (1996). The CYBERosetta architecture. *Software*

Productivity Consortium, Internal Report.

[Deutsch et al., 1999] Deutsch, A., Fernandez, M., Florescu, D., Levy, A., and Suciu, D. (1999). A query language for XML. In *Proceedings of the International Worm Wide Web Conference.*

[Dubes and Jain, 1988] Dubes, R.C. and Jain, A.K. (1988). *Algorithms for Clustering Data*. Prentice Hall.

[Duda and Sheldon, 1994] Duda, A. and Sheldon, M. (1994). Content routing in a network of WAIS servers. In *Proceedings of the IEEE Fourteenth International Conference on Distributed Computing Systems*, pages 124-132.

[Dumais et al., 1997] Dumais, S., Letsche, T., Littman, M., and Landauer, T. (1997). Automatic cross-language retrieval using latent semantic indexing. In *In AAAI Symposium on CrossLanguage Text and Speech Retrieval. American Association for Artificial Intelligence.*

[Dumais, 1994] Dumais, S. T. (1994). Latent semantic indexing (LSI): TREC-3 report. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, pages 219-230.

[EI-Hamdouchi and Willett, 1986] EI-Hamdouchi, A. and Willett, P. (1986). Hierarchic document clustering using ward's method. In *Proceedings of the Ninth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 149-156.

[EI-Hamdouchi and Willett, 1989] EI-Hamdouchi, A. and Willett, P. (1989). Comparison of hierarchic agglomerative clustering methods for document retrieval. *The Computer Journal*, 32(3):220-226.

[Elias, 1975] Elias, P. (1975). Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*., IT-21 (2):194-203.

[Elmasri and Navathe, 1994] Elmasri, R. and Navathe, S. (1994). *Fundamentals of Database Systems*. Addison-Wesley.

[Evans and Ghanemi, 1988] Evans, D. and Ghanemi, S. (1988). Parallel string matching algorithms. *Kybernetes*, 17(3):32-34.

[Federico and Bertoldi, 2002] Federico, M. and Bertoldi, N. (2002). Statistical cross-language information retrieval using n-best query translations. In *Proceedings of the 25th A CM SIGIR Conference on Research and Development in Information Retrieval*, pages 167-174.

[Fetterly et al., 2003] Fetterly, D., Manasse, M., and Najork, M. (2003). On the evolution of clusters of near-duplicate web pages. In *Proceedings of the First Latin American Web Congress*, page 37Ů45.

[Florescu and Kossman, 1999] Florescu, D. and Kossman, D. (1999). Storing and querying XML data using an RDBMS. *IEEE Datd Engineering Bulletin*, 22(3):27-34.

[Flynn, 1972] Flynn, M.J. (1972). Some computer organizations and their effectiveness. *IEEE Transactions on Computers*, 21 (9):948-960.

[Fontaine, 1995] Fontaine, Anne (1995). Sub-element indexing and probabilistic retrieval in the POSTGRES database system. Master's thesis, University of California, Berkeley.

[Foster, 2002] Foster, I. (2002). The grid: A new infrastructure for 21st century science. *Physics Today*, 55(2):42-47.

[Foster et al., 2001] Foster, I., Kesselman, C., and Tuecke, S. (2001). The anatomy of the grid: Enabling scalable virtual organizations. *International Journal of Supercomputing*, 15(1).

[Fox, 1990] Fox, C. (1990). A stop list for general text. *SIGIR Forum*, 24(1):19-35.

[Fox, 1983a] Fox, E.A. (1983a). *Extending the Boolean and Vector Space Models of Information Retrieval with P-Norm Queries and Multiple Concept Types*. PhD thesis, Comell University.

[Fox, 1983b] Fox, E.A. (1983b). Some considerations for implementing the SMART information retrieval system under UNIX. Technical Report TR83-560, Cornell University.

[Frakes and Baeza-Yates, 1993] Frakes, W. and Baeza-Yates, R. (1993). *Information Retrieval: Data Structures and Algorithms*. Prentice-Hall, Inc.

[Francis and Kucera, 1982] Francis, W. and Kucera, H. (1982). *Frequency Analysis of English Usage. Lexicon and Grammer*. Houghton Mifflin.

[Frieder et al., 2000a] Frieder, O., Chowdhury, A., Grossman, D., and Frieder, G. (2000a). Efficiency considerations for scalable information retrieval servers. *Journal of Digital Information.*

[Frieder et al., 2000b] Frieder, O., Chowdhury, A., Grossman, D., and McCabe, M. C. (2000b). On the integration of structured data and text: A review of the sire architecture. In *Proceedings of the DELOS Workshop on Information Seeking, Searching, and Querying in Digital Libraries*.

[Frieder et al., 2003] Frieder, O., Grossman, D., and Chowdhury, A. (2003). On scalable information retrieval systems (invited keynote). In *IEEE Second International Symposium on Network Computing and Applications (NCA)*.

[Fuhr, 1992] Fuhr, Norbert (1992). Probabilistic models in information retrieval. *The Computer Journal*, 35(3):243-255.

[Fung, 1998] Fung, P. (1998). A statistical view on bilingual lexicon extraction: From parallel corpora to non-parallel corpora. In *Machine Translation and the Information Soup, Third Conference of the Association for Machine Translation in the Americas (AMTA)*, Lecture Notes in Computer Science, pages 1-17.

[Gale and Church, 1991] Gale, W. and Church, K. (1991). A program for aligning sentences in bilingual corpora. In *Proceedings of the 29th Annual Meeting of the ACL*, Berkeley, CA.

[Galil, 1979] Galil, Z. (1979). On improving the worst-case running time of the Boyer-Moore string matching algorithm. *Communications of the ACM*, 22(9):505-508.

[Garcia-Molina et al., 1996] Garcia-Molina, H., Gravano, L., and Shivakumar, N. (1996). dscam: Finding document copies across multiple databases. In *Proceedings of the 4th International Conference on Parallel and Distributed Systems (PDIS 96)*.

[Garcia-Molina et al., 1997] Garcia-Molina, H., Papakonstantinou, Y., Quass, D., Rajamaran, A., Sagiv, Y., Ullman, J., Vassalos, V., and Widom, J. (1997). The TSIMMIS approach to mediation: Data models and languages. *Journal of Intelligent Information Systems,* 8.

[Gauch and Wang, 1996] Gauch, S. and Wang, J. (1996). Corpus analysis for TREC-5 query expansion. In *Proceedings of the Fifth Text REtrieval Conference (TREC-5)*, pages 537-546.

[Ghose and Dhawle, 1977] Ghose, A. and Dhawle, A. (1977). Problems of thesaurus construction. *Journal of the American Society for Information Science*, 28(4):211-217.

[Giger, 1988] Giger, H.P. (1988). Concept based retrieval in classical IR systems. In *Eleventh International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 275-289.

[Goda et al., 2001] Goda, K., Tamura, T., Kitsuregawa, M., Chowdhury, A., and Frieder, O. (2001). Query optimization for vector space problems. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 416-417.

[Goharian et al., 2004] Goharian, N., Grossman, D., Raju, N., and Frieder, O. (2004). Migrating information retrieval from the graduate to the undergraduate curriculum. *Journal of Information Systems Education*, 15(1).

[Goldfarb, 1990] Goldfarb, C. (1990). *The SGML Handbook*. Oxford University Press.

[Goldman et al., 1999] Goldman, R., McHugh, J., and Widom., J. (1999). From semistructured data to XML: Migrating the lore data model and query language. In *Proceedings of the Second International Workshop on the Web and Databases (WebDB '99)*, Philadelphia, Pennsylvania.

[Goldsmith, 2001] Goldsmith, J. (2001). Unsupervised learning of the morphology of a natural language. *Computational Linguistics*, 27(2):153-198.

[Gollins and Sanderson, 2004] Gollins, T. and Sanderson, M. (2004). Improving cross language information retrieval with triangulated translation. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval.*

[Gomez, 1988] Gomez, F. (1988). WUP: a parser based on word usage. In *Seventh Annual International Phoenix Conference on Computers and Communications*, pages 445-449.

[Gomez and Segami, 1989] Gomez, E and Segami, C. (1989). The recognition and classification of concepts in understanding scientific texts. *Journal of Experimental Theoretical Artificial Intelligence*, pages 51-77.

[Gomez and Segami, 1991] Gomez, F. and Segami, C. (1991). The recognition and classification of concepts in understanding scientific texts. *IEEE Transactions on Systems, Man and Cybernetics*, 21(3):644-659.

[Gordon, 1988] Gordon, M. (1988). Probabilistic and genetic algorithms for document retrieval. *Communications of the ACM*, 31(10): 1208-1218.

[Gordon, 1997] Gordon, M. (1997). It's 10 a.m. do you know where your documents are? The nature and scope of information retrieval problems in business. *Information Processing and Management*, 33(1): 107-121.

[Grabs et al., 2001] Grabs, T., Böhm, K., and Schek, H.J. (2001). High-level parallelisation in a database cluster: a feasibility study using document services. In *Proceedings of the IEEE Seventeenth International Conference on Data Engineering (ICDE2001).*

[Grauer and Messier, 1971] Grauer, R.T. and Messier, M. (1971). *The SMART Retrieval System—Experiments in Automatic Document Processing*, chapter An Evaluation of Rocchio's Clustering Algorithm, pages 243-264. Prentice Hall.

[Gravano and Garcia-Molina, 1995] Gravano, L. and Garcia-Molina, H. (1995). Generalizing GIOSS to vector-space databases and broker hierarchies. In *Proceedings of the 21st International Conference on Very Large Database Conference*, pages 78-89.

[Greiff, 1996] Greiff, W. (1996). Computationally tractable, conceptually plausible classes of link matrices for the Inquery inference network. Technical Report TR96-66, University of Massachusetts, Amherst.

[Greiff et al., 1997] Greiff, W.R., Croft, W.B., and Turtle, H. (1997). Computationaily tractable probabilistic modelling of boolean operators. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 119-128.

[Gropp and Lusk, 1998] Gropp, W. and Lusk, E (1998). User's guide for MPICH, a portable implementation of MPI. Technical report, Mathematics and Computer Science Division, Argonne National Laboratory, University of Chicago.

[Grossman, 1995] Grossman, D. (1995). *Integrating Structured Data and Text: A Relational Approach.* PhD thesis, George Mason Univeristy.

[Grossman et al., 1997] Grossman, D., Frieder, O., Holmes, D., and Roberts, D. (1997). Integrating structured data and text: A relational approach. *Journal of the American Society for Information Science*, 48(2): 122-132.

[Grossman et al., 1994] Grossman, D., Holmes, D., and Frieder, O. (1994). A parallel DBMS approach to IR. *The Third Text REtrieval Conference*, pages 279-288.

[Grossman et al., 1995] Grossman, D., Holmes, D., and Frieder, O. (1995). A dbms approach to ir in trec-4. In

Proceedings of the Fourth Text REtrieval Conference (TREC-4).

[Guha et al., 1988] Guha, S., Rastogi, R., and Shim, K. (1988). Cure: An efficient clustering algorithm for large databases. In *Proceedings of the 1998 ACM-SIGMOD Special Interest Group on Management of Data*, pages 73-84.

[Guiasu and Shenitzer, 1985] Guiasu, S. and Shenitzer, A. (1985). The principle of maximum entropy. *Mathmatical Intelligencer*, 7(1).

[Gupte and Frieder, 1995] Gupte, A.V. and Frieder, O. (1995). Compression within a contextsensitive commercial random access domain: An industrial case study. *Information Processing and Management*, 31 (4):573-591.

[Gutmann and Bell, 1994] Gutmann, P.C. and Bell, T.C. (1994). A hybrid approach to text compression. In *Proceedings of the Data Compression Conference DCC '94*, pages 225-233.

[Haddouti, 1999] Haddouti, H. (1999). Survey: Multilingual text retrieval and access. *UNESCO Press*.

[Harman, 1988] Harman, D. (1988). Towards interactive query expansion. In *Eleventh International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 321-331.

[Harman, 1992] Harman, D. (1992). Relevance feedback revisited. In *Proceedings of the Fifteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1-10.

[Harman, 1995] Harman, D. (1995). The TREC conferences. In *Proceedings of the Hypertext-Information Retrieval-Multimedia: Synergieeffekte Elektronischer informationssysteme (HIM'95)*, pages 9-28.

[Harman and Buckley, 2004] Harman, D. and Buckley, C. (2004). The NRRC reliable information access (RIA) workshop. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 528-529.

[Hawking et al., 2000] Hawking, D., Bailey, P., and Craswell, N. (2000). Efficient and flexible search using text and metadata. Technical Report TR2000-83, Department of Computer Science CSIRO Matematical and Information Sciences.

[Hawking, 1991] Hawking, D.A. (1991). High speed search of large text bases on the fujitsu cellular array processor. In *Proceedings of the Fourth Australian Supercomputing Conference*, pages 83-90.

[Hawking, 1994a] Hawking, D.A. (1994a). The design and implementation of a parallel document retrieval engine. Technical report.

[Hawking, 1994b] Hawking, D.A. (1994b). Padre-a parallel document retrieval engine. In *Proceedings of the Third Fujitsu Parallel Computing Workshop*.

[Heintze, 1996] Heintze, N. (1996). Scalable document fingerprinting. In *In Proceedings of the USENIX Work-shop on Electronic Commerce*.

[Hiemstra and Kraaij, 1998] Hiemstra, D. and Kraaij, W. (1998). Twenty-one at TREC-7: Adhoc and cross-language track. In *Proceedings of Seventh Text REtrieval Conference (TREC-7)*, pages 227-238.

[Hiemstra et al., 2004] Hiemstra, D., Robertson, S., and Zaragoza, H. (2004). Parsimonious language models for information retrieval. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 178-185.

[Hines and Harris, 1971] Hines, T.C. and Harris, J. L. (1971). Columbia University School of Library Service System for Thesaurus Development and Maintenance. *Information Storage and Retrieval*, 7(1):39-50.

[Hoad and Zobel, 2002] Hoad, T. and Zobel, J. (2002). Methods for identifying versioned and plagiarism documents. *Journal of the American Society for Information Science and Technology (JASIST)*.

[Hodjat et al., 2003] Hodjat, B., Franco, H., Bratt, H., and Precoda, K. (2003). Iterative statistical language model

generation for use with an agent-oriented natural language interlace. In *10th International Conference on Human-Computer Interaction.*

[Hofland, 1996] Hofland, K. (1996). A program for aligning English and Norwegian sentences. *Research in Humanities Computing*, pages 165-178.

[Hollaar and Haskins, 1984] Hollaar, L. and Haskins, R. (1984). Method and system for matching encoded characters. *U.S. Patent Number 4,450,520.*

[Hollaar and Haskins, 1991] Hollaar, L. and Haskins, R. (1991). Implementation and evaluation of a parallel text searcher for very large text databases. In *Proceedings of the 25th International Conference on System Sciences*, volume 1, pages 300-307.

[Holmes, 2004] Holmes, David (2004). Personal communication with David Holmes.

[Horspool, 1983] Horspool, R.N. (1983). Practical fast searching in strings. *Software Practice and Experience*, 10(6):501-510.

[Hull and Grefenstette, 1996] Hull, D. and Grefenstette, G. (1996). Querying across langages: a dictionary-based approach to multilingual information retrieval. In *Proceedings of the Nineteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 46-57.

[Hurson et al., 1990] Hurson, A.R., Miller, L.L., Pakzad, S.H., and Cheng, Jia-bing (1990). Specialized parallel architectures for textual databases. *Advances in Computers*, 30:1-37.

[Ives et al., 1999] Ives, Z. G., Florescu, D., Friedman, M., Levy, A., and Weld, D. S. (1999). An adaptive query execution system for data integration. In *Proceedings of the ACM SIGMOD Conference on Management of Data (SIGMOD '99)*, Philadelphia, Pennsylvania.

[Jelinek and Mercer, 1980] Jelinek, F. and Mercer, R. (1980). *Interpolated Estimation of Markov Source Parameters from Sparse Data*, pages 381-402. Noth-Holland Publishing Company, North Holland, Amsterdam.

[Jensen et al., 2002] Jensen, Eric C., Beitzel, Steven M., Pilotto, Angelo J., Goharian, Nazli, and Frieder, Ophir (2002). Parallelizing the buckshot algorithm for efficient document clustering. In *Proceedings of the Eleventh International Conference on Information and Knowledge Management (CIKM)*, pages 684-686. ACM Press.

[Kamps et al., 2004] Kamps, J., deRijke, M., and Sigurbjoernsson, B. (2004). Length normalization in XML retrieval. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 80-87.

[Kamps et al., 2003] Kamps, J., Monz, C., de Rijke, M., and Sigurbjornsson, B. (2003). The university of amsterdam and CLEF 2003. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign.*

[Kanne and Moerkotte, 2000] Kanne, C. and Moerkotte, G. (2000). Efficient storage of XML data. In *Proceedings Of the 16th International Conference On Data and Knowledge Engineering (ICDE).*

[Kantor, 1994] Kantor, P. (1994). *Information Retrieval Techniques*, volume 29, chapter 2, pages 53-90. Learned Information, Inc.

[Karp and Rabin, 1987] Karp, R. and Rabin, M. (1987). Efficient randomized patternmatching algorithms. *IBM Journal of Research and Development*, 31(2):249-260.

[Katz, 1987] Katz, S.M. (1987). Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech and Signal Processing*, ASSP-35:400-401.

[Khoja and Garside, 1999] Khoja, S. and Garside, R. (1999). Stemming arabic text. Technical report, Computing Department, Lancaster University.

[Kim and Kim, 1990] Kim, Y.W. and Kim, J.H. (1990). A model of knowledge based information retrieval with hierarchical concept graph. *Journal of Documentation*, 46(2):113-136.

[Kishida and Kando, 2003] Kishida, K. and Kando, N. (2003). Two stage refinement of query translation for pivot language approach to cross-lingual information retrieval: A trial at CLEF 2003. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign.*

[Kjeldsen and Cohen, 1987] Kjeldsen, R. and Cohen, P.R. (1987). The evolution and performance of the GRANT system. *IEEE Expert*, 2(2):73-79.

[Kjell et al., 1994] Kjell, B., Woods, W., and Frieder, O. (1994). Discrimination of authorship using visualization. *Information Processing and Management*, 30(1): 141-150.

[Kleinberg, 1999] Kleinberg, J. (1999). Authoritative sources in a hyperlinked environment. *Journal of the ACM*, 46.

[Knuth et al., 1977] Knuth, D., Morris, J., and Pratt, V. (1977). Fast pattern matching in strings. *SIAM Journal of Computing*, 6(2):323-350.

[Kohler, 1993] Kohler, W. (1993). TPC: Transaction Processing performance Council. *Capacity Management Review*, 21(9):5-6.

[Koicz et al., 2004] Kolcz, A., Chowdhury, A., and Alspector, J. (2004). Improved stability of i-match signatures via lexicon randomization. Technical Report Internal Report, AOL.

[Kowalski and Maybury, 2000] Kowalski, G. and Maybury, M. (2000). *Information Storage and Retrieval Systems: Theory. and Implementation*. Kluwer Academic Publishers, Boston, MA.

[Kraft et al., 1994] Kraft, D., Petry, F., Buckles, B., and Sadasivan, T. (1994). The use of genetic programming to build queries for information retrieval. In *Proceedings of the IEEE Symposium on Evolutionary Computation*, pages 468-473.

[Kristensen, 1993] Kristensen, J. (1993). Expanding end-users' query statements for free text searching with a search-aid thesaurus. *Information Processing and Management*, 29(6):733-744.

[Krovetz, 1993] Krovetz, R. (1993). Viewing morphology as an inference process. In *Proceedings of the Sixteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 191-202.

[Krovetz and Croft, 1989] Krovetz, R. and Croft, W. (1989). Word sense disambiguation using machine-readable dictionaries. In *Proceedings of the Twelfth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 127-136.

[Kukich, 1992] Kukich, K. (1992). Techniques for automatically correcting words in text. *ACM Computing Surveys*, 24(4):377-439.

[Kwok, 1990] Kwok, K. (1990). Experiments with a component theory of probabilistic information retrieval based on single terms as document components. *ACM Transactions on Office Information Systems*, 8(4):363-386.

[Kwok, 1989] Kwok, K.L. (1989). A neural network for probabilistic information retrieval. In *Proceedings of the Twelth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 21-30.

[Kwok, 1995] Kwok, K.L. (1995). A network approach to probabilistic information retrieval. *ACM Transactions on Information Systems*, 13(3):324-353.

[Larkey et al., 2002a] Larkey, L., Allan, J., Connell, M., Bolivar, C., and Wade, C. (2002a). UMass at TREC-2002: Cross language and novelty tracks. In *Proceedings of the Text REtrieval and Evaluation Conference (TREC-2002).*

[Larkey et al., 2002b] Larkey, L., Ballesteros, L., and Connell, M.E. (2002b). Improving stemming for arabic information retrieval: Light stemming and co-occurrence analysis. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 275-282.

[Larkey and Connell, 2000] Larkey, L. S. and Connell, M. E. (2000). Arabic information retrieval at umass in TREC-10. In *Proceedings of the Text Retrieval and Evaluation Conference (TREC-2000)*.

[Larsen and Aone, 1999] Larsen, B. and Aone, C. (1999). Fast and effective text mining using linear-time document clustering. In *Proceedings of the 5th ACM-SIGKDD Special Interest Group on Knowledge and Data Discover*, pages 16-22.

[Lavrenko and Croft, 2001] Lavrenko, V. and Croft, W. B. (2001). Relevance-based language models. In *Proceedings of the 24th A CM SIGIR Conference on Research and Development in Information Retrieval*, pages 120-127.

[Lavrenko et al., 2002] Lavrenko, V. A, Choquette, M., and Croft, W. B. (2002). Cross-lingual relevance models. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, page 175-182.

[Lecroq, 1994] Lecroq, T. (1994). Experimental results on string matching. *Software Practice and Experience*, 25(7):727-765.

[Lee, 1995] Lee, D. (1995). Massive parallelism on the hybrid text-retrieval machine. *Information Processing and Management*, 31 (6):815-830.

[Lee et al., 1997] Lee, D., Chuang, H., and Seamons, K. (1997). Document ranking and the vector-space model. *IEEE Software*, 14(2):67-75.

[Lee and Ren, 1996] Lee, D. and Ren, L. (1996). Document ranking on weight-partitioned signature files. *ACM Transactions on Information Systems*, 14(2): 109-137.

[Lee, 1997] Lee, J. (1997). Analysis of multple evidence combinations. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 267-276.

[Lee et al., 1993] Lee, J.H., Kim, M.H., and Lee, Y.J. (1993). Information retrieval based on conceptual distance in is-a hierarchies. *Journal of Documentation*, 49(2):188-207.

[Lee et al., 1994] Lee, J.H., Kim, M.H., and Lee, Y.J. (1994). Ranking documents in thesaurusbased boolean retrieval systems. *Information Processing and Management*, 30(1):79-91.

[Leek et al., 2000] Leek, T., Jin, H., Sista, S., and Schwartz, R. (2000). The BBN crosslingual topic detection and tracking system. In *Working Notes of the Third Topic Detection and Tracking Workshop*.

[Lenat and Guha, 1989] Lenat, D. and Guha, R. (1989). *Building Large Knowledge-Based Systems*. Addison - Wesley.

[Letsche and Berry, 1997] Letsche, T. and Berry, M. (1997). Large-scale information retrieval with latent semantic indexing. *Information Sciences*, 100:105-137.

[Levenshtein, 1966] Levenshtcin, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Sov. Phys. Dokl.*, 6:707-710.

[Levow and Oard, 2000] Levow, G. and Oard, D. (2000). Translinguai topic tracking with prise. In *Working Notes of the Third Topic Detection and Tracking Workshop*.

[Levow et al., 2004] Levow, G., Oard, D., and Resnik, P. (2004). Dictionary-based techniques for cross-language information retrieval. *Information Processing and Management (to appear)*.

[Lewis and Sparck Jones, 1996] Lewis, D. and Sparck Jones, K. (1996). Natural language processing for information retrieval. *Communications of the ACM*, 39(1):92-101.

[Lin and Chen, 2003] Lin, W. and Chen, H. (2003). Merging restults by using predicted retreival effectiveness. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign*.

[Littman and Jiang, 1998] Littman, Michael L. and Jiang, Fan (1998). A comparison of two corpus-based methods for translingual information retrieval. Technical Report CS-98-11, Duke University.

[Liu and Callan, 2003] Liu, J. and Callan, J. (2003). Content-based retrieval in hybrid peerto-peer networks. In *Proceedings of the ACM Conference on Information and Knowledge Management*.

[Liu et al., 2004] Liu, S., Liu, F., Yu, C., and Meng, W. (2004). An effective approach to document retrieval via utilizing WordNet and recognizing phrases. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 266-272.

[Liu and Croft, 2004] Liu, X. and Croft, W.B. (2004). Cluster-based retrieval using language models. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 186-193.

[Liu et al., 1996] Liu, Y., Dantzig, P., Wu, C., Challenger, J., and Ni, L. (1996). A distributed web server and its performance analysison multiple platforms. In *Proceedings of the Sixteenth IEEE International Conference on Distributed Computing Systems*, pages 665-672.

[Loney, 1997] Loney, K. (1997). *Oracle 8 DBA Handbook*. Osborne McGraw-Hill.

[Lovins, 1968] Lovins, J. (1968). Development of a stemming algorithm. *Mechanical Translation and Computational Linguistics*, 11:22-31.

[Lu et al., 1996] Lu, A., Ayoub, M., and Dong, J. (1996). Ad hoc experiments using EUREKA. In *Proceedings of the Fifth Text REtrieval Conference (TREC-5)*, pages 229-239.

[Lu and Callan, 2004] Lu, J. and Callan, J. (2004). Merging retrieval results in hierarchical peer-to-peer networks. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 472-473.

[Lu et al., 1997] Lu, Z., McKinley, K.S., and Cahoon, B. (1997). A performance evaluation of parallel information retrieval on symmetrical multiprocessors. Technical report, Department of Computer Science, University of Massachusetts.

[Lucarella and Morara, 1991] Lucarella, D. and Morara, R. (1991). FIRST: Fuzzy information retrieval systems. *Journal of Information Science*, 17(2):81-91.

[Luk et al., 2002] Luk, R., Leong, H.V., Dillon, T., Chan, A., Croft, B., and Allan, J. (2002). A survey in indexing and searching XML documents. *Journal of the American Society. of Information Science and Technology (JASIST)*, 53(6):415-437.

[Lundquist et al., 1999] Lundquist, C., Frieder, O., Holmes, D., and Grossman, D. (1999). A parallel relational database management system approach to relevance feedback in information retrieval. *Journal of the American Society for Information Science*, 50(5).

[Lundquist et al., 1997] Lundquist, C., Grossman, D., and Frieder, O. (1997). Improving relevance feedback in the vector space model. In *Proceedings of the Sixth ACM Annual Conference on Information and Knowledge Management (CIKM '97)*.

[Lynch and Stonebraker, 1988] Lynch, C. and Stonebraker, M. (1988). Extended user-defined indexing with application to textual databases. In *Proceedings of the Fourteenth International Conference on Very Large Databases*, pages 306-317.

[Ma et al., 2003] Ma, L., Goharian, N., Chowdhury, A., and Chung, M. (2003). Extracting unstructured data from template generated web documents. In *Proceedings of the ACM Conference on Information and Knowledge*

Management.

[MacFarlane et al., 1999] MacFarlane, A., McCann, J.A., and Robertson, S.E. (1999). PLIERS: A parallel information retrieval system using mpi. In *PVM/MPI*, pages 317-324.

[MacFarlane et al., 1997] MacFarlane, A., Robertson, S.E., and McCann, J.A. (1997). Parallel computing in information retrieval-an updated review. *Journal of Documentation*, 53(3):274-315.

[MacKay and Peto, 1995] MacKay, D. and Peto, L. (1995). A hierarchical dirichlet language model. *Natural Language Engineering*, 1(3):289-307.

[Macleod, 1978] Macleod, I. (1978). A relational approach to modular information retrieval systems design. In *Proceedings of the ASIS Annual Meeting*, pages 83-85.

[Macleod, 1979] Macleod, I. (1979). SEQUEL as a language tor document retrieval. *Journal of the American Society for Information Science*, 30(5):243-249.

[Macleod and Robertson, 1991] Macleod, K. and Robertson, W. (1991). A neural algorithm for document clustering. *Information Processing and Management*, 27(4):337-346.

[Mak et al., 1991] Mak, V., Lee, K.C., and Frieder, O. (1991). Exploiting parallelism in pattern matching: An information retrieval application. *ACM Transactions on Information Systems*, 9(1):52-74.

[Maloney and Niv, 1998] Maloney, J. and Niv, M. (1998). TAGARAB: A last accurate arabic name recogniser using high precision morphological analysis. In *Proceedings of the Workshop on Computational Approaches to Semitic Languages*, Montreal, Canada.

[Manning and Schutze, 1999] Manning, C. and Schutze, H. (1999). *Foundations of Statistical Natural Language Processing*. The MIT Press.

[Manning, 1989] Manning, G. (1989). The use of the DAP, a massively parallel computing system, for information retrieval and processing. In *IEE Colloquium on Parallel Techniques for Information Retrieval Digest*, pages 28-32.

[Maron and Kuhns, 1960] Maron, M.E. and Kuhns, J.L. (1960). On relevance, probabilistic indexing and intormation retrieval. *Journal of the Association for Computing Machines*, 7:216-244.

[Martinez et al., 2003] Martinez, J., Roman, J., Fombella, J., Serrano, A., Ruiz, A., Martinez, P., Goni, J., and Gonzalez, J. (2003). Evaluation of MIRACLE approach results for CLEF 2003. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign.*

[Masand and Stanfill, 1994] Masand, B. and Stanfill, C. (1994). An information retrieval testbed on the CM-5. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*, pages 117-122.

[Mayfield and McNamee, 2003] Mayfield, J. and McNamee, P. (2003). Single n-gram stemming. In *Proceedings of the 26th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 415-416.

[Mazur, 1984] Mazur, Z. (1984). On a model of distributed information retrieval systems based on thesauri. *Information Processing and Management*, 20(4):499-505.

[Mazur, 1988] Mazur, Z. (1988). Properties of a model of distributed homogeneous information retrieval based on weighted models. *Information Processing and Management*, 24(5):525-540.

[McCabe et al., 1999] McCabe, M., Chowdhury, A., Grossman, D., and Frieder, O. (1999). A unified environment for fusion of information retrieval approaches. In *Proceedings of the Eighth ACM Conference on Information and Knowledge Management (CIKM).*

[McCabe et al., 2000] McCabe, M. C., Lee, J., Chowdhury, A., Grossman, D., and Frieder, O. (2000). On the design

and evaluation of a multi-dimensional approach to information retrieval. In *Proceedings of the 23rd ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 363-365.

[McCallum and Nigam, 1998] McCallum, A. and Nigam, K. (1998). A comparison of event models for Naïve Bayes text classification. In *AAAI-1998 Learning for Text Categorization Workshop*, pages 41-48.

[McHugh et al., 1997] McHugh, J., Abiteboul, S., Goldman, R., Quass, D., and Widom., J. (1997). Lore: A database management system for semistructured data. *SIGMOD Record*, 26(3):54-66.

[McKenzie et al., 1990] McKenzie, B.J., Harries, R., and Bell, T. (1990). Selecting a hashing algorithm. *Software Practice and Experience*, 20(2):209-224.

[McNally, 1997] McNally, J. (1997). *Informix Unleashed*. Sams.

[McNamee, 2001] McNamee, P. (2001). Experiments in the retrieval of unsegmented japanese text at the NTCIR-2 workshop. In *Proceedings of the Second NTCIR Workshop*, pages 157-162, Tokyo, Japan.

[McNamee and Mayfield, 2002a] McNamee, P. and Mayfield, J. (2002a). Comparing crosslanguage query expansion techniques by degrading translation resources. In *Proceedings of the 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 405-406.

[McNamee and Mayfield, 2002b] McNamee, P. and Mayfield, J. (2002b). Entity extraction without language-specific resources. In *Proceedings of the Sixth Conference on Natural Language Learning (CoNLL-2002)*.

[Melnik et al., 2000] Melnik, S., Garcia-Molina, H., and Paepcke, A. (2000). A mediation infrastructure for digital library services. In *Proceedings of ACM Digital Libraries*, San Antonio, Texas.

[Miller et al., 1999] Miller, D., Leek, T., and Schwartz, R. (1999). A hidden markov model information retrieval system. In *Proceedings of the 22nd Annual International ACM SIGIR Conference on Research and Developmentin Information Retrieval*, pages 214 -221.

[Minker et al., 1972] Minker, J., Wilson, G., and Zimmerman, B. (1972). An evaluation of query expansion by the addition of clustered terms for a document retrieval system. *Information Storage and Retrieval*, 8(6):329-348.

[Minsky, 1975] Minsky, M. (1975). *The Psychology of Computer Vision*, chapter A Framework for Representing Knowledge, pages 211-277. McGraw-Hill Book Company.

[Moffat and Zobel, 1994] Moffat, A. and Zobel, J. (1994). Fast ranking in limited space. In *Proceedings of the Tenth IEEE International Conference on Data Engineering*, pages 428-437.

[Moffat and Zobel, 1996] Moffat, A. and Zobel, J. (1996). Self-indexing inverted files tbr fast text retrieval. *ACM Transactions on Information Systems*, 14(4):349-379.

[Mohan and Willett, 1985] Mohan, K.C. and Willett, P. (1985). Nearest neighbor searching in serial files using text signatures. *Journal of Information Science Principles and Practice*, 11 (1):31-39.

[Navarro et al., 2000] Navarro, G., de Moura, E. Silva, Neubert, M., Ziviani, N., and Baeza-Yates, R. (2000). Adding compression to block addressing inverted indexes. *Journal of Information Retrieval*, pages 49-77.

[Nejdl et al., 2002] Nejdl, W., Wolf, B., Qu, C., Decker, S., Sintek, M., Naeve, A., Nilsson, M., Palmér, M., and Risch, T. (2002). Edutella: A P2P networking infrastructure based on rdf. In *Proceedings of the World Wide Web Conference.*

[Ney et al., 1994] Ney, H., Essen, U., and Kneser, R. (1994). On structuring probabilistic dependencies in stochastic language modeling. *Computer Speech and Language*, 8(1): 1-38.

[Ng et al., 2003] Ng, W., Ooi, B. C., Tan, K., and Zhou, A. (2003). Peerdb: A P2P-based system for distributed data sharing. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE).*

[Niemi and Jarvelin, 1995] Niemi, T. and Jarvelin, K. (1995). A straightforward NF2 relational interface with applications in information retrieval. *Information Processing and Management*, 31(2):215-231.

[Nilsson, 2004] Nilsson, M. (2004). Id3v2 overview.

[Nunzio et al., 2003] Nunzio, G.M. Di, Ferro, N., Melucci, M., and Orio, N. (2003). The University of Padova at CLEF 2003: Experiments to evaluate probabilistic models for automatic stemmer generation and query word translation. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign.*

[Oard, 2003] Oard, D. (2003). When you come to a fork in the road, take it: Multiple futures for CLIR research. In *Proceedings of the SIGIR 2003 Workshop on the future of Cross-Language Information Retrieval.*

[Oard and Diekema, 1998] Oard, D. and Diekema, A. (1998). Cross-language information retrieval. *Annual Review of information Science and Technology (ARIST)*, 33:223-256.

[Oard et al., 2000] Oard, D., Levow, G., and Cabezas, C. (2000). CLEF experiments at the University of Maryland: Statistical stemming and backoff translation strategies. In *Cross Language Information Retrieval and Evaluation: Workshop of Cross-Language Evaluation Forum, CLEF 2000*, pages 176-187, Lisbon, Portugal.

[Oard et al., 2004] Oard, D., Soergel, D., Doermann, D., Huang, X., Murray, G., Wang, J., Ramabhadran, B., Franz, M., Gustman, S., Mayfield, J., Kharevych, L., and Strassel, S. (2004). Building an information retrieval test collection for spontaneous conversational speech. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 41-48.

[Oard, 2004] Oard, Doug (2004). Personal communication with David Grossman.

[Och and Ney, 2003] Och, F. and Ney, H. (2003). A systematic comparison of various statistical alignment models. *Computational Linguistics*, 29(1):19-51.

[Och and Ney, 2000a] Och, F.J. and Ney, H. (2000a). A comparison of alignment models for statistical machine translation. In *Proceedings of COLING 2000*, pages 1086-1090, Saarbrücken, Germany.

[Och and Ney, 2000b] Och, F.J. and Ney, H. (2000b). Improved statistical alignment models. In *Proceedings of the Association of Computing Linguistics*, pages 440-447, Hong Kong, China.

[Olson, 1995] Olson, C. F. (1995). Parallel algorithms for hierarchical clustering. *Parallel Computing*, 21:1313-1325.

[Osborn et al., 1997] Osborn, M., Strzalkowski, T., and Marinescu, M. (1997). Evaluating document retrieval in patent database: A preliminary report. In *Proceedings of the Sixth ACM International Conference on Information and Knowledge Management*, pages 216-221.

[Ozkarahan, 1995] Ozkarahan, E. (1995). Multimedia document retrieval. *Information Processing and Management*, 31(1):113-131.

[Paepcke et al., 2000] Paepcke, A., Baldonado, M., Chang, C.C.K., Cousins, S., and GarciaMolina, H. (2000). Building the Infobus: A review of technical choices in the Stanford Digital Library Project. Technical Report TR-2000-50, Stanford University.

[Pearce and Nicholas, 1993] Pearce, C. and Nicholas, C. (1993). Generating a dynamic hypertext environment with n-gram analysis. In *Procedings of the Second International Conference on Information and Knowledge Management*, pages 148-153.

[Peat and Willett, 1991] Peat, H.J. and Willett, P. (1991). The limitations of term co-occurrence data for query expansion in document retrieval systems. *Journal of the American Society for Information Science*, 42(5):378-383.

[Petry et al., 1993] Petry, EE., Buckles, B., Prabbu, D., and Kraft, D. (1993). Fuzzy information retrieval using genetic algorithms and relevance feedback. In *ASIS '93 Proceedings of the Fifty-Sixth Annual ASIS Meeting*,

pages 122-125.

[Pirkola, 1998] Pirkola, A. (1998). The effects of query structure and dictionary setups in dictionary-based cross-language information retrieval. In *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 55-63.

[Pirkola et al., 2001] Pirkola, A., Hedlund, T., Keskustalo, H., and Järvelin, K. (2001). Dictionary-based cross-language information retrieval: Problems, methods, and research findings. *Information Retrieval*, 4:209-230.

[Pirkola et al., 2003] Pirkola, A., Toivonen, J., Keskustalo, H., Visala, K., and Jarvelin, K. (2003). Fuzzy translation of cross-lingual spelling variants. In *Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 345-352.

[Pogue and Willett, 1987] Pogue, C.A. and Willett, P. (1987). Use of text signatures for document retrieval in a highly parallel environment. *Parallel Computing*, 4(3):259-268.

[Pollock and Zamora, 1984] Pollock, J. and Zamora, A. (1984). Automatic spelling correction in scientific and scholarly text. *Communications of the ACM*, 27(4):358-358.

[Ponte and Croft, 1998] Ponte, J. M. and Croft, W. B. (1998). A language modelling approach to information retrieval. In *Proceedings of the 21st ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 275-281.

[Porter, 1980] Porter, M. (1980). An algorithm for suffix stripping. *Program*, 14(3):130-137.

[Pulley, 1994] Pulley, E. (1994). A preprocessor for integrating structured data and text. Master's thesis, George Mason University.

[Qu et al., 2003a] Qu, Y., Grefenstette, G., and Evans, D. (2003a). Automatic transliteration for japanese-to-english text retrieval. In *Proceedings of the 23rd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 353-360.

[Qu et al., 2003b] Qu, Y., Grefenstette, G., and Evans, D. (2003b). Clairvoyance CLEF-2003 experiments. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign*.

[Quass et al., 1996] Quass, D., Widom, J., Goldman, R., Haas, K., Luo, Q., McHugh, J., Nestorov, S., Rajaraman, A., Rivero, H., Abiteboul, S., Ullman, J., and Wiener., J. (1996). LORE: A lightweight object REpository for semistructured data. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, Montreal, Canada.

[Rabbiner, 1989] Rabbiner, L. (1989). A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257-286.

[Rada et al., 1987] Rada, R., Mili, H., Bicknell, E., and Blettner, M. (1987). Development and application of a metric on semantic nets. *IEEE Transactions on System, Man, and Cybernetics*, 19(1):17-30.

[Raghavan and Garcia-Molina, 2002] Raghavan, S. and Garcia-Molina, H. (2002). Integrating diverse information management systems: A brief survey. Technical Report TR-2001-57, Stanford University.

[Rasmussen and Willett, 1989] Rasmussen, E. and Willett, P. (1989). Efficiency of hierarchic agglomerative clustering using the ICL distributed array processor. *Journal of Documentation*, 45(1): 1-24.

[Ratnasamy et al., 2001] Ratnasamy, S., Francis, P., Handley, M., Karp, R., and Shenker, S. (2001). A scalable content-addressable network. In *Proceedings of the ACM SIGCOMM.*

[Reddaway, 1991] Reddaway, S. (1991). High speed text retrieval from large databases on a massively parallel processor. *Information Processing and Management*, 27(4):311-316.

[Rijsbergen, 1977] Rijsbergen, C.J. Van (1977). A theoretical basis for the use of cooccurrence data in information retrieval. *Journal of Documentation*, 33(2): 106-119.

[Robertson and Walker, 1994] Robertson, S. and Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. In *Proceedings of the Seventeenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 232-241.

[Robertson and Walker, 1997] Robertson, S. and Walker, S. (1997). On relevance weights with little relevance information. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 16-24.

[Robertson et al., 1995] Robertson, S., Walker, S., Beaulieu, M.M., and Gatford, M. (1995). Okapi at TREC-4. In *Proceedings of the Fourth Text REtrieval Conference (TREC-4)*, pages 73-96.

[Robertson, 1977] Robertson, S.E. (1977). The probability ranking principle in IR. *Journal of Documentation*, 33(4):294-304.

[Robertson, 1990] Robertson, S.E. (1990). On term selection for query expansion. *Journal of Documentation*, 46(4):359-364.

[Robertson, 1997] Robertson, S.E. (1997). Overview of the okapi projects. *Journal of Documentation*, 53(1):3-7.

[Robertson and Sparck Jones, 1976] Robertson, S.E. and Sparck Jones, K. (1976). Relevance weighting of search terms. *Journal ofAmerican Society for Information Science*, 27(3): 129-146.

[Rocchio, 1966] Rocchio, J. J. (1966). *Document Retrieval Systems—Optimization and Evaluation*. PhD thesis, Harvard.

[Rocchio, 1971] Rocchio, J. J. (1971). *The SMART Retrieval System Experiments in Automatic Document Processing*, chapter Relevance Feedback in Information Retrieval, pages 313-323. Prentice Hall.

[Rogati and Yang, 2004] Rogati, M. and Yang, Y. (2004). Resource selection for domainspecific cross-lingual IR. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 154-161.

[Rohrs, 2000] Rohrs, C. (2000). Keyword matching in gnutella. http://www.limewire.org/project/www/Keyword Matching.htm.

[Rohrs, 2001] Rohrs, C. (2001). Search result grouping in gnutella. http://www.limewire.org/project/www/ result_grouping.htm.

[Rosenfeld, 2000] Rosenfeld, R. (2000). Two decades of statistical language modeling: Where do we go from here. *Proceedings of the IEEE*, 88(8).

[Ruiz, 2003] Ruiz, M. (2003). Report on CLEF 2003 experiments at UB. In *Proceedings of the CLEF 2003 Cross-Language Text Retrieval System Evaluation Campaign*.

[Ruocco and Frieder, 1997] Ruocco, A. and Frieder, O. (1997). Clustering and classification of large document bases in a parallel environment. *Journal of the American Society for Information Science*, 48(10):932-943.

[Salton, 1969] Salton, G. (1969). A comparison between manual and automatic indexing methods. *Journal of American Documentation*, 20(1):61-71.

[Salton, 1970a] Salton, G. (1970a). Automatic processing of foreign language documents. *Journal of the American Society for Information Science*, 21(3): 187-194.

[Salton, 1970b] Salton, G. (1970b). Automatic text analysis. *Science*, 168(3929):335-342.

[Salton, 1971a] Salton, G. (1971a). *The SMART Retrieval System—Experiments in Automatic Document Processing*, chapter New Experiments in Relevance Feedback, pages 337-354. Prentice Hall.

[Salton, 1971b] Salton, G. (1971b). *The SMARTRetrieval System—Experiments in Automatic Document Processing*,

chapter Negative Response Relevance Feedback, pages 403-411. Prentice Hall.

[Salton, 1971c] Salton, G. (1971c). *The SMART Retrieval System—Experiments in Automatic Document Processing*, chapter Information Analysis and Dictionary Construction, pages 115-142. Prentice Hall.

[Salton, 1971d] Salton, G. (1971d). *The SMART Retrieval System Experiments in Automatic Document Processing*, chapter Relevance feedback and the optimization of retrieval effectiveness, pages 324-336. Prentice Hall.

[Salton, 1988] Salton, G. (1988). Parallel text search methods. *Communications of the ACM*, 31(2):202-214.

[Salton, 1989] Salton, G. (1989). *Automatic Text Processing*. Addison-Wesley.

[Salton and Buckley, 1988] Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management*, 24(5):513-523.

[Salton and Buckley, 1990] Salton, G. and Buckley, C. (1990). Improving retrieval performance by relevance feedback. *Journal of the American Society for Information Science*, 41(4):288-297.

[Salton and Lesk, 1968] Salton, G. and Lesk, M. E. (1968). Computer evaluation of indexing and text processing. *Journal of the ACM*, 15(1):8-36.

[Salton et al., 1975] Salton, G., Yang, C.S., and Wong, A. (1975). A vector-space model for automatic indexing. *Communications of the ACM*, 18(11):613-620.

[Sanderson, 1997] Sanderson, M. (1997). Duplicate detection in the reuters collection. Technical Report TR-1997-5, Department of Computing Science at the University of Glasgow, Glasgow, UK.

[Sanderson and Joho, 2004] Sanderson, M. and Joho, H. (2004). Forming test collections with no system pooling. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 33-40.

[Sanderson and Rijsbergen, 1991] Sanderson, M. and Rijsbergen, C.J. Van (1991). NRT: News Retrieval Tool. *Electronic Publishing*, 4(4):205-217.

[Sang and Meulder, 2003] Sang, E. F. and Meulder, F. D. (2003). Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of CoNLL-2003*, pages 142-147, Edmonton, Canada.

[Schank, 1975] Schank, R.C. (1975). *Conceptual Information Processing*. Oxford, Amsterdam.

[Schank and Lehnert, 1977] Schank, R.C. and Lehnert, W. (1977). *Human and Artificial Intelligence,* chapter Computer Understanding of Stories, pages 135-139. North-Holland, Amsterdam.

[Schek and Pistor, 1982] Schek, H.J. and Pistor, P. (1982). Data structures for an integrated data base management and information retrieval system. In *Proceedings of the Eighth International Conference on Very Large Data Bases*, pages 197-207.

[Schmidt et al., 2000] Schmidt, A., Kersten, M., Windhouwer, M., and Waas, F. (2000). Efficient relational storage and retrieval of XML documents. In *Proceedings of the Third International Workshop on the Web and Databases*, pages 47-52.

[Schutze and Pedersen, 1997] Schutze, H. and Pedersen, J. (1997). A co-occurrence-based thesaurus and two applications to information retrieval. *Information Processing and Management*, 33(3):307-318.

[Schutze and Silverstein, 1997] Schutze, H. and Silverstein, C. (1997). Projections for efficient document clustering. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 74-81.

[Selinger, 1979] Selinger, P. (1979). Access path selection in a relational database management system. *IBM Technical Disclosure Bulletin*, 22(4): 1657-1660.

[SHA 1, 1995] SHA1 (1995). *Secure Hash Standard*. U.S Department of Commerce/National Institute of Standards and Technology (FIPS PUB 180-1).

[Shanmugasundaram et al., 1999] Shanmugasundaram, J., Tufte, K., He, G., Zhang, C., DeWitt, D., and Naughton, J. (1999). Relational databases for querying XML documents: Limitations and opportunities. In *Proceedings of the 25th Conference on Very Large Databases (VLDB)*.

[Sheridan and Ballerini, 1996] Sheridan, P. and Ballerini, J. P. (1996). Experiments in multilingual information retrieval using the SPIDER system. In *Proceedings of the Nineteenth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 58-65.

[Sheridan et al., 1997] Sheridan, P., Wechsler, M., and Schauble, P. (1997). Cross-language speech retrieval: Establishing a baseline performance. In *Proceedings of the 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 99-108.

[Sherman, 1994] Sherman, Franklin (1994). Personal communication with David Grossman.

[Shin, 1998] Shin, D. (1998). BUS: An effective indexing and retrieval scheme in structured documents. In *Proceedings of International Conference on Digital Libraries*.

[Shivakumar and Garcia-Molina, 1996] Shivakumar, N. and Garcia-Molina, H. (1996). Building a scalable and accurate copy detection mechanism. In *Proceedings of the Second International Conference on Theory and Practice of Digital Libraries*.

[Shivakumar and Garcia-Molina, 1998] Shivakumar, N. and Garcia-Molina, H. (1998). Finding near-replicas of documents on the web. In *Proceedings of the Workshop on Web Databases (WebDB 98)*.

[Shuey et al., 1997] Shuey, R., Spooner, D., and Frieder, O. (1997). *The Architecture of Distributed Computer Systems: A Data Engineering Perspective*. Addison Wesley.

[Silvestri et al., 2004] Silvestri, F., Orlando, S., and Perego, R. (2004). Assigning identifiers to documents to enhance to clustering property of fulltext indexes. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 305-312.

[Singhal, 1997] Singhal, A. (1997). *Term Weighting Revisited*. PhD thesis, Cornell University.

[Smeaton et al., 1997] Smeaton, A., Kelledy, F., and Quinn, G. (1997). Ad hoc retrieval using thresholds, wsts for french monolingual retrieval, document-at-a-glance for high precision and triphone windows for spoken documents. In *Proceedings of the Sixth Text Retrieval Conference (TREC-6)*.

[Smeaton and Rijsbergen, 1983] Smeaton, A.F. and Rijsbergen, C.J. Van (1983). The retrieval effects of query expansion on a feedback document retrieval system. *The Computer Journal*, 26(3):239-246.

[Somers, 2001] Somers, H. (2001). Bilingual parallel corpora and language engineering. In *Anglo-Indian Workshop Language Engineering for South-Asian Languages (LESAL)*.

[Song and Croft, 1999] Song, F. and Croft, W.B. (1999). A general language model for information retrieval. In *Proceedings of the Eighth International Conference on Information and Knowledge Management (CIKM '99)*.

[Sparck Jones, 1979a] Sparck Jones, K. (1979a). Experiments in relevance weighting of search terms. *Information Processing and Management*, 15(3):133-144.

[Sparck Jones, 1979b] Sparck Jones, K. (1979b). Search term relevance weighting given little relevance information. *Journal of Documentation*, 35(1):30-48.

[Sparck Jones and Barber, 1971] Sparck Jones, K. and Barber, E.O. (1971). What makes an automatic keyword classification effective? *Journal of the American Society for Information Science*, 22(3):166-175.

[Sparck Jones and Jackson, 1968] Sparck Jones, K. and Jackson, D. (1968). Some experiments in the use of automatically obtained term clusters for retrieval. *Mechanized Information Storage, Retrieval and Dissemination*, pages 203-212.

[Sparck Jones and Willett, 1997] Sparck Jones, K. and Willett, P. (1997). *Readings in Information Retrieval.* Morgan Kaufmann Publishers, Inc.

[Spink, 1994] Spink, A. (1994). Term relevance feedback and query expansion: Relation to design. In *Proceedings of the Seventeenth Annual International ACM SIGIR Conference on Research and Devlopment in Information Retrieval*, pages 81-90.

[Spink, 1995] Spink, A. (1995). Term relevance feedback and mediated database searching: Implications for information retrieval practice and systems design. *Information Processing and Management*, 31 (2): 161-171.

[SQL, 2003] SQL (2003). SQL Standard: Standard database languages -SQL, ISO, IEC 9075 -*.

[Sripanidkulchai et al., 2003] Sripanidkulchai, K., Maggs, B., and Zhang, H. (2003). Efficient content location using interest-based locality in peer-to-peer systems. In *Proceedings of the IEEE INFOCOM.*

[Stanfill, 1990] Stanfill, C. (1990). Partitioned posting files: A parallel inverted file structure for information retrieval. In *Proceedings of the Thirteenth Annual ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 413-428.

[Stanfili and Kahle, 1986] Stanfill, C. and Kahle, B. (1986). Parallel free-text search on the connection machine system. *Communications of the ACM*, 29(12): 1229-1239.

[Stanfill and Thau, 1991] Stanfill, C. and Thau, R. (1991). Information retrieval on the connection machine: 1 to 8192 gigabytes. *Information Processing and Management*, 27(4):285-310.

[Stanfill et al., 1989] Stanfili, C., Thau, R., and Waltz, D. (1989). A parallel indexed algorithm for information retrieval. In *Proceedingx of the Twelfth Annual ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 88-97.

[Steinbach et al., 2000] Steinbach, M., Karypis, G., and Kumar, V. (2000). A comparison of document clustering techniques. In *Proceedings of the Knowledge Data and Discovery. (KDD) Workshop on Text Mining.*

[Stoica et al., 2001] Stoica, I., Morris, R., Karger, D., Kaashoek, F., and Balakrishnan, H. (2001). Chord: A scalable peer-to-peer lookup service for internet applications. In *Proceedings of the ACM SIGCOMM.*

[Stone, 1987] Stone, H. (1987). Parallel querying of large databases: A case study. *Computer*, 20(10): 11-21.

[Stonebraker et al., 1983] Stonebraker, M., Stettner, H., Lynn, N., Kalash, J., and Guttman, Antonin (1983). Document processing in a relational database system. *ACM Transactions on Office Information Systems*, 1 (2):143-158.

[Strzalkowski et al., 1997] Strzalkowski, T., Lin, Fang, and Perez-Carballo, J. (1997). Natural language information retrieval TREC-6 report. In *Proceedings of the Sixth Text REtrieval Conference (TREC-6)*, pages 209-228.

[Sundheim, 1995] Sundheim, B.M. (1995). Overview of results of the MUC-6 evaluation. In *Proceedings of the Sixth Message Understanding Conference (MUC-6)*, pages 13-32.

[Tang et al., 2003] Tang, C., Xu, Z., and Dwarkadas, S. (2003). Peer-to-peer information retrieval using self-organizing semantic overlay networks. In *ACM SIGCOMM.*

[Tang et al., 2002] Tang, C., Xu, Z., and Mahalingam, M. (2002). Peersearch: Efficient information retrieval in peer-to-peer networks. Technical Report HPL-2002-198.

[Teuful, 1988] Teuful, B. (1988). Statistical n-gram indexing of natural language documents. *International Forum of Information and Documentation*, 13(4):3-10.

[Teuful, 1991] Teuful, B. (1991). Office document retrieval. *International Forum of Information and Documentation,* 16(4): 15-19.

[Thorelli, 1962] Thorelli, Lars Erik (1962). Automatic correction of errors in text. *BIT*, 2:45-62.

[Tomasic and Garcia-Molina, 1993] Tomasic, A. and Garcia-Molina, Hector (1993). Performance of inverted indices in shared-nothing distributed text document information retreival systems. In *Proceedings of the Second International Conference on Parallel and Distributed Information Systems*, pages 8-17.

[TREC, 2003] TREC (2003). *NIST Special Publication 500-255 : The Twelfth Text REtrieval Conference (TREC 2003)*. NIST.

[Trotman, 2003] Trotman, A. (2003). Compressing inverted files. *Information Retrieval*, 6:5-19.

[Turski, 1971] Turski, W.M. (1971). On a model of information retrieval systems based on thesaurus. *Information Storage and Retrieval*, 7(201):201-205.

[Turtle, 1991] Turtle, H. (1991). *Inference Networks for Document Retrieval*. PhD thesis, University of Massachusetts, Amherst.

[Uratani and Takeda, 1993] Uratani, N. and Takeda, M. (1993). A fast string-searching algorithm for multiple patterns. *Information Processing and Management*, 29(6):775-791.

[V0.4, 2004] V0.4, Gnutella Protocol Specification (2004). Gnutella protocol specification v0.4.

[V0.6, 2004] V0.6, Gnutella Protocol Specification (2004). Gnutella protocol specification v0.6.

[Vercoustreet al., 2002] Vercoustre, A.-M., Thorn, J.A., Krumpholz, A., Mathieson, I., Wilkins, P., Wu, M., Craswell, N., and Hawking, D. (2002). CSIRO INEX experiments: XML search using PADRE. In *Proceedings of the First Workshop of the INitiative for the Evaluation of XML Retrieval (INEX)*, pages 65-70.

[Veronis, 2000] Veronis, Jean (2000). *Parallel Text Processing*: *Alignment and Use of Translation Corpora.* Kluwer Academic Publishers.

[Vogel et al., 1996] Vogel, S., Ney, H., and Tillmann, C. (1996). HMM-based word alignment in statistical translation. In *Proceedings of COLING 1996*, pages 836-841, Copenhagen.

[Voorhees, 1986] Voorhees, E. (1986). Implementing agglomerative hierarchic clustering algorithms for use in document retrieval. *Information Processing and Management*, 22(6):465-476.

[Voorhees, 1993] Voorhees, E. (1993). On expanding query vectors with lexically related words. In *Proceedings of the Second Text REtrieval Conference (TREC-2)*, pages 223-231.

[Walden and Sere, 1988] Walden, M. and Sere, K. (1988). Free text retrieval on transputer networks. *Microprocessors and Microsystems*, 13(3):179-187.

[Wang et al., 1985] Wang, Y., Vandendorpe, J., and Evens, M. (1985). Relational thesauri in information retrieval. *Journal of the American Society. for Information Science*, 36(1):15-27.

[White and Date, 1989] White, C. and Date, C.J. (1989). *A Guide to SQI/DS*. Addison-Wesley.

[Wilkinson, 1994] Wilkinson, R. (1994). Effective retrieval of structured documents. In *Proceedings of the Seventeenth Annual International ACM SIGIR Conference on Research and Devlopment in Information Retrieval*, pages 311-317.

[Willet, 1988] Wiilet, P. (1988). Recent trends in hierarchical document clustering: A critical review. *Information Processing and Management*, 24(5):577-597.

[Willett, 1988] Wiilett, P. (1988). Recent trends in hierarchic document clustering: A critical review. *Information Processing and Management*, 24(5):577-597.

[Willett, 1990] Willett, P. (1990). Document clustering using an inverted file approach. *Journal of Information Science*, 2:223-231.

[Winograd, 1983] Winograd, T. (1983). *Language as a Cognitive Process: Volume I: Syntax*. Addison-Wesley.

[Witten et al., 1999] Witten, I., Moffat, A., and Bell, T. (1999). *Managing Gigabytes*. Van Nostrand Reinhold.

[Wong et al., 1985] Wong, S.K.M., Wojciech, Z., and Wong, P. (1985). Generalized vector space model in information retrieval. *Proceedings of the Eighth Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 18-25.

[Wu and Salton, 1981] Wu, H. and Salton, G. (1981). The estimation of term relevance weights using relevance feedback. *Journal of Documentation*, 37(4):194-214.

[Xu and Weischedel, 1999] Xu, J. and Weischedel, R. (1999). TREC-9 cross-lingual retrieval at bbn. In *Proceedings of the Text REtrieval and Evaluation Conference* (TREC-9).

[Xu et al., 2001] Xu, J., Weischedel, R., and Nguyen, C. (2001). Evaluating a probabilistic model for cross-lingual information retrieval. In *Proceedings of the 24th ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 105-110.

[Xu et al., 2003] Xu, W., Liu, X., and Gong, Y. (2003). Document clustering based on nonnegative matrix factorization. In *Proceedings of the 26th ACM SIGIR 2003 Conference on Research and Development in Information Retrieval*, pages 267-273.

[Xue and Converse, 2002] Xue, N. and Converse, S. (2002). Combining classifiers for chinese word segmentation. In *Proceedings of the SIGHAN Workshop*.

[Yang and Korfhage, 1993] Yang, J. and Korfhage, R. (1993). Effects of query term weights modification in document retrieval: A study based on a genetic algorithm. In *Proceedings of the Second Annual Symposium on Document Analysis and Information Retrieval*, pages 271-285.

[Yang and Korfhage, 1994] Yang, J.J. and Korfhage, R. (1994). Query modification using genetic algorithms in vector space models. *International Journal of Expert Systems*, 7(2): 165-191.

[Yannakoudakis et al., 1982] Yannakoudakis, E.J., Goyal, P., and Huggill, J.A. (1982). The generation and use of text fragments for data compression. *Information Processing and Management*, 18(1): 15-21.

[Yee et al., 1993] Yee, W., Wong, P., and Lee, D. (1993). Implementations of partial document ranking using inverted files. *Information Processing and Management*, 29(5):647-689.

[Yee and Frieder, 2004] Yee, W.G. and Frieder, O. (2004). The design of pirs, a peer-to-peer information retrieval system. In *Proceedings of the VLDB 2004 Second International Workshop On Databases, Information Systems and Peer-to-Peer Computing*. Springer-Verlag Lecture Notes in Computer Science.

[Yu et al., 2003] Yu, B., Liu, J., and Ong, C.S. (2003). Scalable P2P information retrieval via hierarchical result merging. Technical report, University of Illinois at Urbana-Champaign.

[Yu et al., 1983] Yu, C.T., Buckley, C., Lam, I.I., and Salton, G. (1983). A generalized term dependence model in information retrieval. *Information Technology: Research and Development*, 2(4): 129-154.

[Yu et al., 1989] Yu, C.T., W, Meng, and Park, S. (1989). A framework for effective retrieval. *ACM Transactions on Database Systems*, 14(2): 147-167.

[Yuwono and Lee, 1996] Yuwono, B. and Lee, D. (1996). Search and ranking algorithms for locating resources on the world wide web. In *Proceedings of the Twelfth IEEE International Conference on Data Engineering*, pages 164-171.

[Zadeh, 1965] Zadeh, L. A. (1965). Fuzzy sets. *Information Control*, 8:338-353.

[Zahariev, 2004] Zahariev, M. (2004). Automatic sense disambiguation for acronyms. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 586-587.

[Zamora et al., 1981] Zamora, E.M., Pollock, J.J., and Zamora, A. (1981). The use of trigram analysis for spelling error detection. *Information Processing and Management*, 17(6):305-316.

[Zeng et al., 2004] Zeng, H.J., He, Q.C., Chen, Z., and Ma, W.Y. (2004). Learning to cluster web search results. In *ACM 27th Conference on Research and Development in Information Retrieval (SIGIR)*, pages 210-217.

[Zhai and Lafferty, 2001a] Zhai, C. and Lafferty, J. (2001a). Model-based feedback in the language modeling approach to information retrieval. In *Proceedings of the Tenth International Conference on Information and Knowledge Management (CIKM 2001)*.

[Zhai and Lafferty, 2001b] Zhai, C. and Lafferty, J. (2001b). A study of smoothing methods for language models applied to ad hoc information retrieval. In *Proceedings of the 24th ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 334-342.

[Zhai and Lafferty, 2002] Zhai, C. and Lafferty, J. (2002). Two-stage language models for informatioo retrieval. In *Proceedings of the 25th ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 49-56.

[Zhang et al., 1996] Zhang, T., Ramakrishnan, R., and Livny, M. (1996). Birch: An efficient data clustering method for very large databases. In *Proceedings of 1996 ACM-SIGMOD Special Interest Group on Management of Data*, Montreal, Canada.

[Zhao and Karypis, 2002] Zhao, Y. and Karypis, G. (2002). Evaluations of algorithms for obtaining hierarchical clustering solutions. In *Proceedings of the 2002 ACM International Conference on Information and Knowledge Management (ACM-CIKM)*, Washington D.C.

[Zipf, 1949] Zipf, G.K. (1949). *Human Behaviour and the Principle of Least Effort*. Addison-Wesley.

[Zobel et al., 1992] Zobel, J., Moffat, A., and Sacks-Davis, R. (1992). An efficient indexing technique for full-text database systems. In *Proceedings of the Eighteenth International Conference on Very Large Databases*, pages 352-362.

[Zobel et al., 1995] Zobel, J., Moffat, A., Wilkinson, R., and Sacks-Davis, R. (1995). Efficient retrieval of partial documents. *Information Processing and Management*, 31 (3):361-377.

索　引

索引中的页码为英文原书的页码，与书中边栏的页码一致。

2-Poission（二元泊松分布），37
Antonym（反义词），131

B-Tree（B 树），220
Bitonic sort（双调归并排序算法），267
Boolean query（布尔查询），135，236，241
Boolean request（布尔请求），67-69，86，91，132，138，237，262
Boyer-Moore（波伊尔-摩尔字符串匹配算法，通称 BM 算法），202-203，263
Buckshot clustering（Buckshot 聚类算法，亦称大团聚类算法）111，129

CACM（ACM 通信杂志语料库，这里指的是 ACM 通信文章标题与摘要组成的文档集），76，84，101
Centroid vector （中心向量），109，117-118，285
CISI（发明与科技情报中心语料库），73，101
Collision（冲突），202
Compression（压缩），7，116，183，185-191，208，267，270，283
Connection machine（连接机[①]）258，260，264，267，273
Context term（上下文的词项），128
Cosine，18-19，43，72，90，96，110，113，117-118，128，138，195，198-199，242-243
Cranfield（克兰菲尔德文档集，一个公开的评测文档集的名称），31，33，42-43，77，83，102
CYC（CYC 项目[②]），142

DAP，DAP 机（一种并行机的名称），264，267-268，273
Data integration（数据融合），7，253
Database management system（数据库管理系统），6，28，211-213
DBMS（数据库管理系统的简称），6，28，213-215，222-226，229-232
Dec-Hi ，101
Dice（Dice 系数），18，118
Digital array processor（数字阵列处理器），106，112，261
Disjunctive normal form，（析取范式），135-136
Document length（文档长度），18-19，33-36，38，78，86，104，141，198-199
Document routing（文档分发），1-2，82
DOE（美国能源部语料库），83-84
Dynamic programming（动态规划），143

Extended Boolean retrieval（扩展布尔检索），73，132，137

FIRST（FIRST 系统），88-89
Fitness function（适应度函数），80-83
Frequency distribution（频率分布），189，268
Fuzzy set（模糊集），10，84-86，88，91

Genetic algorithm（遗传算法），81-84
Google，282
GRANT（GRANT 系统），138-139
Group average（组平均），79，109，129

Hash function（散列函数），185，202，229
Hashing（散列），185，199-201，208，232，262，270
Hidden Markov Model（隐马尔科夫模型），157，164
Hypertext（超文本），118，281，285-286

Independence assumptions（独立性假定），21-25，27，42，44，122
Inference network（推理网络），44，57-63，67，90
Information extraction（信息抽取），140，296
Inner product（内积），11，18，33，70，96，118，243
INSPEC（INSPEC 语料库），101，119
Intel（英特尔），269-270，273
Inverted indexing（倒排索引），182

Jaccard（Jaccard 系数），19，82，84，118

K-distance（K-距离），136-137
KMP（KMP 算法），202
Knowledge base（知识库），142，227

① 20 世纪 80 年代早期 MIT 研制的超级计算机系列的名称，主要用于人工智能处理。——译者注

② Douglas Lenat 设立，旨在构建覆盖大部分人类知识的知识库，被誉为“人工智能领域最难的任务之一”。——译者注

KSTEM.，KSTEM 系统（一个词还原系统），141

Latent semantic indexing（隐性语义索引），70，73，90，113，129
Link matrices（链接矩阵），61，65-67
Lovins（Lovins 算法），141
LSI （隐性语义索引），73-74，90

Maximum likelihood（最大似然概率），46-48，52，54
MED（MED 语料库），73，101
Mediator（协同器，xv-xvi），215，251-254
MEDLARS（MEDLAR 语料库），76
MIMD（多指令流多数据流），273
MUC（Message Understanding Conference），144

N-gram（N 元），6，116-117，119，297
Natural language processing（自然语言处理），70，140，296-297
NCR（NCR 公司），211，213
Neural network（神经网络），74，77-79
NLP（自然语言处理），140，142-143
Non-binary independence model（非二值独立模型），21，33
NPL（NPL 语料库），33

OCR（光学字符识别），115

P2P（点对点），276，286-288
PageRank algorithm（PageRank 算法），282
Parameter estimation（参数估计），21，42
Parsing（语言解析），6，118，140-144，146，183，232，235，297
Part-of-speech（词性），140，142-144，252
Polynomial regression（多项式回归），119-120
Potter（Potter 算法），141
Probabilistic retrieval（概率检索），10，21，25，57，73-74，77，98-99，101
PTRANS（物理交通动词类别），142

R-distance（R-距离），135，137
Regression analysis（回归分析），94，119，146
Relevance feedback（相关反馈），41-43，74-79，83，94，96，98-101，104-105，107，113，146，244
Relevance ranking（相关度排序），21，28，57，67，77，86，131-132，134，141，184，223，229，236，241，261，269，283

Semantic network（语义网络），132-134，136-139，146
Signature file（签名文件），201，208，258，262-263
SIMD（单指令流多数据流），257，273
Similarity measure（相似度度量方法），38，108，118-119，125，135-136，146
Singular value decomposition（奇异值分解），70，129
Slots（槽），133
SMART.SMART 系统（很有名的一套检索系统），223
Sort order（排序次序），102-104
Spreading activation（激活扩散），134，136，139，285
SQL，215，219-220，223-224，226，228-231，234-239，242-244，254，270
Staged logistic regression（分段逻辑回归），121
Stemming（词干还原），73，82，117-119，124，127，141，186
Stop word（停用词），141，185，197
Structured data（结构化数据），6-7，144，211-213，223，226，231，235
Synonym（同义词），127，131-132，277

Target term（目标词），127-128
Term frequency（词频），14，16-18，20，32-33，35-38，40-41，63，78，86，183-185，190，197-198，201，229，233，238，242
Term independence（词的独立性），20，98，
Term weights（词权重），12，15，21，25，28，33，42，44，67，69，96，98，104，145，197
Text scanning（文本扫描），7，201-202，258，273
Thesaurus（知识库），10，62，122-123，125-126，128-134，138，146，277-279
TIPSTER（TIPSTER 文档集），73，78，83-84，101，104，119，130，132，229，231
TREC（Text Retrieval Conference），8，19-21，91，97，99，104，112，132，140-141，146，186，196，198-199，229，271，284，292-296
Trigram（三元模型），263
Troponym（共现词），134

Unstructured data（非结构化数据），7

Vector space model（向量空间模型），10-11，17，20，28，32-33，43，69，73-74，77-78，89-90，96，99，116-118，182，195，198
VLSI（超大规模集成电路），259

Wordnet，134，138
World Wide Web（万维网），8，241，258，293，296
WWW，296

Zipf's Law（齐普夫定律），181
Zipfian distribution（齐普夫分布），91，116，129，183

TURING 图灵教育 站在巨人的肩上

读者积分赠书卡

手机号码：_____（此为会员编号）

姓　　名：_____　性别：□男 □女　出生年月：____年__月

通信地址：_____

邮政编码：_____

电子邮件：_____

您购买的图书是：23575 /《信息检索：算法与启发式方法（第2版）》

（49.00元）

您获得的会员积分是：4.9 分

欢迎参加“**有奖DEBUG**”活动。提交本书勘误，每确认一处即可获赠积分5分。详情见图灵网站。

请沿虚线剪下此页，寄回图灵公司，即可成为图灵读者俱乐部的一员（复印无效）。**积分累计，可获赠书**（赠书清单见图灵网站）。

邮政编码：100107

通信地址：北京市朝阳区北苑路13号院1号楼C603

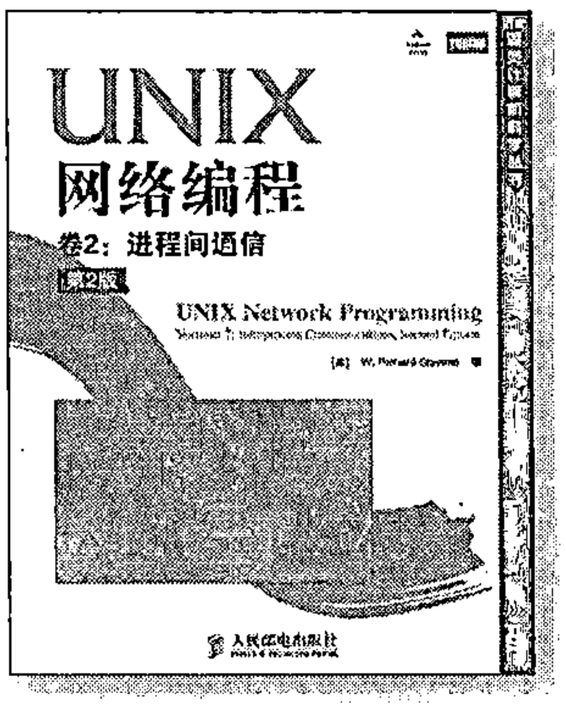
北京图灵文化发展有限公司　图灵读者俱乐部

TURING 图灵教育

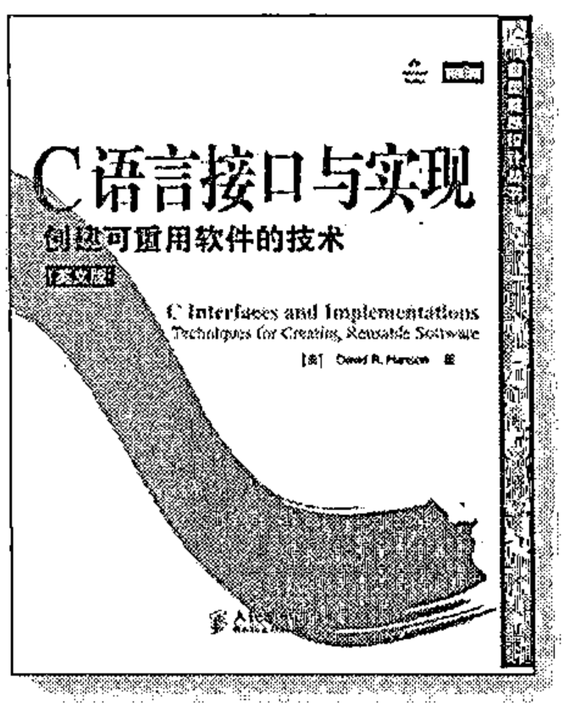
图灵最新重点图书



书名：JavaScript高级程序设计（第2版）
书号：978-7-115-23095-9
- JavaScript经典教程
- Amazon超级畅销书
- 前端开发人员必备



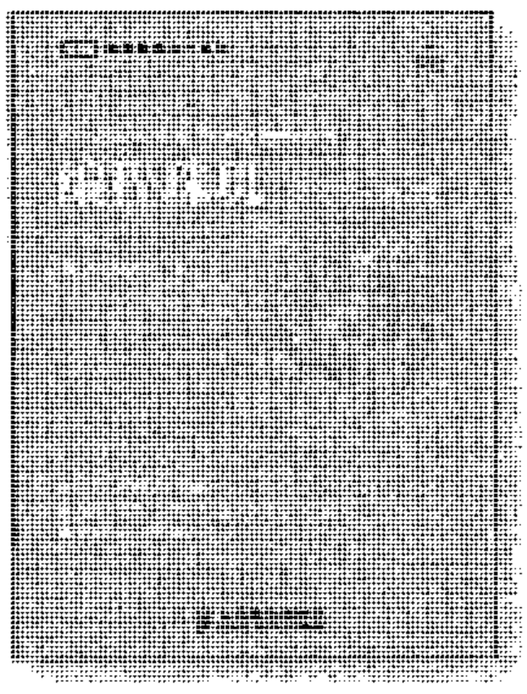
书名：UNIX网络编程 卷2：进程间通信（第2版）
书号：978-7-115-23028-7
- 世界顶级网络专家传世之作
- 网络编程权威参考



书名：C语言接口与实现：创建可重用软件的技术（英文版）
书号：978-7-115-23113-0
- 从C语言新手变成高手的必读之作
- UNIX和网络专家Stevens力荐



书名：人月神话（英文版）
书号：978-7-115-23268-7
- 图灵奖得主总结软件项目实战得失
- 软件工程领域35年畅销不衰的经典
- 软件人业人员不可不读的宝书

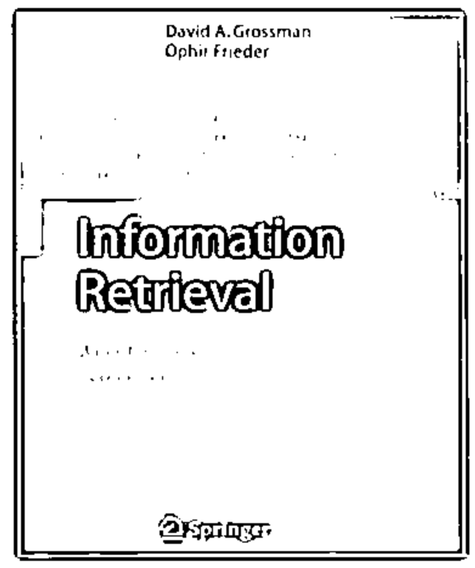


书名：编程珠玑（英文版·第2版）
书号：978-7-115-23260-1
- 计算机科学的不朽经典
- 融深邃思想、实战技术与趣味轶事于一炉的奇书
- 字字珠玑，意味隽永



书名：赢在设计：网页设计如何大幅提升网站收益
书号：978-7-115-22941-0
- 揭示网站设计盈利秘诀

信息检索
算法与启发式方法（第2版）



Information Retrieval
Algorithms and Heuristics
Second Edition

“本书涉及最新的研究成果，语言经得起推敲，还精心准备了大量的实例说明，适合作为研究生和本科生信息检索课程的首选教材。”

——W. Bruce Croft，美国马萨诸塞大学阿默斯特校区计算机系杰出教授

“推荐把本书作为计算机科学专业学生的首选教材，同时也适用于SEO专业人员和Web开发者阅读。将搜索技术、算法和启发式方法运用于他们的项目中。”

——E. Garcia博士，信息技术与服务顾问

随着Google、百度等搜索引擎公司的崛起，信息检索已经成为令人振奋的热门研究领域。

本书从发展的角度描述了ad hoc信息检索，讨论了用来实现大规模数据检索的最新算法，详细介绍了推理网络和系统的效率，并且对每种方法都给出了详细可行的实例。此外，本书整合了结构化和非结构化数据的处理技术，这是其他教材所不具备的。

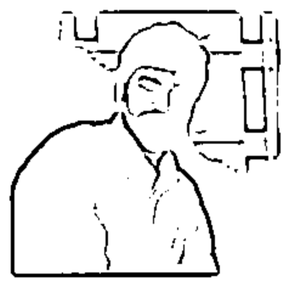
第2版新增加了IR语言模型和跨语言检索，还讨论了许多当前的热点话题，如XML、P2P信息检索、文本查重、文档并行聚类、不同检索策略的融合、信息中间表示等。

本书兼顾了学科广度和主题深度，把握了最新的发展趋势，是信息检索领域的一本名著，更为许多著名高校（如美国普林斯顿大学、罗格斯大学）采用为教材。



David A. Grossman

佐治亚梅森大学博士，现在伊利诺伊理工大学计算机系任教。曾在美国政府部门高级技术服务中心和研究发展办公室担任项目经理。主要研究领域包括信息检索、结构化与非结构化数据集成以及数据挖掘。



Ophir Frieder

乔治敦大学教授、计算机科学系主任。曾任伊利诺伊理工大学计算机系首席教授、学院数据检索实验室主任。ACM会员，IEEE和美国艺术与科学研究院高级会员。他在数据检索系统、通信系统、高性能系统结构等方面均有

Springer

图灵网站 http://www.turingbook.com 热线：(010) 51095186
反馈/投稿/推荐信箱：contact@turingbook.com
有奖勘误：debug@turingbook.com

分类建议：计算机/计算机科学

人民邮电出版社网址 www.ptpress.com.cn

ISBN 978-7-115-23575-6

9 787115 235756 >

ISBN 978-7-115-23575-6

定价：49.00 元